Logik für Informatik

Wintersemester 2024/25

Thomas Schwentick / Thomas Zeume

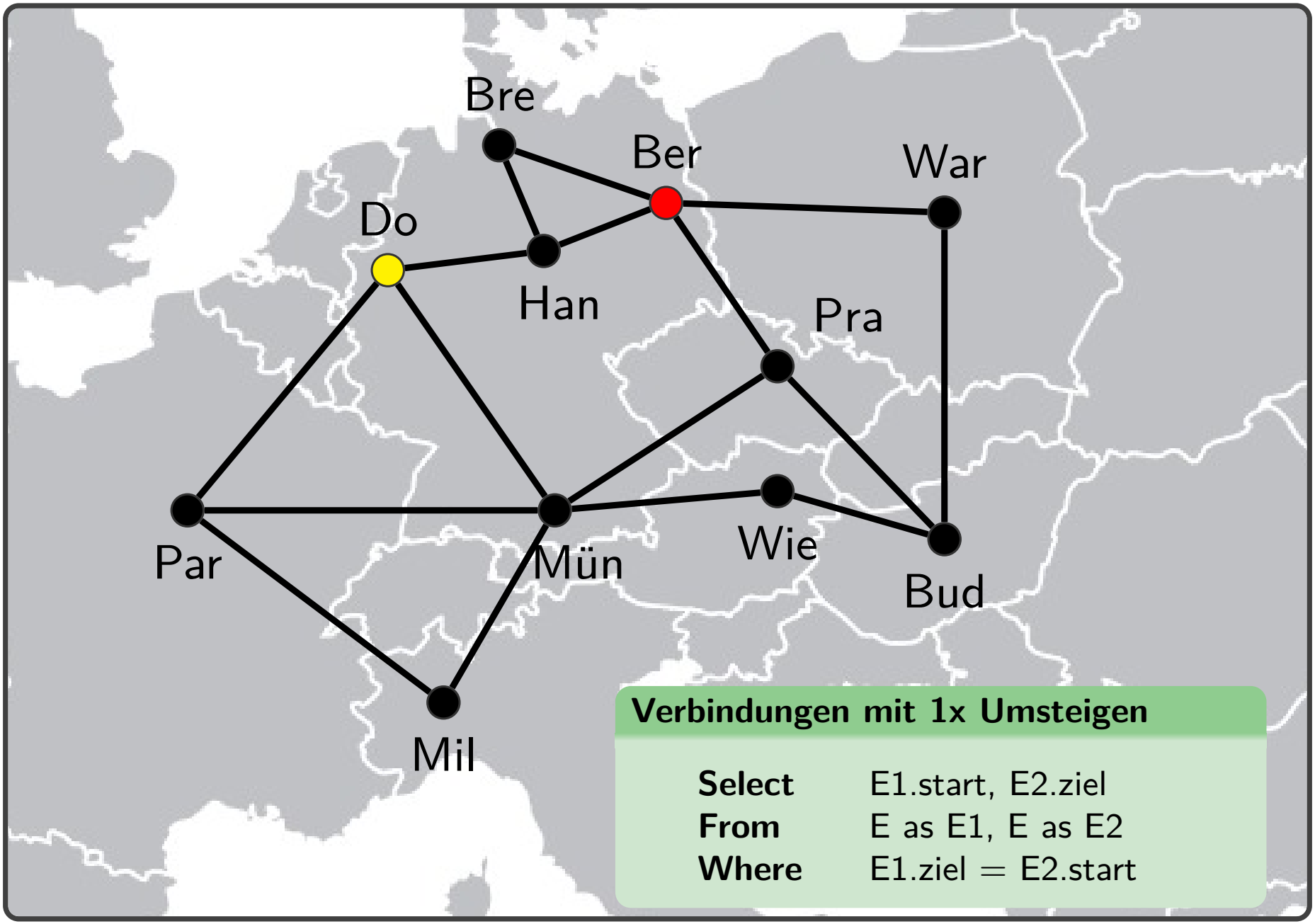
Version vom 7. Oktober 2024, 07:59

Kapitel A.0:

Einführung

Inhalte der Vorlesung

Gründe für formale Logik: Datenbanken

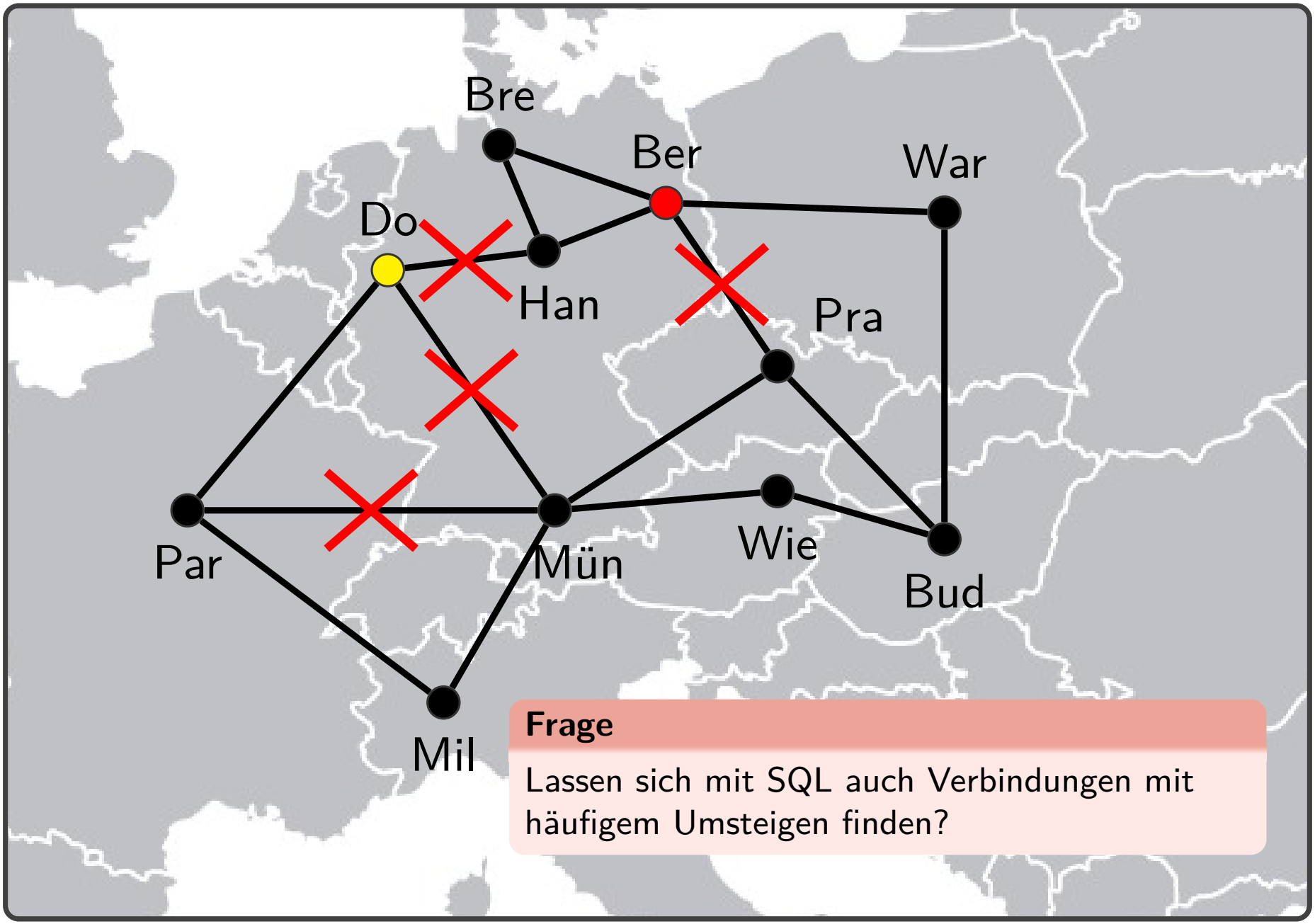


Verbindungen mit 1x Umsteigen

Select	E1.start, E2.ziel
From	E as E1, E as E2
Where	E1.ziel = E2.start

Streik!

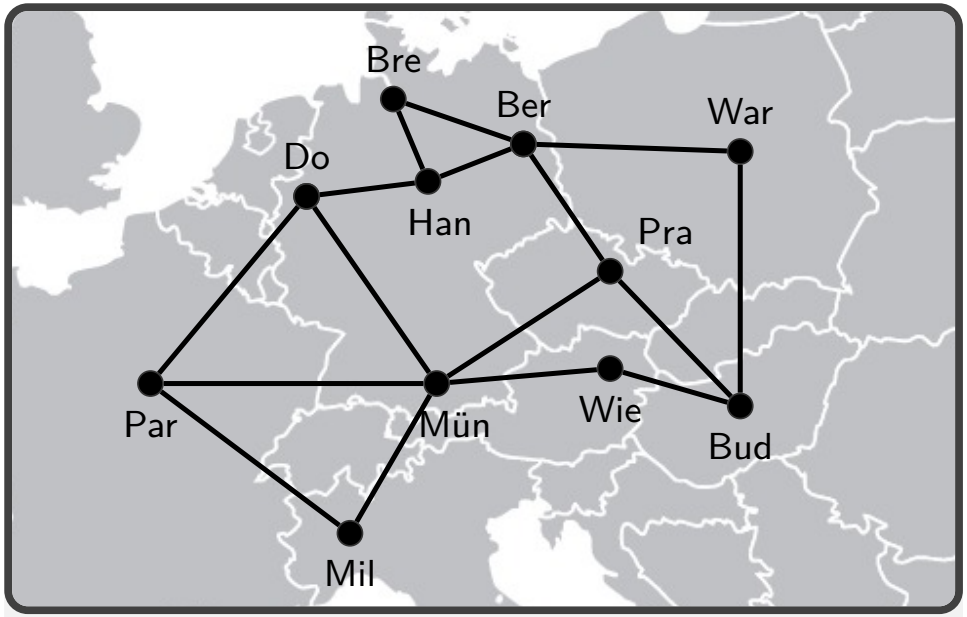
Gründe für formale Logik: Datenbanken



Frage

Lassen sich mit SQL auch Verbindungen mit häufigem Umsteigen finden?

Gründe für formale Logik: Datenbanken



Erreichbarkeitsanfrage

Gegeben: Ein Graph $G = (V, E)$
Gesucht: Alle Paare $(x, y) \in V \times V$, sodass es einen Weg von x nach y in G gibt

Frage

Lässt sich die Erreichbarkeitsanfrage in (Kern-)SQL ausdrücken?

- **Kern-SQL:** Kein Grouping, keine Aggregation, keine Rekursion

- **Idee:** Beantwortung der Frage mit **logischen Methoden**

Fragestellung

↓

Übersetzung in logische Fragestellung

↓

Lösung mit Methoden der Logik

↓

Rückübertragung der Lösung

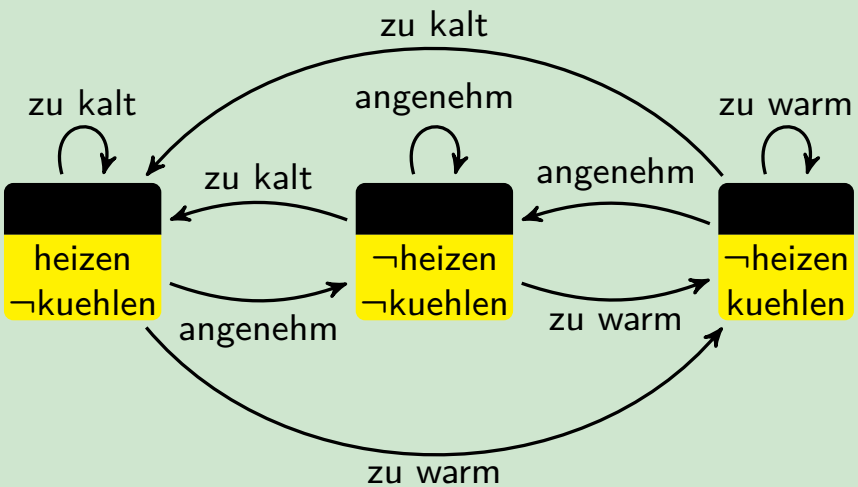
- Passende Logik:
  Prädikatenlogik (kurz: FO)

Gründe für formale Logik: Verifikation von Systemen

Beispiel: Klimasteuerung

- Regelung der Raumtemparatur
- Mögliche Raumzustände: angenehm, zu kalt, zu warm
- Aktionen:
  - falls zu kalt: Heizeinheit aktivieren, Kühleinheit deaktivieren
  - falls zu warm: Heizeinheit deaktivieren, Kühleinheit aktivieren

- Modellierung als formales System:



- Gewünschte Eigenschaft:

  Ist es zu warm, so ist im nächsten Schritt die Kühlung aktiv und die Heizung nicht

Frage

Wie lässt sich automatisiert überprüfen, ob ein System eine Eigenschaft hat?

- **Grundidee:**
  - Formalisiere die Eigenschaft durch eine logische Formel φ
  - Überprüfe, ob das formale Modell M die Formel erfüllt

- Passende Logiken:
  Temporallogiken

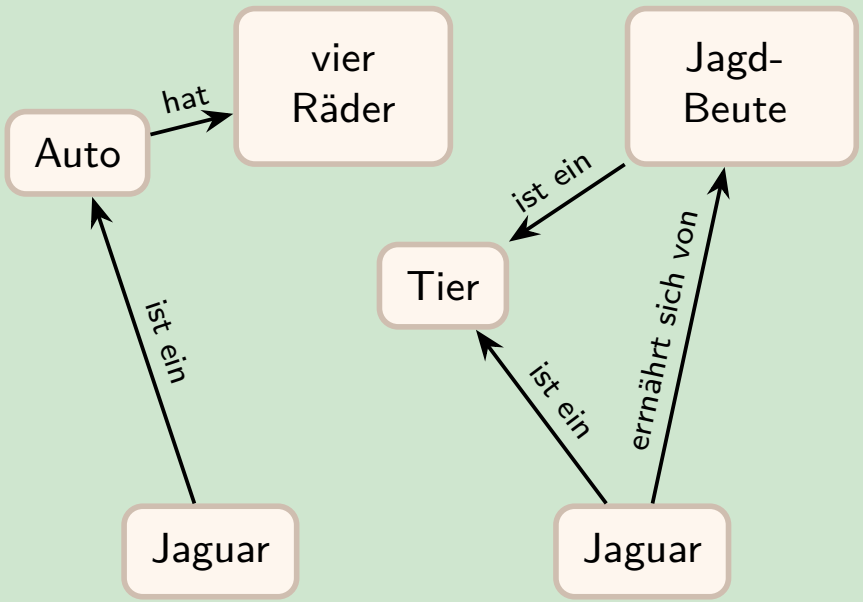
Gründe für formale Logik: Semantische Technologien

Beispiel: Mehrdeutigkeiten

- Wieviele Räder hat ein Jaguar?
- Was frisst ein Jaguar?
- „Gestohlener Jaguar taucht nach 46 Jahren wieder auf“

  ☞ Spiegel online, 18.09.2014

Beispiel: Ontologie



- **Semantische Technologien:** Verbesserung von (Software-)Systemen durch Bereitstellen zusätzlicher „semantischer Informationen“ zur Bedeutung von Begriffen und Konzepten
- **Ansatz:** Trennung von Wissen und Implementierung

- **Schlüsselideen:**
  - Repräsentiere Wissen durch logische Wissensbasen
  - Erschließe neue Zusammenhänge durch logisches Folgern

- Geeignete Logiken:
  Beschreibungslogiken

Idee für diese Folie: Gabriele Kern-Isberner

Logiken dieser Vorlesung: Aussagenlogik

- In der **Aussagenlogik** lassen sich logische Beziehungen von **Aussagen** ausdrücken

Beispiel: Aussagenlogik

- Die aussagenlogische Struktur von

  „Wenn der Server aktiv ist und der Client eine Anfrage sendet, erhält der Client ein Ergebnis“

  lässt sich durch folgende aussagenlogische Formel beschreiben:

  $$(S \wedge C) \rightarrow E$$

- Dabei werden die Elementaraussagen durch aussagenlogische Variablen repräsentiert:
  - S: „Der Server ist aktiv“
  - C: „Der Client stellt eine Anfrage“
  - E: „Der Client erhält ein Ergebnis“

✎ Die Beispielformeln dienen hier nur der Illustration der Logiken
✎ Syntax und Semantik von Formeln werden wir später ausführlicher behandeln

Logiken dieser Vorlesung: Modallogik

- In der **Modallogik**
  - können mehrere Welten (Zustände,...) vorhanden sein, in denen jeweils andere elementare Aussagen gelten können, und
  - es gibt die Möglichkeit, von Welten in andere überzugehen

Beispiel: Modallogik

- Die Eigenschaft eines Systems

  „Wenn der Drucker in der nächsten Phase zum Drucken bereit sein kann, so druckt er mit Sicherheit in einer der drei folgenden Phasen“

  lässt sich durch folgende modallogische Formel ausdrücken:

  $$\Diamond B \rightarrow \Box(D \vee \Box(D \vee \Box D))$$

- Dabei repräsentieren:
  - B: „Der Drucker ist bereit zum Drucken“
  - D: „Der Drucker druckt“

Logiken dieser Vorlesung: Prädikatenlogik

- Die **Prädikatenlogik** ist die ausdrucksstärkste Logik dieser Vorlesung
- Sie erlaubt die Modellierung sehr allgemeiner Situationen und Eigenschaften

Beispiel: Prädikatenlogik

- Die Behauptung

  „Jede Speicherstelle wird initialisiert, bevor sie verwendet wird“

  lässt sich beispielsweise ausdrücken durch:

$$\forall x \exists y \forall z \big((S(x) \wedge V(x,z)) \rightarrow (I(x,y) \wedge y < z) \big)$$

- Dabei „bedeuten“:
  - $S(x)$: „x ist Speicherstelle“
  - $V(x,z)$: „x wird in Schritt z verwendet“
  - $I(x,y)$: „x wird in Schritt y initialisiert“
  - $y < z$: „y ist ein früherer Zeitpunkt als z“

Drei Hauptbegriffe dieser Vorlesung: 1. Modellbeziehung

- Informatik-Systeme lassen sich durch logische Strukturen („Interpretationen“) modellieren
- Systemeigenschaften lassen sich durch logische Formeln spezifizieren
- Zum Testen, ob ein System die fragliche Eigenschaft hat, muss überprüft werden, ob die Struktur ein **Modell** der Formel ist

Modell/Interpretation M		**Formel** φ
Transitionssystem		Sicherheitsbedingung
Datenbank		Anfrage
Datenstruktur	$M \models \varphi$	Invariante
Graph		Eigenschaft
...		...

- Dieser Ansatz hat einen wichtigen Vorteil: **Präzision**
  - Umgangssprachliche Aussagen sind oft mehrdeutig und für Computer schwer verständlich
  - Logische Formeln haben dagegen eine eindeutige Semantik

- In der Logik-Vorlesung werden wir
  - verschiedene, unterschiedlich ausdrucksstarke Logiken betrachten
  - Algorithmen zur Auswertung von Formeln kennenlernen

Drei Hauptbegriffe dieser Vorlesung: 2. Semantisches Folgern

- Wie lässt sich aus einer Wissensbasis Φ neues Wissen ψ **folgern**?
- **Formale Schreibweise:** $\Phi \models \psi$
  - Wenn Φ gilt, gilt auch ψ
  - Für alle Interpretationen M mit $M \models \Phi$ gilt auch $M \models \psi$

Formelmenge Φ	$\Phi \models \psi$	**Formel ψ**
Bekannte Systemeigenschaften		Weitere Eigenschaft
Bekanntes Wissen		Neues Wissen
Axiome		Neues Theorem
...		...

- Zweiter Vorteil des Logik-Ansatzes: **Automatisierung**
  - Ein Ziel: Algorithmen zum Testen von $\Phi \models \psi$
  - Der Entwurf solcher Algorithmen ist einfacher, wenn zu testende Formeln eine möglichst einfache Struktur haben ☞ Normalformen

- In der Logik-Vorlesung werden wir
  - Algorithmen für typische Problemstellungen betrachten
    ➡ Erfüllbarkeit, Folgerungen, ...
  - Normalformen für die betrachteten Logiken kennenlernen

Drei Hauptbegriffe dieser Vorlesung: 3. Syntaktisches Beweisen

- Wie lässt sich aus einer Wissensbasis Φ neues Wissen ψ **beweisen**?
- **Formale Schreibweise:** $\Phi \vdash \psi$
  - Die Formel ψ lässt sich aus Φ beweisen
  - „Beweisen“ bedeutet hier: durch syntaktische Operationen Herleiten (z.B. durch äquivalenzerhaltende Umformungen)

Formelmenge Φ		**Formel ψ**
Bekannte Systemeigenschaften	$\Phi \vdash \psi$	Weitere Eigenschaft
Bekanntes Wissen		Neues Wissen
Axiome		Neues Theorem
...		...

- Die meisten Logiken haben verschiedene Beweissysteme, die jeweils eine eigene Bedeutung von „$\vdash$“ ergeben

- In der Logik-Vorlesung werden wir
  - verschiedene Beweissysteme betrachten
  - die Grenzen des automatischen Beweisens kennenlernen

Wiederkehrende Themen dieser Vorlesung

- Die folgenden Fragen werden für jede der drei betrachteten Logiken eine Rolle spielen

- **Syntax:** Wie sind Formeln aufgebaut?
  - Und: Wie lassen sie sich in eine besonders einfache **Normalform** bringen?
- **Semantik:** Wie wird die Bedeutung von Formeln formal definiert?

- **Modellierung:** Wie lassen sich Situationen modellieren, umgangssprachliche Aussagen in Formeln überführen und Formeln verstehen?

- Wie lassen sich **„logische Schlüsse“** automatisieren?
- Eine besondere Rolle spielen dabei **Beweissysteme** und **Erfüllbarkeitstests**

Gliederung der Vorlesung (Plan)

0. Einführung und Organisation

Teil A: Aussagenlogik

1. Syntax und Semantik der Aussagenlogik
2. Modellierung, Äquivalenzen und Normalformen
3. Erfüllbarkeit aussagenlogischer Formeln
4. Vollständigkeits- und Endlichkeitssatz
5. Praktisches SAT-Solving

Teil B: Modallogik

1. Grundlagen der Modallogik
2. Erfüllbarkeit modallogischer Formeln
3. Weitere Ergebnisse

Teil C: Prädikatenlogik

1. Strukturen, Syntax & Semantik
2. Modellierung und Normalformen
3. Erfüllbarkeit: Grundresolution
4. Prädikatenlogische Resolution
5. Logische Programmierung und Prolog
6. Weitere Ergebnisse

Einige Lernziele der Vorlesung

- **Grundwissen**
  - Verhältnis von Syntax und Semantik
  - Grundlagen von Aussagenlogik, Modallogik und Prädikatenlogik
  - Möglichkeiten und Grenzen der betrachteten Logiken
  - Grundlagen von Beweissystemen
  - Grundlagen der logischen Programmierung
- **Handwerkszeug**
  - Formalisieren und Modellieren mit mathematischer Logik
  - Sicherheit im Umgang mit logischen Formeln
  - Auswahl eines geeigneten logischen Systems für ein gegebenes Problem
  - Umgang mit Beweiskalkülen, insbesondere Resolution, Anwendung auf konkrete Situationen
  - Anwendung der betrachteten Formalismen und Algorithmen

Literatur

- Die Foliensätze der Vorlesung sind in Zusammenarbeit mit **Prof. Dr. Thomas Zeume**, RUB, entstanden

- Begleitlektüre:

[KuK] M. Kreuzer and S. Kühling. *Logik für Informatiker.* Pearson, 2006 (verfügbar in der Uni-Bibliothek: https://elibrary.pearson.de/book/99.150005/9783863267391)

[HR] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems.* Cambridge University Press, New York, NY, USA, 2004

[B] Mordechai Ben-Ari. *Mathematical Logic for Computer Science.* Springer, 2008

[Sch] Uwe Schöning. *Logik für Informatiker.* Spektrum Akademischer Verlag, 5. a. edition, 2000

- Wolfgang Rautenberg. *Einführung in die Mathematische Logik.* Vieweg, 2008
- Anil Nerode and Richard Shore. *Logic for Applications.* Springer, 1993
- Heinz-Dieter Ebbinghaus, Jörg Flum, and Wolfgang Thomas. *Einführung in die mathematische Logik.* Spektrum, Heidelberg, 2007
- D.W. Hoffmann. *Grenzen der Mathematik: eine Reise durch die Kerngebiete der Mathematischen Logik.* Spektrum-Akademischer Vlg, 2011
- Dirk Siefkes. *Formalisieren und Beweisen: Logik für Informatiker.* Vieweg, 1992
- Hans-Peter Tuschik and Helmut Wolter. *Mathematische Logik—kurzgefasst.* Spektrum, 2002

Kapitel A.1:

Syntax und Semantik der Aussagenlogik

Inhalt

1. Einführende Beispiele
2. Syntax und Semantik der Aussagenlogik

Einführende Beispiele

Gründe für formale Logik: Präzision

(1) **Keine zwei Spieler haben in allen Spielen gleich gut gespielt**

= höchstens ein Spieler hat in allen Spielen gleich gut gespielt?

= es gab keine zwei Spieler, für die in jedem Spiel galt, dass beide gleich gut gespielt haben?

(2) **Jede Speicherstelle wird vor der Verwendung initialisiert**

= verwendete Speicherzellen werden vorher initialisiert

= **jede** Speicherstelle wird vorab initialisiert

(3) **Ich fahre im März oder April nach Spanien**

➡ Stimmt das auch, wenn ich am 15.3. **und** am 18.4. nach Spanien fahre?

- Formale Logik hat im Gegensatz zur menschlichen Sprache eine eindeutige Semantik und erlaubt daher, Zusammenhänge präzise zu beschreiben

- Beispiele (1) und (2): Prädikatenlogik ☞ später
- Beispiel (3): Aussagenlogik ☞ jetzt

Gründe für formale Logik: Struktur von Zusammenhängen

Eine schlüssige Überlegung

- Wenn der Zug zu spät kommt und keine Taxis am Bahnhof sind, kommt Jane zu spät zu ihrem Termin

- Der Zug ist zu spät
- Aber Jane ist pünktlich

⇒ Also sind Taxis am Bahnhof

Noch eine schlüssige Überlegung

- Falls es regnet und John seinen Schirm nicht dabei hat, wird er nass

- Es regnet
- Aber John wird nicht nass

⇒ Also hat er seinen Schirm dabei

☞ Beispiele aus [HR]

- Die beiden Beispiele sind sehr ähnlich

- In beiden Fällen haben wir es mit 3 elementaren Aussagen zu tun:

(1)	Der Zug ist zu spät	Es regnet
(2)	Es sind Taxis am Bahnhof	John hat seinen Schirm dabei
(3)	Jane kommt zu spät	John wird nass

- In beiden Fällen ist die Struktur der Argumentation gleich:
  - Falls (1) gilt und (2) nicht gilt, so gilt (3)
  - (1) gilt
  - (3) gilt nicht

  ⇒ Also gilt (2)

- **Beobachtung:**
  - Die Argumentation ist unabhängig vom Inhalt der Aussagen (1) – (3)
  - Es kommt nur auf den logischen Zusammenhang zwischen den Aussagen an

Gründe für formale Logik: Problemlösen

Eine typische Logelei

- **Anna:** „Jan oder ich werden zur Party gehen“
- **Jan:** „Entweder Patrick oder ich werden zur Party gehen“
- **Patrick:** „Entweder Anna oder ich werden zuhause bleiben“

  $\Rightarrow$ D.h. Anna bleibt genau dann zuhause, wenn ich nicht zuhause bleibe.

- Wer geht zur Party?

- Wir kürzen ab:
  - A steht für „Anna wird zur Party gehen“
  - P steht für „Patrick wird zur Party gehen“
  - J steht für „Jan wird zur Party gehen“

- Es ergibt sich die Formel:

$$(J \vee A) \wedge (P \leftrightarrow \neg J) \wedge (\neg A \leftrightarrow \neg\neg P)$$

- Durch Ausprobieren aller Möglichkeiten stellen wir fest:
  - Nur **eine** Zuordnung von Variablen zu Wahrheitswerten macht die Formel wahr:
    - $P \mapsto$ `falsch`
    - $J \mapsto$ `wahr`
    - $A \mapsto$ `wahr`

$\Rightarrow$ Wenn alle drei die Wahrheit gesagt haben, werden Jan und Anna zur Party gehen, und Patrick zuhause bleiben

- **Beobachtung:** Die logische Formalisierung erlaubt die Anwendung systematischer Methoden zur Lösung von Problemen

Syntax und Semantik der Aussagenlogik

Aussagen und Aussagenvariablen

- Aussagenlogik ist die **Logik der Aussagen**
- **Was ist eine Aussage?**

- **Beispiele für Aussagen:**
  - Der Zug ist pünktlich.
  - Alle Marsianer lieben Pizza.
  - Der BVB wird Champions League Sieger der Saison 2024/2025.
  - Jede gerade Zahl, die größer als $\mathbf{2}$ ist, ist die Summe zweier Primzahlen.

- **Keine Aussagen sind:**
  - Geben Sie mir mal das Salz, bitte?
  - Alles Gute!
  - Möge das bessere Team gewinnen!

- Was unterscheidet Aussagen von anderen Sätzen?

  ➡ Aussagen können wahr oder falsch sein

- Gemäß dieser Erkenntnis wählen wir unsere **Abstraktion** von Aussagen:

  etwas, das wahr oder falsch sein kann

- Statt sprachlicher Aussagen verwenden wir **Variablen**, die die Werte `wahr` oder `falsch` annehmen können
- Solche Variablen nennen wir dann **Aussagevariablen**

Aussagenlogik: Syntax

- Wir verwenden **aussagenlogische Variablen** der Art A, B, C sowie A_i, für alle $i \in \mathbb{N}$ ☞ und weitere, wenn nötig
- **AV** bezeichne die Menge aller aussagenlogischen Variablen ☞ A.1.1

Definition:
Syntax aussagenlogischer Formeln

- Die Menge **AL** der **aussagenlogischen Formeln** ist die kleinste Menge, die die folgenden Eigenschaften hat:

(1) $\top$ und $\bot$ sind in AL (**wahr** und **falsch**)

(2) Jede Variable aus AV ist in AL

(3) Sind φ_1 und φ_2 in AL, so auch

- $\neg\varphi_1$, (**Negation**)
- $(\varphi_1 \wedge \varphi_2)$, (**Konjunktion**)
- $(\varphi_1 \vee \varphi_2)$, (**Disjunktion**)

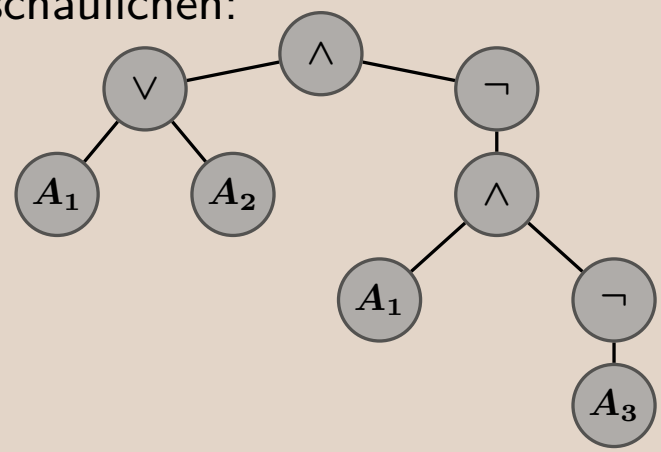
- Formeln der Typen (1) und (2) nennen wir **atomar**

Beispiel

- Aussagenlogische Formeln:
  - $((A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3))$
  - $((A \vee \neg\top) \wedge \neg\neg B)$
  - $(A \wedge \neg A)$

- Keine aussagenlogischen Formeln:
  - $A \wedge \neg B)$
  - $A \wedge B \vee A_2$
  - $((A \vee B))$

- Die Struktur aussagenlogischer Formeln lässt sich durch ihren Syntaxbaum veranschaulichen:



Aussagenlogik: Semantik (1/3)

- Bisher haben wir nur definiert, wie aussagenlogische Formeln aussehen ☞ Syntax
- Jetzt definieren wir, was sie bedeuten ☞ Semantik

- Was soll beispielsweise die Bedeutung der Formel

  $$\varphi = ((A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3))$$

  sein?

- Aus der Einleitung wissen wir schon, dass wir Formeln in gewissen Strukturen („Interpretationen“) M auswerten
- Welcher Art muss „M“ sein, damit wir $M \models \varphi$ sinnvoll definieren können?

- **Idee:** Wenn jeder vorkommenden Variable ein Wahrheitswert zugewiesen ist, dann kann auch einer aussagenlogischen Formel ihr Wahrheitswert zugewiesen werden

- Z.B. hat φ bei Zuordnung $A_1 \mapsto 0$, $A_2 \mapsto 1$, $A_3 \mapsto 0$ den Wahrheitswert 1

  ➡ Ab jetzt: Verwende 1 statt `wahr` und 0 statt `falsch`

Aussagenlogik: Semantik (2/3)

Definition: Wahrheitsbelegung

- Eine **Wahrheitsbelegung** ist eine partielle Funktion $\alpha : \mathrm{AV} \rightharpoonup \{0, 1\}$ ☞ A.1.2
- Eine Belegung α heißt **passend** für eine Formel φ, wenn $\alpha(X)$ für alle in φ vorkommenden Variablen X definiert ist

✎ X ist hier eine „Meta-Variable“, die für eine beliebige aussagenlogische Variable steht

✎ Statt „Wahrheitsbelegung“ sagen wir meist nur „Belegung“

✎ Aussprache der griechischen Buchstaben:

- φ, Φ: „Phi“
- ψ, Ψ: „Psi“
- χ: „Chi“
- α: „Alpha“
- β: „Beta“

Aussagenlogik: Semantik (3/3)

- Die Semantik von φ ordnet jeder passenden Belegung α einen Wahrheitswert $[\![\varphi]\!]_\alpha$ zu

Definition: Semantik aussagenlogischer Formeln

- Sei $\varphi \in \mathrm{AL}$ und α eine zu φ passende Belegung
- Wir definieren $\underline{[\![\varphi]\!]_\alpha}$ induktiv nach der Struktur von φ:

- $[\![\bot]\!]_\alpha \stackrel{\text{def}}{=} 0$, $[\![\top]\!]_\alpha \stackrel{\text{def}}{=} 1$
- $[\![X]\!]_\alpha \stackrel{\text{def}}{=} \alpha(X)$ für Variablen $X \in \mathrm{AV}$

- $$[\![\neg\varphi_1]\!]_\alpha \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } [\![\varphi_1]\!]_\alpha = 0 \\ 0 & \text{falls } [\![\varphi_1]\!]_\alpha = 1 \end{cases}$$

- $$[\![(\varphi_1 \wedge \varphi_2)]\!]_\alpha \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } [\![\varphi_1]\!]_\alpha = 1 \text{ und } [\![\varphi_2]\!]_\alpha = 1 \\ 0 & \text{andernfalls} \end{cases}$$

- $$[\![(\varphi_1 \vee \varphi_2)]\!]_\alpha \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } [\![\varphi_1]\!]_\alpha = 1 \text{ oder } [\![\varphi_2]\!]_\alpha = 1 \\ 0 & \text{andernfalls} \end{cases}$$

- α **erfüllt** φ, falls $[\![\varphi]\!]_\alpha = 1$ ☞ Schreibweise: $\alpha \models \varphi$

Aussagenlogik: Semantik (Beispiel)

Beispiel

- Sei φ die Formel $((A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3))$
- Sei die Belegung α wie folgt gegeben:

X	$\alpha(X)$
A_1	0
A_2	1
A_3	0

- Dann gelten:
  - $[\![A_2]\!]_\alpha = 1$
  - $[\![A_1]\!]_\alpha = [\![A_3]\!]_\alpha = 0$
  - $[\![\neg A_3]\!]_\alpha = 1$
  - $[\![(A_1 \vee A_2)]\!]_\alpha = 1$
  - $[\![(A_1 \wedge \neg A_3)]\!]_\alpha = 0$
  - $[\![\neg(A_1 \wedge \neg A_3)]\!]_\alpha = 1$
  - $[\![((A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3))]\!]_\alpha = 1$
- Also: $\alpha \models ((A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3))$

Wahrheitstabelle (Beispiel)

Beispiel

- Sei φ wieder die Formel $((A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3))$

- Dann ist die Semantik von φ durch die folgende Wahrheitstabelle gegeben:

$\alpha(A_1)$	$\alpha(A_2)$	$\alpha(A_3)$	$[\![\varphi]\!]_\alpha$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

- Jede Zeile der Wahrheitstabelle entspricht also genau einer Belegung α der Variablen von φ

✎ Künftig werden wir in Tabellenköpfen oft statt „$\alpha(X)$“ und „$[\![\varphi]\!]_\alpha$“ nur „X“ und „φ“ schreiben

Abkürzende Schreibweisen

- Wir lassen Klammern weg, wenn dadurch keine Missverständnisse entstehen:
  - Statt

    $$((A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3))$$

    schreiben wir beispielsweise

    $$(A_1 \vee A_2) \wedge \neg(A_1 \wedge \neg A_3)$$

  - Weitere Fälle, in denen Klammern eingespart werden können, lernen wir im nächsten Kapitel kennen

- Wir vereinbaren die logischen Operatoren $\rightarrow$ und $\leftrightarrow$ als abkürzende Schreibweisen wie folgt:
  - $\varphi_1 \rightarrow \varphi_2$ für $\neg\varphi_1 \vee \varphi_2$
  - $\varphi_1 \leftrightarrow \varphi_2$ für

    $$(\varphi_1 \wedge \varphi_2) \vee (\neg\varphi_1 \wedge \neg\varphi_2)$$

- Die entsprechenden Wahrheitstabellen:

φ_1	φ_2	$\varphi_1 \rightarrow \varphi_2$	$\varphi_1 \leftrightarrow \varphi_2$
0	0	1	1
0	1	1	0
1	0	0	0
1	1	1	1

- **Vereinbarung:**
  - Im Folgenden bezeichnet AL weiterhin die in der Definition „Syntax aussagenlogischer Formeln“ definierte Formelmenge
  - Wenn stattdessen von „aussagenlogischen Formeln“ die Rede ist, sind auch die abkürzenden Schreibweisen erlaubt

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Gründe für die Formalisierung mit Logiken
  - Syntax und Semantik der Aussagenlogik
  - Wahrheitstabellen
  - Abkürzende Schreibweisen

Anmerkungen

Bemerkung A.1.1

- Zu definierende **Begriffe** sind durch Unterstreichung und Fettdruck zu erkennen
- Auch Schreibweisen wie $[\![\varphi]\!]_\alpha$ werden so gekennzeichnet
- Später werden diese Schreibweisen dann aber **ohne** Unterstreichung verwendet

Bemerkung A.1.2

- Zur Notation von Funktionen:
  - $X \to Y$: Funktion von Menge X nach Y
  - $X \rightharpoonup Y$: partielle Funktion von Menge X nach Y
  - $A \mapsto B$: A wird auf B abgebildet

Kapitel A.2:

Modellierung, Äquivalenzen und Normalformen

Inhalt

1. Modellierung mit Aussagenlogik
2. Äquivalenzen der Aussagenlogik
3. Normalformen der Aussagenlogik
4. Anhang: Formelgrößen von Normalformen

Modellierung mit Aussagenlogik

Modellierung mit Aussagenlogik: Erstes Beispiel

Beispiel **[Grohe, RWTH Aachen]**

- „Das Fluchtauto war rot oder grün und hatte weder vorne noch hinten ein Nummernschild“

- Elementare Aussagen:
  - R: Das Fluchtauto war rot
  - G: Das Fluchtauto war grün
  - V: Das Fluchtauto hatte vorne ein Nummernschild
  - H: Das Fluchtauto hatte hinten ein Nummernschild

- Gesamtaussage:

$$(R \vee G) \wedge (\neg V \wedge \neg H)$$

- Vorgehen bei der aussagenlogischen Modellierung
  (1) Analyse: Welche aussagenlogischen Variablen mit welcher Bedeutung werden benötigt?
  (2) Aufstellen der aussagenlogischen Formel

- Zu Beachten: Variablen werden normalerweise für elementare Aussagen ohne Verknüpfungen wie **nicht**, **und** und **oder** gewählt

Modellierung mit Aussagenlogik: Zweites Beispiel

Beispiel **[KuK]**

„Platon hatte mit seiner Einschätzung des Sokrates genau dann Recht, wenn Sokrates kein großer Philosoph war. Wenn Sokrates ein großer Philosoph war, dann hatte Aristoteles Recht mit seiner Einschätzung des Platon. Aristoteles hatte nur dann Recht mit seiner Einschätzung des Platon, falls Platon Recht hatte mit seiner Einschätzung des Sokrates.“

- Elementare Aussagen:

P: „Platon hatte Recht mit seiner Einschätzung des Sokrates“

S: „Sokrates war ein großer Philosoph“

A: „Aristoteles hatte Recht mit seiner Einschätzung des Platon“

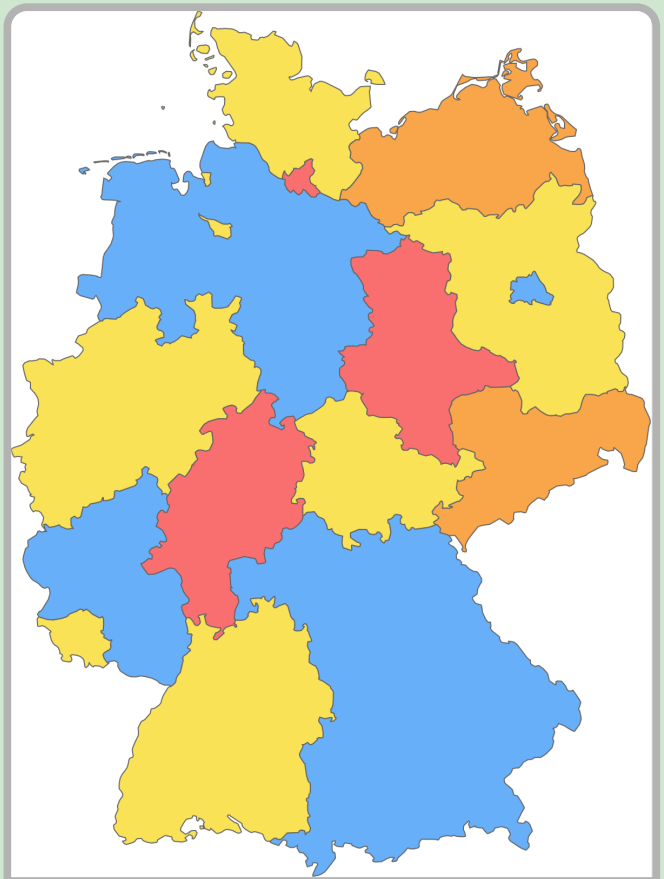
- Teilaussagen:
  - $P \leftrightarrow \neg S$
  - $S \rightarrow A$
  - $A \rightarrow P$

- Gesamtaussage:

$$(P \leftrightarrow \neg S) \wedge (S \rightarrow A) \wedge (A \rightarrow P)$$

Modellierung mit Aussagenlogik: Drittes Beispiel (1/3)

Landkartenfärbung



Creative Commons (CC BY-SA 2.0 DE)
Urheber: (DeStatis); David Liuzzo.
Färbung: Gaetano Geck

- **Landkartenfärbung**: Die Bundesländer Deutschlands lassen sich mit vier Farben so färben, dass benachbarte Länder verschiedene Farben haben
  - ➡ Wir sagen: Es gibt eine **zulässige** 4-Färbung
- **Frage:** Gibt es auch eine zulässige $\mathbf{3}$-Färbung?

- **Lösungsansatz 1:** Probieren + Denken
  - ➡ fehleranfällig, langsam

- **Lösungsansatz 2:** Aussagenlogik

  (1) Konstruiere eine Formel φ sodass

  φ hat eine erfüllende Belegung
  $\Leftrightarrow$ es gibt eine zulässige $\mathbf{3}$-Färbung der Karte

  ➡ Erfüllende Belegungen sollen zulässigen $\mathbf{3}$-Färbungen entsprechen

  (2) Überprüfe, ob φ eine erfüllende Belegung hat
  - ➡ Leicht auf andere Karten übertragbar

- **Hier:** Lösungsansatz 2

Modellierung mit Aussagenlogik: Drittes Beispiel (2/3)

- Unsere Farben:
  rot, blau und gelb

- Aussagenlogische Variablen: eine für jedes Bundesland und jede Farbe
  - R_{NRW} steht für „NRW hat die Farbe rot“
  - B_{NRW} steht für „NRW hat die Farbe blau“
  - Y_{NRW} steht für „NRW hat die Farbe gelb“
  - usw.

- Unsere Formel muss sicherstellen:
  1. Jedes Bundesland wird mit genau einer Farbe gefärbt
  2. Zwei benachbarte Bundesländer werden nicht gleich gefärbt

- Jedes Bundesland, wie bspw. NRW, hat genau eine Farbe:
  - NRW wird mit mindestens einer Farbe gefärbt:
    $$\varphi^{\geq 1}_{\text{NRW}} \stackrel{\text{def}}{=} Y_{\text{NRW}} \vee R_{\text{NRW}} \vee B_{\text{NRW}}$$
  - NRW wird mit höchstens einer Farbe gefärbt:
    $$\begin{aligned}\varphi^{\leq 1}_{\text{NRW}} \stackrel{\text{def}}{=} \quad & \neg(Y_{\text{NRW}} \wedge R_{\text{NRW}}) \\ & \wedge \neg(B_{\text{NRW}} \wedge R_{\text{NRW}}) \\ & \wedge \neg(B_{\text{NRW}} \wedge Y_{\text{NRW}})\end{aligned}$$
  - NRW hat genau eine Farbe:
    $$\varphi^{=1}_{\text{NRW}} \stackrel{\text{def}}{=} \varphi^{\geq 1}_{\text{NRW}} \wedge \varphi^{\leq 1}_{\text{NRW}}$$

- Benachbarte Länder wie NRW und Hessen haben nicht dieselbe Farbe:
  $$\begin{aligned}\varphi^{\neq}_{\text{NRW,HS}} \stackrel{\text{def}}{=} \quad & \neg(Y_{\text{NRW}} \wedge Y_{\text{HS}}) \\ & \wedge \neg(R_{\text{NRW}} \wedge R_{\text{HS}}) \\ & \wedge \neg(B_{\text{NRW}} \wedge B_{\text{HS}})\end{aligned}$$

Modellierung mit Aussagenlogik: Drittes Beispiel (3/3)

- Insgesamt erhalten wir die Formel:

$$\begin{aligned}\varphi \overset{\text{def}}{=} \quad & \varphi^{=1}_{\text{NRW}} \wedge \varphi^{=1}_{\text{BW}} \wedge \varphi^{=1}_{\text{BY}} \wedge \varphi^{=1}_{\text{RP}} \wedge \varphi^{=1}_{\text{SR}} \wedge \varphi^{=1}_{\text{HS}} \wedge \varphi^{=1}_{\text{TH}} \wedge \varphi^{=1}_{\text{SX}} \\ & \wedge \varphi^{=1}_{\text{NI}} \wedge \varphi^{=1}_{\text{SA}} \wedge \varphi^{=1}_{\text{BR}} \wedge \varphi^{=1}_{\text{B}} \wedge \varphi^{=1}_{\text{HH}} \wedge \varphi^{=1}_{\text{HB}} \wedge \varphi^{=1}_{\text{MV}} \wedge \varphi^{=1}_{\text{SH}} \\ & \wedge \varphi^{\neq}_{\text{NRW,HS}} \wedge \varphi^{\neq}_{\text{BW,BY}} \wedge \varphi^{\neq}_{\text{BW,RL}} \wedge \varphi^{\neq}_{\text{BW,HS}} \wedge \varphi^{\neq}_{\text{BY,HS}} \wedge \varphi^{\neq}_{\text{BY,TH}} \\ & \wedge \varphi^{\neq}_{\text{BY,SX}} \wedge \varphi^{\neq}_{\text{SR,RL}} \wedge \varphi^{\neq}_{\text{RL,HS}} \wedge \varphi^{\neq}_{\text{RL,NRW}} \wedge \varphi^{\neq}_{\text{HS,TH}} \wedge \varphi^{\neq}_{\text{HS,NI}} \\ & \wedge \varphi^{\neq}_{\text{TH,SX}} \wedge \varphi^{\neq}_{\text{TH,SA}} \wedge \varphi^{\neq}_{\text{TH,NI}} \wedge \varphi^{\neq}_{\text{SX,BR}} \wedge \varphi^{\neq}_{\text{SX,SA}} \wedge \varphi^{\neq}_{\text{NRW,NI}} \\ & \wedge \varphi^{\neq}_{\text{NI,SH}} \wedge \varphi^{\neq}_{\text{NI,HB}} \wedge \varphi^{\neq}_{\text{NI,HH}} \wedge \varphi^{\neq}_{\text{NI,MV}} \wedge \varphi^{\neq}_{\text{NI,SA}} \wedge \varphi^{\neq}_{\text{NI,BR}} \\ & \wedge \varphi^{\neq}_{\text{SA,BR}} \wedge \varphi^{\neq}_{\text{BR,MV}} \wedge \varphi^{\neq}_{\text{BR,B}} \wedge \varphi^{\neq}_{\text{SH,HH}} \wedge \varphi^{\neq}_{\text{SH,MV}}\end{aligned}$$

- Dies ist eine Abkürzung für eine Konjunktion von über 150 Teilformeln

- **Es gilt:** Die Deutschland-Karte hat eine zulässige $\mathbf{3}$-Färbung
  $\Leftrightarrow$ Es gibt eine Wahrheitsbelegung, die φ wahr macht

- **Frage:** Wie lässt sich herausfinden, ob eine Formel (un-)erfüllbar ist?
  ➡ Nächste Kapitel

Formalisierung umgangssprachlicher Aussagen (1/3)

- Vorsicht bei der Formalisierung umgangssprachlicher Aussagen!

- **Implikationen:**
  „wenn . . . dann . . . “ **vs.** „nur wenn . . . dann . . . “

Beispiel

- „Wenn ich die Studienleistung schaffe, werde ich die Klausur mitschreiben“:
  - **Aussagenlogische Variablen:**
    - S: „Ich die schaffe Studienleistung“
    - K: „Ich werde die Klausur mitschreiben“
  - **Formel:** $S \rightarrow K$

- „**Nur** wenn ich die Studienleistung schaffe, werde ich die Klausur mitschreiben“:
  - **Formel:** $K \rightarrow S$
  - „Vielleicht schreibe ich die Klausur aber auch nicht mit, obwohl ich die Studienleistung geschafft habe. . . “

Formalisierung umgangssprachlicher Aussagen (2/3)

- **Disjunktionen:** ausschließlich oder nicht?

Beispiel

- „oder“ lässt sich oft in das nicht ausschließliche $\vee$ übersetzen:
  - „Morgen oder übermorgen bearbeite ich die Übungsaufgaben“:
    - M: „Morgen bearbeite ich die Übungsaufgaben“
    - U: „Übermorgen bearbeite ich die Übungsaufgaben“
  - Übersetzung: $M \vee U$
  - („Vielleicht auch an beiden Tagen“)

- Manchmal ist es aber ausschließlich gemeint:
  - „Morgen oder übermorgen gebe ich die Übungsaufgaben ab“:

    $$(M \wedge \neg U) \vee (\neg M \wedge U)$$

  - Kürzer:
    - $\neg(M \leftrightarrow U)$ oder $(M \leftrightarrow \neg U)$

- **Sprach-Konvention in dieser Vorlesung:**
  - Für das nicht-exklusive Oder verwenden wir die Formulierung:

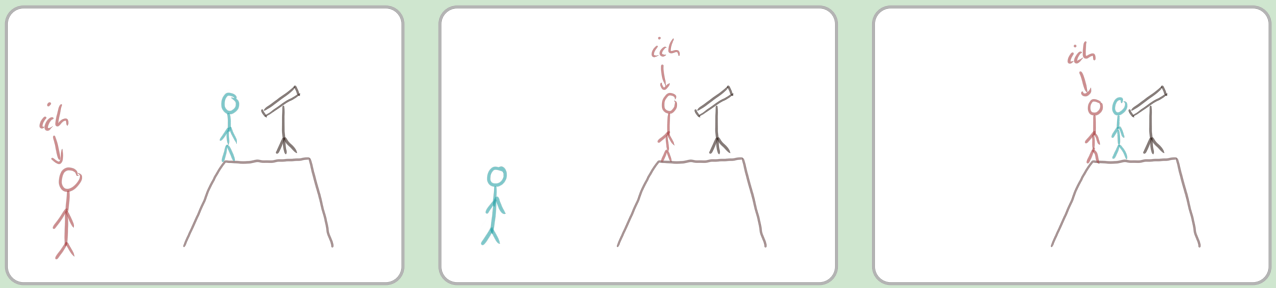
    „A oder B“
  - Für das exklusive Oder verwenden wir die Formulierung:

    „Entweder A oder B“

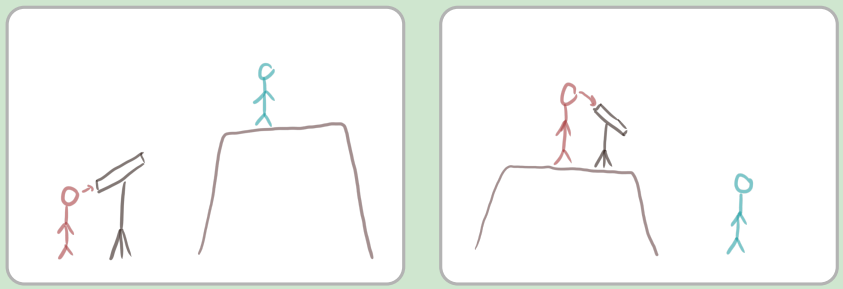
Formalisierung umgangssprachlicher Aussagen (3/3)

Beispiel: Mehrdeutigkeit der Umgangsprache

- „Ich sah den Mann auf dem Berg mit dem Teleskop“
- Es gibt mindestens folgende Interpretationen dieses Satzes:
  - „. . . Berg mit dem Teleskop“:



  - „Ich sah. . . mit dem Teleskop“:



✎ Idee zu diesen drei Folien: Foliensatz von Prof. Kastens, Paderborn

Äquivalenzen der Aussagenlogik

Äquivalenz von Formeln

Beispiel

- Wir betrachten folgende zwei Formeln:
  - $A \wedge (A \vee B)$
  - $(\neg B \vee A) \wedge (\neg A \rightarrow B)$

- Sie sind syntaktisch völlig verschieden
- **Aber:** Sie haben dieselbe Semantik
  - D.h., für alle Wahrheitsbelegungen haben sie denselben Wahrheitswert

Definition: Äquivalenz von Formeln

- Zwei Formeln φ_1, φ_2 sind **äquivalent**, falls für jede zu φ_1 und φ_2 passende Belegung α gilt:

$$\alpha \models \varphi_1 \iff \alpha \models \varphi_2$$

- Schreibweise: $\varphi_1 \equiv \varphi_2$

Beispiel

- $A \wedge (A \vee B)$
  $\equiv (\neg B \vee A) \wedge (\neg A \rightarrow B)$

- Zu beachten: $\varphi_1 = \varphi_2$ und $\varphi_1 \equiv \varphi_2$ sind unterschiedliche Aussagen!

Beispiel

- $A \wedge (A \vee B)$
  $\neq (\neg B \vee A) \wedge (\neg A \rightarrow B)$

- Aber natürlich gilt:

  wenn $\varphi_1 = \varphi_2$ dann $\varphi_1 \equiv \varphi_2$

- Als nächstes lernen wir kennen:
  (1) Allgemeine Äquivalenzen für aussagenlogische Formeln
  (2) Beweismethoden für den Nachweis von Äquivalenzen

- Anschließend: Herleitung der wichtigsten Normalformen für aussagenlogische Formeln mit Hilfe von Äquivalenzen

Allgemeine Äquivalenzen der Aussagenlogik

Satz A.2.1: Äquivalenzen der Aussagenlogik

- Für beliebige aussagenlogische Formeln $\varphi, \varphi_1, \varphi_2, \varphi_3$ gelten:

- **Kommutativität**
  - $\varphi_1 \wedge \varphi_2 \equiv \varphi_2 \wedge \varphi_1$
  - $\varphi_1 \vee \varphi_2 \equiv \varphi_2 \vee \varphi_1$

- **Assoziativität**
  - $(\varphi_1 \wedge \varphi_2) \wedge \varphi_3 \equiv \varphi_1 \wedge (\varphi_2 \wedge \varphi_3)$
  - $(\varphi_1 \vee \varphi_2) \vee \varphi_3 \equiv \varphi_1 \vee (\varphi_2 \vee \varphi_3)$

- **Idempotenz**
  - $\varphi \wedge \varphi \equiv \varphi$
  - $\varphi \vee \varphi \equiv \varphi$

- **Absorption**
  - $\varphi_1 \wedge (\varphi_1 \vee \varphi_2) \equiv \varphi_1$
  - $\varphi_1 \vee (\varphi_1 \wedge \varphi_2) \equiv \varphi_1$

- **Distributivität**
  - $\varphi_1 \vee (\varphi_2 \wedge \varphi_3) \equiv (\varphi_1 \vee \varphi_2) \wedge (\varphi_1 \vee \varphi_3)$
  - $\varphi_1 \wedge (\varphi_2 \vee \varphi_3) \equiv (\varphi_1 \wedge \varphi_2) \vee (\varphi_1 \wedge \varphi_3)$

- **De Morgansche Regeln**
  - $\neg(\varphi_1 \wedge \varphi_2) \equiv \neg\varphi_1 \vee \neg\varphi_2$
  - $\neg(\varphi_1 \vee \varphi_2) \equiv \neg\varphi_1 \wedge \neg\varphi_2$

- **Doppelnegation**
  - $\neg\neg\varphi \equiv \varphi$

- **Tautologie und Kontradiktion**
  - $\varphi \vee \neg\varphi \equiv \top$
  - $\varphi \wedge \neg\varphi \equiv \bot$

- **Dualität**
  - $\neg\top \equiv \bot$
  - $\neg\bot \equiv \top$

Weniger Klammern

- Dank der Assoziativität von Konjunktion und Disjunktion können wir zukünftig einige weitere Klammern weglassen:
  - $\varphi_1 \wedge \varphi_2 \wedge \varphi_3$ statt $\varphi_1 \wedge (\varphi_2 \wedge \varphi_3)$
  - $\varphi_1 \vee \varphi_2 \vee \varphi_3$ statt $\varphi_1 \vee (\varphi_2 \vee \varphi_3)$

- Statt $\varphi_1 \wedge \cdots \wedge \varphi_n$ schreiben wir auch: $\bigwedge_{i=1}^{n} \varphi_i$
- Statt $\varphi_1 \vee \cdots \vee \varphi_n$ schreiben wir auch: $\bigvee_{i=1}^{n} \varphi_i$

- **Aber:** Formeln wie $A \vee B \wedge C$ sind in dieser Veranstaltung nicht erlaubt
- **Denn:** Es ist nicht klar, ob zunächst $\vee$ oder $\wedge$ ausgewertet wird
  - ➡ Die beiden Möglichkeiten führen zu unterschiedlichen Ergebnissen:

$$(A \vee B) \wedge C \not\equiv A \vee (B \wedge C)$$

- Auch Formeln der Art $A \wedge B \rightarrow C \vee D$ sind nicht erlaubt
  - ➡ Stattdessen: $(A \wedge B) \rightarrow (C \vee D)$

Aussagenlogische Äquivalenzen beweisen

- Wir werden zwei Beweismethoden für aussagenlogische Äquivalenzen kennen lernen:
  (1) Beweis mit Hilfe von Wahrheitstabellen und Substitutionen
      ➡ Damit können wir die Äquivalenzen aus Satz A.2.1 beweisen
  (2) Beweis durch Äquivalenzumformungen

Wahrheitstabellen und Substitutionen (1/6)

- Wie lässt sich eine Äquivalenz wie

  $$(\varphi_1 \wedge \varphi_2) \wedge \varphi_3 \equiv \varphi_1 \wedge (\varphi_2 \wedge \varphi_3)$$

  für **beliebige** aussagenlogische Formeln $\varphi_1, \varphi_2, \varphi_3$ beweisen?

- Es gibt unendlich viele verschiedene Formeln, die für $\varphi_1, \varphi_2, \varphi_3$ eingesetzt werden können
- Müssen alle ausprobiert werden?

- Die erste Methode geht viel einfacher vor:
  (1) Ersetze die Formeln $\varphi_1, \varphi_2, \varphi_3$ durch aussagenlogische Variablen A_1, A_2, A_3
  (2) Überprüfe die behauptete Äquivalenz mit einer Wahrheitstabelle
- Für die Beispielformeln ergibt sich:

  $$(A_1 \wedge A_2) \wedge A_3 \equiv A_1 \wedge (A_2 \wedge A_3)$$

- Dann schließen wir
  - aus der Gültigkeit der Äquivalenz für die konkreten Formeln (mit A_1, A_2, A_3)
  - auf die Gültigkeit der Äquivalenz für die allgemeinen Formeln (mit $\varphi_1, \varphi_2, \varphi_3$)

- Ist das so erlaubt?
- Ja, die Korrektheit dieser Methode lässt sich einfach beweisen

  ➡ **Substitutionslemma**

Wahrheitstabellen und Substitutionen (2/6)

Definition: Substitution

- Eine **Substitution** S ist eine partielle Funktion $\mathrm{AV} \rightharpoonup \mathrm{AL}$, die aussagenlogische Variablen auf AL-Formeln abbildet

- $S(\varphi)$ bezeichne die Formel, die aus der AL-Formel φ entsteht, indem jede in φ vorkommende Variable X, für die $S(X)$ definiert ist, durch $S(X)$ ersetzt wird

Beispiel

- Sei $\varphi_1 = \neg(A \vee B)$
- Sei $\varphi_2 = \neg A \wedge \neg B$
- Sei S definiert durch $A \mapsto (C \wedge D)$

- Dann ist:
  - $S(\varphi_1) = \neg((C \wedge D) \vee B)$
  - $S(\varphi_2) = \neg(C \wedge D) \wedge \neg B$

Wahrheitstabellen und Substitutionen (3/6)

Lemma A.2.2: Substitutionslemma

- Sei S eine Substitution
- Seien φ_1, φ_2 AL-Formeln

- Wenn $\varphi_1 \equiv \varphi_2$ gilt,
  dann gilt auch $S(\varphi_1) \equiv S(\varphi_2)$

Beispiel

- Sei $\varphi_1 = \neg(A \vee B)$
- Sei $\varphi_2 = \neg A \wedge \neg B$
- Sei S definiert durch $A \mapsto (C \wedge D)$

- Dann ist:
  - $S(\varphi_1) = \neg((C \wedge D) \vee B)$
  - $S(\varphi_2) = \neg(C \wedge D) \wedge \neg B$

- Es gilt: $\varphi_1 \equiv \varphi_2$ ☞ Wahrheitstabelle
- Die Anwendung des Substitutionslemma ergibt:

  $\neg((C \wedge D) \vee B) \equiv \neg(C \wedge D) \wedge \neg B$

Wahrheitstabellen und Substitutionen (4/6)

Beweisskizze für Satz A.2.1

- Als Beispiel zeigen wir die Assoziativität der Konjunktion, also:

$$(\varphi_1 \wedge \varphi_2) \wedge \varphi_3 \equiv \varphi_1 \wedge (\varphi_2 \wedge \varphi_3)$$

- Durch Angabe einer Wahrheitstabelle stellen wir fest:

$$(A_1 \wedge A_2) \wedge A_3 \equiv A_1 \wedge (A_2 \wedge A_3)$$

- Für beliebige Formeln $\varphi_1, \varphi_2, \varphi_3$ folgt die Behauptung nun durch Anwendung des Substitutionslemmas mit der Substitution:
  - $A_1 \mapsto \varphi_1$
  - $A_2 \mapsto \varphi_2$
  - $A_3 \mapsto \varphi_3$

- Also gilt: $(\varphi_1 \wedge \varphi_2) \wedge \varphi_3 \equiv \varphi_1 \wedge (\varphi_2 \wedge \varphi_3)$

- Der Beweis der anderen Äquivalenzen aus Satz A.2.1 verläuft analog

Wahrheitstabellen und Substitutionen (5/6)

Lemma A.2.3: Substitutionslemma

- Sei S eine Substitution
- Seien φ_1, φ_2 AL-Formeln

- Wenn $\varphi_1 \equiv \varphi_2$ gilt,
  dann gilt auch $S(\varphi_1) \equiv S(\varphi_2)$

Beweis des Substitutionslemmas (1/2)

- OBdA: $S(X)$ für alle X definiert
- Sonst setze $S(X) \stackrel{\text{def}}{=} X$

- Sei α eine Belegung für $S(\varphi_1)$ und $S(\varphi_2)$
- Zu zeigen: $[\![S(\varphi_1)]\!]_\alpha = [\![S(\varphi_2)]\!]_\alpha$

- Sei die Belegung α_S für φ_1 und φ_2 definiert durch $\alpha_S(X) \stackrel{\text{def}}{=} [\![S(X)]\!]_\alpha$
- Wir zeigen gleich:
  $[\![S(\varphi)]\!]_\alpha = [\![\varphi]\!]_{\alpha_S}$ für alle Formeln φ

- Dann folgt für die Belegung α:

$$\begin{aligned} [\![S(\varphi_1)]\!]_\alpha &= [\![\varphi_1]\!]_{\alpha_S} \\ &= [\![\varphi_2]\!]_{\alpha_S} \qquad \text{☞ } \varphi_1 \equiv \varphi_2 \\ &= [\![S(\varphi_2)]\!]_\alpha \end{aligned}$$

- Es bleibt zu zeigen:
  $[\![S(\varphi)]\!]_\alpha = [\![\varphi]\!]_{\alpha_S}$ für alle Formeln φ

Wahrheitstabellen und Substitutionen (6/6)

Lemma A.2.3: Substitutionslemma

- Sei S eine Substitution
- Seien φ_1, φ_2 aussagenlogische Formeln

- Wenn $\varphi_1 \equiv \varphi_2$ gilt,
  dann gilt auch $S(\varphi_1) \equiv S(\varphi_2)$

Beweis des Substitutionslemmas (2/2)

- **Erinnerung:**
  - α ist eine Belegung für $S(\varphi_1)$ und $S(\varphi_2)$
  - $\alpha_S(X) \stackrel{\text{def}}{=} [\![S(X)]\!]_\alpha$ ist eine Belegung für φ_1 und φ_2
- Zeige durch strukturelle Induktion über den Aufbau von Formeln:

  $[\![S(\varphi)]\!]_\alpha = [\![\varphi]\!]_{\alpha_S}$ für alle Formeln φ

Induktionsanfang:

- Für $\varphi = \bot$:

$$[\![S(\bot)]\!]_\alpha = [\![\bot]\!]_\alpha = 0 = [\![\bot]\!]_{\alpha_S}$$

- Für $\varphi = \top$: analog
- Für $\varphi = X$ für eine Variable X:

$$[\![S(X)]\!]_\alpha = \alpha_S(X) = [\![X]\!]_{\alpha_S}$$

Induktionsschritt:

- Für $\varphi = \varphi_1 \wedge \varphi_2$:

$$\begin{aligned}
&[\![S(\varphi_1 \wedge \varphi_2)]\!]_\alpha \\
&\quad = [\![S(\varphi_1) \wedge S(\varphi_2)]\!]_\alpha \\
&\quad = \min\{[\![S(\varphi_1)]\!]_\alpha, [\![S(\varphi_1)]\!]_\alpha\} && \text{☞ Def. } \wedge \\
&\quad = \min\{[\![\varphi_1]\!]_{\alpha_S}, [\![\varphi_1]\!]_{\alpha_S}\} && \text{☞ I.V.} \\
&\quad = [\![\varphi_1 \wedge \varphi_2]\!]_{\alpha_s} && \text{☞ Def. } \wedge
\end{aligned}$$

- Für $\varphi = \varphi_1 \vee \varphi_2$ und $\varphi = \neg\varphi_1$: analog

Äquivalenzumformungen

- Jetzt lernen wir die zweite Methode zum Nachweis von Äquivalenzen kennen

Beispiel

- Nach De Morgan gilt:

$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

- Dann sollte auch gelten:

$$\neg(A \vee B) \wedge C \equiv (\neg A \wedge \neg B) \wedge C$$

- Denn: Es wird lediglich $\neg(A \vee B)$ durch $(\neg A \wedge \neg B)$ ersetzt

- Dass eine solche Schlussweise allgemein zulässig ist, sagt uns das **Ersetzungslemma**

Lemma A.2.4: Ersetzungslemma

- Sei φ_1 eine Formel mit einer Teilformel ψ_1
- Sei ψ_2 eine Formel mit $\psi_2 \equiv \psi_1$

- Wenn φ_2 aus φ_1 entsteht, indem ein Vorkommen von ψ_1 durch ψ_2 ersetzt wird, dann gilt $\varphi_2 \equiv \varphi_1$

Beispiel

- Sei $\varphi_1 = \neg(A \vee B) \wedge C$
- Sei $\psi_1 = \neg(A \vee B)$
- Sei $\psi_2 = (\neg A \wedge \neg B)$

- Wir wissen: $\psi_1 \equiv \psi_2$

- Mit dem Ersetzungslemma folgt:

$$\neg(A \vee B) \wedge C \equiv (\neg A \wedge \neg B) \wedge C$$

- Der Beweis des Ersetzungslemmas ist nicht schwierig, wird aber aus Zeitgründen hier nicht gegeben

Mehrere Äquivalenzumformungen

- Die Relation $\equiv$ ist eine Äquivalenzrelation auf AL
  - Insbesondere ist sie transitiv, also:

    Wenn $\varphi_1 \equiv \varphi_2$ und $\varphi_2 \equiv \varphi_3$ gilt, dann gilt auch $\varphi_1 \equiv \varphi_3$

- Äquivalenzen zwischen Formeln lassen sich also auch nachweisen durch Folgen von Anwendungen der Äquivalenzen aus Satz A.2.1
  - auf die ganze Formel, und
  - auf Teilformeln (unter Zuhilfenahme des Ersetzungslemmas)

Beispiel

- Wir zeigen: $\neg(\neg\varphi_1 \vee \varphi_2) \wedge \varphi_3 \equiv \varphi_1 \wedge \neg(\varphi_2 \vee \neg\varphi_3)$

$$
\begin{aligned}
\neg(\neg\varphi_1 \vee \varphi_2) \wedge \varphi_3 &\equiv (\neg\neg\varphi_1 \wedge \neg\varphi_2) \wedge \varphi_3 && \text{☞ De Morgan}\\
&\equiv (\varphi_1 \wedge \neg\varphi_2) \wedge \varphi_3 && \text{☞ Doppelnegation}\\
&\equiv \varphi_1 \wedge (\neg\varphi_2 \wedge \varphi_3) && \text{☞ Assoziativität von } \wedge\\
&\equiv \varphi_1 \wedge (\neg\varphi_2 \wedge \neg\neg\varphi_3) && \text{☞ Doppelnegation}\\
&\equiv \varphi_1 \wedge \neg(\varphi_2 \vee \neg\varphi_3) && \text{☞ De Morgan}
\end{aligned}
$$

Normalformen der Aussagenlogik

Normalformen

- Die Äquivalenzregeln der Aussagenlogik sind insbesondere hilfreich, um Formeln in eine bestimmte, „günstige“ syntaktische Form umzuwandeln

- Dabei werden zumeist zwei Ziele angestrebt
  - Leichtere Verständlichkeit
  - Garantie einer bestimmten Struktur zur Erleichterung der Anwendung von Algorithmen

- Wenn eine solche „günstige Form“ immer erreicht werden kann, nennen wir sie **Normalform**

- Wir betrachten drei Normalformen:
  - Negationsnormalform
  - Konjunktive Normalform
  - Disjunktive Normalform

Die Negationsnormalform (1/2)

Beispiel

- Lässt sich die Formel

$$\neg(\neg(B \wedge \neg A) \vee \neg A)$$

vereinfachen, ohne die Semantik zu verändern?

- Ja:

$$\neg(\neg(B \wedge \neg A) \vee \neg A) \equiv \bot$$

- Warum sieht man das der Formel nicht sofort an?

➡ Verschachtelte Negationen sind oft schwierig zu verstehen

- Falls eine AL-Formel φ Negationszeichen nur unmittelbar vor Variablen enthält, ist sie in **Negationsnormalform** (kurz: NNF)

Beispiel

- $A_1 \vee \neg(A_2 \vee \neg A_3)$ ist nicht in NNF
- $A_1 \vee (\neg A_2 \wedge A_3)$ ist in NNF

Beispiel

- Die Formel $\neg(\neg(B \wedge \neg A) \vee \neg A)$ lässt sich leicht in Negationsnormalform umformen:

$$\begin{aligned} & \neg(\neg(B \wedge \neg A) \vee \neg A) \\ \equiv\ & \neg\neg(B \wedge \neg A) \wedge \neg\neg A && \text{☞ De Morgan} \\ \equiv\ & (B \wedge \neg A) \wedge A && \text{☞ Doppelneg.} \end{aligned}$$

- Dass die Formel in Negationsnormalform umgeformt werden kann, ist kein Zufall ...

Die Negationsnormalform (2/2)

Satz A.2.5

- Zu jeder AL-Formel φ kann eine äquivalente Formel φ' in NNF berechnet werden

Beweisskizze

- Sei NNF die wie folgt induktiv definierte Funktion:

1. $\mathrm{NNF}(\varphi) = \varphi$ für Formeln φ der Art $\bot$, $\top$, X oder $\neg X$ (für Variablen X)
2. $\mathrm{NNF}((\varphi_1 \vee \varphi_2)) = (\mathrm{NNF}(\varphi_1) \vee \mathrm{NNF}(\varphi_2))$
3. $\mathrm{NNF}((\varphi_1 \wedge \varphi_2)) = (\mathrm{NNF}(\varphi_1) \wedge \mathrm{NNF}(\varphi_2))$
4. $\mathrm{NNF}(\neg\bot) = \top$ und $\mathrm{NNF}(\neg\top) = \bot$
5. $\mathrm{NNF}(\neg\neg\varphi_1) = \mathrm{NNF}(\varphi_1)$
6. $\mathrm{NNF}(\neg(\varphi_1 \vee \varphi_2)) = (\mathrm{NNF}(\neg\varphi_1) \wedge \mathrm{NNF}(\neg\varphi_2))$
7. $\mathrm{NNF}(\neg(\varphi_1 \wedge \varphi_2)) = (\mathrm{NNF}(\neg\varphi_1) \vee \mathrm{NNF}(\neg\varphi_2))$

- Durch Induktion nach der Struktur von φ lässt sich zeigen:
  1. $\mathrm{NNF}(\varphi) \equiv \varphi$
  2. $\mathrm{NNF}(\varphi)$ ist in Negationsnormalform

Konjunktive und Disjunktive Normalform

Definition: KNF, DNF

- Ein **Literal** L ist eine Formel der Form X oder $\neg X$ mit $X \in \mathrm{AV}$

- Eine **disjunktive Klausel** ist eine Disjunktion $\bigvee_{i=1}^{k} L_i$ von Literalen
- Eine **konjunktive Klausel** ist eine Konjunktion $\bigwedge_{i=1}^{k} L_i$ von Literalen

- Eine AL-Formel φ ist in **konjunktiver Normalform (KNF)**, falls sie eine Konjunktion disjunktiver Klauseln ist
- Eine AL-Formel φ ist in **disjunktiver Normalform (DNF)**, falls sie eine Disjunktion konjunktiver Klauseln ist

Beispiel

- $\neg A_2$ ist ein (negatives) Literal
- A_3 ist ein (positives) Literal
- $\neg\neg A_1$ ist kein Literal

- $(A_1 \vee A_2 \vee \neg A_4)$ ist eine disjunktive Klausel
- $(A_3 \wedge A_2 \wedge \neg A_1)$ ist eine konjunktive Klausel

- $(A_1 \vee A_2 \vee \neg A_4) \wedge (\neg A_3 \vee A_4) \wedge A_3$ ist in KNF
- $\neg A_3 \vee (A_1 \wedge A_2) \vee (A_3 \wedge A_2 \wedge \neg A_1)$ ist in DNF

- Beobachtung: KNF-Formeln und DNF-Formeln sind auch in NNF

- Zusätzlich vereinbaren wir:
  - Eine Disjunktion von $\mathbf{0}$ konjunktiven Klauseln ist äquivalent zu $\bot$
  - Eine Konjunktion von $\mathbf{0}$ disjunktiven Klauseln ist äquivalent zu $\top$
  - Die Formeln $\top$ und $\bot$ sind in KNF und DNF

Berechnung von KNF und DNF

Satz A.2.6

- Zu jeder AL-Formel φ gibt es eine äquivalente KNF-Formel φ_1 und eine äquivalente DNF-Formel φ_2
- φ_1 und φ_2 können durch einen Algorithmus berechnet werden

Algorithmus: KNF-Transformation

Eingabe: AL-Formel φ
Ausgabe: AL-Formel φ_1 in KNF mit $\varphi \equiv \varphi_1$

1: Bringe φ in NNF
2: **while** eine der folgenden Regeln ist anwendbar **do**
3: Ersetze eine Teilformel der Form $(\psi_1 \vee (\psi_2 \wedge \psi_3))$ durch $((\psi_1 \vee \psi_2) \wedge (\psi_1 \vee \psi_3))$
4: Ersetze eine Teilformel der Form $((\psi_1 \wedge \psi_2) \vee \psi_3)$ durch $((\psi_1 \vee \psi_3) \wedge (\psi_2 \vee \psi_3))$
5: Entferne überflüssige Klammern mit Hilfe der Assoziativität der Disjunktion und der Konjunktion
6: **end while**

- Die Korrektheit des Algorithmus lässt sich durch Induktion nach φ beweisen
- Der Algorithmus lässt eine gewisse Wahlfreiheit
  ➡ das Ergebnis ist nicht eindeutig bestimmt
- Die Berechnung der DNF verläuft dual
  ☞ $\wedge$ und $\vee$ vertauschen

KNF: Beispiel-Berechnung (1/2)

Beispiel [KuK]

- Sei $\varphi = (A \leftrightarrow (B \wedge \neg C)) \wedge (\neg B \rightarrow A)$
- Ersetzung der Abkürzungen $\leftrightarrow$ und $\rightarrow$ ergibt die Formel:

$$((A \wedge (B \wedge \neg C)) \vee (\neg A \wedge \neg (B \wedge \neg C))) \wedge (B \vee A)$$

- Die Negationsnormalform davon ist:

$$((A \wedge (B \wedge \neg C)) \vee (\neg A \wedge (\neg B \vee C))) \wedge (B \vee A)$$

- Mit dieser Formel geht der Algorithmus in die While-Schleife

KNF: Beispiel-Berechnung (2/2)

Beispiel

$$
\begin{aligned}
& ((A \wedge (B \wedge \neg C)) \vee (\neg A \wedge (\neg B \vee C))) \wedge (B \vee A) \\
\equiv\ & \big((A \wedge (B \wedge \neg C)) \vee \neg A\big) \wedge \big((A \wedge (B \wedge \neg C)) \vee (\neg B \vee C)\big) \\
& \quad \wedge (B \vee A) \\
\equiv\ & (A \vee \neg A) \wedge ((B \wedge \neg C) \vee \neg A) \\
& \quad \wedge \big((A \wedge (B \wedge \neg C)) \vee (\neg B \vee C)\big) \\
& \quad \wedge (B \vee A) \\
\equiv\ & (A \vee \neg A) \wedge (B \vee \neg A) \wedge (\neg C \vee \neg A) \\
& \quad \wedge \big((A \wedge (B \wedge \neg C)) \vee (\neg B \vee C)\big) \\
& \quad \wedge (B \vee A) \\
\equiv\ & (A \vee \neg A) \wedge (B \vee \neg A) \wedge (\neg C \vee \neg A) \\
& \quad \wedge (A \vee \neg B \vee C) \wedge ((B \wedge \neg C) \vee (\neg B \vee C)) \\
& \quad \wedge (B \vee A) \\
\equiv\ & (A \vee \neg A) \wedge (B \vee \neg A) \wedge (\neg C \vee \neg A) \\
& \quad \wedge (A \vee \neg B \vee C) \wedge (B \vee \neg B \vee C) \wedge (\neg C \vee \neg B \vee C) \\
& \quad \wedge (B \vee A)
\end{aligned}
$$

Formelgrößen der Normalformen

- Bei der Umwandlung einer Formel φ in NNF ist die Ziel-Formel φ' nicht viel größer als φ: $|\varphi'| \leq \mathbf{2}|\varphi|$

- Im Gegensatz dazu kann die Umwandlung in DNF oder KNF zu einem wesentlich größeren Zuwachs führen
- Sei $\underline{\textbf{Parity}_n(A_1, \ldots, A_n)}$ die folgende Funktion mit Wertebereich $\{\mathbf{0}, \mathbf{1}\}$:

  Für eine Belegung der Variablen $A_1, \ldots, A_n$ mit $\mathbf{0}$ oder $\mathbf{1}$ gibt Parity_n genau dann den Wert $\mathbf{1}$ aus, wenn ungerade viele Variablen mit $\mathbf{1}$ belegt sind

Satz A.2.7

(a) Es gibt eine aussagenlogische Formel der Größe $O(n^2)$ für $\text{Parity}_n(A_1, \ldots, A_n)$

(b) Jede KNF-Formel für $\text{Parity}_n(A_1, \ldots, A_n)$ hat mindestens 2^{n-1} Klauseln

(c) Jede DNF-Formel für $\text{Parity}_n(A_1, \ldots, A_n)$ hat mindestens 2^{n-1} Klauseln

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Beispiele der Modellierung mit Aussagenlogik
  - Äquivalenzen der Aussagenlogik
  - Normalformen:
    - Negationsnormalform
    - Disjunktive Normalform
    - Konjunktive Normalform

Anhang: Formelgrößen von Normalformen

NNF, DNF, KNF: Formelgröße (1/2)

- Bei der Umwandlung einer Formel φ in NNF ist die Ziel-Formel φ' nicht viel größer als φ: $\quad |\varphi'| \leq \mathbf{2}|\varphi|$

- Im Gegensatz dazu kann die Umwandlung in DNF oder KNF zu einem wesentlich größeren Zuwachs führen

Beispiel

- Sei $\underline{\textbf{Parity}_n}$ die folgende Funktion mit Wertebereich $\{\mathbf{0}, \mathbf{1}\}$:
  - Für eine Belegung α der Variablen $A_1, \ldots, A_n$ gibt Parity_n genau dann den Wert 1 aus, wenn die Anzahl der $i \leq n$ mit $\alpha(A_i) = 1$ ungerade ist

- **Beobachtung:** $\text{Parity}_n(A_1, \ldots, A_n)$ lässt sich durch eine Formel der Größe $O(n^2)$ ausdrücken
- **Idee:** Aus einer AL-Formel $\varphi_n(A_1, \ldots, A_n)$ für Parity_n, lässt sich leicht eine Formel $\varphi_{2n}(A_1, \ldots, A_{2n})$ für Parity_{2n} gewinnen:

$$\varphi_{2n}(A_1, \ldots, A_{2n}) \stackrel{\text{def}}{=} (\varphi_n(A_1, \ldots, A_n) \wedge \neg\varphi_n(A_{n+1}, \ldots, A_{2n})) \\ \vee (\neg\varphi_n(A_1, \ldots, A_n) \wedge \varphi_n(A_{n+1}, \ldots, A_{2n}))$$

- Wir sehen jetzt: Für Parity_n gibt es keine kleine KNF und DNF

NNF, DNF, KNF: Formelgröße (2/2)

Satz A.2.8

(a) Jede KNF-Formel für $\text{Parity}_n(A_1, \ldots, A_n)$ hat mindestens 2^{n-1} Klauseln

(b) Jede DNF-Formel für $\text{Parity}_n(A_1, \ldots, A_n)$ hat mindestens 2^{n-1} Klauseln

Beweisskizze

- Sei $\varphi = \bigvee_{i=1}^{k} \varphi_i$ eine DNF-Formel für $\text{Parity}_n(A_1, \ldots, A_n)$

- **Zwischenbehauptung:**

  Jede Klausel φ_i enthält alle Variablen

- **Annahme:** A_j kommt nicht in φ_i vor
  - Sei α eine Belegung, die φ_i wahr macht

    $\Rightarrow$ Die Belegung α', in der der Wert für A_j geändert wird, macht φ_i auch wahr
  - **Widerspruch!** ☞ Definition von Parity

- Es gibt 2^{n-1} Belegungen der Variablen, die φ wahr machen

  $\Rightarrow$ Für jede dieser Belegungen gibt es eine andere Klausel

  $\Rightarrow$ Behauptung

- KNF: dual

Kapitel A.3:

Erfüllbarkeit aussagenlogischer Formeln

In diesem Kapitel

- Viele algorithmische Probleme lassen sich auf die Frage zurückführen:

  Hat eine gegebene aussagenlogische Formel eine Belegung, die sie wahr macht?

- Beispiel: Im letzten Kapitel haben wir eine aussagenlogische Formel φ für einen Graphen G konstruiert sodass

  φ hat eine erfüllende Belegung
  $\Leftrightarrow$ G hat eine zulässige $\mathbf{3}$-Färbung

- Weitere Beispiele:
  - Existenz eines Matchings, einer Euler-Tour,...
  - Äquivalenz Boolescher Schaltkreise
  - usw.

- In diesem Kapitel lernen wir kennen:
  - Methoden, mit denen sich herausfinden lässt, ob eine gegebene aussagenlogische Formel eine erfüllende Belegung hat

    ➡ Im Allgemeinen ist das ein schwieriges algorithmisches Problem
  - Eine Einschränkung von aussagenlogischen Formeln, für die sich Erfüllbarkeit effizient testen lässt

Das Erfüllbarkeitsproblem

Erfüllbare und allgemeingültige Formeln

Definition:
Modelle, erfüllbar, Tautologien

- Eine Belegung α heißt **Modell** von φ wenn $\alpha \models \varphi$

- Eine Formel φ heißt
  - **erfüllbar**, falls sie ein Modell hat,
  - andernfalls **unerfüllbar**

- Eine Formel φ heißt **allgemeingültig** (oder: **Tautologie**), falls jede zu φ passende Belegung ein Modell von φ ist

- Ist Φ eine Menge von aussagenlogischen Formeln, so heißt α **Modell** von Φ, falls α ein Modell von allen $\varphi \in \Phi$ ist
- Notation: $\alpha \models \Phi$

- Die Begriffe **erfüllbar** und **allgemeingültig** werden für Mengen von Formeln analog definiert

Beispiel

- $(A_1 \wedge \neg A_1)$ ist unerfüllbar
- $(A_1 \vee \neg A_1)$ ist allgemeingültig
- $A_1 \vee A_2$ ist erfüllbar aber nicht allgemeingültig
- $\{(A_1 \vee \neg A_2), A_2, \neg A_1\}$ ist unerfüllbar

- Die Menge der aussagenlogischen Formeln lässt sich in drei disjunkte Mengen aufteilen:

Allgemeingültig

Erfüllbar, aber nicht allgemeingültig

Unerfüllbar

Erfüllbarkeitstest mit Wahrheitstabelle (1/3)

- Wir beschäftigen uns in diesem Kapitel mit dem folgenden algorithmischen Problem:

Problem: SAT

- **Gegeben:** Eine aussagenlogische Formel φ
- **Frage:** Ist φ erfüllbar?
  (Falls ja: Was ist eine erfüllende Belegung?)

- Eine einfache Methode zum Testen der Erfüllbarkeit einer Formel φ:
  - Berechne die Wahrheitstabelle von φ
  - φ ist genau dann erfüllbar, wenn in der Spalte für φ eine $\mathbf{1}$ vorkommt

- Das ist ein **semantischer Erfüllbarkeitstest**
  - Denn alle möglichen Modelle werden ausprobiert
  - Der Test beruht also darauf, die Semantik von φ vollständig zu bestimmen

Erfüllbarkeitstest mit Wahrheitstabelle (2/3)

Beispiel

- $\varphi_1 = (A \vee C) \wedge (B \vee D)$

A	B	C	D	φ_1
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

Beispiel

- $\varphi_2 = (A \wedge B \wedge C) \wedge (\neg B \vee D) \wedge (\neg B \vee \neg D)$

A	B	C	D	φ_2
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Erfüllbarkeitstest mit Wahrheitstabelle (3/3)

- **Problem:** Wahrheitstabellen werden sehr schnell sehr groß
- **Genauer:** Für Formeln mit n Variablen hat die Wahrheitstabelle $\mathbf{2}^{n}$ Zeilen

- Falls die Formel erfüllbar ist, wird eine erfüllende Belegung eventuell schneller gefunden
- Falls sie unerfüllbar ist, müssen aber alle $\mathbf{2}^{n}$ Belegungen ausprobiert werden

- **Wir suchen:** Methoden, die die Erfüllbarkeit auf Basis der **Syntax** der Formeln testen
- **Hoffnung:** Auch Unerfüllbarkeit lässt sich effizienter nachweisen

(Zu) Naiver Ansatz: Erfüllbarkeitstest mit DNF

- Wie schwierig ist es, die Erfüllbarkeit von Formeln in DNF zu testen?

Beispiel

$$(A \wedge \neg B \wedge C \wedge D) \vee (\neg A \wedge B \wedge C \wedge \neg D)$$

- Sehr einfach: es genügt, eine erfüllbare konjunktive Klausel zu finden
- Ein möglicher Erfüllbarkeitstest wäre also:
  (1) Wandle φ in eine äquivalente Formel φ' in DNF um
  (2) Teste die Erfüllbarkeit von φ'

- **Problem:** Wie wir wissen, kann die in Schritt (1) berechnete Formel φ' sehr lang werden
  - Bei der Parity-Funktion ist die DNF exponentiell größer als die kleinste Formel
  - Das ist zwar ein Extremfall, aber...

- ... Formeln, die in Anwendungen auftreten, haben häufig eine lange DNF

- **Typisch:** Bei der Modellierung von Situationen ergibt sich eine Menge von Teilformeln, die alle gleichzeitig gelten müssen
  ➡ Das führt in der Regel zu Formeln mit einer relativ kurzen KNF, aber einer langen DNF

➡ **Im Folgenden:** Erfüllbarkeitstests für aussagenlogische Formeln in KNF

Erfüllbarkeitstest mit KNF

- Unsere Vorgehensweise zum Testen der Erfüllbarkeit einer aussagenlogischen Formel φ ist also:
  (1) Wandle φ in eine äquivalente Formel φ' in KNF um
  (2) Teste die Erfüllbarkeit von φ'

- Im Folgenden testen wir stets die Erfüllbarkeit von Formeln in KNF
- **Ab jetzt:** „Klausel“ steht immer für disjunktive Klauseln

- Klar: eine Formel in KNF ist erfüllbar, wenn es eine Belegung gibt, die alle ihre Klauseln wahr macht
  ➡ Erfüllbarkeitstests für KNF-Formeln suchen nach einer erfüllenden Belegung für eine Menge von Klauseln

- **Als nächstes:**
  - Ein Erfüllbarkeitstest für eingeschränkte Formeln
  - Dann: Ein allgemeiner Test

Hornformeln und der Markierungsalgorithmus

Klauseln in Implikationsform (1/2)

- **Erinnerung:** $\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi$
- **Beobachtung:** (Disjunktive) Klauseln lassen sich immer auch als **Implikationen** schreiben

Beispiel

- $(A_1 \vee \neg A_2 \vee \neg A_3 \vee A_4)$ ist äquivalent zu:

  $$(\neg A_1 \wedge A_2) \rightarrow (\neg A_3 \vee A_4)$$

- Aber auch zu:

  $$(A_2 \wedge A_3) \rightarrow (A_1 \vee A_4)$$

- Bei geschickter Sortierung können wir immer eine Implikation ohne (explizite) Negationen finden

  ➡ Implikationsform

Beispiel

- Eine zur Klausel

  $$(A \vee \neg C \vee \neg B \vee E \vee \neg D)$$

  äquivalente Formel in Implikationsform ist

  $$(C \wedge B \wedge D) \rightarrow (A \vee E)$$

Klauseln in Implikationsform (2/2)

Definition: Implikationsform

- Sei $K = \neg X_1 \vee \cdots \vee \neg X_k \vee X_{k+1} \vee \cdots \vee X_m$ eine Klausel, wobei alle X_i aussagenlogische Variablen sind
- Die **Implikationsform** für K ist dann

$$\Big(\bigwedge_{i=1}^{k} X_i\Big) \rightarrow \Big(\bigvee_{j=k+1}^{m} X_j\Big)$$

- Um eine Klausel in Implikationsform zu erreichen, werden die Literale also zunächst umsortiert

- Die Konjunktion $\bigwedge_{i=1}^{k} X_i$ heißt **Prämisse**
- Die Disjunktion $\bigvee_{j=k+1}^{m} X_j$ heißt **Konsequenz**

- Spezialfälle von Klauseln in Implikationsform:
  - leere Prämisse: $\top \rightarrow (\bigvee_{i=1}^{k} X_i)$
  - leere Konsequenz: $(\bigwedge_{i=1}^{k} X_i) \rightarrow \bot$

Hornklauseln und Hornformeln

Definition: Horn-Klauseln und -Formeln

- Eine Klausel $(L_1 \vee \cdots \vee L_k)$ heißt **Horn-Klausel**, falls sie höchstens ein positives Literal L_i hat
- Eine Formel in KNF heißt **Horn-Formel**, falls alle ihre Klauseln Horn-Klauseln sind

✎ Horn-Formeln sind nach dem Logiker Alfred Horn (1918-2001) benannt

Beispiel

- $(A_1 \vee \neg A_2 \vee \neg A_3) \wedge (\neg A_1 \vee A_3 \vee A_4)$
  ist keine Horn-Formel
- $(A_1 \vee \neg A_2 \vee \neg A_3) \wedge A_2 \wedge (\neg A_1 \vee \neg A_3 \vee \neg A_4)$
  ist eine Horn-Formel

- In der Implikationsform von Horn-Klauseln enthält also die Konsequenz höchstens ein Literal

Beispiel

- $(A_1 \vee \neg A_2 \vee \neg A_3) \wedge A_2 \wedge (\neg A_1 \vee \neg A_3 \vee \neg A_4)$

  hat die Implikationsform

  $((A_2 \wedge A_3) \rightarrow A_1) \wedge A_2 \wedge ((A_1 \wedge A_3 \wedge A_4) \rightarrow \bot)$

Erfüllbarkeit von Horn-Formeln: Der Markierungsalgorithmus (1/2)

- **Frage:** Wie lässt sich testen, ob eine Horn-Formel φ erfüllbar ist?

- **Intuitive Überlegung:**
  - In Hornklauseln sind Literale „überwiegend“ negativ
  - Ansatz zum Test der Erfüllbarkeit:

    Versuche, so viele Variablen wie möglich mit $\mathbf{0}$ zu belegen

- Algorithmische Grundidee des **Markierungsalgorithmus**:

  (1) Berechne (und **markiere**) die Variablen, die unbedingt mit $\mathbf{1}$ belegt werden **müssen**

  (2) Belege die übrigen Variablen mit $\mathbf{0}$ und teste, ob diese Belegung die Formel wahr macht

- Für die Umsetzung von Schritt (1) nutzen wir folgende Beobachtungen:
  - Wenn A eine Klausel in φ ist, dann muss A mit $\mathbf{1}$ belegt werden
  - Wenn $(A \wedge B) \rightarrow C$ eine Klausel in φ ist und wir bereits wissen, dass A und B mit $\mathbf{1}$ belegt werden müssen, dann muss auch C mit $\mathbf{1}$ belegt werden

Erfüllbarkeit von Horn-Formeln: Der Markierungsalgorithmus (2/2)

Markierungsalgorithmus

Eingabe: Horn-Formel φ
Ausgabe: „erfüllbar“, falls φ erfüllbar, sonst „unerfüllbar“

1: Markiere alle Variablen X, für die es eine Klausel X in φ gibt
2: **while** es gibt Klauseln der Art
$(X_1 \wedge \cdots \wedge X_k) \rightarrow X$
in φ, für die
(1) $X_1, \ldots, X_k$ markiert sind und
(2) X noch nicht markiert ist
do
3: Wähle eine solche Klausel und markiere X
4: **end while**
5: **if** es gibt eine Klausel der Art
$(X_1 \wedge \cdots \wedge X_k) \rightarrow \bot$
in φ für die $X_1, \ldots, X_k$ markiert sind
then
6: Ausgabe „unerfüllbar“
7: **else**
8: Ausgabe „erfüllbar“
9: **end if**

Beispiel

- Formel φ:

$$(\neg A_2 \vee A_4) \wedge (\neg A_4 \vee \neg A_2 \vee A_5) \wedge (\neg A_2 \vee \neg A_3 \vee A_4 \vee \neg A_1) \wedge (\neg A_5 \vee \neg A_4) \wedge A_2$$

- In Implikationsform übersetzt:

$$(A_2 \rightarrow A_4) \wedge ((A_4 \wedge A_2) \rightarrow A_5) \wedge ((A_1 \wedge A_2 \wedge A_3) \rightarrow A_4) \wedge ((A_5 \wedge A_4) \rightarrow \bot) \wedge A_2$$

- Markierte Variablen:
  - A_2
  - A_4
  - A_5

- φ ist unerfüllbar
  ➡ wegen Klausel $(A_5 \wedge A_4) \rightarrow \bot$

Korrektheit des Markierungsalgorithmus (1/3)

Satz A.3.1

- Der Markierungsalgorithmus löst das Erfüllbarkeitsproblem für Horn-Formeln φ in Laufzeit $O(|\varphi|)$

  ☞ bei geeigneter Implementierung

Beweis

- Sei φ eine Horn-Formel mit n Variablen

- Zur Laufzeit:
  - Klar: der Algorithmus terminiert nach höchstens n Durchläufen der WHILE-Schleife
    - Denn: in jedem Durchlauf wird eine Variable markiert
  - Bei geschickter Implementierung ergibt sich daraus eine $O(|\varphi|)$-Laufzeit-Schranke

- Für die Korrektheit zeigen wir, dass die folgenden Aussagen äquivalent sind:

  (1) Der Markierungsalgorithmus hat die Ausgabe „erfüllbar“
  (2) Die Formel φ ist erfüllbar

Korrektheit des Markierungsalgorithmus (2/3)

Beweis (Fortsetzung)

- Wir zeigen zuerst:

  Wenn der Algorithmus „erfüllbar“ ausgibt,
  dann ist φ erfüllbar

- Sei die Ausgabe des Algorithmus also „erfüllbar“

- **Idee:** Definiere eine erfüllende Belegung α mit Hilfe der durch den Algorithmus markierten Variablen
- Wir definieren:

$$\alpha(X) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } X \text{ markiert ist} \\ 0 & \text{andernfalls} \end{cases}$$

- **Zeige:** α erfüllt φ
- Sei dazu K eine beliebige Klausel von φ
- **Zu zeigen:** α erfüllt K

- Fallunterscheidung nach dem Typ von K

- **1. Fall:** K ist vom Typ

$$(X_1 \wedge \cdots \wedge X_k) \rightarrow X$$

  - Falls $\alpha(X) = 1$ ist auch $[\![K]\!]_\alpha = 1$
  - Falls $\alpha(X) = 0$, dann ist $[\![X_1 \wedge \cdots \wedge X_k]\!]_\alpha = 0$

    ☞ sonst wäre X markiert worden
  - Also: $[\![K]\!]_\alpha = 1$

- **2. Fall:** K ist vom Typ

$$(X_1 \wedge \cdots \wedge X_k) \rightarrow \bot$$

  - Dann ist $[\![X_1 \wedge \cdots \wedge X_k]\!]_\alpha = 0$

    ☞ sonst wäre die Ausgabe „unerfüllbar“
  - Also gilt wieder: $[\![K]\!]_\alpha = 1$

➡ Da alle Klauseln durch α wahr werden, ist φ erfüllbar

Korrektheit des Markierungsalgorithmus (3/3)

Beweis (Fortsetzung)

- Wir zeigen jetzt:

  Wenn φ erfüllbar ist,
  dann gibt der Algorithmus „erfüllbar“ aus

- Sei also φ erfüllbar und α eine beliebige erfüllende Belegung für φ

- Wir zeigen durch Induktion nach m:

  $(*)$ für jedes m sind nach m WHILE-Durchläufen nur Variablen X mit $\alpha(X) = 1$ markiert

- $m = 0$: alle vor der WHILE-Schleife markierten Variablen kommen in Klauseln mit nur einem (positiven) Literal vor
- Diese Variablen müssen unter α den Wert 1 erhalten

- Induktionschritt: von m zu $m + 1$:

- Sei X im $(m + 1)$-sten Durchlauf durch die Klausel $K = (X_1 \wedge \cdots \wedge X_k) \rightarrow X$ markiert
  $\Rightarrow X_1, \ldots, X_k$ sind nach m Durchläufen markiert
  $\Rightarrow \alpha(X_i) = 1$, für alle $i = 1, \ldots, k$

  ☞ Induktion

- Wegen $\alpha \models K$, muss also $\alpha(X) = 1$ gelten

- Damit haben wir die Induktionsbehauptung gezeigt
- Nach der WHILE-Schleife sind also nur Variablen X mit $\alpha(X) = 1$ markiert

- Sei nun K eine Klausel von φ der Form

  $$(X_1 \wedge \cdots \wedge X_k) \rightarrow \bot$$

- Es gilt: $[\![X_1 \wedge \cdots \wedge X_k]\!]_\alpha = 0$
  $\Rightarrow$ nicht für alle X_i gilt $\alpha(X_i) = 1$
  $\Rightarrow$ nicht alle X_i sind markiert

$\Rightarrow$ Der Algorithmus gibt „erfüllbar“ aus

Der aussagenlogische Resolutionskalkül

Grundidee der aussagenlogischen Resolution

- Aussagenlogische Resolution ist ein **Kalkül** für das Testen von **Unerfüllbarkeit** beliebiger aussagenlogischer Formeln

- Ein **Kalkül** ist eine Menge von (syntaktischen) Regeln, mit denen sich aus gegebenen Aussagen neue Aussagen herleiten lassen
- Ziel ist meist eine „gewünschte Formel“ abzuleiten
- Es gibt Kalküle für viele verschiedene Logiken

- **Grundidee der Resolution:**
  - Leite aus den Klauseln einer Formel φ in KNF neue Klauseln ab, die wahr sein müssen, wenn φ wahr ist
  - Ableitung neuer Klauseln mit einer sehr einfachen Regel: die Bildung von **Resolventen**

- Wir werden zeigen:
  - Wenn die leere Klausel hergeleitet werden kann, dann ist φ unerfüllbar
  - Wenn die leere Klausel nicht hergeleitet werden kann, dann ist φ erfüllbar

- **Zunächst:** Eine praktischere Notation für Klauseln

Klauseln und Klauselmengen (1/2)

- Klar: Semantik einer Klausel hängt nicht ab von
  - der Reihenfolge der Literale
  - der Anzahl der Vorkommen eines bestimmten Literals

Beispiel

$(A \vee C \vee \neg B \vee A) \quad \equiv (\neg B \vee C \vee A)$

- Wir repräsentieren Klauseln deshalb oft nur durch die **Menge ihrer Literale**

- Statt

  $$(A \vee C \vee \neg B \vee A)$$

  schreiben wir beispielsweise

  $$\{A, \neg B, C\}$$

- Eine Formel φ in KNF repräsentieren wir dann durch ihre **Klauselmenge** $\mathcal{K}(\varphi)$, also die Menge ihrer Literalmengen

Beispiel

- Die Formel

  $$\varphi = (A \vee \neg B \vee A) \wedge (C \vee B \vee \neg A)$$

  hat die Klauselmenge

  $$\mathcal{K}(\varphi) = \{\{A, \neg B\}, \{C, B, \neg A\}\}$$

✎ Der Begriff „Klausel“ wird also sowohl für Disjunktionen von Literalen (in Formelschreibweise) als auch für Literalmengen verwendet

➡ Was gemeint ist, sollte jeweils durch den Kontext klar sein

Klauseln und Klauselmengen (2/2)

- Die Erfüllbarkeit von KNF-Formeln φ wollen wir durch Testen der Erfüllbarkeit ihrer Klauselmenge $\mathcal{K}(\varphi)$ überprüfen

- Eine Formel φ in KNF wird durch eine Belegung α erfüllt, wenn α in jeder Klausel mindestens ein Literal wahr macht

- Entsprechend: Eine Klauselmenge $\mathcal{K}$ wird durch eine Belegung α **erfüllt**, wenn es in jeder Klausel $K \in \mathcal{K}$ ein Literal L gibt, für das $\alpha \models L$ gilt
- Wir nennen $\mathcal{K}$ **erfüllbar**, wenn es eine Belegung gibt, die $\mathcal{K}$ erfüllt

- Es gibt zwei Spezialfälle:

- Die **leere Klausel** $\emptyset = \{\}$ enthält keine Literale
  - Es gibt keine Belegung α, die diese Klausel erfüllt
  - Insofern entspricht sie der Formel $\bot$
  - Statt $\bot$ verwenden wir aber im Folgenden die im Zusammenhang der Resolution übliche Notation $\square$ für die leere Klausel

- Die **Klauselmenge** $\emptyset$ wird andererseits durch **jede** Belegung erfüllt
  - Denn: für jede Belegung α gilt, dass jede Klausel $K \in \emptyset$ durch α wahr gemacht wird
  - Insofern entspricht die leere Klauselmenge der Formel $\top$

Aussagenlogische Resolutionsbeweise (1/3)

- **Ziel:** Ein einfacher Weg um für eine Klauselmenge $\mathcal{K}$ weitere Klauseln K zu finden, sodass:

  Wenn $\mathcal{K}$ erfüllbar ist, dann auch $\mathcal{K} \cup \{K\}$
- **Ansatz:** Bilde K aus zwei geeigneten $K_1, K_2 \in \mathcal{K}$

- **Erinnerung:** Ein **Literal** ist eine Variable X oder eine negierte Variable $\neg X$
- Wir definieren $\underline{\neg L}$ für Literale L wie folgt:

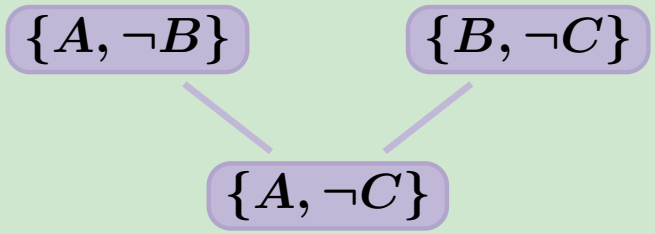
$$\neg L \stackrel{\text{def}}{=} \begin{cases} X & \text{falls } L = \neg X \\ \neg X & \text{falls } L = X \end{cases}$$

Definition: Resolvente

- Seien K_1, K_2 zwei Klauseln
- Eine Klausel K heißt **Resolvente** von K_1 und K_2 falls es ein Literal L gibt, sodass
  - $L \in K_1$ und $\neg L \in K_2$ und
  - $K = (K_1 \setminus \{L\}) \cup (K_2 \setminus \{\neg L\})$

Beispiel

- $\{A, \neg B\}$ und $\{B, \neg C\}$ können zu $\{A, \neg C\}$ resolviert werden
- Visualisierung:



- $\{A, B\}$ und $\{\neg A, \neg B\}$ können nicht zu $\square$ resolviert werden

  ➡ Das lässt die Definition von Resolventen nicht zu!

- Wir sagen auch: Die Resolvente K entsteht aus K_1 und K_2 in einem **Resolutionsschritt**
- Eine Menge von Resolutionsschritten nennen wir **Resolutionsbeweis**

Aussagenlogische Resolutionsbeweise (2/3)

- **Wir werden gleich sehen:** Für eine Klauselmenge $\mathcal{K}$ und eine Resolvente K zweier Klauseln aus $\mathcal{K}$ gilt:

  Wenn $\mathcal{K}$ erfüllbar ist,
  dann ist auch $\mathcal{K} \cup \{K\}$ erfüllbar

  ☞ Resolutionslemma

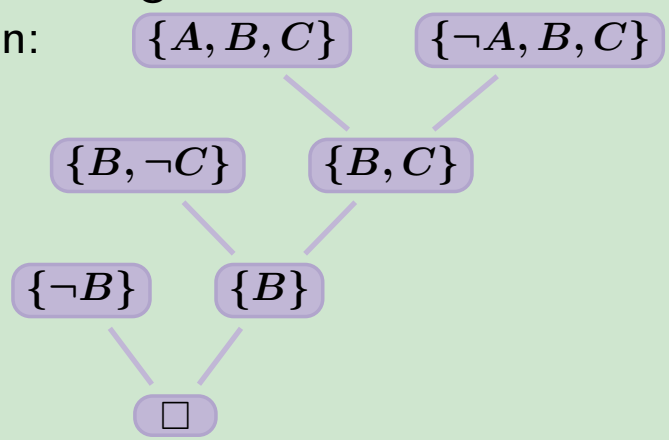
- **Zunächst:** Was bringt uns das?

Beispiel

- Aus der Klauselmenge

$$\mathcal{K} = \{\{A, B, C\}, \{\neg A, B, C\}, \{B, \neg C\}, \{\neg B\}\}$$

lässt sich folgender Resolutionsbeweis gewinnen:



- Es folgt: Die Klauselmenge $\mathcal{K}$ ist unerfüllbar
- **Denn:**
  - **Annahme:** $\mathcal{K}$ ist erfüllbar
  - Dann: Auch ihre Resolventen sind erfüllbar
  - Also: auch $\square$ ist erfüllbar

    ➡ **Widerspruch!**

➡ Mit Resolutionsbeweisen lässt sich also die Unerfüllbarkeit von Klauselmengen nachweisen

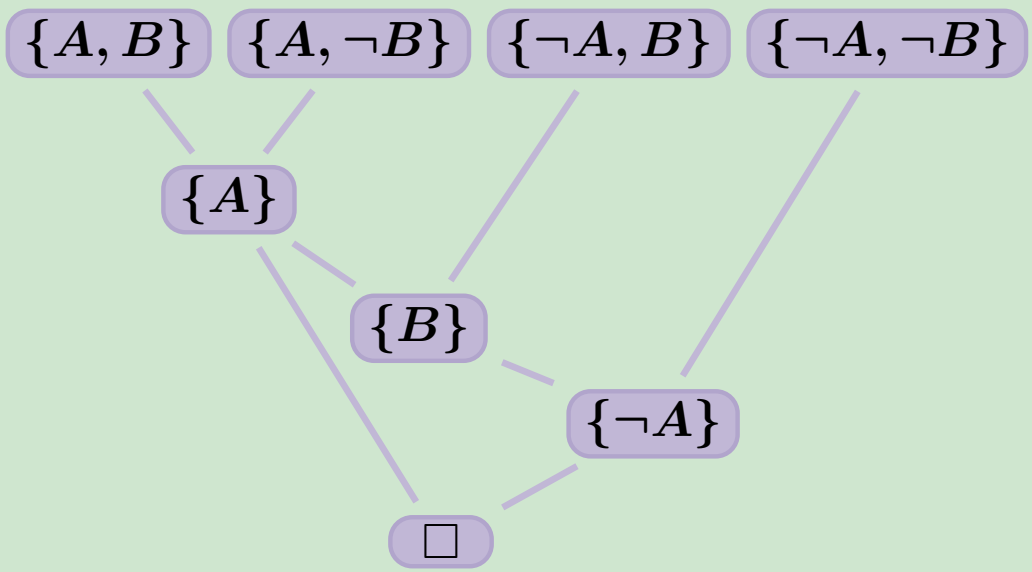
- Als nächstes:

  (1) Noch ein Beispiel
  (2) Beweis des Resolutionslemmas
  (3) Korrektheit und Vollständigkeit von Resolutionsbeweisen

Aussagenlogische Resolutionsbeweise (3/3)

- **Bemerkung:** Klauseln dürfen in Resolutionsbeweisen mehrfach zur Bildung von Resolventen verwendet werden
  - ➡ Sowohl gegebene Klauseln als auch durch Resolutionsschritte entstandene Klauseln
- **Insbesondere:** Resolutionsbeweise müssen keine baumartige Struktur haben

Beispiel



➡ Auch diese Klauselmenge ist unerfüllbar

Das aussagenlogische Resolutionslemma

Lemma A.3.2: Resolutionslemma

- Seien K_1 und K_2 Klauseln aus einer Klauselmenge $\mathcal{K}$
- Sei K eine Resolvente aus K_1 und K_2

- Dann gilt für alle Wahrheitsbelegungen α:

  Wenn $\alpha \models K_1$ und $\alpha \models K_2$
  dann $\alpha \models K$

- Insbesondere gilt: $\mathcal{K}$ ist genau dann erfüllbar, wenn $\mathcal{K} \cup \{K\}$ erfüllbar ist

Beweis

- Seien:
  - $K_1 \stackrel{\text{def}}{=} \{L_1, \ldots, L_k, X\}$
  - $K_2 \stackrel{\text{def}}{=} \{L'_1, \ldots, L'_\ell, \neg X\}$
  - $K \stackrel{\text{def}}{=} \{L_1, \ldots, L_k, L'_1, \ldots, L'_\ell\}$

- Sei α eine Belegung, die K_1 und K_2 wahr macht

- **Zu zeigen:** $\alpha \models K$

- Wir unterscheiden zwei Fälle:

- **1. Fall:** $\alpha(X) = 1$
  - Wegen $\alpha \models K_2$ gilt dann: $\alpha \models \{L'_1, \ldots, L'_\ell\}$
  - Also auch: $\alpha \models K$

- **2. Fall:** $\alpha(X) = 0$
  - Wegen $\alpha \models K_1$ gilt dann: $\alpha \models \{L_1, \ldots, L_k\}$
  - Also auch: $\alpha \models K$

Der aussagenlogische Resolutionssatz

- Wir wollen für Klauselmengen $\mathcal{K}$ zeigen:
  - Wenn sich die leere Klausel $\square$ herleiten lässt, dann ist $\mathcal{K}$ unerfüllbar
  - Wenn sich die leere Klausel $\square$ nicht herleiten lässt, dann ist $\mathcal{K}$ erfüllbar

- Für den Nachweis hilfreich:

  Die Menge $\mathbf{Res}^\infty(\mathcal{K})$ aller Klauseln, die sich aus $\mathcal{K}$ durch Anwendung von Resolutionsschritten erreichen lassen

- Dazu definieren wir:
  - $\underline{\mathbf{Res}(\mathcal{K})} \stackrel{\text{def}}{=} \mathcal{K} \cup \{K \mid K \text{ ist Resolvente zweier Klauseln aus } \mathcal{K}\}$
  - $\underline{\mathbf{Res}^0(\mathcal{K})} \stackrel{\text{def}}{=} \mathcal{K}$
  - $\underline{\mathbf{Res}^{\ell+1}(\mathcal{K})} \stackrel{\text{def}}{=} \mathbf{Res}(\mathbf{Res}^\ell(\mathcal{K}))$, für jedes $\ell \geq 0$
  - $\underline{\mathbf{Res}^\infty(\mathcal{K})} \stackrel{\text{def}}{=} \bigcup_{\ell \geq 0} \mathbf{Res}^\ell(\mathcal{K})$

Satz A.3.3: Resolutionssatz

- Eine endliche Klauselmenge $\mathcal{K}$ ist genau dann unerfüllbar, wenn $\square \in \mathbf{Res}^\infty(\mathcal{K})$

Beweisidee

- Wir zeigen:

  (1) Wenn $\square \in \mathbf{Res}^\infty(\mathcal{K})$, dann ist $\mathcal{K}$ unerfüllbar

  (2) Wenn $\mathcal{K}$ unerfüllbar, dann $\square \in \mathbf{Res}^\infty(\mathcal{K})$

- Beweisdetails: Im Anhang

Ein Erfüllbarkeitstest für AL-Formeln

- Aus dem Resolutionssatz ergibt sich der folgende Algorithmus zum Testen der Erfüllbarkeit aussagenlogischer Formeln

Resolutionsalgorithmus

Eingabe: AL-Formel φ in KNF
Ausgabe: „erfüllbar“, falls φ erfüllbar ist, sonst „unerfüllbar“

1: $\mathcal{K} := \mathcal{K}(\varphi)$
2: **repeat**
3: $\quad \mathcal{K}' := \mathcal{K}$
4: $\quad \mathcal{K} := \mathbf{Res}(\mathcal{K}')$
5: **until** $\mathcal{K} = \mathcal{K}'$ oder $\square \in \mathcal{K}$
6: **if** $\square \in \mathcal{K}$ **then**
7: $\quad$ Ausgabe „unerfüllbar“
8: **else**
9: $\quad$ Ausgabe „erfüllbar“
10: **end if**

- Die Korrektheit dieses Algorithmus folgt aus dem Resolutionssatz

- Der Algorithmus terminiert immer, da die Größe der Menge $\mathcal{K}$ begrenzt ist:
  - Aus n Variablen von φ lassen sich $\leq 4^n$ Klauseln bilden
  - Also: $|\mathcal{K}| \leq 4^n$

- Wie aufwändig ist der Resolutionsalgorithmus?
  - Obere Schranke: maximal 4^n Schleifendurchläufe
  - Untere Schranke: es gibt (Familien von) Formeln, für die bei n Variablen $2^{\Omega(n)}$ viele Schleifendurchläufe nötig sind

Beispiel-Berechnung für den Resolutionsalgorithmus

Beispiel

- Eingabe: $\varphi = (A \vee B \vee C) \wedge \neg B \wedge (B \vee \neg A \vee C) \wedge (B \vee \neg C)$

- $\mathcal{K} = \{\{A, B, C\}, \{\neg A, B, C\}, \{B, \neg C\}, \{\neg B\}\}$
- $\mathbf{Res}^1(\mathcal{K}) = \mathcal{K} \cup \{\{B, C\}, \{\neg C\}, \{A, C\}, \{\neg A, C\}, \{A, B\}, \{\neg A, B\}\}$
- $\mathbf{Res}^2(\mathcal{K}) = \mathbf{Res}^1(\mathcal{K}) \cup \{\{A\}, \{\neg A\}, \{B\}, \{C\}\}$
- $\mathbf{Res}^3(\mathcal{K}) = \mathbf{Res}^2(\mathcal{K}) \cup \{\square\}$

$\Rightarrow \varphi$ ist unerfüllbar

Erfüllbarkeitsäquivalente Umformungen

Erfüllbarkeitsäquivalente Formeln (1/2)

- **Bislang:** Für den Erfüllbarkeitstest werden Formeln φ in eine **äquivalente** Formel φ' in KNF umgeformt
- **Beobachtung:** Um zu testen, ob φ erfüllbar ist, genügt

  Berechne eine Formel φ' in KNF, die genau dann erfüllbar ist, wenn φ erfüllbar ist
- Das führt zu folgender Definition:

Definition:
Erfüllbarkeitsäquivalenz

- Zwei Formeln φ und ψ heißen **erfüllbarkeitsäquivalent**, wenn gilt:
  - φ und ψ sind beide erfüllbar, oder
  - φ und ψ sind beide nicht erfüllbar

- Wir betrachten zwei Situationen, in denen das nützlich ist:
  - Umwandlung von Formeln in erfüllbarkeitsäquivalente Horn-Formeln
  - Umwandlung in kurze erfüllbarkeitsäquivalente KNF-Formeln

Beispiel

- Die KNF-Formel

$$\varphi = (\neg A \vee B) \wedge (B \vee C) \wedge (\neg B \vee \neg C)$$

ist keine Horn-Formel

- Durch Ersetzen von C durch $\neg C$ und $\neg C$ durch C, kann sie aber in eine erfüllbarkeitsäquivalente Formel umgewandelt werden:

$$\varphi' = (\neg A \vee B) \wedge (B \vee \neg C) \wedge (\neg B \vee C)$$

- Die Erfüllbarkeit von φ' (und von φ) kann dann mit dem Markierungsalgorithmus getestet werden

Erfüllbarkeitsäquivalente Formeln (2/2)

- **Erinnerung:** Bei der Umwandlung in KNF kann eine exponentiell lange Formel entstehen ☞ Parity
- Es kann aber immer leicht eine erfüllbarkeitsäquivalente Formel in KNF berechnet werden, die nur wenig länger als die Ausgangsformel ist

Satz A.3.4

- Es gibt einen Algorithmus, der aussagenlogische Formeln φ in linearer Zeit in linear große erfüllbarkeitsäquivalente Formeln φ' in KNF umwandelt

Beweisidee

- **Ansatz:** Verwende für jede Teilformel ψ von φ eine zusätzliche Variable, die genau dann wahr werden soll, wenn ψ wahr wird

Beispiel

- Sei $\varphi = A_1 \vee (\neg A_2 \wedge (\neg A_3 \vee A_4))$
- Wir verwenden neue Variablen:
  - B_1 für $(\neg A_3 \vee A_4)$
  - B_2 für $(\neg A_2 \wedge (\neg A_3 \vee A_4))$
  - B_3 für φ

- Es ergibt sich die zu φ erfüllbarkeitsäquivalente Formel

$$\begin{aligned}&(B_1 \leftrightarrow (\neg A_3 \vee A_4))\\ &\wedge\ (B_2 \leftrightarrow (\neg A_2 \wedge B_1))\\ &\wedge\ (B_3 \leftrightarrow (A_1 \vee B_2))\\ &\wedge\ B_3\end{aligned}$$

  die sich durch Auflösen der Äquivalenzen leicht (äquivalent!) in KNF bringen lässt

Zusammenfassung und Ausblick

- **In diesem Vorlesungskapitel haben wir Erfüllbarkeit untersucht:**

- Einige eingeschränkte Klassen aussagenlogischer Formeln lassen sich effizient auf Erfüllbarkeit testen:
  - Horn-Formeln mit dem Markierungsalgorithmus
    ☞ das haben wir gesehen
  - 2-SAT-Formeln ☞ hier nicht gesehen

- Aussagenlogische Resolution liefert einen syntaktischen Unerfüllbarkeitstest
  - Der **Resolutionsalgorithmus**
    - löst das Erfüllbarkeitsproblem für aussagenlogische Formeln und
    - geht dabei weniger naiv vor als die Wahrheitstabellen-Methode
  - **Aber:** Auch der Resolutionsalgorithmus hat im schlimmsten Fall exponentielle Laufzeit

- **Ein kleiner Ausblick:**

- Für das allgemeine ausagenlogische Erfüllbarkeitsproblem ist kein effizienter Algorithmus bekannt!
- Ob es für SAT einen Algorithmus mit polynomieller Laufzeit gibt, ist äquivalent zum berühmtesten offenen Problem der Informatik: **dem P-NP-Problem**
  - Wenn es für SAT einen polynomiellen Algorithmus gibt, ist $\mathrm{P} = \mathrm{NP}$
  - Wenn es für SAT keinen polynomiellen Algorithmus gibt, ist $\mathrm{P} \neq \mathrm{NP}$

➡ Näheres hierzu in der Vorlesung *Theoretische Informatik*

- In der Praxis lässt sich SAT mittlerweile sehr gut lösen
  ➡ Mehr dazu in Kapitel A.5 *Praktisches SAT-Solving*

Anhang: Vollständigkeit des Resolutionskalküls

Der Resolutionssatz der Aussagenlogik: Beweis (1/2)

Beweisidee (Fortsetzung)

- Wir zeigen zuerst die **Korrektheit des Resolutionskalküls**:

  wenn $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$, dann ist $\mathcal{K}$ unerfüllbar

- Wir nehmen also an: $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$

  $\Rightarrow$ Es gibt ein ℓ mit $\square \in \mathbf{Res}^{\ell}(\mathcal{K})$

  $\Rightarrow$ Da keine Belegung die Klausel $\square$ wahr machen kann, ist $\mathbf{Res}^{\ell}(\mathcal{K})$ unerfüllbar

- Aus dem Resolutionslemma folgt:
  - Für jede Klauselmenge $\mathcal{K}'$ gilt:

    $\mathcal{K}'$ erfüllbar $\Rightarrow$ $\mathbf{Res}(\mathcal{K}')$ erfüllbar
  - Kontraposition:

    $\mathbf{Res}(\mathcal{K}')$ unerfüllbar $\Rightarrow$ $\mathcal{K}'$ unerfüllbar

- Also: $\mathbf{Res}^{\ell}(\mathcal{K})$ unerfüllbar

  $\Rightarrow$ $\mathbf{Res}^{\ell-1}(\mathcal{K})$ unerfüllbar

  $\Rightarrow$ ...

  $\Rightarrow$ $\mathbf{Res}^{0}(\mathcal{K})$ unerfüllbar

  $\Rightarrow$ $\mathcal{K}$ unerfüllbar

Der Resolutionssatz der Aussagenlogik: Beweis (2/2)

Beweisidee (Fortsetzung)

- Wir geben jetzt die Beweisidee für die **Vollständigkeit des Resolutionskalküls**:

  wenn $\mathcal{K}$ unerfüllbar ist,
  dann ist $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$

- Der Beweis ist eine Induktion nach der Anzahl n der Variablen in $\mathcal{K}$

- $n = 0$:
  - Da $\mathcal{K}$ keine Variablen enthält, aber unerfüllbar ist, muss $\mathcal{K}$ die leere Klausel $\square$ enthalten

- Der Induktionsschritt ($n-1 \rightarrow n$) konstruiert aus einer unerfüllbaren Klauselmenge $\mathcal{K}$ mit Variablen $\{A_1, \ldots, A_n\}$ zwei Klauselmengen $\mathcal{K}_1$ und $\mathcal{K}_2$ **ohne** die Variable A_n
  - $\mathcal{K}_1$ entsteht aus $\mathcal{K}$ durch Belegung von A_n mit 1
  - $\mathcal{K}_2$ entsteht aus $\mathcal{K}$ durch Belegung von A_n mit 0

- Dann wird wie folgt argumentiert:
  1. $\mathcal{K}_1$ und $\mathcal{K}_2$ sind beide unerfüllbar ☞ klar
  2. Es gibt Resolutionsbeweise für die Unerfüllbarkeit von $\mathcal{K}_1$ und $\mathcal{K}_2$ ☞ Induktionsvoraussetzung
  3. Ein Resolutionsbeweis für die Unerfüllbarkeit von $\mathcal{K}$ lässt sich aus diesen Resolutionsbeweisen gewinnen

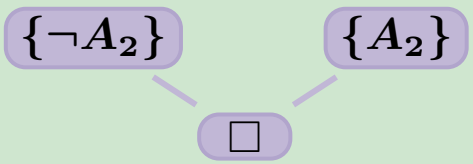
Vollständigkeit des Resolutionskalküls: Illustration

Beispiel

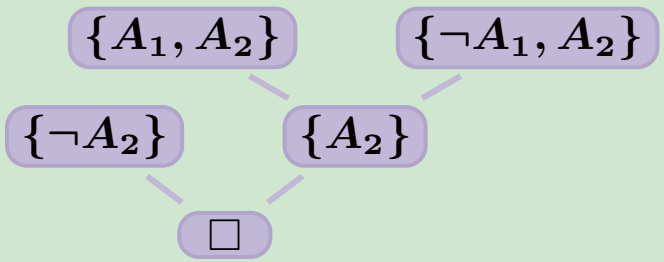
- $\mathcal{K} = \{\{A_1, A_2, A_3\}, \{\neg A_1, A_2, A_3\}, \{A_2, \neg A_3\}, \{\neg A_2\}\}$

- Sei $\mathcal{K}_1$ die Menge der Klauseln, die sich aus $\mathcal{K}$ durch $A_3 \mapsto 1$ ergeben:
  $\mathcal{K}_1 = \{\{A_2\}, \{\neg A_2\}\}$
- Sei $\mathcal{K}_2$ analog definiert mit $A_3 \mapsto 0$:
  $\mathcal{K}_2 = \{\{A_1, A_2\}, \{\neg A_1, A_2\}, \{\neg A_2\}\}$

- Ein Resolutionsbeweis zu $\mathcal{K}_1$:

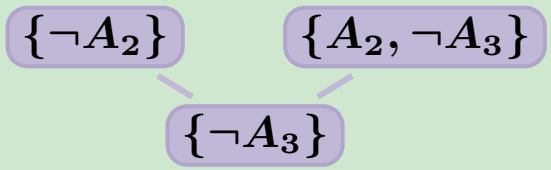


- Ein Resolutionsbeweis zu $\mathcal{K}_2$:

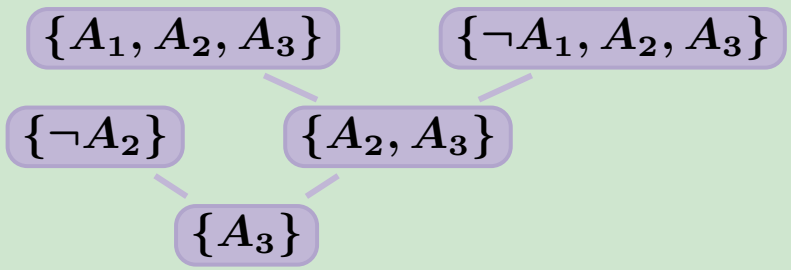


- Also: $\square \in \mathbf{Res}^1(\mathcal{K}_1)$, $\square \in \mathbf{Res}^2(\mathcal{K}_2)$

- Aus dem Beweis für $\square \in \mathbf{Res}^1(\mathcal{K}_1)$ lässt sich ein Beweis für $\{\neg A_3\} \in \mathbf{Res}^1(\mathcal{K})$ gewinnen:



- Aus dem Beweis für $\square \in \mathbf{Res}^2(\mathcal{K}_2)$ lässt sich ein Beweis für $\{A_3\} \in \mathbf{Res}^2(\mathcal{K})$ gewinnen:



- Also: $\{\neg A_3\}, \{A_3\} \in \mathbf{Res}^2(\mathcal{K})$
  $\Rightarrow \square \in \mathbf{Res}^3(\mathcal{K})$

Vollständigkeit des Resolutionskalküls: Beweisdetails (1/2)

Beweis (Fortsetzung)

- Wir zeigen die **Vollständigkeit des Resolutionskalküls**:
  - wenn $\mathcal{K}$ unerfüllbar
    dann $\square \in \mathbf{Res}^\infty(\mathcal{K})$
- Der Beweis ist eine Induktion nach der Anzahl n der Variablen in $\mathcal{K}$
- $n = 0$:
  - Da $\mathcal{K}$ keine Variablen enthält, aber unerfüllbar ist, muss $\mathcal{K}$ die leere Klausel $\square$ enthalten
- $n-1 \to n$:
  - Sei also $\mathcal{K}$ unerfüllbar mit Variablen $\{A_1, \ldots, A_n\}$
  - Sei $\mathcal{K}_1$ die Klauselmenge, die aus $\mathcal{K}$ entsteht, indem
    - Klauseln, in denen A_n (positiv) vorkommt, gestrichen werden, und
    - $\neg A_n$ aus allen Klauseln entfernt wird
  - Sei dual dazu $\mathcal{K}_2$ die Klauselmenge, die aus $\mathcal{K}$ entsteht, indem
    - Klauseln, in denen $\neg A_n$ vorkommt, gestrichen werden, und
    - A_n aus allen Klauseln entfernt wird
  - **Behauptung**: $\mathcal{K}_1$ und $\mathcal{K}_2$ sind unerfüllbar
    - Denn: andernfalls könnte eine erfüllende Belegung α_1 von $\mathcal{K}_1$ durch $\alpha_1(A_n) \stackrel{\text{def}}{=} 1$ oder eine erfüllende Belegung α_2 von $\mathcal{K}_2$ durch $\alpha_2(A_n) \stackrel{\text{def}}{=} 0$ zu einer erfüllenden Belegung von $\mathcal{K}$ erweitert werden
  - Mit Induktion folgt: $\square \in \mathbf{Res}^\infty(\mathcal{K}_1)$ und $\square \in \mathbf{Res}^\infty(\mathcal{K}_2)$

Vollständigkeit des Resolutionskalküls: Beweisdetails (2/2)

Beweis **(Fortsetzung)**

- Wir wissen also: $\square \in \mathbf{Res}^{\infty}(\mathcal{K}_1)$ und $\square \in \mathbf{Res}^{\infty}(\mathcal{K}_2)$
- Es gibt deshalb ein k, so dass gilt:

  $\square \in \mathbf{Res}^{k}(\mathcal{K}_1)$ und $\square \in \mathbf{Res}^{k}(\mathcal{K}_2)$
- **Behauptung:** $\square \in \mathbf{Res}^{k}(\mathcal{K})$ oder $\{\neg A_n\} \in \mathbf{Res}^{k}(\mathcal{K})$
  - Denn: für jede Klausel K aus $\mathcal{K}_1$ gilt:
    - K kommt in $\mathcal{K}$ vor oder
    - $K \cup \{\neg A_n\}$ kommt in $\mathcal{K}$ vor
  - Daraus folgt, dass aus jedem Resolutionsbeweis für $\mathcal{K}_1$ ein Resolutionsbeweis für $\mathcal{K}$ gewonnen werden kann, indem, wenn nötig, Klauseln um $\neg A_n$ ergänzt werden (siehe Beispiel)
  - Insbesondere wird aus dem Resolutionsbeweis für $\square$ dann ein Resolutionsbeweis für $\square$ oder $\{\neg A_n\}$

- Analog lässt sich mit Hilfe von $\mathcal{K}_2$ zeigen:
  $\square \in \mathbf{Res}^{k}(\mathcal{K})$ oder $\{A_n\} \in \mathbf{Res}^{k}(\mathcal{K})$
- Also gilt insgesamt:
  - $\square \in \mathbf{Res}^{k}(\mathcal{K})$ oder
  - $\{A_n\} \in \mathbf{Res}^{k}(\mathcal{K})$ und $\{\neg A_n\} \in \mathbf{Res}^{k}(\mathcal{K})$
- In jedem Fall folgt:

  $\square \in \mathbf{Res}^{k+1}(\mathcal{K})$

Kapitel A.4:

Vollständigkeits- und Endlichkeitssatz

Inhalt

1. Erfüllbarkeit, Allgemeingültigkeit, Folgerungen
2. Semantisches Folgern und syntaktisches Beweisen
3. Der Endlichkeitssatz
4. Anhang: Der Beweis des Endlichkeitssatzes

Nutzen von Erfüllbarkeitstests

- **Im letzten Kapitel:** Ein zuverlässiger Erfüllbarkeitstest für aussagenlogische Formeln:

  ➡ **Der Resolutionsalgorithmus**

- Er entscheidet für eine gegebene aussagenlogische Formel φ (in KNF), ob φ erfüllbar ist

- **In diesem Kapitel:**

- Viele Fragestellungen lassen sich auf die Erfüllbarkeit einer aussagenlogischen Formel zurückführen, z.B.:
  - ob eine endliche Menge von Formeln erfüllbar ist,
  - ob eine gegebene Formel allgemein gültig ist, und
  - ob eine gegebene Formel aus einer endlichen Menge von Formeln folgt

- Ein fundamentaler Zusammenhang zwischen semantischem Folgern und syntaktischem Beweisen

  ☞ **Der Vollständigkeitssatz**

- Die Erfüllbarkeit unendlicher Formelmengen lässt sich auf die Erfüllbarkeit endlicher Formelmengen zurückführen

  ☞ **Der Endlichkeitssatz**

Erfüllbarkeit, Allgemeingültigkeit, Folgerungen

Erfüllbarkeit endlicher Formelmengen (1/5)

- Klar: eine endliche Menge $\{\varphi_1, \ldots, \varphi_n\}$ von Formeln ist genau dann erfüllbar, wenn $\varphi_1 \wedge \cdots \wedge \varphi_n$ erfüllbar ist
- Wir betrachten dazu jetzt ein umfangreicheres Beispiel

Beispiel: Eine ehemalige Übungsaufgabe

- Es sollen n Werte $a_1, \ldots, a_n$ in m Speicherstellen $s_1, \ldots, s_m$ abgelegt werden, wobei n und m natürliche Zahlen sind
- Dabei kann in jeder Speicherstelle jeweils höchstens ein Wert gespeichert werden

(a) Modelliere die beschriebene Situation durch eine Menge $\Phi_{n,m}$ aussagenlogischer Formeln

(b) Zeige mittels Resolution, dass es unmöglich ist, drei Datenwerte in zwei Speicherstellen abzulegen

Erfüllbarkeit endlicher Formelmengen (2/5)

Allgemeiner Lösungsansatz

Variablen für atomare Aussagen:

- A_{ij} für $i \in \{1, \ldots, n\}$ und $j \in \{1, \ldots, m\}$:

  „a_i wird in Speicherstelle s_j abgelegt“

Formeln zur Modellierung der Situation:

- „Es sollen n Werte $a_1, \ldots, a_n$ abgelegt werden“
- In anderen Worten: Jedes a_i wird in mindestens einer Speicherstelle abgelegt
- Also:

$$\varphi_i^m \stackrel{\text{def}}{=} \bigvee_{j=1}^{m} A_{ij}$$

für jedes $i \in \{1, \ldots, n\}$

- „Dabei kann in jeder Speicherstelle jeweils höchstens ein Wert abgespeichert werden“
- In anderen Worten: In jeder Speicherstelle j wird höchstens ein Wert abgelegt
- Also:

$$\psi_j^n \stackrel{\text{def}}{=} \bigwedge_{\substack{i_1, i_2 \in \{1, \ldots, n\} \\ i_1 \neq i_2}} \neg(A_{i_1 j} \wedge A_{i_2 j})$$

für jedes $j \in \{1, \ldots, m\}$

- Für (a) ergibt sich insgesamt die Formelmenge

$$\Phi_{n,m} = \{\varphi_i^m \mid i \leq n\} \cup \{\psi_j^n \mid j \leq m\}$$

Erfüllbarkeit endlicher Formelmengen (3/5)

Lösungsansatz für 3 Werte und 2 Speicherstellen

(b) Zeige mittels Resolution, dass es unmöglich ist, drei Datenwerte in zwei Speicherstellen abzulegen

- **Also:** Wir müssen die Unerfüllbarkeit von $\Phi_{3,2}$ nachweisen:

$$\begin{aligned}\Phi_{3,2} =& \{A_{11} \vee A_{12}, A_{21} \vee A_{22}, A_{31} \vee A_{32}\} \\ & \cup \{\neg(A_{1,1} \wedge A_{2,1}), \neg(A_{1,1} \wedge A_{3,1}), \neg(A_{2,1} \wedge A_{3,1}), \\ & \quad \neg(A_{1,2} \wedge A_{2,2}), \neg(A_{1,2} \wedge A_{3,2}), \neg(A_{2,2} \wedge A_{3,2})\}\end{aligned}$$

- $\Phi_{3,2}$ ist genau dann unerfüllbar, wenn

$$\begin{aligned}\chi_{3,2} \stackrel{\text{def}}{=} & (A_{11} \vee A_{12}) \wedge (A_{21} \vee A_{22}) \wedge (A_{31} \vee A_{32}) \\ & \wedge \neg(A_{1,1} \wedge A_{2,1}) \wedge \neg(A_{1,1} \wedge A_{3,1}) \wedge \neg(A_{2,1} \wedge A_{3,1}) \\ & \wedge \neg(A_{1,2} \wedge A_{2,2}) \wedge \neg(A_{1,2} \wedge A_{3,2}) \wedge \neg(A_{2,2} \wedge A_{3,2})\end{aligned}$$

unerfüllbar ist

- Diese Formel entspricht der Klauselmenge

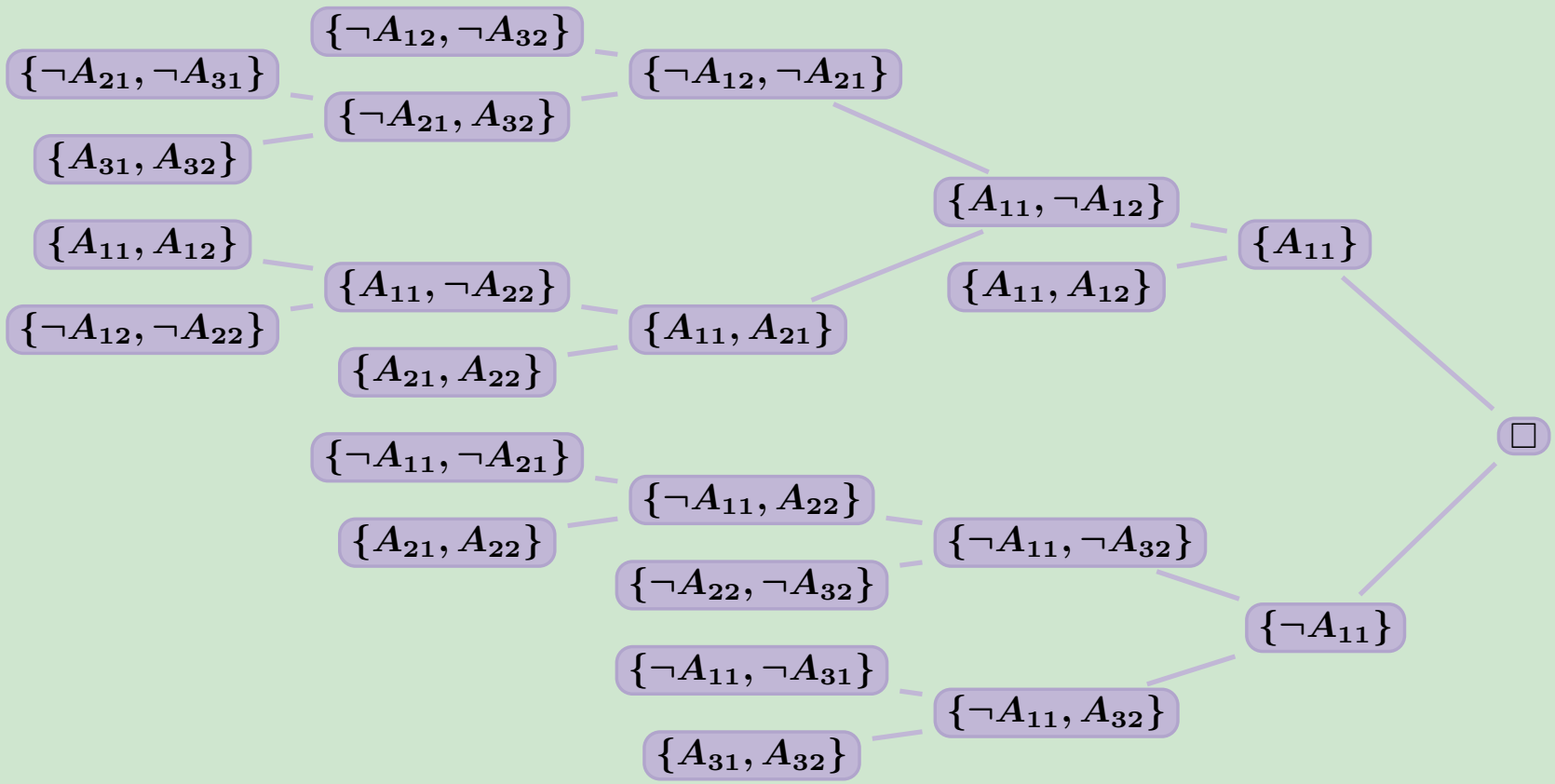
$$\begin{aligned}\mathcal{K}_{3,2} \stackrel{\text{def}}{=} & \{\{A_{11}, A_{12}\}, \{A_{21}, A_{22}\}, \{A_{31}, A_{32}\}, \\ & \{\neg A_{11}, \neg A_{21}\}, \{\neg A_{11}, \neg A_{31}\}, \{\neg A_{21}, \neg A_{31}\}, \\ & \{\neg A_{12}, \neg A_{22}\}, \{\neg A_{12}, \neg A_{32}\}, \{\neg A_{22}, \neg A_{32}\}\}\end{aligned}$$

Erfüllbarkeit endlicher Formelmengen (4/5)

Lösung durch Resolution

- Wir müssen also die Unerfüllbarkeit der Klauselmenge $\mathcal{K}_{3,2}$ zeigen:

$$\{\{A_{11}, A_{12}\}, \{A_{21}, A_{22}\}, \{A_{31}, A_{32}\},$$
$$\{\neg A_{11}, \neg A_{21}\}, \{\neg A_{11}, \neg A_{31}\}, \{\neg A_{21}, \neg A_{31}\},$$
$$\{\neg A_{12}, \neg A_{22}\}, \{\neg A_{12}, \neg A_{32}\}, \{\neg A_{22}, \neg A_{32}\}\}$$



Erfüllbarkeit endlicher Formelmengen (5/5)

Antwort

- Da $\square$ herleitbar ist, ist $\boldsymbol{\chi_{3,2}}$ unerfüllbar
- Also ist auch $\boldsymbol{\Phi_{3,2}}$ unerfüllbar
- Also können drei Werte nicht in zwei Speicherstellen gespeichert werden

☞ Keine allzu große Überraschung...

Allgemeingültige Formeln (1/2)

- **Erinnerung:** Eine Formel φ ist allgemeingültig, wenn $[\![\varphi]\!]_\alpha = 1$ für jede passende Belegung α
- Erfüllbarkeit und Allgemeingültigkeit hängen eng zusammen

Beobachtung

- Eine Formel φ ist genau dann allgemeingültig, wenn $\neg\varphi$ unerfüllbar ist

Beweis

- Wir zeigen zuerst: φ ist allgemeingültig $\Rightarrow$ $\neg\varphi$ unerfüllbar

  Sei φ allgemeingültig

  $\Rightarrow$ Für jede passende Belegung α ist $[\![\varphi]\!]_\alpha = 1$ ☞ Def. von allgemeingültig

  $\Rightarrow$ Für jede passende Belegung α ist $[\![\neg\varphi]\!]_\alpha = 0$ ☞ Def. von $[\![\neg\varphi]\!]_\alpha$

  $\Rightarrow$ Kein α ist Modell von $\neg\varphi$ ☞ Def. Modell

  $\Rightarrow$ $\neg\varphi$ ist unerfüllbar ☞ Def. erfüllbar

- Jetzt zeigen wir: $\neg\varphi$ unerfüllbar $\Rightarrow$ φ allgemeingültig

  Sei $\neg\varphi$ unerfüllbar

  $\Rightarrow$ $\neg\varphi$ hat kein Modell ☞ Def. unerfüllbar

  $\Rightarrow$ Für alle passenden α ist $[\![\neg\varphi]\!]_\alpha = 0$ ☞ Def. Modell

  $\Rightarrow$ Für alle passenden α ist $[\![\varphi]\!]_\alpha = 1$ ☞ Def. von $[\![\neg\varphi]\!]_\alpha$

  $\Rightarrow$ φ ist allgemeingültig ☞ Def. von allgemeingültig

Allgemeingültige Formeln (2/2)

Beispiel

- **Szenario:** Wir führen einen Beweis in dem eine Fallunterscheidung anhand von Bedingungen A, B, C, D gemacht wird
- Unsere Fallunterscheidung:
  - **1. Fall:** keine der Bedingungen gilt
  - **2. Fall:** D gilt, aber nicht A
  - **3. Fall:** A gilt, aber nicht C
  - **4. Fall:** B gilt, aber nicht D
  - **5. Fall:** C gilt
- **Frage:** Sind alle Fälle berücksichtigt?

- Dies führt zur Frage, ob
  - $\varphi = (\neg A \wedge \neg B \wedge \neg C \wedge \neg D) \vee (D \wedge \neg A) \vee (A \wedge \neg C) \vee (B \wedge \neg D) \vee C$

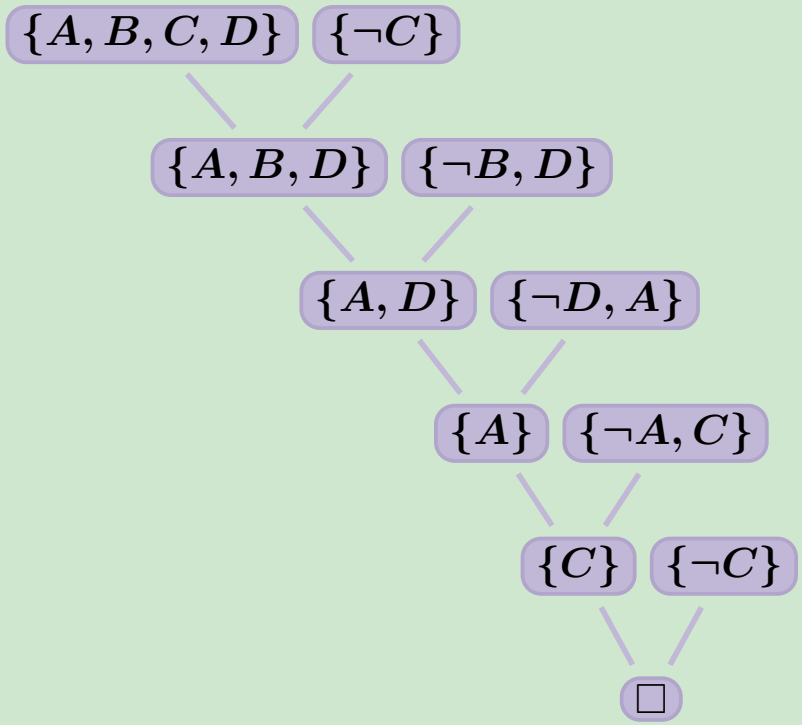
  allgemeingültig ist

- Das ist äquivalent zur Frage, ob

  $\neg\varphi \equiv (A \vee B \vee C \vee D) \wedge (\neg D \vee A) \wedge (\neg A \vee C) \wedge (\neg B \vee D) \wedge \neg C$

  unerfüllbar ist

- Resolutionsbeweis für die Unerfüllbarkeit von $\neg\varphi$:



- □ ist ableitbar, also $\neg\varphi$ unerfüllbar

- Also ist φ allgemeingültig und die Fallunterscheidung erfasst alle möglichen Fälle

Folgerungen (1/3)

- Eine der wichtigsten Anwendungen von Erfüllbarkeitstests ist das Überprüfen von Folgerungen

- **Grundsituation:**
  - **Gegeben:** Eine Menge Φ von Formeln und eine weitere Formel ψ
  - **Frage:** "Folgt" die Formel ψ aus der Formelmenge Φ?

    ➡ Also: Zieht das Gelten von Φ das Gelten von ψ zwangsläufig nach sich?

Beispiel

- Die Formel B ist immer wahr, wenn die Menge $\{A, A \rightarrow B\}$ wahr ist

Definition: Folgerung

- Sei Φ eine Menge von AL-Formeln und ψ eine AL-Formel

- ψ **folgt aus** $\Phi \overset{\text{def}}{\Leftrightarrow}$ jede zu $\Phi \cup \{\psi\}$ passende Belegung, die Modell von Φ ist, ist auch Modell von ψ

- Wir schreiben: $\Phi \models \psi$

Beispiel

- $\{A, A \rightarrow B\} \models B$

✎ Zu beachten: die Menge Φ kann in dieser Definition auch unendlich sein

✎ Wenn $\Phi = \{\varphi\}$ nur aus einer einzigen Formel besteht, schreiben wir auch $\varphi \models \psi$

Folgerungen (2/3)

- Die folgende Proposition formuliert den Zusammenhang zwischen Folgerungen und Erfüllbarkeit bzw. Allgemeingültigkeit

Proposition A.4.1

- Sei Φ eine Menge von AL-Formeln und ψ eine AL-Formel
- Dann gilt:

(a) $\Phi \models \psi \iff \Phi \cup \{\neg\psi\}$ ist unerfüllbar
(b) Falls Φ endlich ist, gilt

$$\Phi \models \psi \iff \Big(\bigwedge_{\varphi\in\Phi} \varphi\Big) \to \psi \text{ ist allgemeingültig}$$

Beweisskizze

- Teil (a) folgt direkt aus der Definition von „$\Phi \models \psi$“
- Teil (b) folgt direkt aus der Semantik der Implikation und der Definition von „$\Phi \models \psi$“

Teil (a) im Detail:

“$\Rightarrow$”:

- Gelte $\Phi \models \psi$
- Angenommen, $\Phi \cup \{\neg\psi\}$ ist erfüllbar
  - Dann gäbe es eine Belegung α mit $\alpha \models \Phi \cup \{\neg\psi\}$
  - Aber: Dann gilt $\Phi \not\models \psi$ nicht, Widerspruch!
- Also ist $\Phi \cup \{\neg\psi\}$ unerfüllbar

“$\Leftarrow$”:

- Gelte $\Phi \cup \{\neg\psi\}$ ist unerfüllbar
- Sei α eine Belegung mit $\alpha \models \Phi$
- Zu zeigen: $\alpha \models \psi$
  - Es gilt $\alpha \not\models \neg\psi$, da $\Phi \cup \{\neg\psi\}$ unerfüllbar ist
  - Und damit: $\alpha \models \psi$
- Es folgt: $\Phi \models \psi$

Folgerungen (3/3)

- Wir haben jetzt also eine **semantische Folgerungsbeziehung** $\Phi \models \psi$ definiert:
  - Ist Φ eine Formelmenge und ψ eine Formel, so folgt ψ semantisch aus Φ, falls „in allen Situationen, in denen Φ gilt, auch ψ gilt“
  - Im Falle der Aussagenlogik sind die „Situationen“ gerade die passenden Belegungen

- Wie für Erfüllbarkeit gilt auch hier:

  Alle „Situationen“ auszuprobieren kann sehr aufwändig sein

- Deshalb: Syntaktische Methoden zum Schlussfolgern

Semantisches Folgern und syntaktisches Beweisen

Semantisches Folgern und syntaktisches Beweisen

- **Jetzt: Syntaktisches Beweisen**

- $\Phi \vdash \psi$ soll bedeuten, dass ψ aus Φ durch syntaktische Operationen hergeleitet (bewiesen) werden kann
- Die **genaue** Bedeutung von „$\vdash$“ hängt dabei jeweils von einem spezifischen **Beweissystem** ab

- Unsere Mindestanforderung an ein solches Beweissystem ist, dass es **korrekt** ist:

$$\Phi \vdash \psi \Rightarrow \Phi \models \psi$$

- Idealerweise sollte ein Beweissystem auch noch **vollständig** sein:

$$\Phi \models \psi \Rightarrow \Phi \vdash \psi$$

- Resolution liefert ein solches Beweissystem: den Resolutionskalkül

- Wir wissen, dass äquivalent sind:
  - $\Phi \models \psi$
  - $\Phi \cup \{\neg\psi\}$ ist unerfüllbar
- Und: Unerfüllbarkeit kann syntaktisch dem Resolutionskalkül überprüft werden

- Wir definieren deshalb:

$$\underline{\Phi \vdash \psi} \quad \overset{\text{def}}{\Leftrightarrow} \quad \square \in \mathbf{Res}^{\infty}(\mathcal{K}(\Phi \cup \{\neg\psi\}))$$

- Dabei: $\mathcal{K}(\Phi \cup \{\neg\psi\})$ bezeichnet die Menge aller Klauseln, die sich aus den Formeln in $\Phi \cup \{\neg\psi\}$ ergeben, wenn sie in KNF gebracht werden
- Wir sagen: ψ ist aus Φ beweisbar

- Auch hier: Φ darf eine unendliche Menge sein

Vollständigkeitssatz der Aussagenlogik (Endliche Version)

- **Erinnerung:** $\Phi \vdash \psi \overset{\text{def}}{\Leftrightarrow} \square \in \mathbf{Res}^{\infty}(\mathcal{K}(\Phi \cup \{\neg\psi\}))$

- Aus dem Resolutionssatz folgt unmittelbar:

Satz A.4.2: Vollständigkeitssatz der Aussagenlogik (Endliche Version)

- Sei Φ eine endliche Menge aussagenlogischer Formeln
- Sei ψ eine aussagenlogische Formel

- Dann sind äquivalent:
  - $\Phi \models \psi$
  - $\Phi \vdash \psi$

Beweis

- Sei $\Phi = \{\varphi_1, \ldots, \varphi_n\}$
- Wir setzen $\varphi \overset{\text{def}}{=} \varphi_1 \wedge \cdots \wedge \varphi_n$ und erhalten:

$$
\begin{aligned}
\Phi \models \psi &\iff \Phi \cup \{\neg\psi\} \text{ unerfüllbar} && \text{☞ Proposition A.4.1}\\
&\iff \varphi \wedge \neg\psi \text{ unerfüllbar} \\
&\iff \square \in \mathbf{Res}^{\infty}(\mathcal{K}(\varphi \wedge \neg\psi)) && \text{☞ Resolutionssatz}\\
&\iff \square \in \mathbf{Res}^{\infty}(\mathcal{K}(\Phi \cup \{\neg\psi\})) \\
&\iff \Phi \vdash \psi && \text{☞ Definition}
\end{aligned}
$$

Folgern, Beweisen und Erfüllbarkeit

Rezept zum Folgern mit dem Resolutionskalkül

- Mal angenommen, wir fragen uns: Folgt aus den Formeln $A \to B$ und $B \to C$ die Formel $A \to C$?
- Notation:

$$\{A \to B, B \to C\} \stackrel{?}{\models} A \to C$$

- Gemäß Proposition A.4.1 ist das äquivalent zu:

  Ist die Formelmenge $\{A \to B, B \to C, \neg(A \to C)\}$ unerfüllbar?

- Wenn wir mit Hilfe des Resolutionskalküls herausfinden, dass diese Menge unerfüllbar ist, also

$$\{A \to B, B \to C\} \vdash A \to C$$

  dann folgt aus der Korrektheit des Resolutionskalküls

$$\{A \to B, B \to C\} \models A \to C$$

Beispiel: Eine alte Klausuraufgabe (1/2)

Aufgabenstellung

Anton hat gerade seine Logik-Klausur geschrieben. Wie immer diskutiert er im Anschluss mit seinen Kommilitonen über die einzelnen Aufgaben. Dabei gelangt er zu folgenden Überlegungen:

1. Für ein Bestehen der Klausur muss er mindestens eine der drei Aufgaben richtig gelöst haben.
2. Aufgabe 3 konnte er nur dann richtig bearbeiten, wenn er eine der ersten beiden richtig gelöst hat.
3. Er hat die ersten beiden Aufgaben entweder beide richtig oder beide falsch gelöst.

(a) Übersetze Antons Überlegungen in aussagenlogische Formeln.

(b) Zeige mittels aussagenlogischer Resolution, dass Anton nur dann die Klausur besteht, wenn er die erste Aufgabe richtig gelöst hat.

Beispiellösung

(a) Es ergeben sich:

- A_i: Anton hat Aufgabe i richtig gelöst
- B: Anton hat die Klausur bestanden

- $\varphi_1 \stackrel{\text{def}}{=} B \rightarrow (A_1 \vee A_2 \vee A_3)$
- $\varphi_2 \stackrel{\text{def}}{=} A_3 \rightarrow (A_1 \vee A_2)$
- $\varphi_3 \stackrel{\text{def}}{=} (\neg A_1 \vee A_2) \wedge (A_1 \vee \neg A_2)$

(b)

- Die Aussage „Anton besteht nur dann die Klausur, wenn er die erste Aufgabe richtig gelöst hat.“ entspricht der Formel

  $$\psi \stackrel{\text{def}}{=} B \rightarrow A_1$$

- Es muss überprüft werden, ob gilt:

  $$\{\varphi_1, \varphi_2, \varphi_3\} \models \psi$$

- Dazu muss die Unerfüllbarkeit von

  $$\{\neg B \vee A_1 \vee A_2 \vee A_3, \neg A_3 \vee A_1 \vee A_2, (\neg A_1 \vee A_2) \wedge (A_1 \vee \neg A_2), B \wedge \neg A_1\}$$

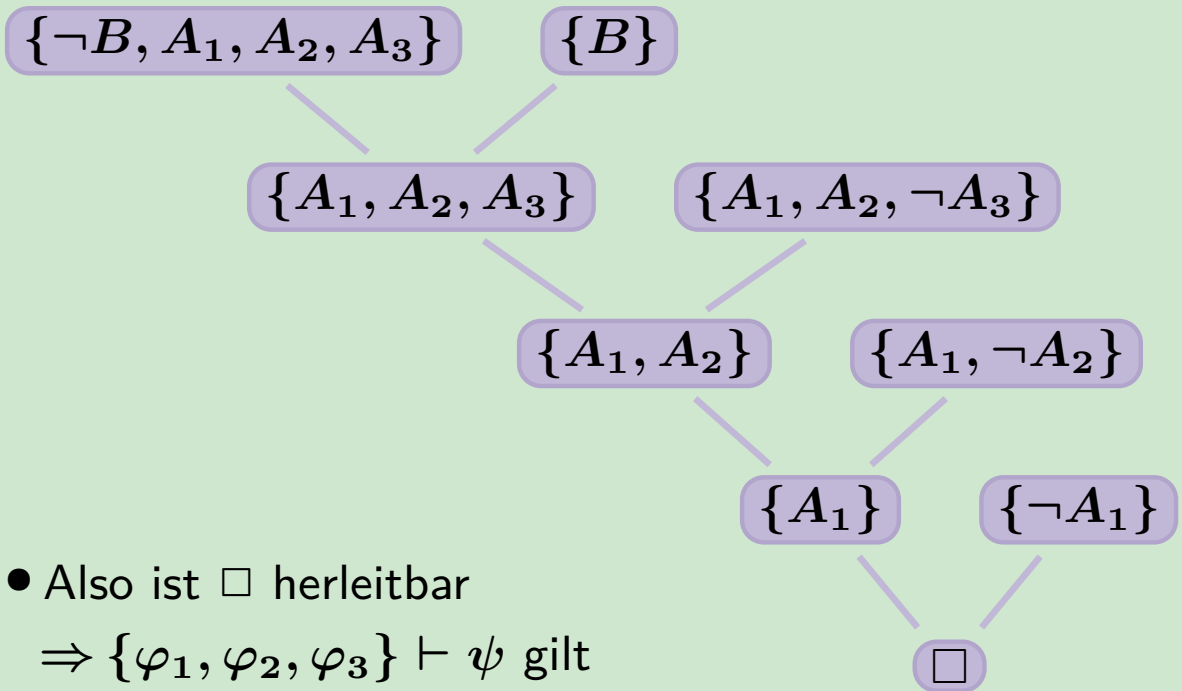
  gezeigt werden

Beispiel: Eine alte Klausuraufgabe (2/2)

Beispiellösung

- Zeige die Unerfüllbarkeit von

$$\{\neg B \vee A_1 \vee A_2 \vee A_3, \neg A_3 \vee A_1 \vee A_2, \\ (\neg A_1 \vee A_2) \wedge (A_1 \vee \neg A_2), B \wedge \neg A_1\}$$



- Also ist $\square$ herleitbar

  $\Rightarrow \{\varphi_1, \varphi_2, \varphi_3\} \vdash \psi$ gilt

  $\Rightarrow \{\varphi_1, \varphi_2, \varphi_3\} \models \psi$ gilt ☞ Vollständigkeitssatz

  $\Rightarrow$ Anton hat nur bestanden, wenn er Aufgabe 1 richtig gelöst hat

Der Endlichkeitssatz

Erfüllbarkeit für unendliche Mengen von AL-Formeln

- Satz A.4.2 stellt einen fundamentalen Zusammenhang her zwischen
  - semantischem Folgern ($\Phi \models \psi$) und
  - syntaktischem Beweisen ($\Phi \vdash \psi$)
- Allerdings ist er nur für endliche Mengen Φ formuliert

- Gilt er auch für beliebige Mengen aussagenlogischer Formeln?

- Wir sehen dazu als nächstes:

  Die Erfüllbarkeit **unendlich vieler** Formeln lässt sich auf die Erfüllbarkeit **endlich vieler** Formeln zurückführen

- **Erinnerung:** Eine Menge V heißt **abzählbar**, wenn es eine Bijektion $\pi : V \to \mathbb{N}$ gibt

Satz A.4.3: Endlichkeitssatz

- Sei V eine abzählbare Menge von aussagenlogischen Variablen
- Für eine unendliche Menge Φ von AL-Formeln über V sind äquivalent

  (i) Φ ist erfüllbar

  (ii) Jede endliche Teilmenge von Φ ist erfüllbar

- Der Endlichkeitssatz ist einer der wichtigsten Sätze der Aussagenlogik
- Er wird oft auch **Kompaktheitssatz** genannt

- **Als nächstes:**

  (1) Eine Anwendung des Endlichkeitssatzes

  (2) Der abzählbare Vollständigkeitssatz

  (3) Der Beweis des Endlichkeitssatzes

Eine Anwendung des Endlichkeitssatzes der Aussagenlogik

Satz A.4.4

- Sei $G = (V, E)$ ein ungerichteter Graph mit einer abzählbaren Knotenmenge V
- Falls jeder endliche Teilgraph von G eine zulässige 3-Färbung hat, dann hat auch G eine zulässige 3-Färbung

Beweis

- Sei G ein Graph, für den jeder endliche Teilgraph eine zulässige 3-Färbung hat
- Sei $V = \{v_1, v_2, \ldots\}$ und seien c_1, c_2, c_3 die erlaubten Farben

- Wir verwenden aussagenlogische Variablen $A_{i,j}$ für $i \in \mathbb{N}$ und $j \in \{1, 2, 3\}$ mit der Bedeutung:

  „Knoten v_i wird mit Farbe c_j gefärbt“

- „Knoten v_i hat mindestens eine Farbe“

$$\varphi_i \stackrel{\text{def}}{=} A_{i,1} \vee A_{i,2} \vee A_{i,3}$$

für jedes $i \in \mathbb{N}$

- „Knoten v_i und v_ℓ haben verschiedene Farben“

$$\psi_{i,\ell} \stackrel{\text{def}}{=} \bigwedge_{j=1}^{3} \neg(A_{i,j} \wedge A_{\ell,j})$$

für jedes $i \neq \ell$

- Sei $\Phi \stackrel{\text{def}}{=} \{\varphi_i \mid i \in \mathbb{N}\} \cup \{\psi_{i,\ell} \mid (v_i, v_\ell) \in E\}$

- Klar: Färbungen von G mit 3 Farben entsprechen Belegungen und es gilt:

$$G \text{ 3-färbbar} \iff \Phi \text{ erfüllbar}$$

- Jede endliche Teilmenge von Φ spricht über einen endlichen Teilgraphen von G und ist daher erfüllbar

$\Rightarrow \Phi$ ist erfüllbar $\Rightarrow G$ ist 3-färbbar

(Abzählbarer) Vollständigkeitssatz der Aussagenlogik

Satz A.4.5: Vollständigkeitssatz der AL (abzählbar)

- Sei Φ eine Menge aussagenlogischer Formeln über einer **abzählbaren** Menge aussagenlogischer Variablen
- Sei ψ eine aussagenlogische Formel

- Dann sind äquivalent:

  (i) $\Phi \models \psi$
  (ii) $\Phi \vdash \psi$

Beweis: (i) $\Rightarrow$ (ii)

- Es gelte zunächst (i): $\Phi \models \psi$

  $\Rightarrow \Phi \cup \{\neg\psi\}$ ist unerfüllbar

  $\Rightarrow$ Es gibt eine unerfüllbare endliche Teilmenge Φ' von $\Phi \cup \{\neg\psi\}$ ☞ Satz A.4.3

  $\Rightarrow \square \in \mathbf{Res}^{\infty}(\mathcal{K}(\Phi'))$ ☞ Satz A.3.3

  $\Rightarrow \square \in \mathbf{Res}^{\infty}(\mathcal{K}(\Phi \cup \{\neg\psi\}))$ ☞ da $\Phi' \subseteq \Phi \cup \{\neg\psi\}$

  $\Rightarrow \Phi \vdash \psi$ ☞ nach Definition

Beweis: (ii) $\Rightarrow$ (i)

- Gelte umgekehrt (ii): $\Phi \vdash \psi$
- Also: $\square \in \mathbf{Res}^{k}(\mathcal{K}(\Phi \cup \{\neg\psi\}))$, für ein k

- $\square$ ist resolvierbar durch zwei Klauseln aus $\mathbf{Res}^{k-1}(\mathcal{K}(\Phi \cup \{\neg\psi\}))$
- Diese Klauseln sind resolvierbar durch (maximal) 4 Klauseln aus $\mathbf{Res}^{k-2}(\mathcal{K}(\Phi \cup \{\neg\psi\}))$
- ...
- Es folgt: $\square$ hat einen Resolutionsbeweis mit maximal 2^{k+1} Klauseln aus $\Phi \cup \{\neg\psi\}$

- Sei Φ' die Menge von Formeln aus $\Phi \cup \{\neg\psi\}$, die in diesem Beweis verwendet werden

  $\Rightarrow \Phi'$ ist unerfüllbar ☞ Satz A.3.3

  $\Rightarrow \Phi \cup \{\neg\psi\}$ ist unerfüllbar

  $\Rightarrow \Phi \models \psi$ ☞ nach Definition

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Der Resolutionskalkül lässt sich auch verwenden, um
    - Erfüllbarkeit für endliche Mengen von Formeln zu testen,
    - Allgemeingültigkeit von Formeln zu testen, und
    - Semantische Schlussfolgerungen zu überprüfen
  - Wir können die Methoden zum Nachweis der Unerfüllbarkeit von Formeln auch auf unendliche Formelmengen erweitern
  - Die entscheidende Verbindung stellt dabei der Endlichkeitssatz bereit
  - Die Hauptanwendung des Endlichkeitssatzes ist der **Vollständigkeitssatz**, der die Äquivalenz des semantischen Folgerns und des syntaktischen Beweisens für unendliche Mengen aussagenlogischer Formeln zeigt

Anhang: Der Beweis des Endlichkeitssatzes

Beweis des Endlichkeitssatzes der Aussagenlogik (1/3)

Satz A.4.3: Endlichkeitssatz (Wdh.)

- Sei V eine abzählbare Menge von aussagenlogischen Variablen
- Für eine unendliche Menge Φ von AL-Formeln über V sind äquivalent:

(i) Φ ist erfüllbar

(ii) Jede endliche Teilmenge von Φ ist erfüllbar

Beweis

- (i) $\Rightarrow$ (ii) ist offensichtlich

- Gelte also nun (ii)
- **Zu zeigen:** Φ ist erfüllbar

- Wir können annehmen:
  - Φ verwendet nur Variablen der Art A_i mit $i \in \mathbb{N}$
  - Φ enthält keine Formeln $\varphi \neq \psi$ mit $\varphi \equiv \psi$

- Für jedes $n \geq 1$ sei Φ_n die Menge aller Formeln aus Φ, die nur Variablen aus $\{A_1, \ldots, A_n\}$ verwenden

- Für jedes n gibt es nur 2^{2^n} verschiedene Wahrheitstabellen für $A_1, \ldots, A_n$
- Es gibt also nur endlich viele nicht äquivalente Formeln über $\{A_1, \ldots, A_n\}$

  $\Rightarrow \Phi_n$ ist endlich

- Da nach Annahme (ii) jede endliche Teilmenge von Φ eine erfüllende Belegung hat, ist also jedes Φ_n erfüllbar

- **Unser Plan:**

  (1) Wir wählen für jedes Φ_n eine erfüllende Belegung α_n

  (2) Wir konstruieren daraus induktiv eine Belegung α aller Variablen

  (3) Dann zeigen wir: $\alpha \models \Phi$

Beweis des Endlichkeitssatzes der Aussagenlogik (2/3)

Beweis (Fortsetzung)

- Sei α_n eine erfüllende Belegung für die Menge Φ_n
- **Beobachtung:** $\alpha_i \models \Phi_n$ für $i \geq n$
- Denn: $\Phi_i \supseteq \Phi_{i-1} \supseteq \cdots$

- **Idee:** Definiere α auf $A_1, \ldots, A_n$ so, dass es mit unendlich vielen der α_i mit $i \geq n$ übereinstimmt

- **Ansatz:** Gehe induktiv vor
  - Definiere α schrittweise für die Variablen $A_1, A_2, \ldots$
  - Verwalte Hilfsmengen $\mathcal{B}_n$ von Belegungen sodass für alle $n \geq 0$:
    - $\mathcal{B}_n$ ist unendlich groß
    - α stimmt mit allen Belegungen in $\mathcal{B}_n$ auf den Variablen $A_1, \ldots, A_n$ überein

- $\alpha(A_n)$ wird mit Hilfe einer Menge $\mathcal{B}_{n-1}$ „verbleibender Belegungen“ definiert

Induktionsanfang:

- Sei $\mathcal{B}_0 \stackrel{\text{def}}{=} \{\alpha_i \mid i \in \mathbb{N}\}$

Induktionsschritt:

- Definition von $\alpha(A_n)$:

$$\alpha(A_n) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } \alpha_i(A_n) = 1 \text{ für unendlich viele } \alpha_i \in \mathcal{B}_{n-1} \\ 0 & \text{andernfalls} \end{cases}$$

- Definition der verbleibenden Belegungen $\mathcal{B}_n$:

$$\mathcal{B}_n \stackrel{\text{def}}{=} \{\alpha_i \in \mathcal{B}_{n-1} \mid \alpha(A_n) = \alpha_i(A_n)\}$$

- $\mathcal{B}_n$ enthält also alle $\alpha_i \in \mathcal{B}_{n-1}$, für die α_i mit α auf $A_1, \ldots, A_n$ übereinstimmt
- Nach Konstruktion: $\mathcal{B}_n$ ist unendlich groß

Beweis des Endlichkeitssatzes der Aussagenlogik (3/3)

Beweis (Fortsetzung)

- **Zeige nun: $\alpha \models \Phi$** ☞ Schritt 3

- Sei dazu $\varphi \in \Phi$ beliebig
- Sei A_n die Variable in φ mit dem größten n
  $\Rightarrow \varphi \in \Phi_n$
  $\Rightarrow \alpha_i \models \varphi$, für alle $i \geq n$

- Alle Belegungen in $\mathcal{B}_n$ stimmen mit α auf $A_1, \ldots, A_n$ überein
- Insbesondere gibt es also eine Belegung α_j mit $j \geq n$ sodass

$$\alpha(A_1) = \alpha_j(A_1), \ldots, \alpha(A_n) = \alpha_j(A_n)$$

- Es gilt $\alpha_j \models \Phi_j$, deshalb auch $\alpha_j \models \Phi_n$ und somit $\alpha_j \models \varphi$
- Folglich: $\alpha \models \varphi$

- Da dies für jede Formel $\varphi \in \Phi$ gilt, folgt: $\alpha \models \Phi$

Kapitel B.1:

Grundlagen der Modallogik

Einführende Beispiele

Ein klassisches Logikrätsel: Muddy Children

Beispiel

Grundsituation:

- Alfred, Beate und Chris haben bei strömendem Regen draußen gespielt
- Sie wissen, dass sie jetzt möglicherweise ein schmutziges Gesicht haben
- Kein Kind kann sehen, ob es selbst ein schmutziges Gesicht hat
- Jedes Kind sieht, welches der anderen Kinder ein schmutziges Gesicht hat

Erste Interaktion:

- Die Eltern sagen: „Mindestens einer von euch hat ein schmutziges Gesicht“

Zweite Interaktion:

- Die Eltern fragen: „Wer weiß, ob er ein schmutziges Gesicht hat?“
- Kein Kind meldet sich

Dritte Interaktion:

- Die Eltern fragen: „Wer weiß, ob er ein schmutziges Gesicht hat?“
- Kein Kind meldet sich

Vierte Interaktion:

- Die Eltern fragen: „Wer weiß, ob er ein schmutziges Gesicht hat?“
- Alle Kinder sagen:

  „Ich habe ein schmutziges Gesicht!“

- **Wie ist dies zu erklären?**

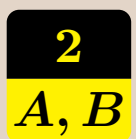
Muddy Children: Modellierung (1/2)

- Wie lässt sich dieses Szenario modellieren?

- Repräsentiere „Alfred hat ein schmutziges Gesicht“ durch die aussagenlogische Variable A
- Analog: B und C für Beate und Chris

- In dem Szenario sind **verschiedene** Wahrheitsbelegungen dieser Variablen und ihr Verhältnis zueinander relevant
- **Ansatz:** Repräsentiere jede mögliche Wahrheitsbelegung durch eine mögliche „Welt“
- Wir geben den Welten Namen, da wir gleich über mehrere Welten reden

- Die Welt, in der Alfred und Beate schmutzige Gesichter haben, aber Chris nicht:



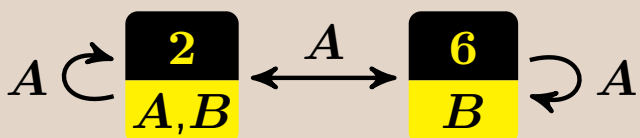
- Die Welt hat den Namen „$\mathbf{2}$“

Muddy Children: Modellierung (2/2)

- In der Welt $\begin{array}{|c|}\hline \mathbf{2} \\ \hline A, B \\ \hline \end{array}$ sieht Alfred, dass Beate ein schmutziges Gesicht hat und Chris ein sauberes
- Da Alfred nicht weiß, ob er selbst ein schmutziges Gesicht hat, hält er in dieser Welt zwei Welten für möglich:
  - $\begin{array}{|c|}\hline \mathbf{2} \\ \hline A, B \\ \hline \end{array}$, falls er ein schmutziges Gesicht hat
  - $\begin{array}{|c|}\hline \mathbf{6} \\ \hline B \\ \hline \end{array}$, falls er kein schmutziges Gesicht hat
- Das stellen wir graphisch so dar:

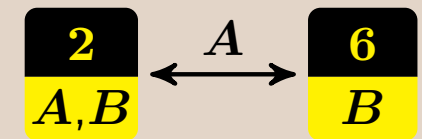


- Umgekehrt hält Alfred in Welt $\mathbf{6}$ auch Welt $\mathbf{2}$ für möglich, also:

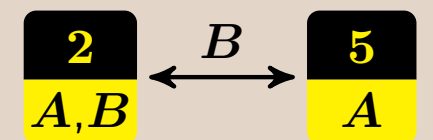


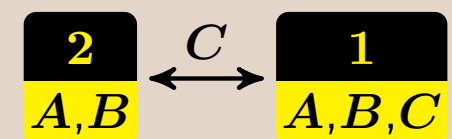
- Für den Rest des Muddy Children Beispiels gilt:
  - Jede Welt hat je eine Selbst-Schleife bezüglich A, B und C
  - Zur Übersichtlichkeit zeichnen wir diese Schleifen aber nicht ein
  - Diese Konvention gilt nur für dieses Beispiel!

- Also:



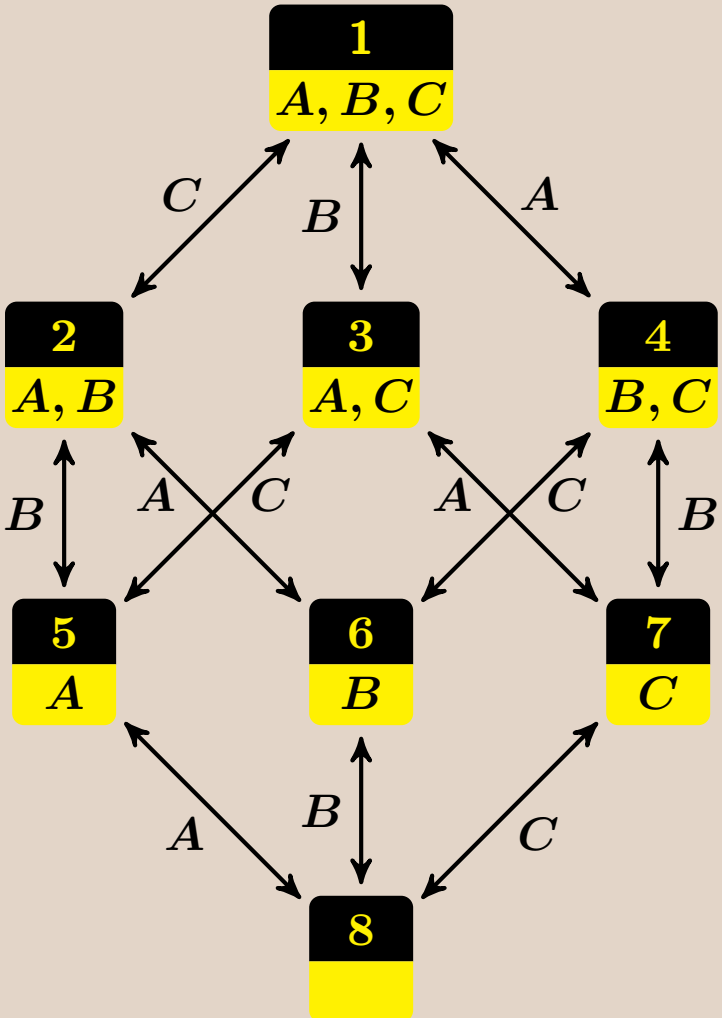
- Entsprechend, aus Sicht von Beate bzw Chris:





Muddy Children: Grundszenario (1/2)

- Insgesamt ergibt sich das Diagramm $\mathcal{K}_1$:



- In der Geschichte befinden wir uns in Welt 1, in der alle Kinder schmutzig sind

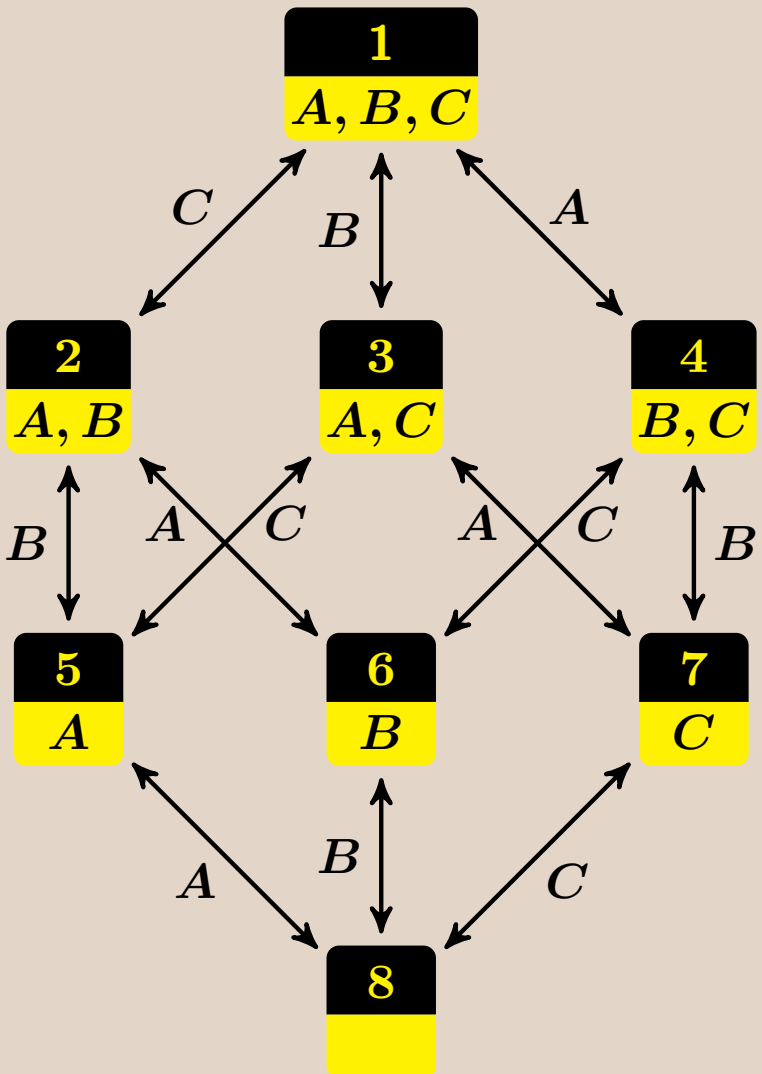
- Alfred hält in dieser Situation die Welten 1 und 4 für möglich
- Insbesondere:
  (a) Alfred hält es für möglich, dass $\neg A$ gilt
  (b) Alfred weiß, dass B gilt
  ☞ B gilt in allen Welten, die er für möglich hält

- Kompakte Notation:
  - $(\mathcal{K}_1, i) \models \Diamond_A \varphi$: „In Welt i hält Alfred es für möglich, dass φ gilt“
    ☞ Sprechweise: „Diamond“
  - $(\mathcal{K}_1, i) \models \Box_A \varphi$: „In Welt i weiß Alfred, dass φ gilt“
    ☞ Sprechweise: „Box“

- Die beiden Feststellungen über Alfreds Wissen lassen sich dann also wie folgt formalisieren:
  (a) $(\mathcal{K}_1, 1) \models \Diamond_A \neg A$
  (b) $(\mathcal{K}_1, 1) \models \Box_A B$

Muddy Children: Grundszenario (2/2)

- Diagramm $\mathcal{K}_1$:



- Was weiß Alfred in Welt 1 über das Wissen von Beate?

- Da er Welt 4 für möglich hält, hält er es für möglich, dass Beate die Welt 7 für möglich hält
- Also gilt zum Beispiel:

$$(\mathcal{K}_1, 1) \models \Diamond_A \Diamond_B (\neg A \wedge \neg B \wedge C)$$

- Alfred weiß in Welt 1, dass Beate keine andere als die Welten $1, 3, 4, 7$ für möglich hält
- Da in all diesen Welten C gilt, weiß er, dass Beate weiß, dass C gilt:

$$(\mathcal{K}_1, 1) \models \Box_A \Box_B C$$

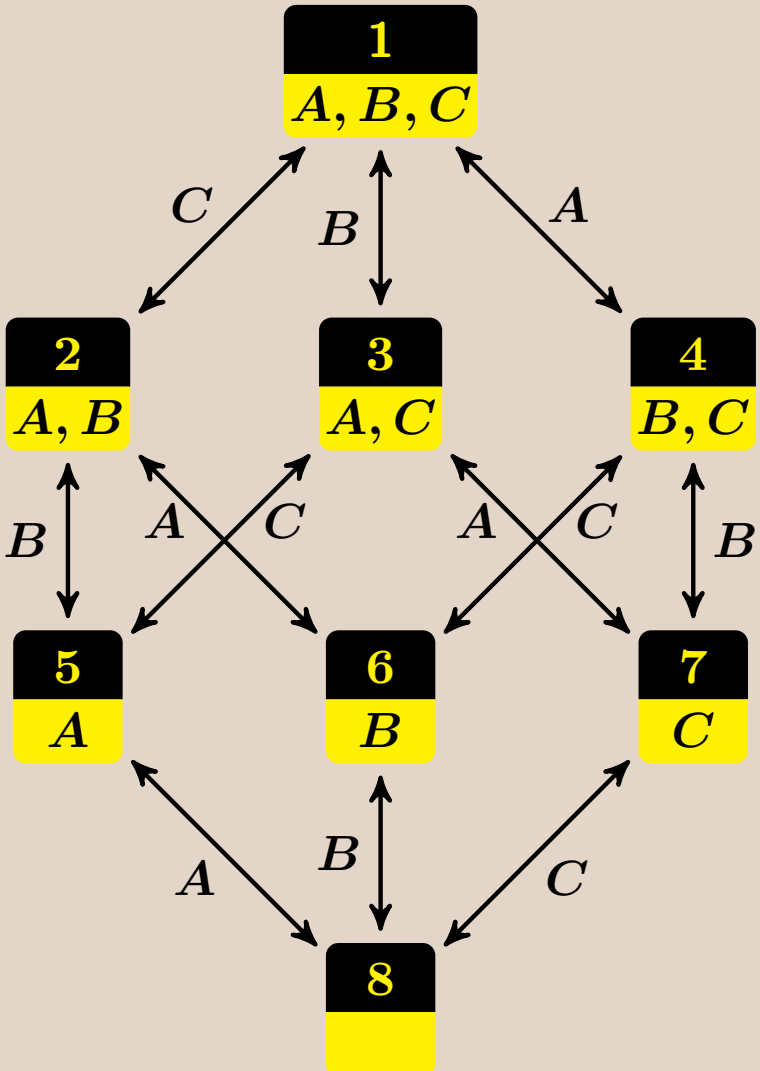
- Außerdem:

$$(\mathcal{K}_1, 1) \models \Diamond_A \Diamond_B \Diamond_C (\neg A \wedge \neg B \wedge \neg C)$$

- Jetzt können wir die Geschichte genauer analysieren

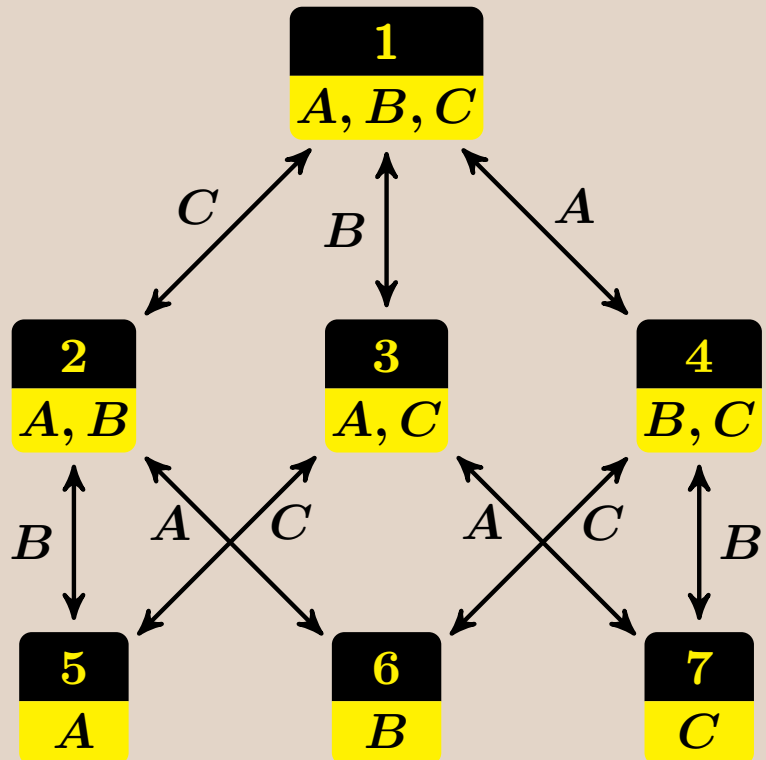
Muddy Children: Nach der ersten Interaktion (1/2)

- Diagramm $\mathcal{K}_1$:



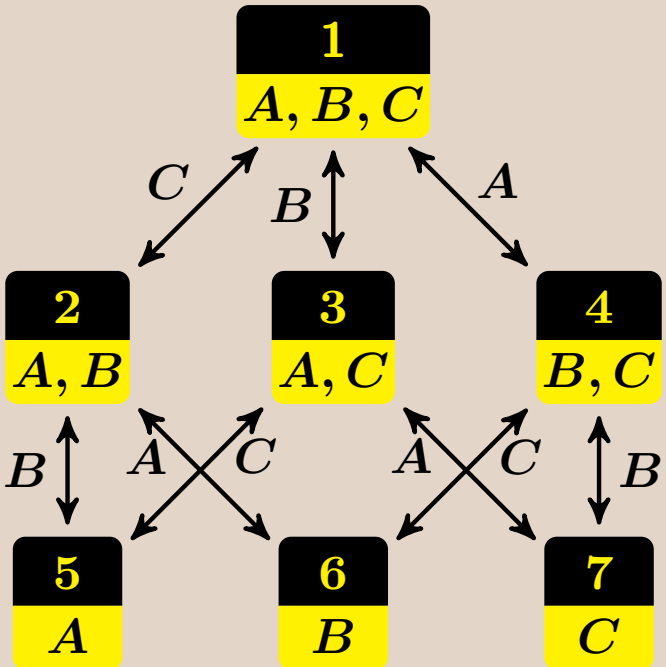
- Erste Interaktion: Mindestens ein Kind ist schmutzig
  - ➡ alle Kinder wissen, dass Welt 8 unmöglich ist

- Welt 8 muss im Modell also nicht mehr berücksichtigt werden und wir erhalten das veränderte Diagramm $\mathcal{K}_2$:



Muddy Children: Nach der ersten Interaktion (2/2)

- Diagramm $\mathcal{K}_2$:



- Das wirkt sich zum Beispiel auf das Wissen von Alfred in Welt 1 aus:

$$(\mathcal{K}_2, 1) \not\models \Diamond_A \Diamond_B \Diamond_C (\neg A \wedge \neg B \wedge \neg C)$$

- Außerdem gilt beispielsweise:
  - Chris weiß in Welt 7, dass er ein schmutziges Gesicht hat: ☞ da Welt 8 nicht möglich ist

$$(\mathcal{K}_2, 7) \models \Box_C C$$

  - In Welt 4 weiß Chris aber nicht, ob er ein schmutziges Gesicht hat: ☞ da er auch Welt 6 für möglich hält

$$(\mathcal{K}_2, 4) \models \neg \Box_C C$$

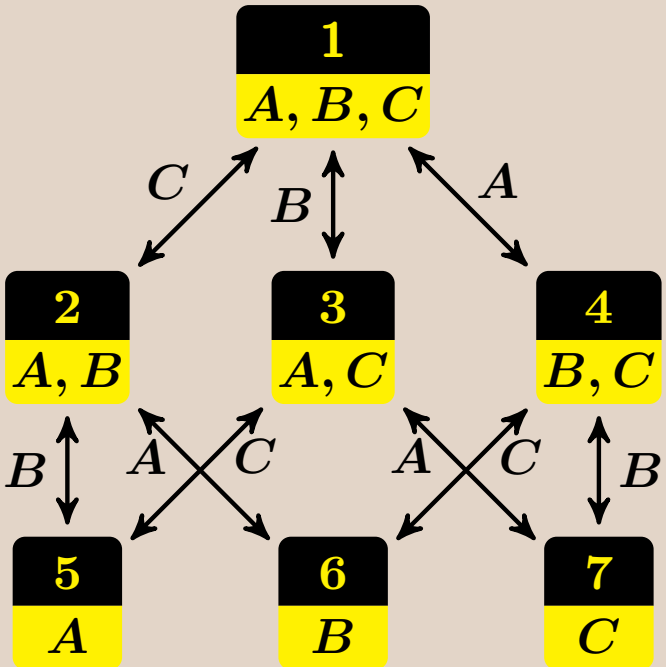
- Zusammen gilt also:

$$(\mathcal{K}_2, 4) \models \Box_B((B \wedge \neg \Box_C C) \vee (\neg B \wedge \Box_C C))$$
$$\equiv \Box_B(B \leftrightarrow \neg \Box_C C)$$

- In Welt 4 kann Beate also schließen, ob sie schmutzig ist, wenn sie weiß, ob Chris weiß, dass er schmutzig ist

Muddy Children: Nach der zweiten Interaktion (1/2)

- Diagramm $\mathcal{K}_2$:

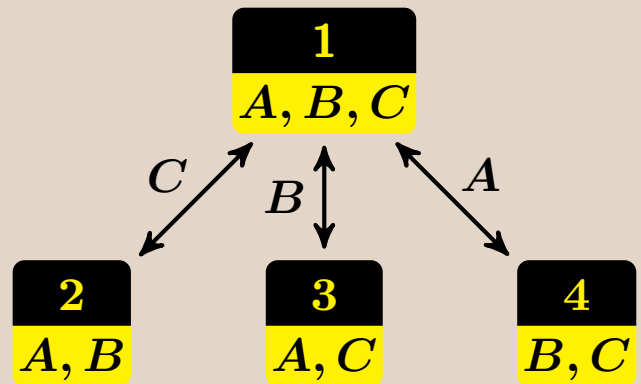


- Da nach der zweiten Interaktion kein Kind geantwortet hat, wusste noch kein Kind, ob es schmutzig ist

- Da Chris nicht wusste, ob er schmutzig ist, hält in Welt 4 Beate Welt 7 nicht mehr für möglich
- Und: In Welt 4 hält Chris Welt 6 nicht mehr für möglich
- Analog für die Welten 2 und 3

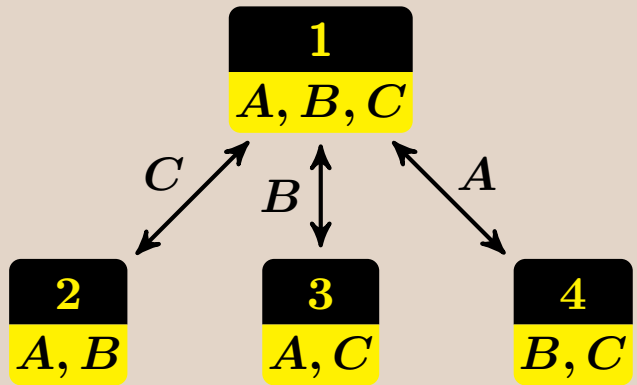
- Deswegen wissen nach der zweiten Interaktion alle Kinder, dass mindestens zwei Kinder schmutzig sind

- Das relevante Wissen wird also durch das folgende Diagramm $\mathcal{K}_3$ beschrieben:



Muddy Children: Nach der zweiten Interaktion (2/2)

- Diagramm $\mathcal{K}_3$:



- Es gilt:

$$(\mathcal{K}_3, 1) \models \Box_A((A \wedge \neg\Box_B B) \vee (\neg A \wedge \Box_B B))$$
$$\equiv \Box_A(A \leftrightarrow \neg\Box_B B)$$

- Aus der Antwort von Beate auf die zweite Frage (in der dritten Interaktion) kann Alfred also schließen, ob er schmutzig ist

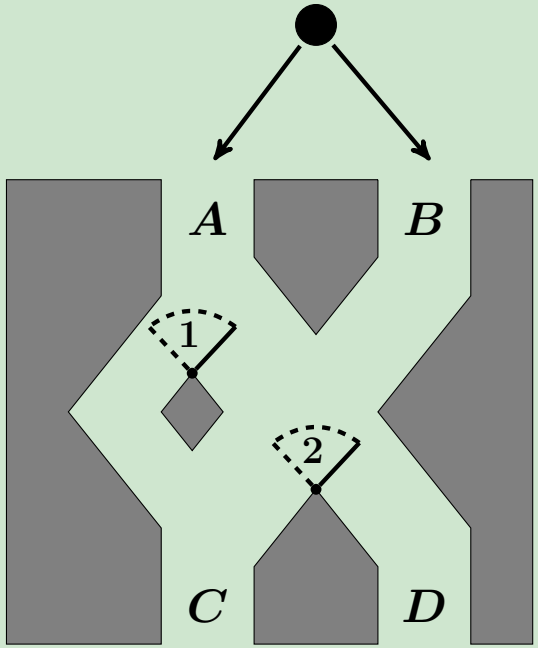
- Nachdem die Eltern zum zweiten Mal ohne Antwort gefragt haben, ist in Welt 1 deshalb keines der drei Kinder mehr unsicher:

  alle wissen, dass sie schmutzig sind

Ein zweites Beispiel: Ein Murmelspiel

Beispiel

- Wir betrachten ein **Murmelspiel**:



- **Szenario:**
  - Eine Murmel kann in A oder B eingeworfen werden
  - Zwei Kippschalter 1 und 2 beeinflussen die Bahn der Kugel
  - Die Murmel wird am Ausgang C oder D ausgegeben

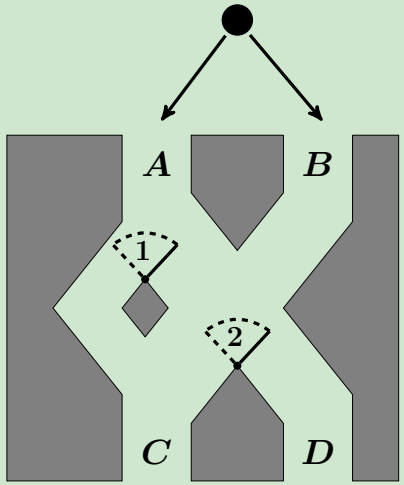
- **Grundstellung:** beide Schalter leiten die nächste Kugel nach links
- **Schalterverhalten:** Nachdem die Murmel einen Schalter passiert hat, wird dieser gekippt

- Wird z.B. in der Grundstellung eine Murmel in B eingeworfen, so
  - passiert sie Schalter 2 und wird bei C ausgegeben und
  - anschließend schaltet Schalter 2 um
- Wird nun noch eine Murmel in B eingeworfen, so
  - wird sie bei D ausgegeben und
  - Schalter 2 kehrt in die Ausgangsposition zurück

- **Frage:** Ist es möglich, ausgehend von der Grundstellung, zwei Murmeln so einzuwerfen, dass die dritte Murmel auf jeden Fall den Ausgang D nimmt?

Murmelspiel: Modellierung (1/2)

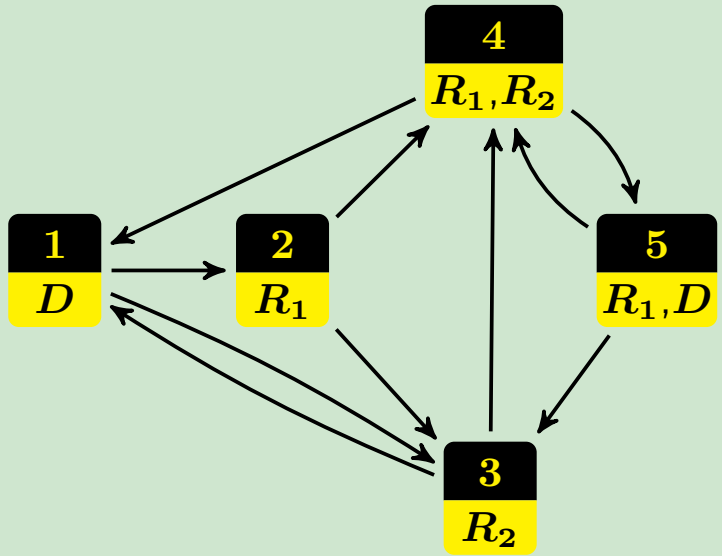
- **Murmelspiel:**



- Das Murmelspiel können wir durch ein Diagramm $\mathcal{K}_M$ modellieren
  - Die **Welten** repräsentieren mit Hilfe von Variablen den aktuellen Zustand des Spiels
  - Die **Kanten** repräsentieren Übergänge zwischen den Welten durch Einwurf einer Murmel

- **Variablen:**
  - R_1: Schalter 1 führt nach rechts
  - R_2: Schalter 2 führt nach rechts
  - D: die letzte Kugel rollte in Ausgang D

- Es ergibt sich das Diagramm $\mathcal{K}_M$:

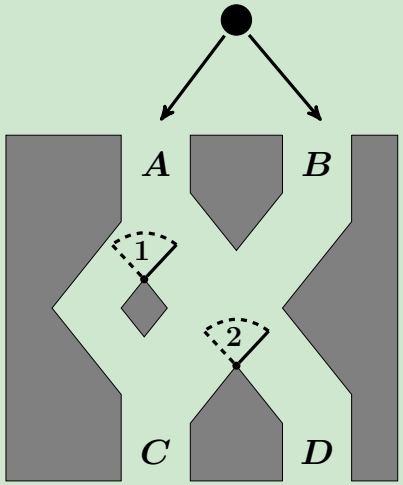


- Die Grundstellung wird durch Welt 1 repräsentiert

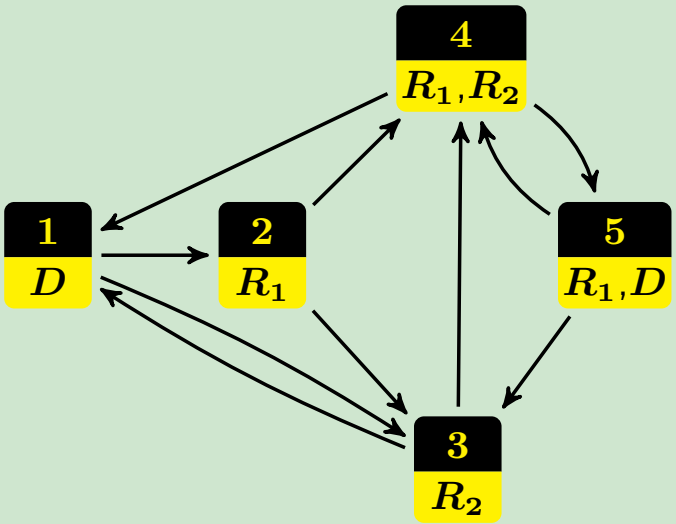
- Zu beachten: es kommen nicht alle Variablenbelegungen als Welten vor

Murmelspiel: Modellierung (2/2)

- **Murmelspiel:**



- **Modellierung:**



- Unsere Frage war:
  - Ist es in der Grundstellung möglich zwei Murmeln so einzuwerfen, dass die dritte Murmel auf jeden Fall den Ausgang D nimmt?

- Wir führen folgende Notation ein:
  - $(\mathcal{K}_M, s) \models \Diamond\varphi$: In einem Schritt kann das System von Welt s aus in eine Welt kommen, in der φ gilt
  - $(\mathcal{K}_M, s) \models \Box\varphi$: In einem Schritt kommt das System von Welt s aus zwangsläufig in eine Welt, in der φ gilt

- Unsere Frage lautet also:

  Gilt $(\mathcal{K}_M, 1) \models \Diamond\Diamond\Box D$?

- Wir beobachten:
  - $(\mathcal{K}_M, 4) \models \Box D$

    $\Rightarrow (\mathcal{K}_M, 2) \models \Diamond\Box D$

    $\Rightarrow (\mathcal{K}_M, 1) \models \Diamond\Diamond\Box D$
- Also ist die Antwort auf unsere Frage: „ja“

Gemeinsamkeiten der beiden Beispiele

- Beide Beispiele haben einiges gemeinsam
- Wir modellieren jeweils
  - verschiedene mögliche Situationen durch verschiedene Welten mit Wahrheitsbelegungen
  - Zusammenhänge der Situationen durch Übergänge (Kanten) zwischen den Welten

- Allerdings haben die Kanten in den beiden Beispielen unterschiedliche Bedeutungen:
  - **Muddy Children:** eine mit K markierte Kante von Welt x nach Welt y bedeutet, dass Kind K in Welt x die Welt y für möglich hält
  - **Murmelspiel:** eine Kante von Welt x nach Welt y bedeutet, dass das System durch Einwurf einer Murmel von Zustand x in Zustand y übergehen kann

- Wir führen jetzt eine passende Logik ein

  ➡ **Modallogik**

Inhalt

1. Einführende Beispiele
2. Syntax und Semantik der Modallogik
3. Äquivalenzen und Normalformen
4. Modellierung in Modallogik: Noch ein Beispiel
5. Anhang: Und noch ein Modellierungsbeispiel

Syntax und Semantik der Modallogik

Modallogik: Syntax

- **Vereinbarung zur Vereinfachung:**
  - Zukünftig betrachten wir nur noch Strukturen, in denen die Kanten nicht beschriftet sind
  - Entsprechend verwenden wir nur $\Diamond$ und $\Box$, aber nicht $\Diamond_A, \Diamond_B, \ldots$

Definition: Syntax der Modallogik

- Die Menge ML der Formeln der Modallogik ist induktiv wie folgt definiert:

(1) $\top$ und $\bot$ und jede Variable aus AV sind in ML

(2) Sind φ und ψ in ML, so auch $\neg\varphi$, $(\varphi \wedge \psi)$ und $(\varphi \vee \psi)$

(3) Ist φ in ML, so auch $\Diamond\varphi$ und $\Box\varphi$

✎ Die Bedeutung des Zeichens $\Box$ in der Modallogik hat nichts mit der Bedeutung in Resolutionsbeweisen der Aussagenlogik (und der Prädikatenlogik) zu tun

Beispiel

- Beispielformeln aus ML:
  - $\Diamond\Diamond\Box D$
  - $((\Diamond A \vee \Box B) \wedge (\Diamond\Box(C \vee \Diamond A)))$

- **Konvention:** $\Box$, $\Diamond$ binden stärker als $\vee, \wedge$
  - Also: Die Operatoren $\Box$ und $\Diamond$ benötigen keine Klammern
  - Beispiel: $\Box A \wedge C \equiv (\Box A) \wedge C$

- Wir verwenden weiterhin die definierten Junktoren $\rightarrow$ und $\leftrightarrow$ mit der gleichen Bedeutung wie in der Aussagenlogik

✎ Wenn wir von „modallogischen Formeln“ sprechen, sind abkürzende Schreibweisen (auch das Weglassen unnötiger Klammern) erlaubt

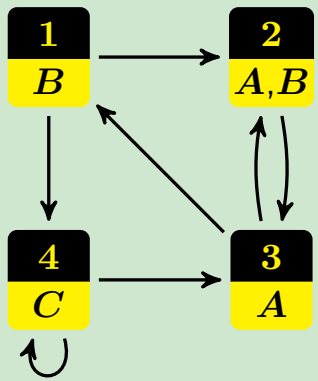
✎ ML bezeichnet die in der formalen Definition definierte Formelmenge

Modallogik: Semantik (1/2)

- Modallogische Formeln werden in **Kripkestrukturen** interpretiert

Beispiel

- Kripkestruktur $\mathcal{K}$:



Definition: Kripkestruktur

- Eine **Kripkestruktur** $\mathcal{K} = (V, E, P)$ besteht aus
  - einer Grundmenge V von **Welten**
  - einer 2-stelligen **Übergangsrelation** $E \subseteq V \times V$, sowie
  - einer Funktion P, die jeder Welt s eine endliche Menge $P(s)$ aussagenlogischer Variablen zuordnet

- **Welten** nennen wir oft auch **Zustände**

- Die Menge $P(s)$ repräsentiert die Wahrheitsbelegung in s:
  - $P(s)$ enthält alle Variablen, die in s wahr sind
  - Formal entspricht $P(s)$ der Wahrheitsbelegung α_s via:

$$\alpha_s(X) = 1 \overset{\text{def}}{\Leftrightarrow} X \in P(s)$$

Beispiel (Fortsetzung)

- $\mathcal{K} \overset{\text{def}}{=} (V, E, P)$ mit
  - $V = \{1, 2, 3, 4\}$
  - $E = \{(1,2), (1,4), (2,3), (3,1), (3,2), (4,3), (4,4)\}$
  - $P(1) = \{B\}$, $P_1(2) = \{A, B\}$, $P(3) = \{A\}$, $P_1(4) = \{C\}$

Modallogik: Semantik (2/2)

Definition: Semantik der Modallogik

- Sei $\mathcal{K} = (V, E, P)$ eine Kripkestruktur und $s \in V$ eine Welt

- Wir definieren $(\mathcal{K}, s) \models \varphi$ induktiv wie folgt:

  - $(\mathcal{K}, s) \models \top$,
  - $(\mathcal{K}, s) \not\models \bot$,
  - $(\mathcal{K}, s) \models X$, falls $X \in P(s)$,

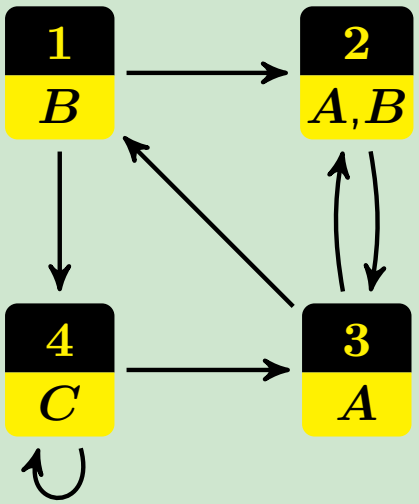
  - $(\mathcal{K}, s) \models \neg\psi$, falls $(\mathcal{K}, s) \not\models \psi$,
  - $(\mathcal{K}, s) \models (\chi \wedge \psi)$, falls $(\mathcal{K}, s) \models \chi$ und $(\mathcal{K}, s) \models \psi$,
  - $(\mathcal{K}, s) \models (\chi \vee \psi)$, falls $(\mathcal{K}, s) \models \chi$ oder $(\mathcal{K}, s) \models \psi$,

  - $(\mathcal{K}, s) \models \Diamond\psi$, falls es eine Welt s' gibt mit: $(s, s') \in E$ und $(\mathcal{K}, s') \models \psi$,
  - $(\mathcal{K}, s) \models \Box\psi$, falls für jede Welt s' gilt: falls $(s, s') \in E$ so $(\mathcal{K}, s') \models \psi$

- Gilt $(\mathcal{K}, s) \models \varphi$, so heißt $(\mathcal{K}, s)$ **Modell** von φ
- Die Begriffe **erfüllbar**, **allgemeingültig** etc. sind analog zur Aussagenlogik definiert

Beispiel

- Kripkestruktur $\mathcal{K}$:



- Es gilt:

$(\mathcal{K}, 1) \models \Diamond\Box A \wedge \Box\Diamond(A \wedge \neg B)$

Modallogik: Auswertung von Formeln

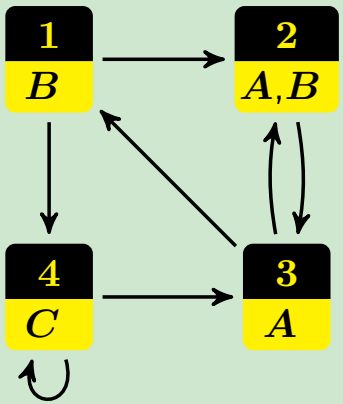
- Wie lässt sich herausfinden, ob eine modallogische Formel φ in einer Welt s gilt?
  - ➡ „bottom-up“ gemäß der Definition der Semantik

Satz B.1.1

- Für eine ML-Formel φ und eine Kripkestruktur $\mathcal{K}$ lässt sich in $O(|\varphi||\mathcal{K}|)$ Schritten die Menge M aller Welten s von $\mathcal{K}$ berechnen, für die $(\mathcal{K}, s) \models \varphi$ gilt

Beispiel

- Kripkestruktur $\mathcal{K}$:



- Sei $\varphi = \Diamond\Box A \wedge \Box\Diamond(A \wedge \neg B)$

- Berechne die Welten s von $\mathcal{K}$ mit $(\mathcal{K}, s) \models \varphi$:

	1	2	3	4
A	✗	✓	✓	✗
B	✓	✓	✗	✗
$\neg B$	✗	✗	✓	✓
$A \wedge \neg B$	✗	✗	✓	✗
$\Diamond(A \wedge \neg B)$	✗	✓	✗	✓
$\Box\Diamond(A \wedge \neg B)$	✓	✗	✗	✗
$\Box A$	✗	✓	✗	✗
$\Diamond\Box A$	✓	✗	✓	✗
φ	✓	✗	✗	✗

Äquivalenzen und Normalformen

Äquivalenz und Folgerung

- Dass zwei Formeln äquivalent sind bzw. dass eine Formel aus einer anderen folgt notieren wir wie in der Aussagenlogik

Definition: Äquivalenz und Folgerung

- Seien φ, ψ ML-Formeln

- φ und ψ sind **äquivalent** wenn für alle Kripkestrukturen $\mathcal{K}$ und alle Welten s gilt:

$$(\mathcal{K}, s) \models \varphi \iff (\mathcal{K}, s) \models \psi$$

☞ Schreibweise: $\varphi \equiv \psi$

- ψ **folgt aus** φ wenn für alle Kripkestrukturen $\mathcal{K}$ und alle Welten s mit $(\mathcal{K}, s) \models \varphi$ gilt $(\mathcal{K}, s) \models \psi$

☞ Schreibweise: $\varphi \models \psi$

Modallogik: Äquivalenzen und Folgerungen

Satz B.1.2

- Für ML-Formeln gelten alle Äquivalenzen aus Satz 2.1
- Außerdem gilt für ML-Formeln φ, ψ:
- **Dualität**
  - $\neg\Box\varphi \equiv \Diamond\neg\varphi$
  - $\neg\Diamond\varphi \equiv \Box\neg\varphi$
- **Distributivität**
  - $\Box(\varphi \wedge \psi) \equiv \Box\varphi \wedge \Box\psi$
  - $\Diamond(\varphi \vee \psi) \equiv \Diamond\varphi \vee \Diamond\psi$
- **Distribution**
  - $\Box(\varphi \to \psi) \models \Box\varphi \to \Box\psi$
  - $\Box(\varphi \to \psi) \models \Diamond\varphi \to \Diamond\psi$
  - $\Diamond(\varphi \to \psi) \equiv \Box\varphi \to \Diamond\psi$
- **Notwendigkeit**

  Ist φ allgemein gültig, so auch $\Box\varphi$

Folgerung B.1.3

- Zu jeder ML-Formel φ gibt es eine äquivalente Formel φ' in **Negationsnormalform (NNF)**, in der die Negation $\neg$ nur unmittelbar vor Variablen vorkommt

Nachweis von Äquivalenzen und Folgerungen (1/2)

Satz B.1.4

- Für ML-Formeln gelten alle Äquivalenzen aus Satz 2.1
- Außerdem gilt für ML-Formeln φ, ψ:
- **Dualität**
  - $\neg\Box\varphi \equiv \Diamond\neg\varphi$
  - $\neg\Diamond\varphi \equiv \Box\neg\varphi$
- **Distributivität**
  - $\Box(\varphi \wedge \psi) \equiv \Box\varphi \wedge \Box\psi$
  - $\Diamond(\varphi \vee \psi) \equiv \Diamond\varphi \vee \Diamond\psi$
- **Distribution**
  - $\Box(\varphi \to \psi) \models \Box\varphi \to \Box\psi$
  - $\Box(\varphi \to \psi) \models \Diamond\varphi \to \Diamond\psi$
  - $\Diamond(\varphi \to \psi) \equiv \Box\varphi \to \Diamond\psi$
- **Notwendigkeit**

  Ist φ allgemein gültig, so auch $\Box\varphi$

- Das Substitutionslemma und das Ersetzungslemma der Aussagenlogik gelten in gleicher Form auch für die Modallogik

Lemma B.1.5: Substitutionslemma der Modallogik

- Seien φ_1 und φ_2 modallogische Formeln
- Sei S eine Substitution $\mathrm{AV} \to \mathrm{ML}$
- Wenn $\varphi_1 \equiv \varphi_2$ gilt,
  dann gilt auch $S(\varphi_1) \equiv S(\varphi_2)$

Lemma B.1.6: Ersetzungslemma der Modallogik

- Sei φ_1 eine modallogische Formel mit einer Teilformel ψ_1
- Sei ψ_2 eine modallogische Formel mit $\psi_2 \equiv \psi_1$
- Wenn φ_2 aus φ_1 entsteht, indem ein Vorkommen von ψ_1 durch ψ_2 ersetzt wird, dann gilt $\varphi_2 \equiv \varphi_1$

Nachweis von Äquivalenzen und Folgerungen (2/2)

Beweisskizze

- Als Beispiel für den Beweis der Aussagen aus Satz B.1.4 zeigen wir:

$$\Box(\varphi \to \psi) \models \Box\varphi \to \Box\psi$$

- Das Substitutionslemma gilt auch für $\models$ statt $\equiv$: wenn $\varphi_1 \models \varphi_2$ gilt, dann gilt auch $S(\varphi_1) \models S(\varphi_2)$

- Es genügt also zu zeigen:

$$\Box(A \to B) \models \Box A \to \Box B$$

- Sei also $\mathcal{K} = (V, E, P)$ eine Kripkestruktur und $s \in V$
- Angenommen $(\mathcal{K}, s) \models \Box(A \to B)$

- Zu zeigen:

$$(\mathcal{K}, s) \models \Box A \to \Box B$$

- Wir unterscheiden zwei Fälle:

- **1. Fall:** $(\mathcal{K}, s) \models \Box B$
  - Dann folgt direkt $(\mathcal{K}, s) \models \Box A \to \Box B$ nach Definition von $\to$

- **2. Fall:** $(\mathcal{K}, s) \not\models \Box B$
  - Dann gibt es eine Welt $s' \in V$ mit $(s, s') \in E$ und $(\mathcal{K}, s') \not\models B$
  - Wegen $(\mathcal{K}, s) \models \Box(A \to B)$ gilt für alle Welten $s'' \in V$ mit $(s, s'') \in E$:

$$(\mathcal{K}, s'') \models A \to B$$

  - Insbesondere gilt also:

$$(\mathcal{K}, s') \models A \to B$$

$$\Rightarrow (\mathcal{K}, s') \not\models A$$
$$\Rightarrow (\mathcal{K}, s) \not\models \Box A$$
$$\Rightarrow (\mathcal{K}, s) \models \Box A \to \Box B$$

Modellierung in Modallogik: Noch ein Beispiel

Modallogische Modellierung: Noch ein Beispiel (1/3)

Eine ehemalige Übungsaufgabe

- Dr. Akula, Medizinerin in einem Krankenhaus, bekommt einen Patienten, der mit Kopf- und Bauchschmerzen eingeliefert wird. Der Patient wird am Montag eingeliefert und soll am Freitag wieder von seinem Vater abgeholt werden. Er muss nun also vier Nächte in der Klinik bleiben. Am Freitag muss er vollständig gesund sein.

- Dr. Akula hat drei Medikamente zur Auswahl, von denen sie jeweils ein Medikament in einer Nacht wirken lässt. Die Medikamente wirken folgendermaßen:
  - Medikament $\mathbf{1}$ vertauscht die Symptome. Es gilt also, dass der Patient nach der Anwendung Bauchschmerzen hat, genau dann wenn er vorher Kopfschmerzen hatte, und dass er nach der Anwendung Kopfschmerzen hat, wenn er vorher Bauchschmerzen hatte.
  - Medikament $\mathbf{2}$ verursacht auf jeden Fall Kopfschmerzen, heilt aber Bauchschmerzen. Wenn der Patient allerdings keine Bauchschmerzen hatte, bekommt er davon welche.
  - Medikament $\mathbf{3}$ macht den Patienten völlig gesund, wenn er nur ein Symptom hat. Sind beide Symptome vorhanden oder ist der Patient gesund, hat er nach der Anwendung sowohl Kopf- als auch Bauchschmerzen.

(a) Konstruiere eine Kripkestruktur $\mathcal{K}$, die die beschriebene Situation modelliert. Gebe auch an, welche Welt s die Situation am Montag repräsentiert.

Nun hat Dr. Akula das Problem, dass sie am Mittwoch eine wichtige Konferenz besuchen muss, so dass ihr Kollege Dr. Hyde die Medizin für die dritte Nacht (von Mittwoch auf Donnerstag) verabreicht. Dieser ist jedoch reichlich inkompetent und wird zufällig eines der drei Medikamente verabreichen.

(b) Gebe eine Formel an, die aussagt, dass Dr. Akula den Patienten bis Freitag heilen kann, egal, was Dr. Hyde macht. Gilt die Formel in $(\mathcal{K}, s)$?

Modallogische Modellierung: Noch ein Beispiel (2/3)

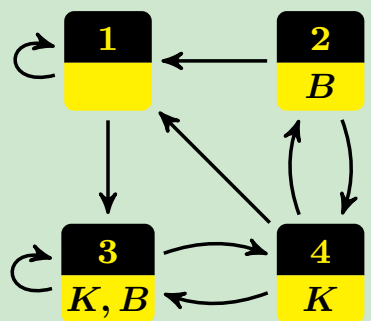
Eine ehemalige Übungsaufgabe

- Medikament $\mathbf{1}$ vertauscht die Symptome. Es gilt also, dass der Patient nach der Anwendung Bauchschmerzen hat genau dann wenn er vorher Kopfschmerzen hatte und dass er nach der Anwendung Kopfschmerzen hat, wenn er vorher Bauchschmerzen hatte.
- Medikament $\mathbf{2}$ verursacht auf jeden Fall Kopfschmerzen, heilt aber Bauchschmerzen. Wenn der Patient allerdings keine Bauchschmerzen hatte, bekommt er davon welche.
- Medikament $\mathbf{3}$ macht den Patienten völlig gesund, wenn er nur ein Symptom hat. Sind beide Symptome vorhanden oder ist der Patient gesund, hat er nach der Anwendung sowohl Kopf-, als auch Bauchschmerzen.

Lösung zu (a)

(a) Konstruiere eine Kripkestruktur $\mathcal{K}$, die die beschriebene Situation modelliert. Gebe auch an, welche Welt s die Situation am Montag repräsentiert.

- Um dieses Problem zu modellieren, führen wir zwei aussagenlogische Variablen ein:
  - K: der Patient hat Kopfschmerzen
  - B: der Patient hat Bauchschmerzen
- In der Kripkestruktur gibt es:
  - eine Welt für jede Kombination der Variablen
  - Übergänge entsprechend der Anwendung der Medikamente
- Kripkestruktur $\mathcal{K}$:



- Die Situation am Montag wird durch Welt $\mathbf{3}$ repräsentiert, der Patient hat beide Symptome.

Modallogische Modellierung: Noch ein Beispiel (3/3)

Eine ehemalige Übungsaufgabe

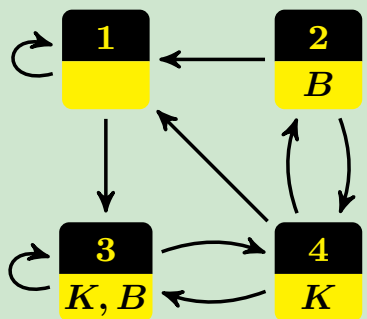
- Medikament 1 vertauscht die Symptome. Es gilt also, dass der Patient nach der Anwendung Bauchschmerzen hat genau dann wenn er vorher Kopfschmerzen hatte und dass er nach der Anwendung Kopfschmerzen hat, wenn er vorher Bauchschmerzen hatte.
- Medikament 2 verursacht auf jeden Fall Kopfschmerzen, heilt aber Bauchschmerzen. Wenn der Patient allerdings keine Bauchschmerzen hatte, bekommt er davon welche.
- Medikament 3 macht den Patienten völlig gesund, wenn er nur ein Symptom hat. Sind beide Symptome vorhanden oder ist der Patient gesund, hat er nach der Anwendung sowohl Kopf-, als auch Bauchschmerzen.

Lösung zu (b)

- Nun hat Dr. Akula das Problem, dass sie am Mittwoch eine wichtige Konferenz besuchen muss, so dass ihr Kollege Dr. Hyde die Medizin für die dritte Nacht (von Mittwoch auf Donnerstag) verabreicht. Dieser ist jedoch reichlich inkompetent und wird zufällig eines der drei Medikamente verabreichen.

(b) Gebe eine Formel an, die aussagt, dass Dr. Akula den Patienten bis Freitag heilen kann, egal, was Dr. Hyde macht. Gilt die Formel in $(\mathcal{K}, s)$?

- Zur Erinnerung:



- Die Formel ist $\Diamond\Diamond\Box\Diamond(\neg K \wedge \neg B)$
- Sie ist in Welt $s = 3$ erfüllt. Dr. Akula kann die Medizin 2 in der ersten Nacht geben (führt in Welt 4), dann Medizin 1 in der zweiten Nacht (führt in Welt 2), dann hat Dr. Hyde nur die Möglichkeit, in die Welten 1 oder 4 zu kommen. Und dann kann der Patient mit Medizin 1 oder Medizin 3 geheilt werden.

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Kripkestrukturen und modallogische Formeln anhand verschiedener Beispiele
  - Syntax und Semantik der Modallogik
  - Methoden für den Nachweis der Äquivalenz modallogischer Formeln

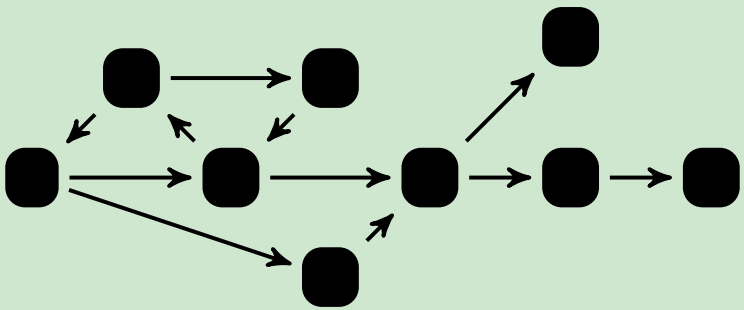
Literatur

- Das Muddy-Children-Beispiel ist, in ähnlicher Form, dem Skript von Rintanen und Wölfl, **Modal logics: theory and applications**, Uni Freiburg, 2004, entnommen

Anhang: Und noch ein Modellierungsbeispiel

Modallogische Modellierung: Ein Spiel

Ein Spiel



- Zwei Spieler ziehen abwechselnd eine Spielfigur von einem Knoten über eine gerichtete Kante zu einem anderen Knoten
- Wer nicht ziehen kann, verliert

- Wir fassen die Knoten des Graphen als Welten einer Kripkestruktur auf

  ☞ Alle Mengen $P(s)$ sind leer

- Aussagen über das Spiel können dann durch modallogische Formeln ausgedrückt werden

- „Der Spieler, der am Zug ist, kann nicht ziehen“: $\Box\bot$
  - Denn:
    - In jeder Welt gilt die Formel $\top$
    - Wenn es von einer Welt s einen Übergang in eine andere Welt gibt, so gilt $(\mathcal{K}, s) \models \Diamond\top$

    $\Rightarrow \Box\bot \equiv \neg\Diamond\top$ drückt also aus, dass es keinen Übergang in eine andere Welt gibt
  - Insbesondere drückt $\Box\bot$ also aus, dass der Spieler, der **nicht** am Zug ist, gewinnt

- „Der Spieler am Zug kann in einem Zug gewinnen“: $\Diamond\Box\bot$

- „Der Spieler am Zug kann in zwei Zügen gewinnen“: $\Diamond\Box\Diamond\Box\bot$

- „Egal was der Spieler, der am Zug ist, als erstes macht, er kann auf jeden Fall im nächsten Zug gewinnen“: $\Box\Box\Diamond\Box\bot$

Kapitel B.2:

Erfüllbarkeit modallogischer Formeln

Einleitung

Einleitung

- **Ziel dieses Kapitels:**

  Beweiskalkül und Erfüllbarkeitstest für die Modallogik

- **Naiver Ansatz:** Versuche ein Verfahren für die Aussagenlogik auf die Modallogik zu erweitern
- Welche Erfüllbarkeitstests kennen wir für ausssagenlogische Formeln φ?

- **Wahrheitstabelle:**

  Werte $\alpha \models \varphi$ für **alle** Belegungen α aus

- **Problem:** Es gibt unendlich viele Kandidaten-Kripkestrukturen für eine gegebene modallogische Formel

- **Resolution:**
  - Kalkül für Unerfüllbarkeit von Klauselmengen $\mathcal{K}$
  - $\mathcal{K}$ unerfüllbar gdw. $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$
- **Problem:** Erweiterung auf Modallogik nicht offensichtlich

- **Also:** Bei allen Methoden ist unklar, wie sie für die Modallogik erweitert werden könnten

- **Unser Ansatz:** Ein weiterer Beweiskalkül für die Aussagenlogik

  ➡ Dieser lässt sich auf die Modallogik erweitern...

Inhalt

1. Ein Tableaukalkül für die Aussagenlogik
2. Ein Tableaukalkül für die Modallogik
3. Korrektheit des Tableaukalküls

Ein Tableaukalkül für die Aussagenlogik

AL-Tableaukalkül: Beispiel für die Herangehensweise

Beispiel

- Ist die folgende Formel erfüllbar?

$$(\neg A \wedge ((B \vee A) \wedge (\neg B \vee A))) \vee (A \wedge ((\neg A \wedge B) \vee \neg B))$$

➡ Dafür muss eine der Teilformeln erfüllbar sein

(1) Ist $\neg A \wedge ((B \vee A) \wedge (\neg B \vee A))$ erfüllbar?

- Dafür müssten die Formeln $\neg A$ und $((B \vee A) \wedge (\neg B \vee A))$ simultan erfüllbar sein
- Dafür müssten $\neg A$, $B \vee A$ und $\neg B \vee A$ simultan erfüllbar sein

- Zwei Fälle:

  (1a) $\neg A$, B und $\neg B \vee A$ simultan erfüllbar - zwei Unterfälle:

  (1ai) $\neg A$, B und $\neg B$ simultan erfüllbar: 

  (1aii) $\neg A$, B und A simultan erfüllbar: 

  (1b) $\neg A$, A und $\neg B \vee A$ simultan erfüllbar: 

- $\neg A \wedge ((B \vee A) \wedge (\neg B \vee A))$ ist also nicht erfüllbar

(2) Ist $A \wedge ((\neg A \wedge B) \vee \neg B)$ erfüllbar?

- Sind A und $(\neg A \wedge B) \vee \neg B$ simultan erfüllbar? Zwei Fälle:

  (2a) A und $\neg A \wedge B$ simultan erfüllbar: 

  (2b) A und $\neg B$ simultan erfüllbar:

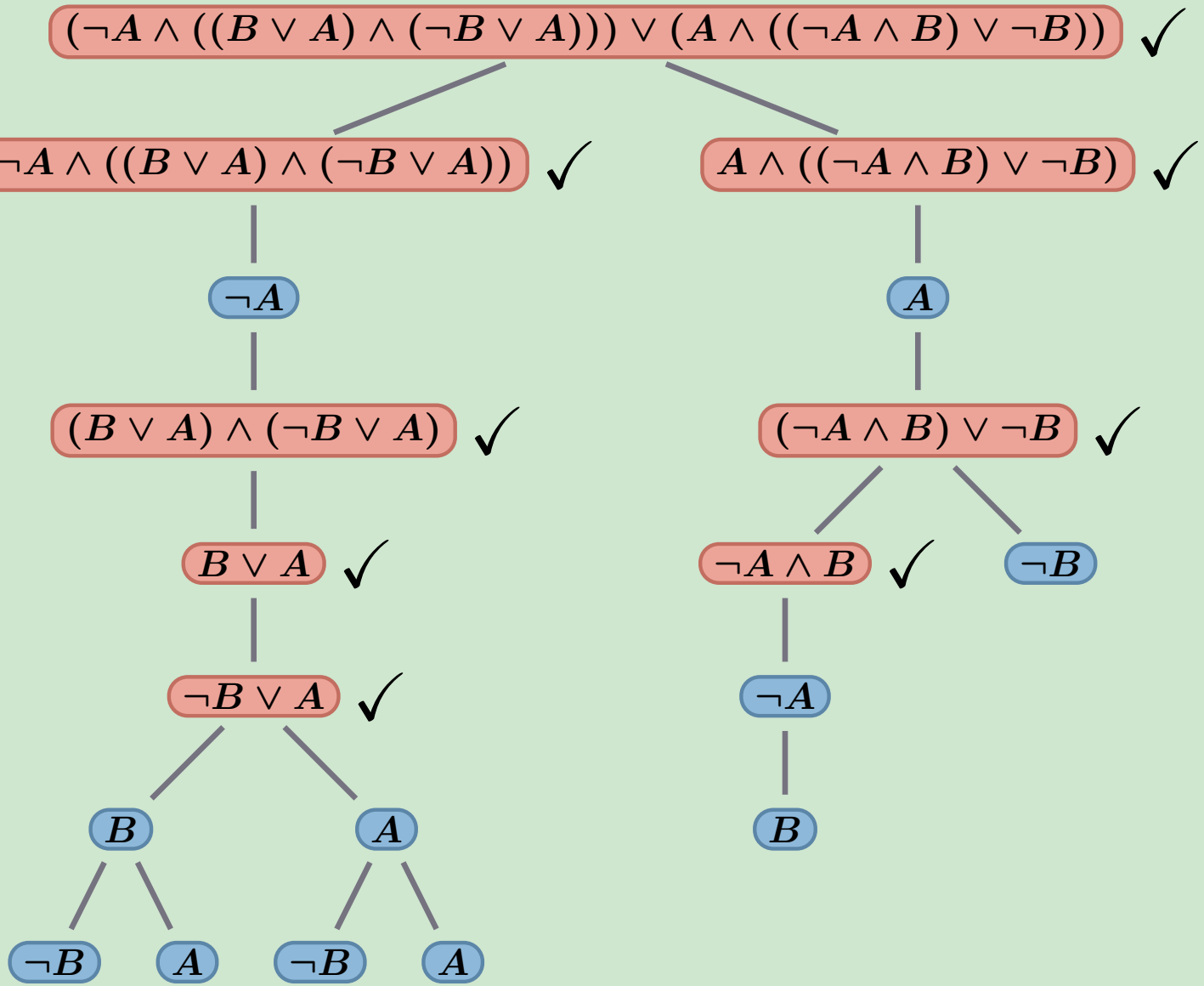


- Also ist die Gesamtformel erfüllbar

- Dieses Vorgehen wird ziemlich unübersichtlich
- **Tableaukalkül**: Systematische und übersichtliche Notation für die obige Grundidee

AL-Tableaukalkül: Beispiel

Beispiel



AL-Tableaukalkül: Regeln

- Ein aussagenlogisches **Tableau** T ist ein Baum, dessen Knoten mit aussagenlogischen Formeln beschriftet sind
- Ein Tableau für eine AL-Formel φ in **Negationsnormalform** und ohne Vorkommen von $\top$ und $\bot$ kann wie folgt konstruiert werden

- T_v: Teilbaum, der am Knoten v von T hängt
- P_v: Pfad vom Knoten v von T zur Wurzel

- **Zunächst:** Erzeuge die Wurzel des Baumes und beschrifte sie mit φ
- **Dann:** Wende eine der beiden folgenden Regeln an, solange wie das möglich ist:

- $\vee$-**Regel:**
  - Wähle einen unmarkierten Knoten v mit einer Formel der Form $\varphi_1 \vee \varphi_2$ und markiere ihn
  - Hänge an jedes Blatt u von T_v zwei neue Knoten u_1 und u_2 als Kinder an, und beschrifte sie mit φ_1 bzw. φ_2

- $\wedge$-**Regel:**
  - Wähle einen unmarkierten Knoten v mit einer Formel der Form $\varphi_1 \wedge \varphi_2$ und markiere ihn
  - Hänge an jedes Blatt u von T_v ein neues Kind u_1 an, das mit φ_1 beschriftet ist, und an u_1 ein weiteres Kind u_2, das mit φ_2 beschriftet ist

AL-Tableaukalkül: Bedeutung

- Ein Tableau heißt **saturiert**, wenn alle Knoten, deren Formel kein Literal ist, markiert sind

- **Frage:** Woran lässt sich an einem saturierten Tableau T zu φ ablesen, ob φ erfüllbar ist?

- Dazu betrachten wir die Blätter von T:
  - Wir nennen ein Blatt v von T **geschlossen**, wenn auf dem Weg P_v von v zur Wurzel eine Variable X und ihre Negation $\neg X$ vorkommt

    ➡ Geschlossene Blätter entsprechen also widersprüchlichen Pfaden
  - Wir nennen ein Blatt **offen**, wenn es nicht geschlossen ist

- Ein saturiertes Tableau heißt **offen**, wenn es ein offenes Blatt hat, andernfalls **geschlossen**

- **Fakt:** Eine Formel φ ist genau dann erfüllbar, wenn φ ein offenes saturiertes Tableau T hat

  ☞ Das werden wir noch beweisen!

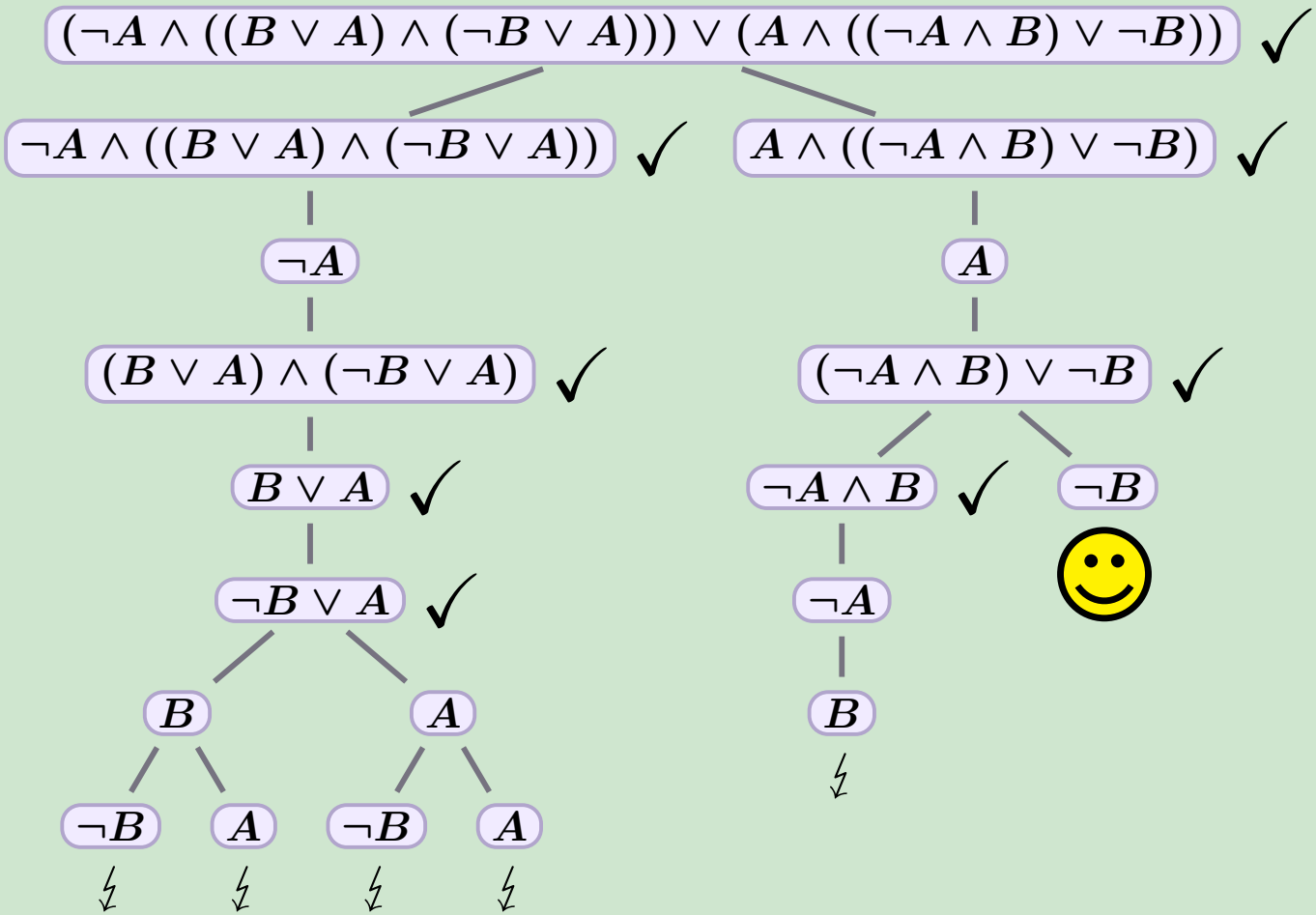
- Und: wenn es **ein** solches Tableau für φ gibt, so haben **alle** saturierten Tableaus für φ ein offenes Blatt

  ➡ Es kann strukturell verschiedene, saturierte Tableaus geben: je nach Reihenfolge der Knotenbearbeitung

- Eine erfüllende Belegung für φ lässt sich wie folgt konstruieren:
  - Wähle ein offenes Blatt v von T
  - Definiere eine Wahrheitsbelegung α so, dass sie alle Literale auf dem Weg von v zur Wurzel wahr macht

AL-Tableaukalkül: Beispiel (Fortsetzung)

Beispiel



- Erfüllende Belegung: $A \mapsto 1, B \mapsto 0$ ☞ B.2.1
- Einfachere Notation: ↯ statt 

AL-Tableaukalkül: Korrektheit, Vollständigkeit, Termination

Satz B.2.1

(a) Termination:

Für jede AL-Formel φ in NNF erzeugt der Tableaukalkül nach endlich vielen Schritten ein saturiertes Tableau $\boldsymbol{T}$ der Tiefe $\leq |\varphi| - 1$ mit $\leq 2^{|\varphi|}$ Knoten

(b) Korrektheit:

(i) Für jedes offene Blatt eines saturierten Tableaus induzieren die Literale seines Pfades eine erfüllende Belegung für φ

(ii) Ist φ unerfüllbar, so ist jedes saturierte Tableau $\boldsymbol{T}$ zu φ geschlossen

(c) Vollständigkeit:

Ist φ erfüllbar, so ist jedes saturierte Tableau $\boldsymbol{T}$ zu φ offen

✎ Bemerkungen zu Teil (b)

- Die in (i) genannte Belegung weist möglicherweise nicht allen Variablen von φ einen Wert zu
  - **Aber:** jede Vervollständigung macht φ wahr
- Teil (ii) folgt direkt aus (i):
  - Wenn das Tableau nicht geschlossen wäre, gäbe es nach (i) eine erfüllende Belegung

- Wir werden Satz B.2.1 am Ende dieses Kapitels beweisen

Ein Tableaukalkül für die Modallogik

Erfüllbarkeit von modallogischen ML-Formeln: Definition

Definition:
Erfüllbarkeit von modallogischen Formeln

- Eine ML-Formel φ ist **erfüllbar**, wenn es eine Kripkestruktur $\mathcal{K}$ und eine Welt s in $\mathcal{K}$ gibt, so dass $(\mathcal{K}, s) \models \varphi$

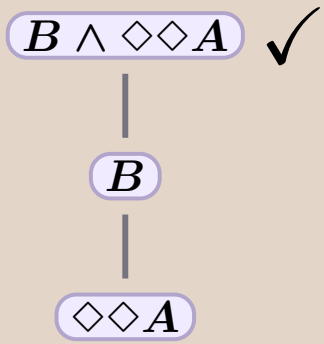
- Aussagenlogischer Tableaukalkül:
  - Entscheide Erfüllbarkeit
  - Liefere für erfüllbare Formeln eine erfüllende Belegung

- Anforderung an den modallogischen Tableaukalkül:
  - Entscheide Erfüllbarkeit
  - Liefere für erfüllbare Formeln eine Kripkestruktur und eine Welt, in der die Formel wahr wird

Tableaukalkül für ML: Grundidee

- Ist die Formel $B \wedge \Diamond\Diamond A$ erfüllbar?

- Wir können die $\wedge$-Regel des AL-Tableaukalküls einmal anwenden:



- Aber wie soll es dann weiter gehen?
- Um $\Diamond\Diamond A$ wahr zu machen, muss es eine Welt geben, die in einem Schritt erreichbar ist und in der $\Diamond A$ gilt

- **Idee:** Wir ergänzen jeden Knoten des Tableaus um die Angabe einer Welt
- Knotenbeschriftungen sind also von der Form s, φ

  ➡ Intendierte Bedeutung: φ gilt in s

- Um die Modaloperatoren aufzulösen, erlauben wir das Hinzufügen neuer Welten

- **Umgang mit $\Diamond$:**
  - Wenn wir einen mit $s, \Diamond\varphi$ beschrifteten Knoten markieren, fügen wir
    - einen mit $(s, s') \in E$ beschrifteten Knoten ein

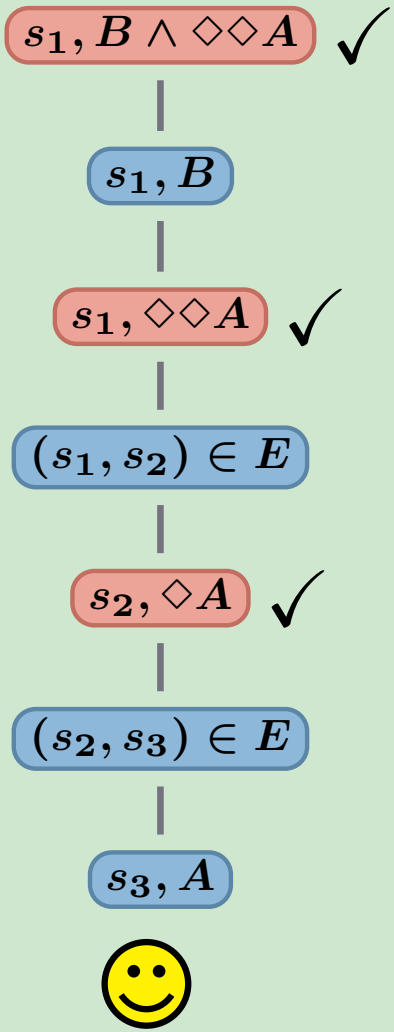
      ☞ für eine neue Welt s'
    - und darunter einen mit s', φ beschrifteten Knoten

      ☞ Details folgen noch. . .

Tableaukalkül für ML: Erstes Beispiel

Beispiel

- Ist $B \wedge \Diamond\Diamond A$ erfüllbar?

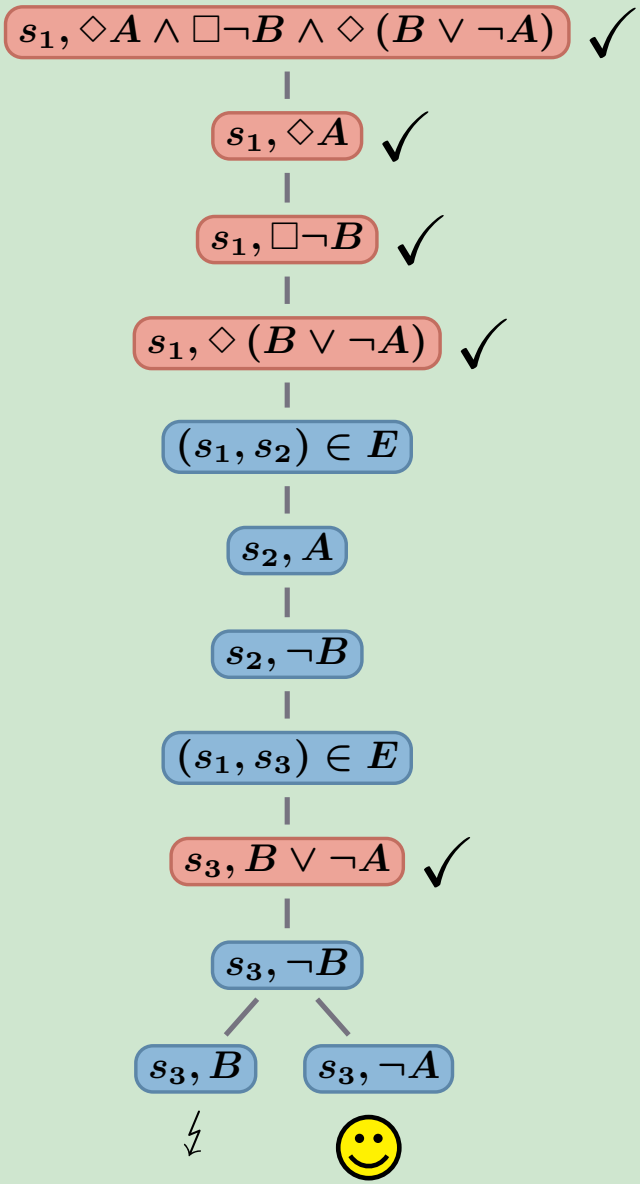


- Also gilt φ in Welt s_1 der folgenden Kripkestruktur und ist deshalb erfüllbar:



Tableaukalkül für ML: Zweites Beispiel

Ist $\Diamond A \wedge \Box\neg B \wedge \Diamond(B \vee \neg A)$ erfüllbar?



- **Umgang mit $\Box$:**
  - Wird ein Knoten v mit Beschriftung $s, \Box\varphi$ bearbeitet, so muss an jedes Blatt u von T_v für jeden Knoten w von P_u mit einer Beschriftung $(s, s') \in E$ ein mit s', φ beschrifteter Knoten angefügt werden
  - Ist v ein Knoten mit Beschriftung $s, \Diamond\varphi$, der markiert wird, und ist w ein mit $(s, s') \in E$ beschrifteter Knoten, der dabei neu eingefügt wird,
    - dann muss für jeden schon markierten Knoten z mit Beschriftung $s, \Box\psi$ in P_w ein mit s', ψ beschrifteter Knoten an jedes Blatt von T_w angefügt werden

Modallogischer Tableaukalkül: Definition (1/3)

- Der modallogische Tableaukalkül entsteht aus dem aussagenlogischen Tableaukalkül durch folgende Ergänzungen:

- Jeder Knoten des Tableaus wird durch ein Paar s_i, ψ beschriftet
- Die Wurzel wird mit s_1, φ beschriftet

- Knoten, die mit einer Formel der Typen
  - s_i, X
  - $s_i, \neg X$
  - $s_i, \varphi_1 \vee \varphi_2$
  - $s_i, \varphi_1 \wedge \varphi_2$

  beschriftet sind, werden wie im AL-Tableaukalkül behandelt

  ➡ Neu eingefügte Knoten werden dabei mit Welt s_i beschriftet

- Knoten, die mit Formeln $\Box\varphi$ und $\Diamond\varphi$ beschriftet sind, werden gemäß der folgenden $\Box$-Regel bzw. der $\Diamond$-Regel behandelt

Modallogischer Tableaukalkül: Definition (2/3)

- **$\Box$-Regel:**
  - Wähle einen unmarkierten Knoten v mit einer Formel der Form $s_i, \Box\psi$ und markiere ihn
  - Hänge an jedes Blatt u von T_v für jede Welt s_j, für die $(s_i, s_j) \in E$ in P_u vorkommt, einen neuen Knoten s_j, ψ an
    - ✎ Falls es mehrere solche s_j gibt, müssen die angehängten Knoten unterhalb von u einen Weg bilden

- **$\Diamond$-Regel:**
  - Wähle einen unmarkierten Knoten v mit einer Formel der Form $s_i, \Diamond\psi$ und markiere ihn
  - Wähle eine Welt s_j, die noch nicht in T vorkommt
  - Für jedes Blatt u unterhalb von v:
    - Hänge einen neuen markierten Knoten mit Beschriftung $(s_i, s_j) \in E$ und darunter einen neuen Knoten mit Beschriftung s_j, ψ an
    - Hänge für jeden schon markierten und mit $s_i, \Box\psi'$ beschrifteten Knoten in P_u einen Knoten s_j, ψ' an jedes Blatt von T_u an
      - ✎ Und zwar wieder so, dass die angehängten Knoten unterhalb von u einen Weg bilden

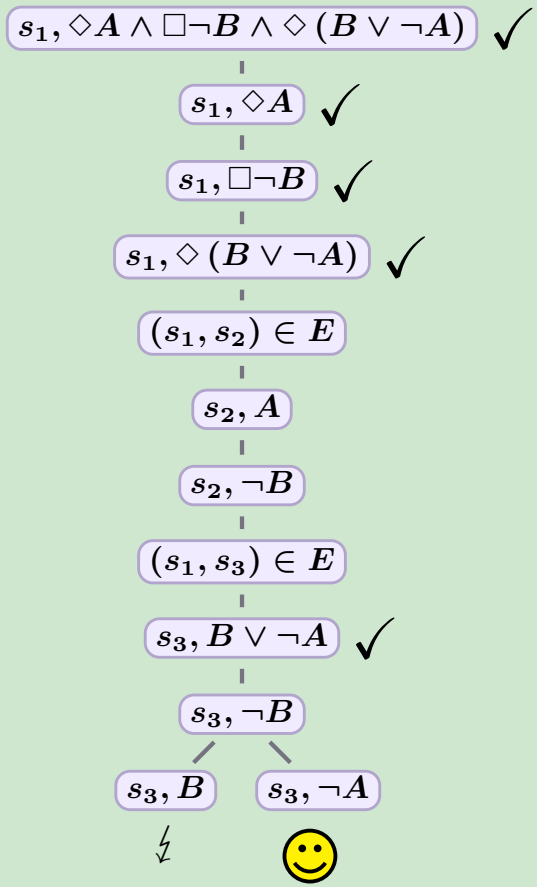
Modallogischer Tableaukalkül: Definition (3/3)

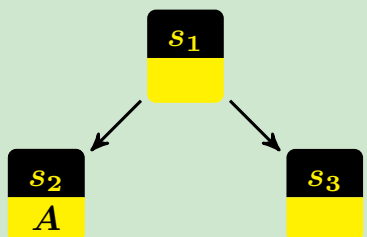
- Ein Blatt v ist **geschlossen**, wenn auf dem Weg von v zur Wurzel zwei Knoten s_i, X und $s_i, \neg X$ vorkommen
- Die restlichen Begriffe sind analog zum Fall der Aussagenlogik

- Aus einem offenen Blatt u kann wie folgt ein Modell für φ gewonnen werden:
  - Konstruiere die Welten und Kanten einer Kripkestruktur $\mathcal{K}$ gemäß der in P_u vorkommenden Knoten $(s_i, s_j) \in E$
  - Für jedes i definieren wir die Wahrheitsbelegung α_i (und damit die Menge $P(s_i)$) durch:

$$\alpha_i(X) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } s_i, X \text{ in } P_u \text{ vorkommt} \\ 0 & \text{andernfalls} \end{cases}$$

Beispiel





ML-Tableaukalkül: Korrektheit, Vollständigkeit, Termination

- Der ML-Tableaukalkül terminiert und ist korrekt und vollständig

Satz B.2.2

(a) Termination:

Für jede ML-Formel φ erzeugt die Tableau-Methode nach endlich vielen Schritten ein saturiertes Tableau $\boldsymbol{T}$

(b) Korrektheit:

(i) Für jedes offene Blatt eines saturierten Tableaus induzieren die (Welt,Literal)-Paare und die Kanten $(\boldsymbol{s_i}, \boldsymbol{s_j})$ seines Pfades ein Modell für φ

(ii) Ist φ unerfüllbar, so ist jedes saturierte Tableau $\boldsymbol{T}$ zu φ geschlossen

(c) Vollständigkeit:

Ist φ erfüllbar, so ist jedes saturierte Tableau $\boldsymbol{T}$ zu φ offen

- Damit haben wir das angestrebte Ziel erreicht: eine Methode zum Testen der Erfüllbarkeit modallogischer Formeln

Tableaukalkül für ML: Drittes Beispiel

- Mit dem modallogischen Tableaukalkül lassen sich natürlich auch modallogische Äquivalenzen und Folgerungen testen

Beispiel

- Wir betrachten die Aussage:

  Wenn $\Box(\varphi \to \psi)$ gilt,
  dann auch $\Diamond\varphi \to \Diamond\psi$

- Zum Nachweis zeigen wir die Unerfüllbarkeit von

  $$\Box(\varphi \to \psi) \wedge \neg(\Diamond\varphi \to \Diamond\psi)$$

- Aufgrund des Substitutionslemmas genügt es dafür zu zeigen, dass

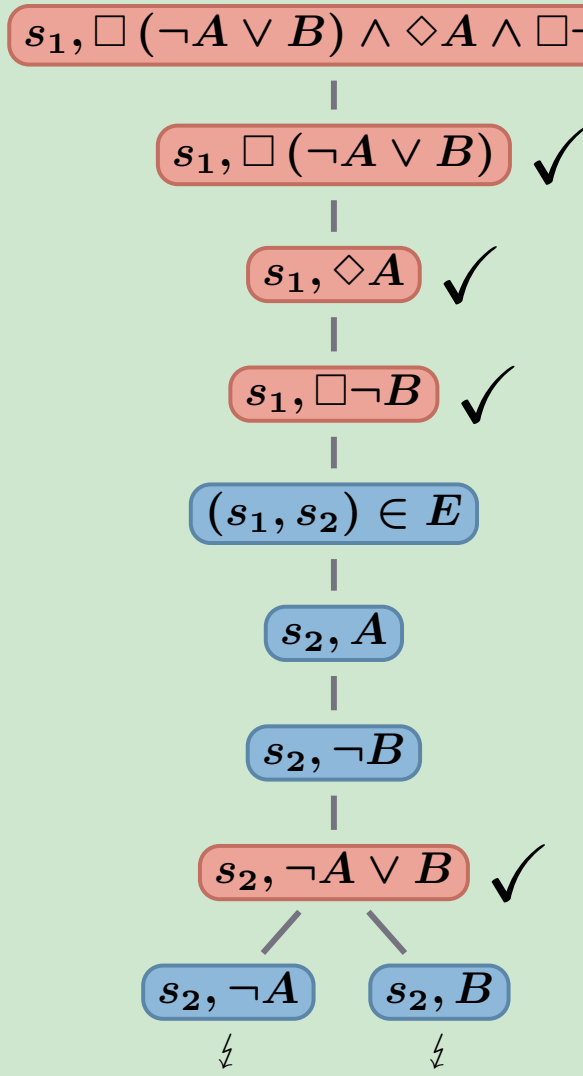
  $$\chi \stackrel{\text{def}}{=} \Box(\neg A \vee B) \wedge \neg(\neg\Diamond A \vee \Diamond B)$$

  unerfüllbar ist

- Dazu bringen wir χ in NNF

  $$\Box(\neg A \vee B) \wedge \Diamond A \wedge \Box\neg B$$

  und wenden den Tableaukalkül an



- Also ist χ unerfüllbar $\Rightarrow$ Behauptung

Korrektheit des Tableaukalküls

Tableaukalkül: Korrektheit, Vollständigkeit, Termination

- Wir beweisen die Korrektheit des aussagenlogischen Tableaukalküls
- Der Beweis für den modallogischen Tableaukalkül ist ähnlich

Satz B.2.1 **(Wdh.)**

(a) Termination:

Für jede AL-Formel φ in NNF erzeugt der Tableaukalkül nach endlich vielen Schritten ein saturiertes Tableau $\boldsymbol{T}$ der Tiefe $\leq |\varphi| - \mathbf{1}$ mit $\leq \mathbf{2}^{|\varphi|}$ Knoten

(b) Korrektheit:

(i) Für jedes offene Blatt eines saturierten Tableaus induzieren die Literale seines Pfades eine erfüllende Belegung für φ

(ii) Ist φ unerfüllbar, so ist jedes saturierte Tableau $\boldsymbol{T}$ zu φ geschlossen

(c) Vollständigkeit:

Ist φ erfüllbar, so ist jedes saturierte Tableau $\boldsymbol{T}$ zu φ offen

Terminierung des Tableaukalkül: Beweisidee

Satz B.2.1 **(Wdh.)**

(a) Termination:

Für jede AL-Formel φ in NNF erzeugt der Tableaukalkül nach endlich vielen Schritten ein saturiertes Tableau $\boldsymbol{T}$ der Tiefe $\leq |\varphi| - \mathbf{1}$ mit $\leq \mathbf{2}^{|\varphi|}$ Knoten

Beweisidee

- **Klar:** Im Tableau zu φ kommen nur Teilformeln von φ vor
- Außerdem: Jede Teilformel von φ kommt auf jedem Pfad höchstens so oft vor, wie sie in φ vorkommt

  $\Rightarrow$ Jeder Pfad hat höchstens so viele Knoten wie φ Vorkommen von Teilformeln hat

- Die Anzahl der Vorkommen von Teilformeln von φ in φ ist $\leq |\varphi|$ ☞ Induktion über den Aufbau von φ

  $\Rightarrow$ Die Tiefe jedes Pfades ist $\leq |\varphi| - \mathbf{1}$

  $\Rightarrow \boldsymbol{T}$ hat höchstens $\mathbf{2}^{|\varphi|}$ Knoten, da $\boldsymbol{T}$ binär ist

- Die Tableau-Methode terminiert also nach höchstens $\mathbf{2}^{|\varphi|}$ Regelanwendungen

Korrektheit des Tableaukalkül: Beweisidee (1/3)

Satz B.2.1 **(Wdh.)**

(b) Korrektheit:

(i) Für jedes offene Blatt eines saturierten Tableaus induzieren die Literale seines Pfades eine erfüllende Belegung für φ

(ii) Ist φ unerfüllbar, so ist jedes saturierte Tableau T zu φ geschlossen

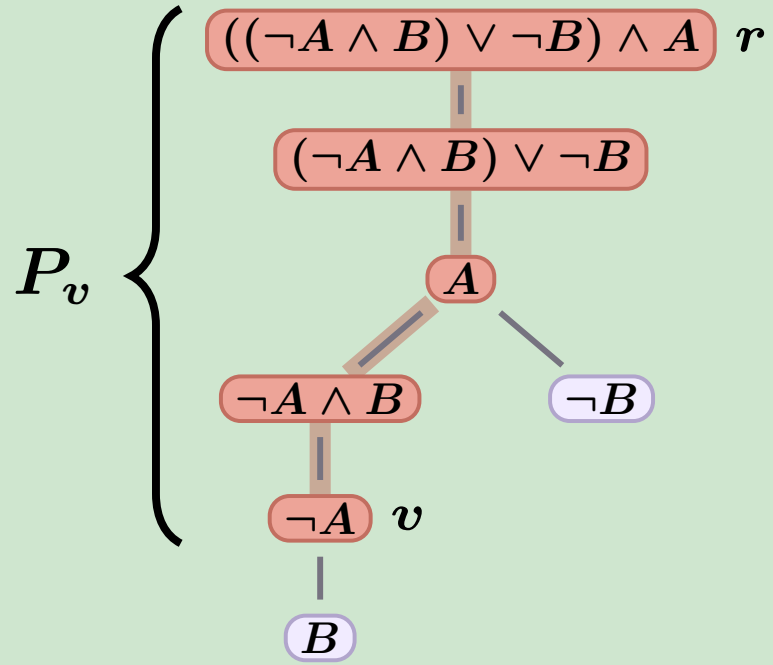
- Wir benötigen die folgende Notation
- Sei φ eine AL-Formel, T ein Tableau zu φ und v ein Knoten von T

  ➡ T muss nicht saturiert sein!

- Die Formel, mit der ein Knoten v beschriftet ist, bezeichnen wir durch φ_v
- Als **Pfad P_v von v** bezeichnen wir den Weg von v zur Wurzel r des Baumes

  ☞ einschließlich v und r
- Mit F_v bezeichnen wir die Menge der Formeln, mit denen Knoten in P_v beschriftet sind

Beispiel



- $\varphi_v = \neg A$
- $F_v = \{\neg A, \neg A \wedge B, A,$
  $(\neg A \wedge B) \vee \neg B,$
  $((\neg A \wedge B) \vee \neg B) \wedge A\}$

Korrektheit des Tableaukalkül: Beweisidee (2/3)

Satz B.2.1 **(Wdh.)**

(b) Korrektheit:

(i) Für jedes offene Blatt eines saturierten Tableaus induzieren die Literale seines Pfades eine erfüllende Belegung für φ

(ii) Ist φ unerfüllbar, so ist jedes saturierte Tableau T zu φ geschlossen

Beweisidee

- Sei u ein offenes Blatt eines saturierten Tableaus T zu einer Formel φ

- Zu u definieren wir eine Belegung α für die Variablen von φ:

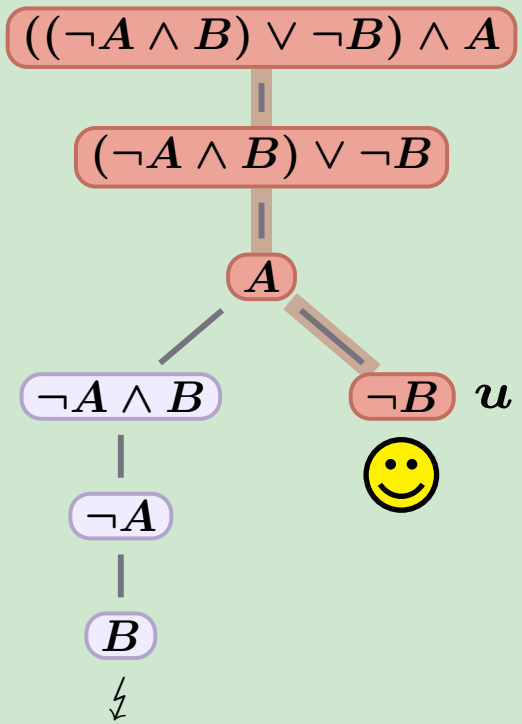
$$\alpha(X) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } X \text{ in } F_u \text{ ist} \\ 0 & \text{andernfalls} \end{cases}$$

✎ Im zweiten Fall ist $\neg X$ in F_u oder weder X noch $\neg X$ sind in F_u

- **Behauptung:** $\alpha \models \varphi$
- **Zeige:** Für alle Knoten w von P_u gilt $\alpha \models \varphi_w$

➡ Durch Induktion nach dem Abstand von w zu u

Beispiel



- $\alpha(A) = 1, \alpha(B) = 0$

Korrektheit des Tableaukalkül: Beweisidee (3/3)

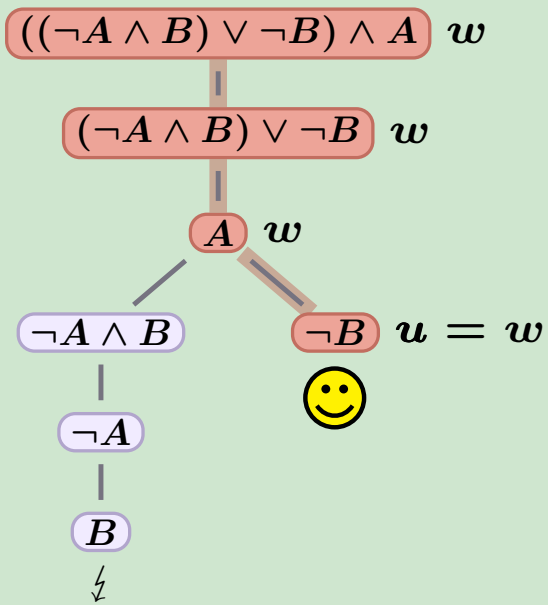
Beweisidee (Fortsetzung)

- Belegung α für Knoten u:

$$\alpha(X) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{falls } X \text{ in } F_u \text{ ist} \\ 0 & \text{andernfalls} \end{cases}$$

- **Zeige:** für alle Knoten w von P_u gilt $\alpha \models \varphi_w$

Beispiel



- $\alpha(A) = 1, \alpha(B) = 0$

- **Induktionsanfang:** $w = u$
  - Klar, da φ_u ein Literal ist, das nach Definition von α wahr gemacht wird

- **Induktionsschritt:**
  - Also: für alle Knoten z auf P_u von u bis w gilt: $\alpha \models \varphi_z$
  - Wir unterscheiden 3 Fälle:

  - **1. Fall:** φ_w ist Literal
    - Dann gilt $\alpha \models \varphi_w$ nach Konstruktion von α

  - **2. Fall:** $\varphi_w = \varphi_1 \vee \varphi_2$
    $\Rightarrow \varphi_1$ oder φ_2 kommt zwischen w und u vor
    $\Rightarrow$ Eine der Formeln ist nach Induktion wahr unter α

  - **3. Fall:** $\varphi_w = \varphi_1 \wedge \varphi_2$
    $\Rightarrow \varphi_1$ und φ_2 kommen zwischen w und u vor
    $\Rightarrow$ Sie sind nach Induktion wahr unter α

  - Es folgt also jeweils: $\alpha \models \varphi_w$

$\Rightarrow \alpha \models \varphi \Rightarrow$ Korrektheit des Tableaukalkül

Vollständigkeit des Tableaukalkül: Beweisidee (1/2)

Satz B.2.1 **(Wdh.)**

(c) Vollständigkeit:

Ist φ erfüllbar, so ist jedes saturierte Tableau T zu φ offen

Beweisidee

- Wir zeigen für jede Belegung α und jedes (nicht notwendigerweise saturierte) Tableau T:

  $(*)$ Falls $\alpha \models \varphi$, gibt es ein Blatt u von T mit $\alpha \models F_u$

$\Rightarrow$ In einem saturierten Tableau für eine erfüllbare Formel gibt es ein offenes Blatt

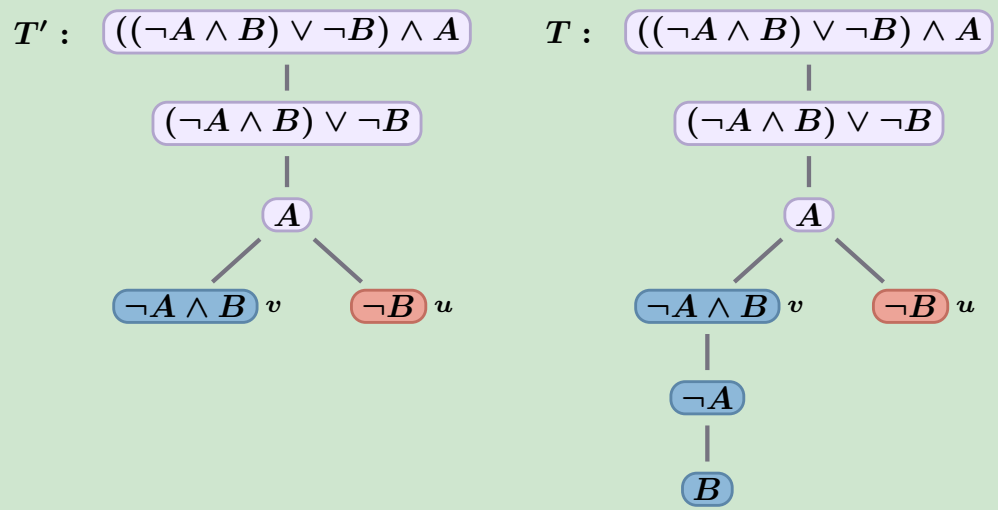
- Sei also α eine erfüllende Belegung für φ
- Der Beweis ist eine Induktion nach der Anzahl der Knoten von T

- **Induktionsanfang:** T besteht nur aus φ ✓

- **Induktionsschritt:**
  - Sei T durch Anwendung einer Regel aus T' auf einen Knoten v entstanden
  - Induktion: In T' gibt es ein Blatt u mit $\alpha \models F_u$

  - **Fall A:** u ist auch ein Blatt von T ✓
  - **Fall B:** An u wurden neue Knoten angehängt

Illustration von Fall A



Vollständigkeit des Tableaukalkül: Beweisidee (2/2)

Beweisidee (Fortsetzung)

- Knoten v: Hier wird die Regel angewendet
- Knoten u: Blatt von Tableau T'

- **Fall B.1:** $\vee$-Regel
  $\Rightarrow \varphi_v$ ist von der Form $\varphi_1 \vee \varphi_2$
  $\Rightarrow \alpha \models \varphi_1$ oder $\alpha \models \varphi_2$
  ☞ Semantik von $\vee$
  $\Rightarrow$ Für die Kinder u_1 und u_2 von u gilt: $\alpha \models \varphi_{u_1}$ oder $\alpha \models \varphi_{u_2}$
  $\Rightarrow \alpha \models L_{u_1}$ oder $\alpha \models L_{u_2}$
  $\Rightarrow (*)$

- **Fall B.2:** $\wedge$-Regel
  $\Rightarrow \varphi_v$ ist von der Form $\varphi_1 \wedge \varphi_2$
  $\Rightarrow \alpha \models \varphi_1$ und $\alpha \models \varphi_2$
  ☞ da $\alpha \models \varphi_v$
  $\Rightarrow$ Für das Kind u_1 und den Enkel u_2 von u gilt: $\alpha \models \varphi_{u_1}$ und $\alpha \models \varphi_{u_2}$
  $\Rightarrow \alpha \models L_{u_2}$
  $\Rightarrow (*)$

$\Rightarrow$ Der Tableau-Kalkül ist vollständig

Illustration von Fall B.1

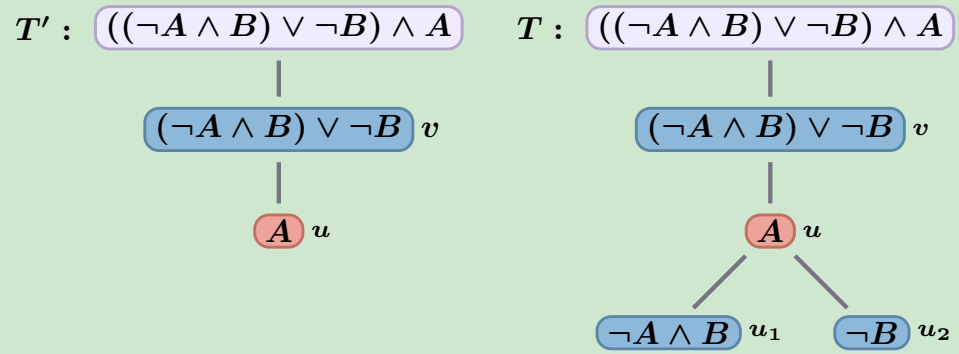
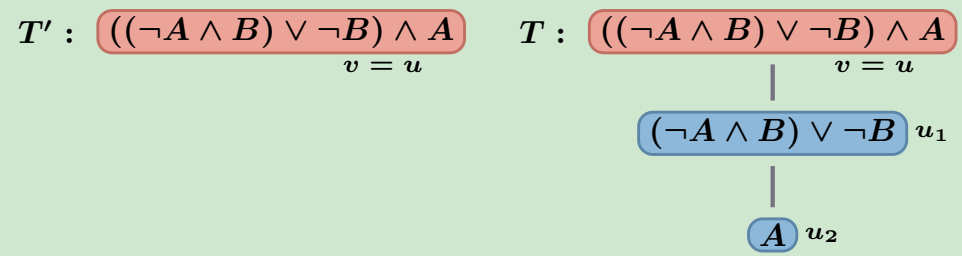


Illustration von Fall B.2



- Damit haben wir Korrektheit, Vollständigkeit, und Terminierung des aussagenlogischen Tableaukalküls nachgewiesen
- Der Beweis für den modallogischen Tableaukalkül ist analog

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Der **aussagenlogische Tableaukalkül**
    - ist eine weitere Methode zum Testen der Erfüllbarkeit aussagenlogischer Formeln
    - liefert für erfüllbare Formeln auch immer eine erfüllende Belegung
  - Der **modallogische Tableaukalkül**
    - ist eine Methode zum Testen der Erfüllbarkeit modallogischer Formeln
    - liefert für erfüllbare Formeln auch immer ein Modell; also eine Kripkestruktur $\mathcal{K}$ und eine Welt s von $\mathcal{K}$ in der die Formel gilt

Anmerkungen

Bemerkung B.2.1

- Widersprüchliche Pfade können geschlossen werden, sobald sich ein Widerspruch ergibt
  - Im Beispiel also zum Beispiel schon beim rechten Kind von $\neg B \vee A$

Kapitel B.3:

Bisimulationen und weitere Eigenschaften

Einleitung

Einleitung

- In diesem Kapitel beschäftigen wir uns mit
  - der Ähnlichkeit von Kripkestrukturen und Bisimulationen
  - der Struktur von Modellen modallogischer Formeln
  - der möglichen Bedeutung der Modaloperatoren

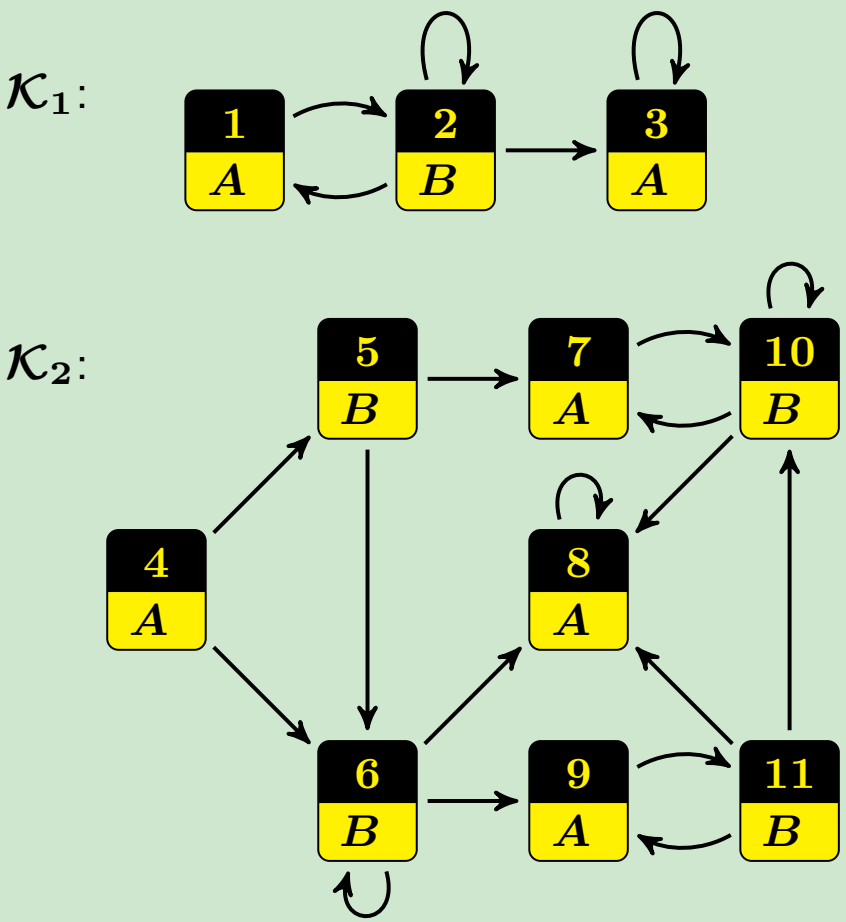
Inhalt

1. Verhalten von Welten: Das Bisimulationsspiel
2. Bisimulationen: Definition und Berechnung
3. Einfache Modelle
4. Bedeutung der Modaloperatoren
5. Anhang

Verhalten von Welten: Das Bisimulationsspiel

Ähnlichkeit in Kripkestrukturen (1/2)

Beispiel



- Kripkestrukturen werden bei der automatischen Verifikation zur Modellierung von Systemen verwendet

- In diesem Zusammenhang kann es interessant sein zu wissen, ob zwei Kripkestrukturen von gegebenen Welten aus das gleiche Systemverhalten haben
- So lässt sich möglicherweise für das **Model Checking** eine große Kripkestruktur durch eine kleinere ersetzen

  ➡ Zeit- und Platzersparnis

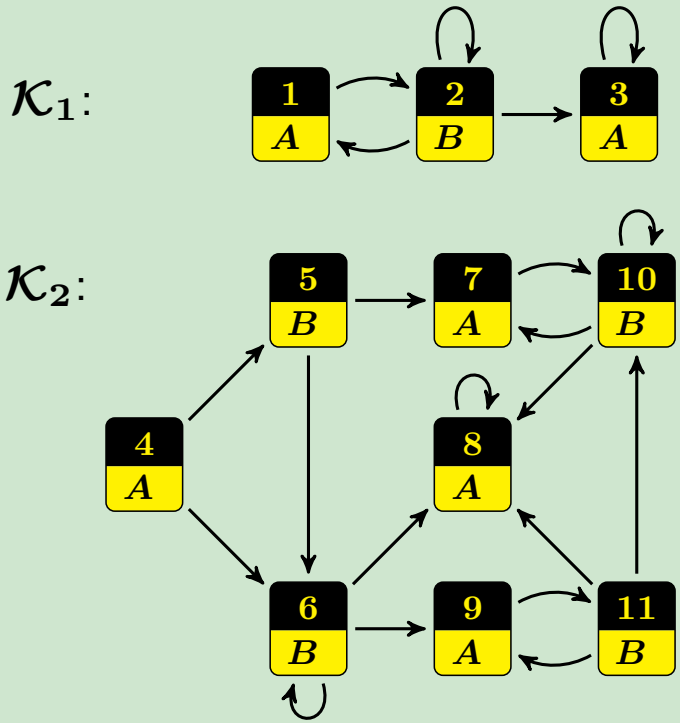
- Wir werden jetzt zunächst versuchen, den Begriff „das gleiche Systemverhalten aufweisen“ genauer zu fassen

- **Vereinbarung:**
  - Wir betrachten in diesem Abschnitt nur Kripkestrukturen, in denen aus jeder Welt mindestens eine Kante führt

  ➡ Das vereinfacht die Darstellung, die Resultate gelten aber auch allgemeiner

Ähnlichkeit in Kripkestrukturen (2/2)

Beispiel



- Wann ist das Systemverhalten zweier Welten in Kripkestrukturen gleich?

- Modallogische Formeln beziehen sich nur auf die jeweils geltenden Propositionen
  - ➡ Betrachte nicht die „Namen“, sondern nur Wahrheitsbelegungen von Welten

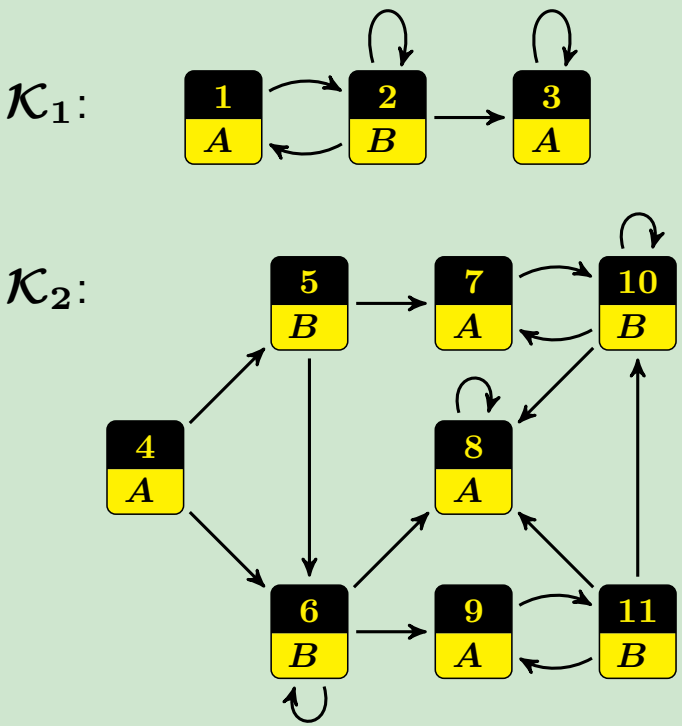
- Zwei Welten werden wir als äquivalent ansehen, wenn sie dieselbe Wahrheitsbelegung und die gleichen „Möglichkeiten für weiteres Verhalten“ haben

- Im Beispiel:
  - Von $\mathcal{K}_1, 3$ und $\mathcal{K}_2, 8$ aus ist das Systemverhalten offensichtlich äquivalent
  - Wie ist es von $\mathcal{K}_1, 2$ und $\mathcal{K}_2, 10$ aus?
  - Und von $\mathcal{K}_1, 2$ und $\mathcal{K}_2, 6$?
  - Und von $\mathcal{K}_1, 1$ und $\mathcal{K}_2, 4$?

- Wir formalisieren „Möglichkeiten für weiteres Verhalten“ durch ein 2-Personen-Spiel:
  - Spieler 1 versucht Unterschiede im Systemverhalten nachzuweisen
  - Spieler 2 versucht dies zu verhindern
- Eine Gewinnstrategie für Spieler 2 soll genau dann existieren, wenn die möglichen Verhaltensweisen der beiden Strukturen gleich sind

Das Bisimulationsspiel (1/2)

Beispiel



Definition: Bisimulationsspiel

- Seien $\mathcal{K}_1 = (V_1, E_1, P_1)$, $\mathcal{K}_2 = (V_2, E_2, P_2)$ Kripke-Strukturen

- Eine **Spielposition** ist ein Paar (s_1, s_2) von **Spielsteinen** mit $s_1 \in V_1$, $s_2 \in V_2$

- Eine **Spielrunde** verläuft, ausgehend von einer Spielsituation (s_1, s_2) wie folgt:
  - Spieler 1 (Anton) wählt einen der beiden Spielsteine und verschiebt ihn entlang einer Kante
  - Spieler 2 (Vera) verschiebt danach den anderen Stein entlang einer Kante
  - Liegen die Spielsteine danach auf Welten t_1 und t_2, so ist (t_1, t_2) die neue Spielposition

- **Anton gewinnt**, wenn in der aktuellen Spielposition (t_1, t_2) gilt: $P_1(t_1) \neq P_2(t_2)$
- **Vera gewinnt**, wenn Anton nicht mehr ziehen kann oder Anton (unendlich lange) nicht gewinnt

Das Bisimulationsspiel (2/2)

Beispiel

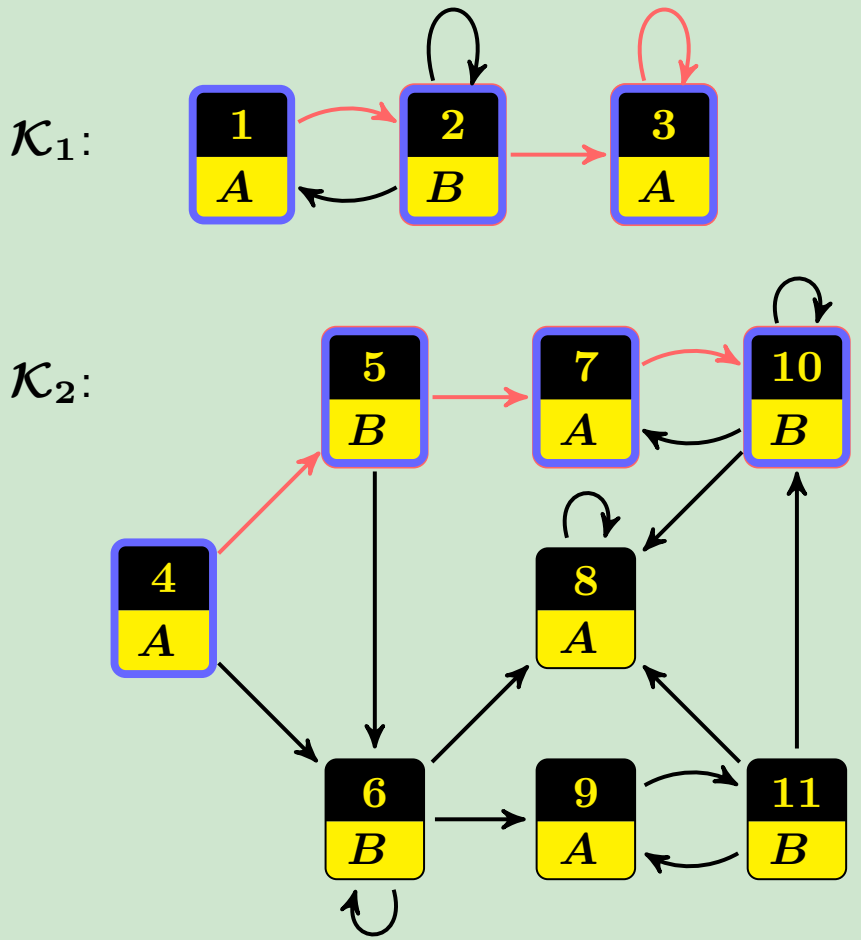
- Ein möglicher Spielverlauf von $(1, 4)$ aus:

- **Runde 1:**
  - Anton zieht von 4 nach 5
  - Vera zieht von 1 nach 2
  - Neue Position: $(2, 5)$

- **Runde 2:**
  - Anton zieht von 2 nach 3
  - Vera zieht von 5 nach 7
  - Neue Position: $(3, 7)$

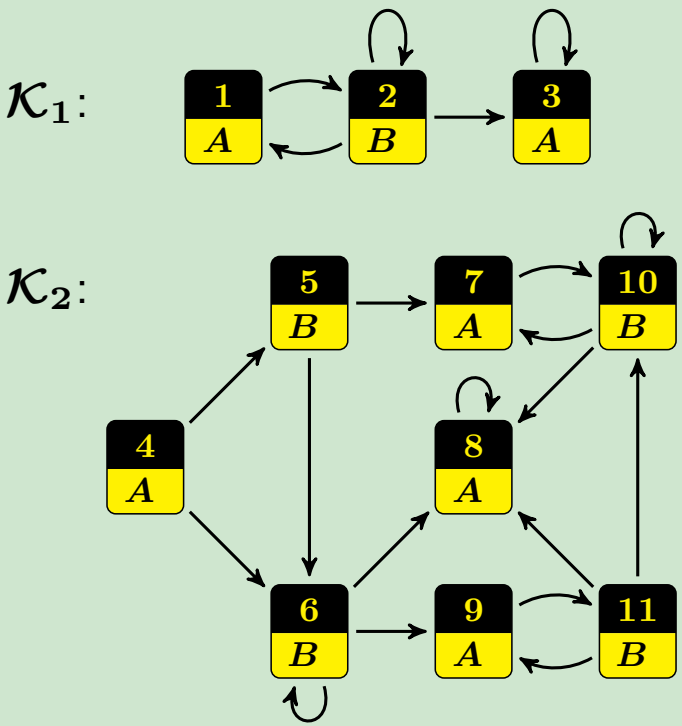
- **Runde 3:**
  - Anton zieht von 3 nach 3
  - Vera zieht von 7 nach 10
  - Neue Position: $(3, 10)$
  - $P_1(3) = \{A\} \neq \{B\} = P_2(10)$

$\Rightarrow$ Anton gewinnt



Gewinnpositionen für Vera

Beispiel



- Wir nennen eine Position (s_1, s_2) eine **Gewinnposition für Vera**, wenn Vera von (s_1, s_2) aus eine Gewinnstrategie hat
  - also, wenn Vera von (s_1, s_2) aus gewinnen kann, egal wie Anton spielt

Definition: $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$

- Seien $\mathcal{K}_1, \mathcal{K}_2$ Kripkestrukturen
- Die Menge aller Gewinnpositionen (s_1, s_2) für Vera im Bisimulationsspiel zu $\mathcal{K}_1$ und $\mathcal{K}_2$ bezeichnen wir mit $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$

- Falls $\mathcal{K}_1$ und $\mathcal{K}_2$ durch den Kontext klar sind, schreiben wir auch einfach $S_\sim$ statt $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$

- Wir werden $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ später die **maximale Bisimulation zu $\mathcal{K}_1$ und $\mathcal{K}_2$** nennen

- Warum ist $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ interessant?

Gewinnpositionen und Modallogik (1/2)

- $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ ist unter anderem aufgrund des folgenden Satzes interessant
- Er besagt, dass für Paare (s_1, s_2) von Welten aus $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ genau die selben modallogischen Formeln gelten

Satz B.3.1

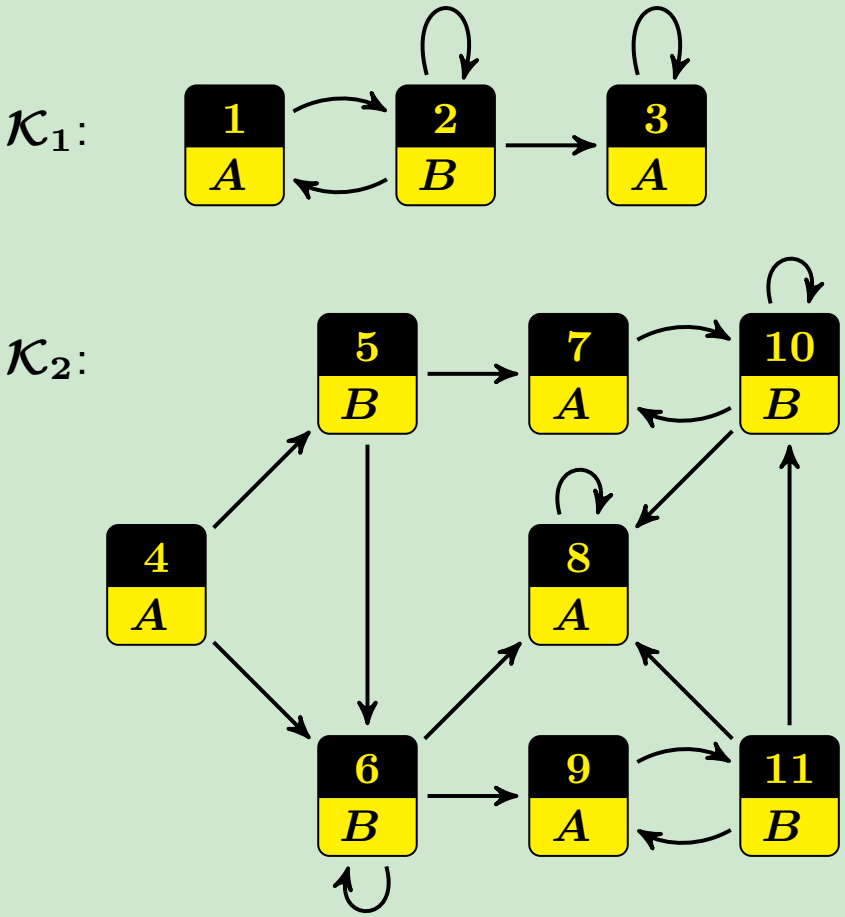
- *Seien $\mathcal{K}_1, \mathcal{K}_2$ Kripkestrukturen mit Welten s_1, s_2*

- *Wenn $(s_1, s_2) \in S_\sim(\mathcal{K}_1, \mathcal{K}_2)$, dann gilt für alle modallogischen Formeln φ:*

$$\mathcal{K}_1, s_1 \models \varphi \iff \mathcal{K}_2, s_2 \models \varphi$$

- Falls $(s_1, s_2) \in S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ gilt, kann also getestet werden, ob eine modallogische Formel in $\mathcal{K}_1, s_1$ wahr ist, indem sie in $\mathcal{K}_2, s_2$ ausgewertet wird
- Wenn $\mathcal{K}_2$ kleiner als $\mathcal{K}_1$ ist, kann das einen Effizienzgewinn ergeben

Beispiel



- $(1, 4) \notin S_\sim(\mathcal{K}_1, \mathcal{K}_2)$, denn:
  - $\mathcal{K}_1, 1 \models \Box\Diamond(A \wedge \Diamond A)$
  - $\mathcal{K}_2, 4 \not\models \Box\Diamond(A \wedge \Diamond A)$

Gewinnpositionen und Modallogik (2/2)

Satz B.3.1 (Wiederholung)

- Seien $\mathcal{K}_1, \mathcal{K}_2$ Kripkestrukturen mit Welten s_1, s_2

- Wenn $(s_1, s_2) \in S_\sim(\mathcal{K}_1, \mathcal{K}_2)$, dann gilt für alle modallogischen Formeln φ:

$$\mathcal{K}_1, s_1 \models \varphi \iff \mathcal{K}_2, s_2 \models \varphi$$

Beweisskizze

- Es gelte: $(s_1, s_2) \in S_\sim(\mathcal{K}_1, \mathcal{K}_2)$
- **Zeige:** Für jede modallogische Formel φ gilt:

$$\mathcal{K}_1, s_1 \models \varphi \Rightarrow \mathcal{K}_2, s_2 \models \varphi$$

- Umkehrung: analog

- **Ansatz:** Induktion nach der Struktur von φ

- $\varphi = X, \top, \bot$: ✓ ☞ da $P_1(s_1) = P_2(s_2)$
- φ ist Boolesche Kombination: ✓
  ☞ nach Induktion

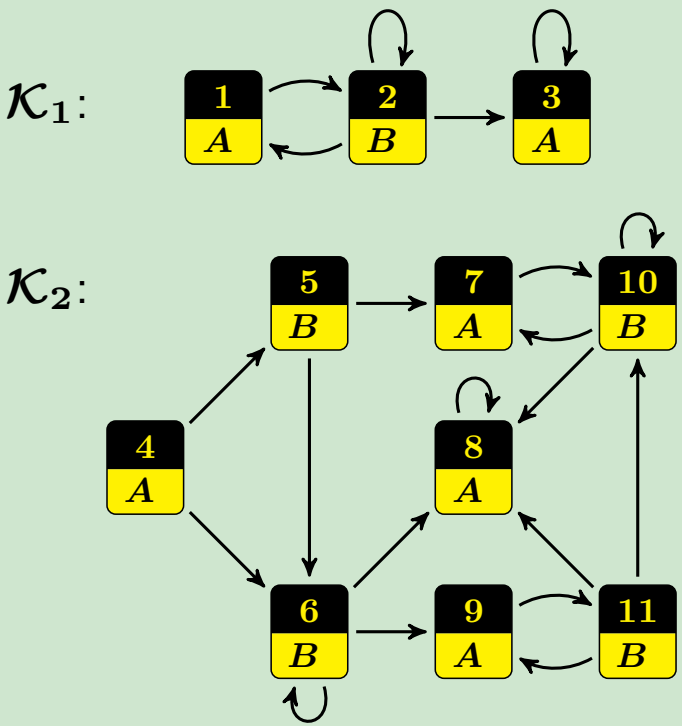
- Für $\varphi = \Diamond\psi$:
  - Es gelte $\mathcal{K}_1, s_1 \models \Diamond\psi$
    $\Rightarrow$ es gibt t_1 mit $(s_1, t_1) \in E_1$ und $\mathcal{K}_1, t_1 \models \psi$
  - Wenn Anton in der Position (s_1, s_2) von s_1 nach t_1 in $\mathcal{K}_1$ ziehen würde, könnte Vera gemäß ihrer Gewinnstrategie zu einem t_2 ziehen, und hätte von (t_1, t_2) aus wieder eine Gewinnstrategie
    $\Rightarrow$ es gibt ein t_2 mit $(s_2, t_2) \in E_2$ und $(t_1, t_2) \in S_\sim(\mathcal{K}_1, \mathcal{K}_2)$
  - Mit Induktion folgt: $\mathcal{K}_2, t_2 \models \psi$
    ☞ ψ Teilformel von φ
  - Es gibt also t_2 in $\mathcal{K}_2$ mit $(s_2, t_2) \in E_2$ und $\mathcal{K}_2, t_2 \models \psi$
    $\Rightarrow \mathcal{K}_2, s_2 \models \Diamond\psi$ ☞ Semantik von $\Diamond$

- $\varphi = \Box\psi \equiv \neg\Diamond\neg\psi$ lässt sich auf die anderen Fälle zurückführen

Bisimulationen: Definition und Berechnung

Bisimulationen (1/3)

Beispiel



- Wie lässt sich herausfinden, ob eine Position (s_1, s_2) eine Gewinnposition für Vera ist?
  ➡ Wie lässt sich $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ berechnen?

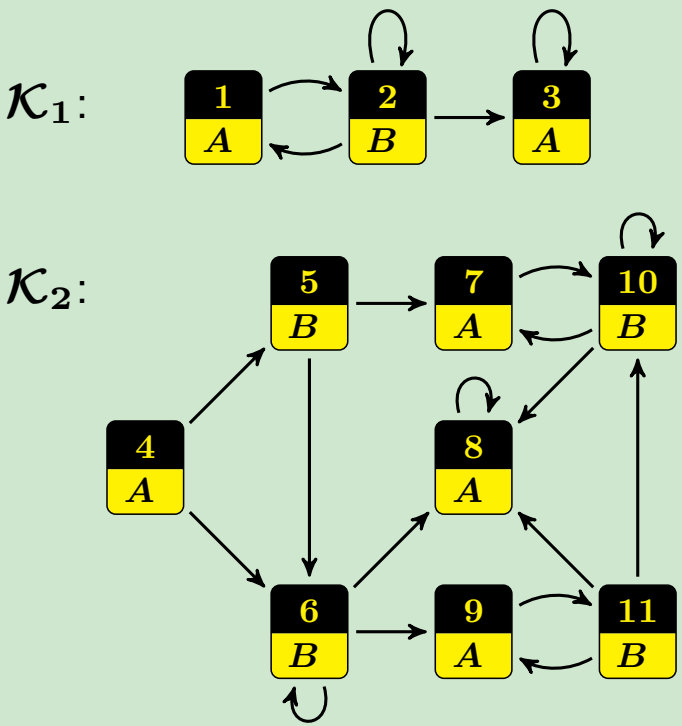
- **Naiver Ansatz:** Rekursion
  - $(s_1, s_2) \in S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ wenn $P_1(s_1) = P_2(s_2)$ und Vera für jede Zugmöglichkeit von Anton im nächsten Zug wieder eine Gewinnposition (t_1, t_2) erreichen kann

- Bevor wir einen Algorithmus zur Berechnung von $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ betrachten, werden wir $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ zuerst besser kennen lernen

- Dazu definieren wir als nächstes den Begriff der **Bisimulation**
- **Idee:** Eine Bisimulation ist eine Menge S von Gewinnpositionen für Vera, die sich „gegenseitig untermauern“
  - Das heißt: Vera kann von einer Gewinnposition in S immer wieder in eine Gewinnposition in S kommen

Bisimulationen (2/3)

Beispiel



- Die Relation

$$S = \{(1, 7), (2, 10), (3, 8)\}$$

ist eine Bisimulation zwischen $\mathcal{K}_1$ und $\mathcal{K}_2$

- Denn es gilt für alle $(s_1, s_2) \in S$:
  - $A \in P_1(s_1) \iff A \in P_2(s_2)$
  - $B \in P_1(s_1) \iff B \in P_2(s_2)$
  - Für alle t_1 mit $(s_1, t_1) \in E$ gibt es t_2 mit $(s_2, t_2) \in E_2$ und $(t_1, t_2) \in S$, und
  - Für alle t_2 mit $(s_2, t_2) \in E_2$ gibt es t_1 mit $(s_1, t_1) \in E$ und $(t_1, t_2) \in S$

Bisimulationen (3/3)

Definition: Bisimulation

- Seien $\mathcal{K}_1 = (V_1, E_1, P_1)$ und $\mathcal{K}_2 = (V_2, E_2, P_2)$ Kripkestrukturen

- Eine Relation $S \subseteq V_1 \times V_2$ heißt **Bisimulation** zwischen $\mathcal{K}_1$ und $\mathcal{K}_2$, falls für alle $(s_1, s_2) \in S$ gilt:

(Atom) $P_1(s_1) = P_2(s_2)$

(Hin) für alle t_1 mit $(s_1, t_1) \in E_1$ gibt es t_2 mit $(s_2, t_2) \in E_2$ und $(t_1, t_2) \in S$, und

(Her) für alle t_2 mit $(s_2, t_2) \in E_2$ gibt es t_1 mit $(s_1, t_1) \in E_1$ und $(t_1, t_2) \in S$

- Wir werden jetzt sehen, dass $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ die Bezeichnung **maximale Bisimulation** wirklich „verdient“

Lemma B.3.2

- *$S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ ist eine Bisimulation*

Beweisidee

- Bedingung (Atom) ist für Gewinnpositionen erfüllt
- (Hin) und (Her) gelten, da Vera im Bisimulationsspiel in jedem Zug wieder eine Position in $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ erzwingen kann

Lemma B.3.3

- *Ist S eine Bisimulation zwischen $\mathcal{K}_1$ und $\mathcal{K}_2$, so gilt $S \subseteq S_\sim(\mathcal{K}_1, \mathcal{K}_2)$*

Beweisidee

- Die Bisimulationseigenschaft garantiert, dass Vera so spielen kann, dass immer nur Positionen aus S erreicht werden

  $\Rightarrow$ Vera hat eine Gewinnstrategie auf allen Positionen von S

  $\Rightarrow S \subseteq S_\sim(\mathcal{K}_1, \mathcal{K}_2)$

Gewinnpositionen: Wrap-up

- Wir wissen also jetzt, dass die Menge $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ der Gewinnpositionen die eindeutig bestimmte maximale Bisimulation ist, die alle Bisimulationen zwischen $\mathcal{K}_1$ und $\mathcal{K}_2$ umfasst

- Wenn $(s_1, s_2) \in S_\sim(\mathcal{K}_1, \mathcal{K}_2)$ gilt,
  - schreiben wir auch: $(\mathcal{K}_1, s_1) \sim (\mathcal{K}_2, s_2)$
  - und sagen auch: s_1 und s_2 sind **bisimilar**

- Wenn $\mathcal{K}_1$ und $\mathcal{K}_2$ durch den Kontext klar sind, schreiben wir auch einfach $s_1 \sim s_2$

Bisimulationen: Berechnung (1/4)

- Der folgende Algorithmus berechnet für zwei gegebene endliche Kripkestrukturen $\mathcal{K}_1, \mathcal{K}_2$ die maximale Bisimulation $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$

- Er basiert auf einer sehr einfachen Grundidee:
  - Berechne induktiv die Menge N aller Paare (s_1, s_2) von Welten, die **nicht bisimilar** sind

- Die Menge N wird mit allen Paaren (s_1, s_2) initialisiert, die verschiedene Wahrheitsbelegungen haben

  ☞ Diese sind nicht bisimilar

- Danach werden die Transitionen betrachtet und es wird systematisch nach Gründen gesucht, warum ein Paar (s_1, s_2) **nicht** bisimilar sein könnte
  - Solche Paare werden zu N hinzugefügt

✎ Der Algorithmus ist sehr ähnlich zum Minimierungsalgorithmus für deterministische endliche Automaten in der Veranstaltung **Grundbegriffe der Theoretischen Informatik**

Bisimulationen: Berechnung (2/4)

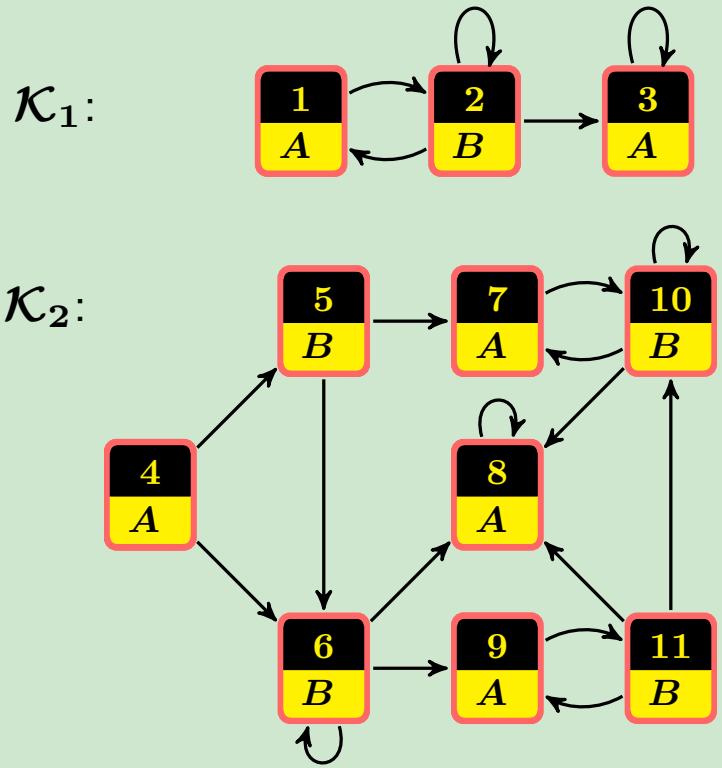
Bisimulationsalgorithmus

Eingabe: $\mathcal{K}_1 = (V_1, E_1, P_1)$, $\mathcal{K}_2 = (V_2, E_2, P_2)$, endlich
Ausgabe: $S = S_\sim(\mathcal{K}_1, \mathcal{K}_2)$

1: $N := \{(s_1, s_2) \in V_1 \times V_2 \mid P_1(s_1) \neq P_2(s_2)\}$
2: **repeat**
3: $\quad N' := \emptyset$
4: $\quad$ **for** (s_1, s_2) **in** $(V_1 \times V_2) - N$ **do**
5: $\quad\quad$ **if** es gibt $t_1 \in V_1$, so dass $(s_1, t_1) \in E_1$ und
für alle $t_2 \in V_2$ mit $(s_2, t_2) \in E_2$ gilt $(t_1, t_2) \in N$
then
6: $\quad\quad\quad N' := N' \cup \{(s_1, s_2)\}$
7: $\quad\quad$ **end if**
8: $\quad\quad$ **if** es gibt $t_2 \in V_2$, so dass $(s_2, t_2) \in E_2$ und
für alle $t_1 \in V_1$ mit $(s_1, t_1) \in E_1$ gilt $(t_1, t_2) \in N$
then
9: $\quad\quad\quad N' := N' \cup \{(s_1, s_2)\}$
10: $\quad\quad$ **end if**
11: $\quad$ **end for**
12: $\quad N := N \cup N'$
13: **until** $N' = \emptyset$
14: **return** $S := (V_1 \times V_2) - N$

Bisimulationen: Berechnung (3/4)

Beispiel



	4	5	6	7	8	9	10	11
1	3 $t_2{=}5$	0	0		1 $t_1{=}2$		0	0
2	0	2 $t_1{=}3$		0	0	0		
3	1 $t_2{=}5$	0	0	1 $t_2{=}10$		1 $t_2{=}11$	0	0

- **Runde 0:**
  - Die Initialisierung „markiert“ 12 Paare ☞ bewegt sie nach N
- **Runde 1:**
  - $(1, 8)$ wird markiert mit $t_1 = 2$
  - $(3, 4)$ wird markiert mit $t_2 = 5$
  - $(3, 7)$ wird markiert mit $t_2 = 10$
  - $(3, 9)$ wird markiert mit $t_2 = 11$
- **Runde 2:**
  - $(2, 5)$ wird markiert mit $t_1 = 3$
- **Runde 3:**
  - $(1, 4)$ wird markiert mit $t_2 = 5$
- **Runde 4:**
  - Keine weiteren Markierungen

➡ $S_\sim(\mathcal{K}_1, \mathcal{K}_2) = \{(1,7), (1,9), (2,6), (2,10), (2,11), (3,8)\}$

Bisimulationen: Berechnung (4/4)

Satz B.3.4

- *Der Bisimulationsalgorithmus berechnet* $S_\sim(\mathcal{K}_1, \mathcal{K}_2)$

Beweisidee

- Durch Induktion nach der Rundenzahl lässt sich leicht zeigen: N enthält immer **nur** nicht bisimilare Paare (s_1, s_2)

- Umgekehrt lässt sich auch zeigen: N enthält irgendwann **alle** nicht bisimilaren Paare (s_1, s_2)
- Dazu kann gezeigt werden:
  - Von jeder Spielposition (s_1, s_2) aus hat genau einer der beiden Spieler eine Gewinnstrategie
  - Wenn Anton von (s_1, s_2) aus eine Gewinnstrategie hat, gibt es eine Zahl k, so dass Anton nach höchstens k Runden gewinnt, egal was Vera macht
  - Dann wird (s_1, s_2) spätestens im k-ten Durchlauf der REPEAT-Schleife in N aufgenommen

☞ Induktion

- Ähnliche Überlegungen wie im Beweis von Satz B.3.4 führen zu der Erkenntnis, dass für endliche Strukturen auch die Umkehrung von Satz B.3.1 gilt

Satz B.3.5

- *Seien* $\mathcal{K}_1, \mathcal{K}_2$ *endliche Kripkestrukturen mit Welten* s_1, s_2
- *Wenn für alle modallogischen Formeln* φ *gilt*

$$\mathcal{K}_1, s_1 \models \varphi \iff \mathcal{K}_2, s_2 \models \varphi$$

dann gilt auch $s_1 \sim s_2$

Bisimulationen: Minimierung von Kripkestrukturen

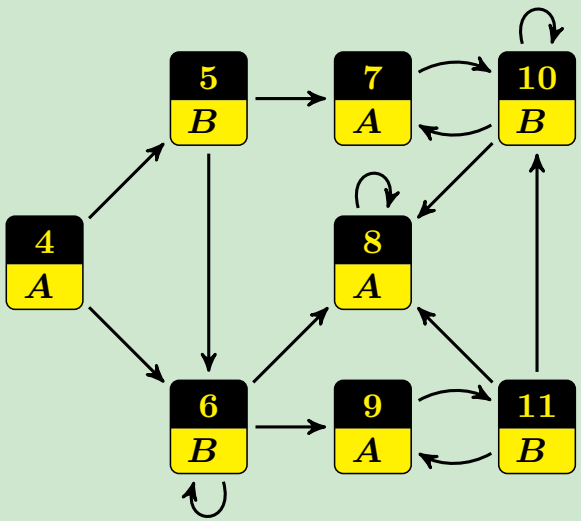
- Mit dem Bisimulationsalgorithmus lässt sich zu $\mathcal{K}, s$ ein $\mathcal{K}', s'$ finden, sodass
  - $s \sim s'$, und
  - die Kripkestruktur $\mathcal{K}'$ ist kleinstmöglich

➡ Modallogische Formeln können dann in $\mathcal{K}', s'$ statt $\mathcal{K}, s$ ausgewertet werden

- **Idee:** Wende den Bisimulationsalgorithmus auf $(\mathcal{K}, \mathcal{K})$ an
  ☞ Entferne zunächst nicht-erreichbare Welten

Beispiel

$\mathcal{K}$:



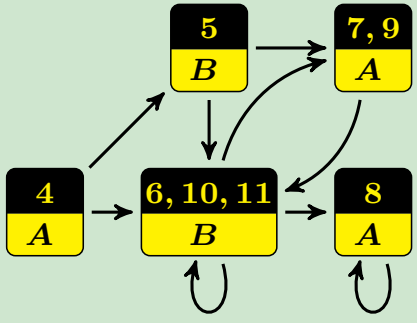
$s = 4$

	5	6	7	8	9	10	11
4	0	0	3 $t_1{=}5$	1 $t_1{=}5$	3 $t_1{=}5$	0	0
5	—	2 $t_2{=}8$	0	0	0	2 $t_2{=}8$	2 $t_2{=}8$
6	—	—	0	0	0		
7	—	—	—	1 $t_1{=}10$		0	0
8	—	—	—	—	1 $t_2{=}11$	0	0
9	—	—	—	—	—	0	0
10	—	—	—	—	—	—	

- Die Welten in $\{7, 9\}$ und $\{6, 10, 11\}$ können zusammengelegt werden, da sie sich gleich verhalten

  ➡ **Quotientenstruktur**

- Es ergibt sich die Quotientenstruktur $\mathcal{K}'$:



$s' = 4$

Einfache Modelle

Modallogik hat die Endliches-Modell-Eigenschaft

- Wir kennen aus dem letzten Kapitel bereits einen Erfüllbarkeitstest für modallogische Formeln:

  ➡ **Tableaukalkül**

- Wird der Tableaukalkül für eine erfüllbare, modallogische Formel φ angewendet, liefert er immer auch ein oder mehrere Modelle für φ
- Diese Modelle sind immer endlich

- Insbesondere gilt also:
  - Jede erfüllbare modallogische Formel hat ein endliches Modell
- Wir sagen auch: Modallogik hat die Endliches-Modell-Eigenschaft

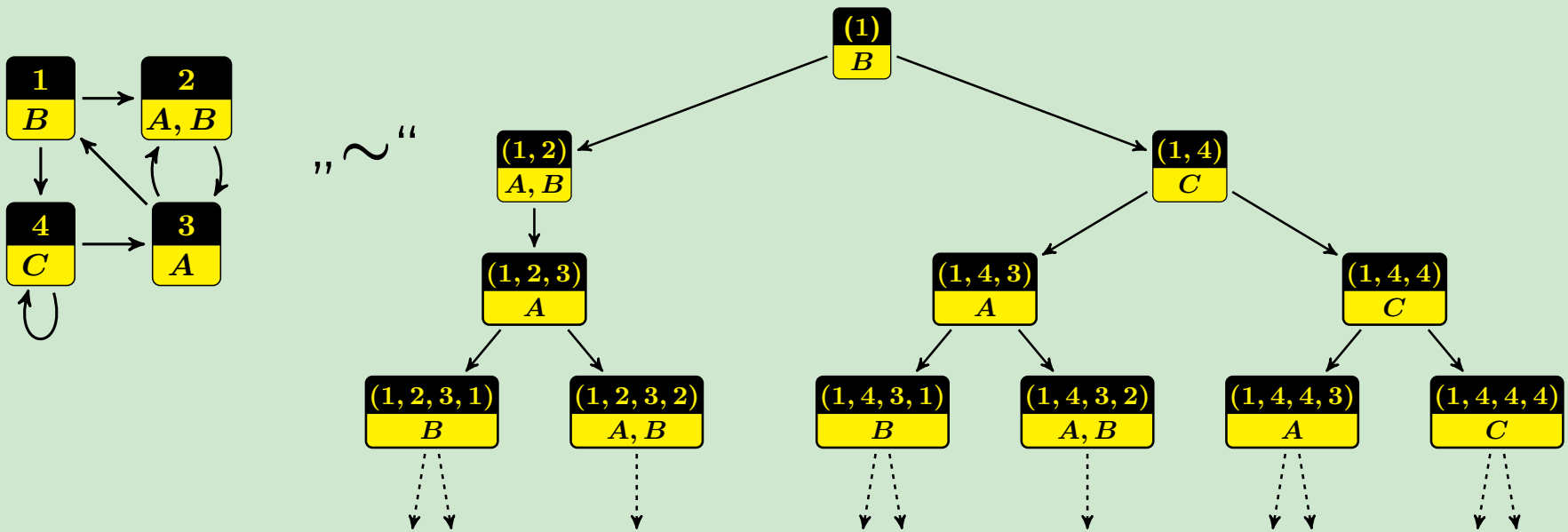
- Wir werden jetzt sehen, dass Modallogik sogar noch eine stärkere Eigenschaft hat

Modallogik hat die Baummodelleigenschaft (1/3)

- Wir nennen eine Kripkestruktur $\mathcal{K} = (V, E, P)$ einen **Baum**, wenn (V, E) ein gerichteter Baum mit einer Wurzel ist, in dem alle Kanten von der Wurzel weg orientiert sind
  - ➡ Wir erlauben hier auch unendliche Bäume!

- Wir zeigen jetzt:
  - Für modallogische Formeln „sieht jede Kripkestruktur wie ein Baum aus“
  - Genauer: Eine modallogische Formel kann nicht zwischen einer Welt und ihrer “Abwicklung” unterscheiden

Beispiel: Abwicklung einer Welt



Modallogik hat die Baummodelleigenschaft (2/3)

Weg, Abwicklung

- Ein (endlicher) **Weg** in einer Kripkestruktur $\mathcal{K} = (V, E, P)$ ist ein Tupel $(s_1, \ldots, s_k)$ von Welten aus V mit $(s_i, s_{i+1}) \in E$, für alle $i < k$

- Die **Abwicklung** $T_{\mathcal{K},s}$ von $\mathcal{K}$ von der Welt s aus ist die Kripkestruktur (V', E', P') mit:
  - $V' \stackrel{\text{def}}{=}$ Menge aller endlichen Wege von $\mathcal{K}$, die in s beginnen
  - $E' \stackrel{\text{def}}{=}$ Menge aller (w, w'), für die w' ein Weg ist, der w um einen Knoten verlängert
  - $P'((s_1, \ldots, s_k)) \stackrel{\text{def}}{=} P(s_k)$

Lemma B.3.6

- *Für jedes $\mathcal{K}$ und jedes s gilt:*

$$(\mathcal{K}, s) \sim (T_{\mathcal{K},s}, (s))$$

Beweisidee

- $\{(s_k, (s_1, \ldots, s_k)) \mid (s_1, \ldots, s_k)$ ist Weg von $\mathcal{K}$, $s_1 = s\}$ ist eine Bisimulation zwischen $\mathcal{K}$ und $T_{\mathcal{K},s}$

Folgerung B.3.7

- *Für jedes $\mathcal{K}$, jedes s und jede modallogische Formel φ gilt:*

$$\mathcal{K}, s \models \varphi \iff T_{\mathcal{K},s}, (s) \models \varphi$$

Modallogik hat die Baummodelleigenschaft (3/3)

- Es gilt sogar noch etwas mehr
- Sei $\mathbf{md}(\varphi)$ die Verschachtelungstiefe der modalen Operatoren $\Diamond$ und $\Box$ in φ

Beispiel

$\mathrm{md}((\Diamond A \vee \Box B) \wedge (\Diamond\Box(C \vee \Diamond A))) = 3$

- Für eine Kripkestruktur $\mathcal{K}$, eine Welt s und $k \geq 0$ sei $T^k_{\mathcal{K},s}$ der endliche Baum, der nur aus den oberen $k+1$ Ebenen von $T_{\mathcal{K},s}$ besteht

  ☞ Wege der Länge $\leq k$

Satz B.3.8

- *Für jede modallogische Formel φ, jede passende Kripkestruktur $\mathcal{K}$ und jede Welt s von $\mathcal{K}$ gilt:*

$$\mathcal{K}, s \models \varphi \iff T^{md(\varphi)}_{\mathcal{K},s}, (s) \models \varphi$$

Folgerung B.3.9

- *Modallogik hat die endliche Baummodelleigenschaft, d.h., für jede erfüllbare modallogische Formel φ gibt es eine Kripkestruktur $\mathcal{K}$ und eine Welt s mit:*
  - *$\mathcal{K}$ ist ein endlicher Baum, und*
  - *$\mathcal{K}, s \models \varphi$*

- Damit lassen sich Grenzen der Ausdrucksstärke der Modallogik zeigen

Beispiel

- In Modallogik kann nicht ausgedrückt werden: Eine Welt liegt in einem Kreis der Länge 3
- **Genauer:** Es gibt keine modallogische Formel φ für die äquivalent sind
  - $(\mathcal{K}, s_1) \models \varphi$
  - Es gibt $s_2, s_3 \in V$ mit $(s_1, s_2), (s_2, s_3), (s_3, s_1) \in E$

Baummodelleigenschaft: Abschließende Bemerkungen

- Wie lassen sich baumförmige Modelle finden?
  - **Ansatz 1:** Eine genaue Analyse des Tableaukalküls ergibt, dass dieser für erfüllbare Formeln auch immer baumartige Modelle liefert
  - **Ansatz 2:** Nach Satz B.3.8 genügt es alternativ auch, zu einer gegebenen Formel φ alle Bäume der Tiefe $\mathrm{md}(\varphi)$ zu überprüfen

- Beide Ansätze führen zu Algorithmen mit exponentieller Laufzeit

- Es lässt sich zeigen, dass beide Verfahren mit polynomiellem Speicherplatzaufwand implementiert werden können

- Es lässt sich weiter zeigen, dass das Erfüllbarkeitsproblem für Modallogik vollständig für die Komplexitätsklasse PSPACE ist
- Zur Erinnerung: Das Erfüllbarkeitsproblem für die Aussagenlogik ist vollständig für NP

➡ Das Erfüllbarkeitsproblem der Modallogik ist vermutlich schwieriger als das Erfüllbarkeitsproblem für die Aussagenlogik

- Was das genau bedeutet, wird in der Vorlesung **Komplexitätstheorie** erklärt

Bedeutung der Modaloperatoren

Modallogik: Mögliche Bedeutung der Operatoren

- Modallogik lässt sich in ziemlich unterschiedlichen Situationen verwenden
- In verschiedenen Zusammenhängen werden dabei die Operatoren $\Diamond$ und $\Box$ unterschiedlich mit Bedeutung versehen

- Einige Beispiele: ☞ aus [Huth, Ryan]

$\Diamond\varphi$	φ ist möglicherweise wahr
$\Box\varphi$	φ ist sicher wahr

$\Diamond\varphi$	φ ist erlaubt
$\Box\varphi$	φ muss gelten

$\Diamond\varphi$	Irgendwann in der Zukunft des Systems wird φ wahr sein
$\Box\varphi$	In der Zukunft des Systems wird φ immer wahr sein

$\Diamond\varphi$	Das System hat einen möglichen nächsten Schritt, nach dem φ gilt
$\Box\varphi$	Nach dem nächsten Schritt des Systems gilt φ

Bedeutungen, Tautologien, Kripke-Strukturen (1/3)

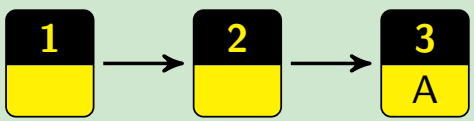
Beispiel

- Wir betrachten die folgende mögliche Bedeutung von $\Diamond\varphi$ und $\Box\varphi$ etwas näher:

$\Diamond\varphi$	Irgendwann in der Zukunft des Systems wird φ wahr sein
$\Box\varphi$	In der Zukunft des Systems wird φ immer wahr sein

- In einer Kripkestruktur soll eine Kante von einer Welt x zu einer Welt y dann bedeuten, dass Welt y in der Zukunft von Welt x liegt

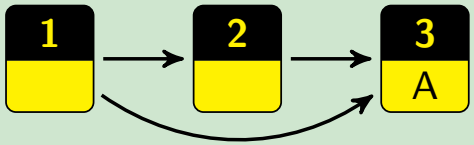
- Dann darf also die folgende Situation nicht in einer Kripkestruktur auftreten:



- Welt 3 liegt in der Zukunft von Welt 2, und Welt 2 in der Zukunft von Welt 1

  $\Rightarrow$ Welt 3 müsste auch in der Zukunft von Welt 1 liegen

- Es müsste also noch eine weitere Kante vorkommen:



- Wenn die modallogischen Operatoren mit der „Zukunftsbedeutung“ versehen werden, sind also nur Kripkestrukturen mit einer transitiven Kantenrelation sinnvoll

- Dadurch ergibt sich auch, dass manche Formeln, die für beliebige Kripkestrukturen nicht allgemeingültig sind, für die „Zukunftsbedeutung“ immer gelten, z.B.: $\varphi_1 = \Box\psi \rightarrow \Box\Box\psi$
- Die folgende Formel muss aber bei „Zukunftsbedeutung“ nicht immer gelten: $\varphi_2 = \Box\psi \rightarrow \Diamond\psi$

Bedeutungen, Tautologien, Kripke-Strukturen (2/3)

- Neben den allgemeingültigen modallogischen Formeln gibt es also je nach der Bedeutung, die mit $\Diamond$ und $\Box$ verbunden wird, möglicherweise weitere Formeln, die immer gelten

- Für die beiden schon betrachteten Beispielformeln
  - $\varphi_1 = \Box\psi \rightarrow \Box\Box\psi$
  - $\varphi_2 = \Box\psi \rightarrow \Diamond\psi$

  zeigt die folgende Tabelle, wie ihre „Tautologie-Eigenschaft“ von der intendierten Bedeutung der Operatoren abhängt:

$\Diamond\varphi$	$\Box\varphi$	Es gilt immer:
Das System hat einen möglichen nächsten Schritt, nach dem φ gilt	Nach dem nächsten Schritt des Systems gilt φ	—
Irgendwann in der Zukunft des Systems wird φ wahr sein	φ wird zukünftig immer wahr sein	φ_1
φ ist erlaubt	φ muss gelten	φ_2
φ ist möglicherweise wahr	φ ist sicher wahr	φ_1, φ_2

- In manchen Fällen lässt sich sicher darüber streiten, welche Formeln immer gelten müssen... ☞ siehe z.B. [Huth, Ryan]

Bedeutungen, Tautologien, Kripke-Strukturen (3/3)

- Der Zusammenhang zwischen graphentheoretischen Eigenschaften von Kripkestrukturen und Formeln, die in diesen Strukturen immer gelten, wird in der **Korrespondenztheorie** untersucht
- Sie verwendet „Rahmen“: Kripke-Strukturen ohne Belegung der aussagenlogischen Variablen

Rahmen

- Ein **Rahmen** $\mathcal{R} = (V, E)$ besteht aus
  - einer Grundmenge V von **Welten**, und
  - einer 2-stelligen **Übergangsrelation** $E \subseteq V \times V$

- Für einen Rahmen $\mathcal{R} = (V, E)$ und eine modallogische Formel φ definieren wir:
  - $\mathcal{R} \models \varphi \overset{\text{def}}{\Leftrightarrow}$ für jede Kripkestruktur $\mathcal{K} = (V, E, P)$ und jede Welt s von $\mathcal{K}$ gilt $\mathcal{K}, s \models \varphi$

Satz B.3.10

(a) *Die folgenden Aussagen sind für einen Rahmen $\mathcal{R} = (V, E)$ äquivalent:*

- E *ist transitiv*
- $\mathcal{R} \models \Box\varphi \to \Box\Box\varphi$, *für alle* $\varphi \in \mathrm{ML}$
- $\mathcal{R} \models \Box A \to \Box\Box A$

(b) *Die folgenden Aussagen sind für einen Rahmen $\mathcal{R} = (V, E)$ äquivalent:*

- E *ist reflexiv*
- $\mathcal{R} \models \Box\varphi \to \varphi$, *für alle* $\varphi \in \mathrm{ML}$
- $\mathcal{R} \models \Box A \to A$

(c) *Die folgenden Aussagen sind für einen Rahmen $\mathcal{R} = (V, E)$ äquivalent:*

- *Jede Welt hat eine ausgehende Kante*
- $\mathcal{R} \models \Box\varphi \to \Diamond\varphi$, *für alle* $\varphi \in \mathrm{ML}$
- $\mathcal{R} \models \Box A \to \Diamond A$

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**

- Bisimulationen stellen eine Beziehung zwischen Welten in Kripkestrukturen her, die sich bezüglich des Systemverhaltens nicht wesentlich unterscheiden

- Modallogik kann zwischen bisimilaren Welten nicht unterscheiden
  - Das eröffnet die prinzipielle Möglichkeit, Formeln in kleineren, äquivalenten Strukturen auszuwerten
  - Und erfüllbare modallogische Formeln haben immer sehr einfach strukturierte Modelle: endliche Bäume
- Bisimulationen lassen sich effizient berechnen

- Mit den modallogischen Operatoren lassen sich verschiedene Bedeutungen verbinden

- Je nach intendierter Bedeutung ergeben sich
  - verschiedene „anwendungsspezifische Tautologien“ und
  - Kripkestrukturen mit spezifischen graphentheoretischen Eigenschaften

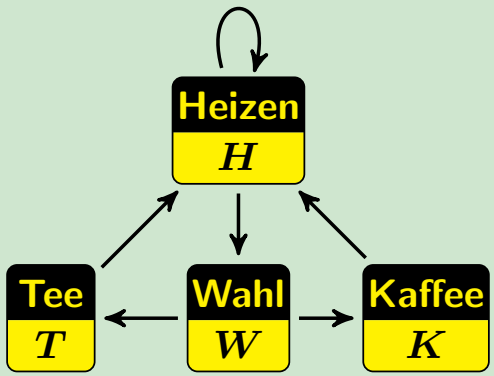
Anhang

Exkurs: Bisimulationen vs. „lineare Läufe“

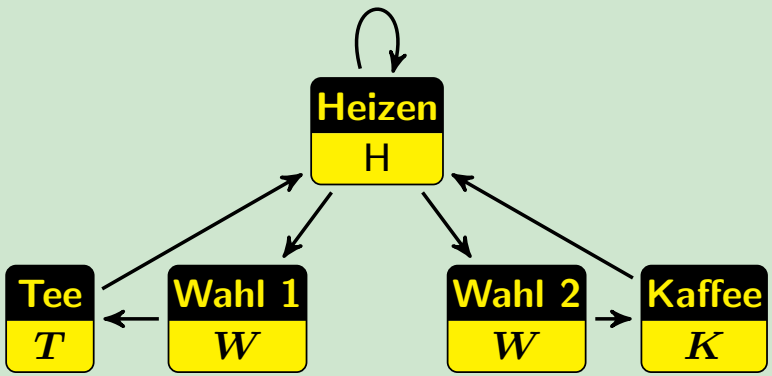
Beispiel

- Die beiden folgenden Kripkestrukturen, modellieren die Verhaltensmöglichkeiten zweier Kaffeeautomaten:

$\mathcal{K}_1$:



$\mathcal{K}_2$:



- In beiden Strukturen gibt es dieselben Folgen von Propositionsmengen, die durchlaufen werden können
  - alle Folgen, die durch Iterationen von Konkatenationen von $\{H\}^+, \{W\}, \{T\}$ und $\{H\}^+, \{W\}, \{C\}$ entstehen
    - $\{H\}^+$ steht dabei für beliebig viele, aber mindestens ein Auftreten von $\{H\}$

- Es gilt aber: $\mathcal{K}_1, \text{Heizen} \not\sim \mathcal{K}_2, \text{Heizen}$, denn:
  - $\mathcal{K}_1, \text{Heizen} \models \Diamond(\Diamond T \wedge \Diamond C)$
  - $\mathcal{K}_2, \text{Heizen} \not\models \Diamond(\Diamond T \wedge \Diamond C)$
- Intuitive Erklärung: $\mathcal{K}_1$ ermöglicht eine spätere Verzweigungsmöglichkeit in Welten mit $\{T\}$ und $\{C\}$ als $\mathcal{K}_2$

- Mit Hilfe der Bisimulation lassen sich also feinere Unterschiede zwischen Kripkestrukturen erkennen als durch die Betrachtung von „Läufen“

Kapitel C.1:

Grundlagen der Prädikatenlogik

Einleitende Beispiele

Einleitendes Beispiel: Mord in Schloss Dreadsbury (1/4)

Beispiel

- Ein Arzt, ein Butler und ein Chauffeur leben in Schloss Dreadsbury und sie sind die einzigen Personen, die dort wohnen. Ein Bewohner von Schloss Dreadsbury hat den dortigen Arzt ermordet.
- Wir wissen: Ein Mörder hasst sein Opfer immer, ist aber niemals reicher als sein Opfer. Der Chauffeur hasst niemanden, den der Arzt hasst. Der Arzt hasst jeden, der nicht der Butler ist. Der Butler hasst jeden, der nicht reicher als der Arzt ist. Der Butler hasst jeden, den der Arzt hasst. Niemand hasst jeden. Der Arzt ist nicht auch Butler.

- Wer ist der Mörder?

- Wir werden das Rätsel heute nicht lösen
- Aber wir werden die obigen Zusammenhänge in logische Formeln übersetzen

- Das ginge im Prinzip auch mit der Aussagenlogik...
- ... **aber:** Prädikatenlogik ermöglicht eine elegantere Übersetzung der „Beziehungen“ zwischen den Personen

Einleitendes Beispiel: Mord in Schloss Dreadsbury (2/4)

- „Ein Arzt, ein Butler und ein Chauffeur leben in Schloss Dreadsbury“:

$$D(a) \wedge D(b) \wedge D(c)$$

- „sie sind die einzigen Personen, die dort wohnen“:

$$\forall x \Big(D(x) \rightarrow \big(x = a \vee x = b \vee x = c \big) \Big)$$

- „Ein Bewohner von Schloss Dreadsbury hat den dortigen Arzt ermordet“:

$$\exists x \Big(D(x) \wedge K(x, a) \Big)$$

- „Ein Mörder hasst sein Opfer immer“:

$$\forall x \forall y \Big(K(x, y) \rightarrow H(x, y) \Big)$$

- „Ein Mörder . . . ist aber niemals reicher als sein Opfer“:

$$\forall x \forall y \Big(K(x, y) \rightarrow \neg R(x, y) \Big)$$

- **Abkürzungen:**
  - Konstanten:
    - a steht für den Arzt
    - b steht für den Butler
    - c steht für den Chauffeur
  - Prädikate:
    - $D(x)$ steht für „x ist Bewohner von Schloss Dreadsbury“
    - $K(x, y)$ steht für „x hat y ermordet“
    - $H(x, y)$ steht für „x hasst y“
    - $R(x, y)$ steht für „x ist reicher als y“

Einleitendes Beispiel: Mord in Schloss Dreadsbury (3/4)

- „Der Chauffeur hasst niemanden, den der Arzt hasst“:

  $$\forall x \Big(H(a,x) \rightarrow \neg H(c,x) \Big)$$

- „Der Arzt hasst jeden, der nicht der Butler ist“:

  $$\forall x \Big(\neg (x = b) \rightarrow H(a,x) \Big)$$

- „Der Butler hasst jeden, der nicht reicher als der Arzt ist“:

  $$\forall x \Big(\neg R(x,a) \rightarrow H(b,x) \Big)$$

- „Der Butler hasst jeden, den der Arzt hasst“:

  $$\forall x \Big(H(a,x) \rightarrow H(b,x) \Big)$$

- „Niemand hasst jeden“:

  $$\neg \exists x \forall y H(x,y)$$

- „Der Arzt ist nicht auch Butler“: $\neg(a = b)$

- **Abkürzungen:**
  - Konstanten:
    - a steht für den Arzt
    - b steht für den Butler
    - c steht für den Chauffeur
  - Prädikate:
    - $D(x)$ steht für „x ist Bewohner von Schloss Dreadsbury“
    - $K(x,y)$ steht für „x hat y ermordet“
    - $H(x,y)$ steht für „x hasst y“
    - $R(x,y)$ steht für „x ist reicher als y“

Einleitendes Beispiel: Mord in Schloss Dreadsbury (4/4)

- **Frage:** Was soll eine Formel

$$\forall x \forall y \Big(K(x,y) \rightarrow H(x,y) \Big)$$

„bedeuten“ und unter welchen Umständen lässt sie sich auswerten?

- Wir benötigen:
  - Eine **Grundmenge**, die festlegt, welche Werte die Variablen annehmen können
  - Konkrete **Relationen**, die angeben, welche Elemente der Grundmenge in K- und H-Beziehung stehen

Beispiel

- Eine Grundmenge:

$$\{\text{Agatha}, \text{Ben}, \text{Chris}, \text{Diana}\}$$

- Mögliche Beziehungen:

D
Agatha
Ben
Chris

K	
Agatha	Ben
Chris	Diana

H	
Agatha	Ben
Ben	Agatha
Chris	Diana

R	
Agatha	Ben
Ben	Chris

- Unter diesen Umständen sollte die Formel

$$\forall x \forall y \Big(K(x,y) \rightarrow H(x,y) \Big)$$

gelten

- Zusammenstellung einer Grundmenge und von Relationen über der Grundmenge: **Struktur**
- Formeln der Prädikatenlogik werden in Strukturen ausgewertet

☞ zumindest im Prinzip

- **Zunächst:** Verschiedenen Arten von Strukturen
- **Danach:** Formalisierung der Syntax und Semantik von Prädikatenlogik

Inhalt

1. Einleitende Beispiele
2. Strukturen
3. Syntax der Prädikatenlogik
4. Semantik der Prädikatenlogik

Strukturen

Repräsentation von Daten: Informatik vs. Mathematik

- Informatiker:innen müssen Daten/Informationen zur Verarbeitung durch Computer repräsentieren
- Verschiedene Zwecke erfordern verschiedene Repräsentationen

- Kodierungen von Daten:
  - . . . im Rechner: Bitfolgen
  - . . . in Dateien: Daten lassen sich als Strings/Textdateien repräsentieren

- Datenstrukturen:
  - Komfortablere Repräsentation von Daten, die geeigneter für die Manipulation „innerhalb von Programmen“ ist
  - Bspw. Arrays, Pointerstrukturen, etc.

- Datenbanken: „externe“ Speicherung von Informationen
- Daten im Web. . .

- **Ziel:** Logische Formeln sollen über Daten reden können, bspw.
  - ➡ Spezifikation von Daten
  - ➡ Anfragen an Daten
- **Dazu:** Modellierung von „Daten“ durch mathematische Objekte

- Die mathematischen Objekte, die wir zur Repräsentation von Daten verwenden, nennen wir **Strukturen**
- Strukturen bestehen aus
  - **Relationen**,
  - **Funktionen** und
  - **Konstanten**

Relationen, Funktionen und Konstanten

- **Zur Erinnerung:**
  - Eine n-**stellige Relation** über einer Grundmenge A ist eine Teilmenge von

$$A^n = \underbrace{A \times \cdots \times A}_{n \text{ Faktoren}}$$

Beispiel

- Strikte Ordnung der natürlichen Zahlen:
  - $A = \mathbb{N}$
  - $R = \{(m, n) \in \mathbb{N}^2 \mid m < n\}$
    $= \{(0,1), (0,2), (1,2), \ldots\}$

- Logik-Studierende:
  - $A =$ Menge aller Strings über $\{a, \ldots, z\}$,
  - $R = \{(x, y) \in A^2 \mid x$ ist Vorname, y ist Nachname einer Person der Logik-Vorlesung$\}$

- Wir notieren n-**stellige Funktionen** über einer Grundmenge A als $f : A^n \to A$

Beispiel

- Addition über den natürlichen Zahlen:
  - $f_+ : \mathbb{N}^2 \to \mathbb{N}$
  - $f_+(n, m) = n + m$

- Verkettete Listen lassen sich als 1-stellige Funktionen repräsentieren:

  $f(x) \stackrel{\text{def}}{=}$ Element, auf das der Zeiger von x zeigt

- Neben Relationen und Funktionen verwenden wir auch **Konstanten**
- Konstanten sind eigentlich 0-stellige Funktionen über der Grundmenge A

  ➡ Abbildung $() \mapsto c$ mit $c \in A$

Strukturen

- Für die meisten Modellierungen ist es vorteilhaft, mehr als eine Relation oder Funktion zur Verfügung zu haben
  - ➡ Wir verwenden Strukturen, die aus mehreren Relationen, Funktionen und Konstanten bestehen dürfen

Definition: Struktur

- Eine **Struktur** $\mathcal{A}$ besteht aus
  - einer Grundmenge $A \neq \emptyset$,
  - Relationen $R_1, \ldots, R_k$ über A,
  - Funktionen $f_1, \ldots, f_m$ über A und
  - Konstanten $c_1, \ldots, c_p$ aus A
- Wir schreiben $\mathcal{A}$ dann als

$$(A, R_1, \ldots, R_k, f_1, \ldots, f_m, c_1, \ldots, c_p)$$

- **Bemerkungen:**
  - Wir betrachten nur Strukturen mit nicht-leerer Grundmenge

- Wir betrachten nur Strukturen mit **endlich vielen** Relationen und Funktionen
- Es ist auch $k = 0$, $m = 0$ oder $p = 0$ möglich

- Statt „Grundmenge“ sagen wir oft auch „Universum“

Strukturen: Schloss Dreadsbury

- Um die Beispiel-Formeln auswerten zu können, brauchen wir eine Struktur mit
  - einer Grundmenge U,
  - Relationen D, K, H, R und
  - Konstanten a, b, c

Schloss Dreadsbury: Beispielstruktur

- Sei $\mathcal{A} = (U, D, K, H, R, a, b, c)$ die wie folgt definierte Struktur:
  - Sei U die Menge

    $\{\text{Agatha}, \text{Ben}, \text{Chris}, \text{Diana}\}$
  - Sei D die (1-stellige) Relation

    $\{\text{Agatha}, \text{Ben}, \text{Chris}\}$
  - Sei K die Relation

    $\{(\text{Agatha}, \text{Ben}), (\text{Chris}, \text{Diana})\}$
  - Sei H die Relation

    $\{(\text{Agatha}, \text{Ben}), (\text{Ben}, \text{Agatha}),$
    $(\text{Chris}, \text{Diana})\}$
  - Sei R die Relation

    $\{(\text{Agatha}, \text{Ben}), (\text{Ben}, \text{Chris})\}$
  - Sei $a =$ Agatha (Ärztin),
    $b =$ Ben (Butler),
    $c =$ Chris (Chauffeur)

✎ Die Relationen sind hier in der Mengenschreibweise angegeben, die Tabellen in der Einleitung drücken die gleichen Mengen aus

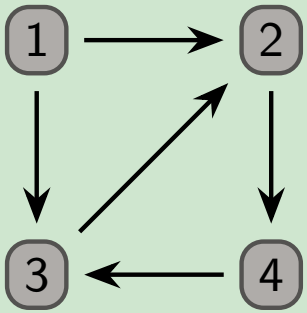
Beispiel-Formeln

- Welche der folgenden Beispielformeln gelten in $\mathcal{A}$?
  - $\forall x \forall y (K(x, y) \rightarrow H(x, y))$ ✓
  - $\forall x (H(a, x) \rightarrow \neg H(c, x))$ ✓
  - $\forall x (\neg (x = b) \rightarrow H(a, x))$ ✘

Strukturen: Graphen

Beispiel: (Gerichtete) Graphen

- Graph G:



- $\mathcal{A}_G = (V, E)$, wobei
  - $V = \{1, 2, 3, 4\}$ und
  - $E = \{(1, 2), (1, 3), (3, 2), (2, 4), (4, 3)\}$

Graphen: Beispielformeln

- Was drücken die folgenden Formeln aus? Gelten sie in G?

- $\forall x \exists y E(x, y)$
  - Jeder Knoten hat mindestens eine ausgehende Kante
  - ✓

- $\forall x \forall y (E(x, y) \leftrightarrow E(y, x))$
  - Die Kantenrelation ist symmetrisch
  - ✘

Strukturen: Algebraische Strukturen (1/2)

- In der Mathematik werden häufig Strukturen verwendet, die Funktionen enthalten

Beispiel: (Peano-)Arithmetik

- Die Arithmetik lässt sich durch die Struktur $(\mathbb{N}, <, +, \times, \mathbf{0}, \mathbf{1})$ repräsentieren:
  - $\mathbb{N}$ ist die Menge der natürlichen Zahlen
  - $<$ ist die (strikte) lineare Ordnung auf $\mathbb{N}$
  - $+$, $\times$ sind die Addition und Multiplikation auf $\mathbb{N}$
  - $\mathbf{0}$ und $\mathbf{1}$ sind Konstanten

- **Beispielformeln:**
  - $\forall x \forall y (+(x, y) = +(y, x))$
    - Die Addition ist kommutativ
    - ✓
  - $\forall x \exists y \exists z (x < y \wedge \times(z, z) = y)$
    - Es gibt unendlich viele Quadratzahlen
    - ✓

- **Bemerkung:** Bei Formeln zur Arithmetik verwenden wir meist die übliche Infixnotation für arithmetische Operationen
  - ➡ Wir schreiben also $z \times z = y$ statt $\times(z, z) = y$

Strukturen: Algebraische Strukturen (2/2)

Beispiel: Gruppen

- Gruppen lassen sich durch Strukturen der Art $(G, \circ, e)$ repräsentieren, wobei
  - G die Menge der Elemente ist,
  - $\circ$ eine 2-stellige Funktion (die Gruppenverknüpfung) ist und
  - e eine Konstante ist, das neutrale Element

- **Beispielformel:**
  - $\forall x \exists y (\circ(x, y) = e)$
    - Jedes Element hat ein rechtsinverses Element
    - ✓ ☞ gilt in jeder Gruppe

Strukturen: „Informatik-Strukturen“ (1/3)

Beispiel: Zeichenketten

- Gegeben sei der String $s = bacabba$
- Der String kann durch die Struktur

  $$(P, \leq, Z_a, Z_b, Z_c, \ell, r)$$

  repräsentiert werden, mit:
  - $P = \{1, 2, 3, 4, 5, 6, 7\}$ ☞ Menge der Positionen
  - $\leq$ = übliche lineare Ordnung auf P
  - $Z_a = \{2, 4, 7\}$
  - $Z_b = \{1, 5, 6\}$
  - $Z_c = \{3\}$

    ☞ Mengen der Positionen mit einem a, b oder c
  - ℓ: Funktion die eine Position (außer 1) auf den linken Nachbarn abbildet, z.B.: $\ell(3) = 2$

    ☞ $\ell(1) = 1$
  - r: analog für rechte Nachbarn

- **Beispielformel:**
  - $\exists x(Z_a(x) \wedge Z_b(r(x)) \wedge Z_c(r(r(x))))$
    - Der Teilstring abc kommt in einem String vor
    - ✘

Strukturen: „Informatik-Strukturen“ (2/3)

Relationale Datenbanken: Beispielrelation und -anfrage

Angestellte

Name	Persnr	Gehalt	Abteilung
Ochs	17	48.000	4
Faninal	12	63.000	12
Lerchenau	8	90.000	13
Schmitt	14	72.000	12

- Finde die Namen aller Angestellten der Abteilung 12, die mehr als 70.000 Euro verdienen

Anfrage in SQL

```
SELECT Name
FROM Angestellte
WHERE Gehalt > 70.000
        AND ABT = 12
```

- Die Anfragesprache SQL entspricht ziemlich genau der Prädikatenlogik

Relationale Datenbanken als Strukturen

- $\mathcal{D} = (U, R_1, \ldots, R_k)$
- U: die Menge der in $\mathcal{D}$ vorkommenden Datenwerte
- $R_1, \ldots, R_k$: Die Relationen über U

  ☞ „Tabellen“
- Weitere „eingebaute“ Relationen wie $<$

Anfrage in Prädikatenlogik

- Zur Anfrage korrespondierende prädikatenlogische Formel:

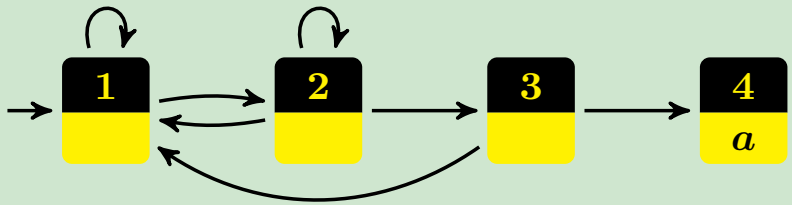
$$\exists y_1 \exists y_2 \, (\textsc{Angestellte}(x, y_1, y_2, 12) \wedge y_2 > 70.000)$$

  - Die Formel hat eine **freie Variable** x und ergibt die Namen aller Angestellten in Abteilung 12 mit einem Gehalt > 70.000
  - Antwort: {Schmitt}

Strukturen: „Informatik-Strukturen“ (3/3)

Beispiel: Transitionssysteme

- **Erinnerung:** Transitionssysteme bestehen aus Zuständen, **Transitionen** zwischen den Zuständen und Propositionen
- Transitionssystem $\mathcal{T}$:



- Dieses Transitionssystem kann durch die Struktur

  $$(Z, E, P_a, s)$$

  repräsentiert werden, wobei
  - $Z = \{1, 2, 3, 4\}$ ☞ Zustände
  - $E = \{(1,1), (1,2), (2,1), (2,2), (2,3), (3,1), (3,4)\}$ ☞ Transitionen
  - $P_a = \{4\}$ ☞ Zustände mit Eigenschaft a
  - $s = 1$ ☞ Anfangszustand

Transitionssysteme: Beispielformel

- $\forall x \exists y \exists z (E(x,y) \wedge E(y,z) \wedge P_a(z))$
  - Von jedem Zustand aus kann mit zwei Transitionen ein Zustand erreicht werden, in dem die Eigenschaft a gilt
  - ✘

Eingaben für algorithmische Probleme

- Eingabe-Instanzen für algorithmische Probleme lassen sich auch durch logische Strukturen modellieren

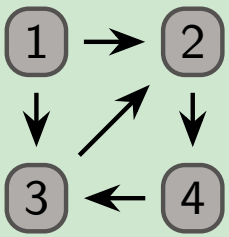
Erreichbarkeitsproblem

Gegeben: Ein Graph $G = (V, E)$ sowie ein Startknoten $s \in V$ und ein Zielknoten $t \in V$

Frage: Gibt es einen Weg von s nach t in G?

Beispiel

- Eingabe für das Erreichbarkeitsproblem:
  - Graph G:



  - Startknoten: $s = 1$
  - Zielknoten: $t = 4$

Beispiel: Eingabe als Struktur

- Die Eingabe G, s, t lässt sich durch die Struktur

$$\mathcal{A} = (V^{\mathcal{A}}, E^{\mathcal{A}}, s^{\mathcal{A}}, t^{\mathcal{A}})$$

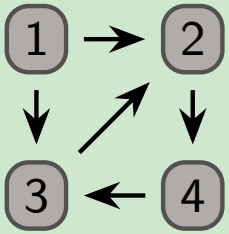
beschreiben, wobei

- $V^{\mathcal{A}} = \{1, 2, 3, 4\}$ und
- $E^{\mathcal{A}} = \{(1, 2), (1, 3), (3, 2), (2, 4), (4, 3)\}$
- $s^{\mathcal{A}} = 1$
- $t^{\mathcal{A}} = 4$

Signaturen (1/3)

Beispiel

- Eingabe für das Erreichbarkeitsproblem:
  - Graph G:



  - Startknoten: $s = 1$
  - Zielknoten: $t = 4$

- Als Struktur: $\mathcal{A} = (V^{\mathcal{A}}, E^{\mathcal{A}}, s^{\mathcal{A}}, t^{\mathcal{A}})$ mit
  - $V^{\mathcal{A}} = \{1, 2, 3, 4\}$ und
  - $E^{\mathcal{A}} = \{(1, 2), (1, 3), (3, 2), (2, 4), (4, 3)\}$
  - $s^{\mathcal{A}} = 1$
  - $t^{\mathcal{A}} = 4$

- Formeln sprechen jeweils über Strukturen eines gewissen „Typs“
  - ➡ Also z.B. über Erreichbarkeitsprobleme oder über Strings
- Der „Typ“ legt jeweils fest
  - welche Relationssymbole, Funktionssymbole und Konstantensymbole in Formeln verwendet werden können
  - welche **Stelligkeit** diese Symbole haben

Signaturen (2/3)

- Eine **Signatur** ist
  - eine Menge von Relations-, Funktions- und Konstantensymbolen,
  - zusammen mit einer Stelligkeit für Relations- und Funktionssymbole

- Die Signatur $\mathcal{S}(\mathcal{A})$ einer Struktur $\mathcal{A}$ ist die Menge der Symbole zu den in $\mathcal{A}$ vorkommenden Relationen, Funktionen und Konstanten
- Eine Struktur $\mathcal{A}$ ist **passend** zu einer Signatur σ, wenn $\sigma \subseteq \mathcal{S}(\mathcal{A})$

Beispiel

- Die Signatur von Erreichbarkeitsproblemen ist $\{E, s, t\}$, wobei
  - E ein 2-stelliges Relationssymbol ist
  - s, t Konstantensymbole sind

- Die Signatur von Zeichenketten über dem Alphabet $\{a, b, c\}$ ist $\{\leq, Z_a, Z_b, Z_c, \ell, r\}$, wobei
  - $\leq$ ein 2-stelliges Relationssymbol ist,
  - Z_a, Z_b, Z_c jeweils 1-stellige Relationssymbole sind
  - ℓ und r 1-stellige Funktionen sind

Signaturen (3/3)

- Wir müssen unterscheiden zwischen
  - konkreten Relationen, Funktionen und Konstanten, die in konkreten Strukturen vorkommen und
  - Relationssymbolen, Funktionssymbolen und Konstantensymbolen, die in Signaturen (und in Formeln) vorkommen und für beliebige Relationen, Funktionen und Konstanten stehen

- **Notation:** Die einem Relationssymbol R zugeordnete konkrete Relation in einer Struktur $\mathcal{A}$ bezeichnen wir mit $R^{\mathcal{A}}$

- Das Superskript $\mathcal{A}$ lassen wir meistens weg, wenn aus dem Zusammenhang klar ist,
  - dass eine konkrete Relation gemeint ist, und
  - welche konkrete Struktur gemeint ist

Syntax der Prädikatenlogik

Syntax der Prädikatenlogik (1/2)

- **Zunächst:** Welche **Terme** lassen sich aus Variablen, Konstanten und Funktionssymbolen bilden?
- **Dann:** Welche prädikatenlogischen Formeln lassen sich über einer Signatur bilden?

- Sei **PV** eine abzählbare Menge **prädikatenlogischer Variablen**
- PV enthalte mindestens x, y, z sowie x_i, y_i, z_i, für alle $i \in \mathbb{N}$

Definition: $\mathcal{S}$-Terme

- Sei $\mathcal{S}$ eine Signatur
- Die Menge $\mathbf{PT}(\mathcal{S})$ der **$\mathcal{S}$-Terme** ist induktiv wie folgt definiert:
  - Jede Variable aus PV ist ein $\mathcal{S}$-Term
  - Jedes Konstantensymbol aus $\mathcal{S}$ ist ein $\mathcal{S}$-Term
  - Ist f ein k-stelliges Funktionssymbol aus $\mathcal{S}$ und sind $t_1, \ldots, t_k$ $\mathcal{S}$-Terme, so ist $f(t_1, \ldots, t_k)$ ein $\mathcal{S}$-Term

- $\mathcal{S}$-Terme, die keine Variablen verwenden, nennen wir **Grundterme**
- $\mathbf{GT}(\mathcal{S}) \stackrel{\text{def}}{=}$ Menge aller Grundterme aus $\mathrm{PT}(\mathcal{S})$

Beispiel

- Für $\mathcal{S} = \{R, f, c\}$ mit 2-stelligem f, sind $\mathcal{S}$-Terme zum Beispiel: $c, x, y, f(c, c), f(c, x), f(y, f(x, x)), f(c, f(c, c)), \ldots$
- Dabei sind $c, f(c, c), f(c, f(c, c))$ Grundterme

- Wenn $\mathcal{S}$ durch den Kontext klar ist, sagen wir einfach „Terme“ statt „$\mathcal{S}$-Terme“

Syntax der Prädikatenlogik (2/2)

Definition: Syntax der Prädikatenlogik

- Sei $\mathcal{S}$ eine Signatur
- Die Menge **PL($\mathcal{S}$)** der **prädikatenlogischen $\mathcal{S}$-Formeln** ist induktiv wie folgt definiert:

(1) Ist R ein k-stelliges Relationssymbol aus $\mathcal{S}$ und sind $t_1, \ldots, t_k$ $\mathcal{S}$-Terme, so ist $R(t_1, \ldots, t_k)$ in $\mathrm{PL}(\mathcal{S})$

(2) Sind $\varphi, \varphi_1, \varphi_2$ in $\mathrm{PL}(\mathcal{S})$, so auch
   (a) $\neg\varphi$
   (b) $(\varphi_1 \wedge \varphi_2)$
   (c) $(\varphi_1 \vee \varphi_2)$

(3) Ist x eine Variable und φ in $\mathrm{PL}(\mathcal{S})$, so sind auch
   (a) $\exists x\ \varphi$ und
   (b) $\forall x\ \varphi$
   in $\mathrm{PL}(\mathcal{S})$

- $\exists$ heißt **Existenzquantor**, $\forall$ heißt **Allquantor**

- Wie immer sparen wir Klammern ein wenn möglich

- Formeln vom Typ (1) heißen **atomar** (oder: **Atome**)
- Formeln, die nur durch (1) und (2) gebildet werden können, heißen **quantorenfrei**

- Atomare und negierte atomare Formeln werden auch **Literale** genannt

Signatur einer Formel

- Wenn $\mathcal{S}$ durch den Zusammenhang klar ist (oder unwichtig), schreiben wir einfach PL statt $\mathrm{PL}(\mathcal{S})$
- Umgekehrt: die **Signatur $\mathcal{S}(\varphi)$** einer Formel φ ist die Menge der in φ vorkommenden Relations-, Funktions- und Konstantensymbole

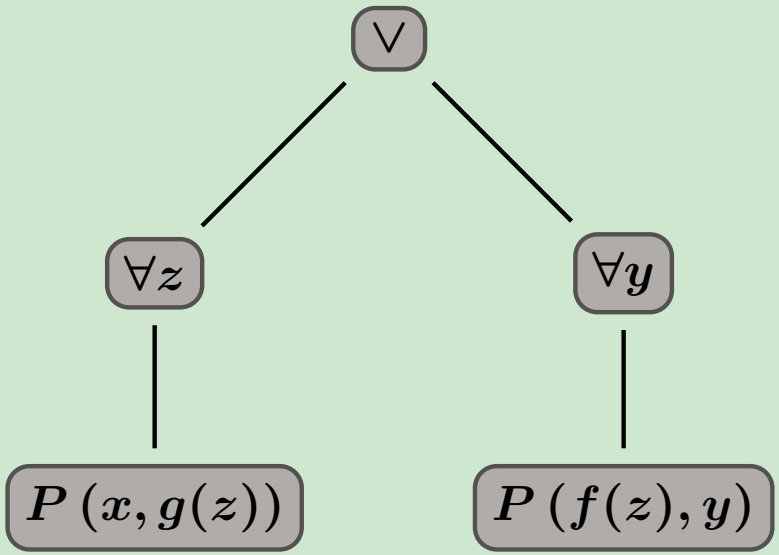
- Wenn die Signatur nicht explizit angegeben ist, gelten folgende Konventionen
  - Variablen sind kleine Buchstaben vom Ende des Alphabets
  - Konstantensymbole sind kleine Buchstaben vom Anfang des Alphabets
  - Relationssymbole sind große Buchstaben
  - Funktionssymbole sind kleine Buchstaben „aus der Mitte des Alphabets“ wie f, g, h

Syntax der Prädikatenlogik: Reichweite von Quantoren

- Der Aufbau prädikatenlogischer Formeln lässt sich durch Syntaxbäume illustrieren

Beispiel

- $\forall z\ P(x, g(z)) \vee \forall y\ P(f(z), y)$ hat den Syntaxbaum:



✎ Das äußere Klammerpaar ist hier weggelassen...

- Wie weit reicht die „Wirkung“ von Quantoren?
  - Bezieht sich $\forall z$ noch auf $P(f(z), y)$ oder nicht mehr?
  - Hat also $\vee$ oder $\forall z$ eine stärkere Bindungsstärke?

➡ Implizit in der Syntax-Definition beantwortet

- Die „äußerste Operation“, durch die die Formel entstanden ist, ist das „oder“
  - ➡ Andernfalls wäre anders geklammert:

$$\forall z \Big(P(x, g(z)) \vee \forall y\ P(f(z), y) \Big)$$

- Das Atom $P(f(z), y)$ befindet sich also **nicht** im Wirkungsbereich von $\forall z$

- Die Quantorenreichweite beschränkt sich auf die unmittelbar anschließende Teilformel
  - ➡ Sie reicht nicht immer bis zum Ende der Formel!

Gebundene und freie Variablen

- Ein Vorkommen einer Variablen x in einer Formel φ ist in $\exists x\ \varphi$ im Wirkungsbereich des Quantors $\exists x$ ☞ analog für Allquantoren

- Jedes Vorkommen einer Variablen x in einer Formel im Wirkungsbereich eines Quantors $\exists x$ oder $\forall x$ heißt **gebunden**
  ☞ durch die jeweiligen Quantoren
- Jedes andere Vorkommen einer Variablen in einer Formel ist **frei**

- Die **freien Variablen** einer Formel sind alle Variablen, die mindestens einmal frei vorkommen

- Eine Formel ist **geschlossen**, wenn sie keine freien Vorkommen von Variablen enthält
- Die Schreibweise $\varphi(x_1, \ldots, x_k)$ soll implizit bedeuten, dass φ keine anderen freien Variablen hat, als die in $x_1, \ldots, x_k$ genannten
  ☞ aber möglicherweise weniger

Beispiel

- Sei $\varphi_1 \stackrel{\text{def}}{=} \forall z\ P(x, g(z)) \vee R(x) \vee \forall y\ P(f(x), y)$
  - Sämtliche Vorkommen von x in φ_1 sind frei, die Vorkommen von y und z gebunden

- Sei $\varphi_2 \stackrel{\text{def}}{=} P(x) \vee \exists x\ R(x, y)$
  - x kommt sowohl frei als auch gebunden vor
  - Die Menge der freien Variablen dieser Formel ist also $\{x, y\}$

- Sei
$$\varphi_3 \stackrel{\text{def}}{=} \forall x\ \Big(\forall y \exists x\ E(y, x) \wedge E(x, y)\Big) \wedge E(x, x)$$
- frei: x in $E(x, x)$ und y in $E(x, y)$
- gebunden:
  - x in $E(y, x)$ (durch $\exists x$)
  - x in $E(x, y)$ (durch $\forall x$)
  - y in $E(y, x)$ (durch $\forall y$)

Sprechweisen: Relationen und Prädikate

- In unseren Definitionen verwenden wir Relationssymbole und Relationen
- Diese heißen auch Prädikatssymbole und Prädikate
  - ➡ Das ist die traditionellere Bezeichnung: Daher kommt der Name „Prädikatenlogik“

- In Formeln verwenden wir ausschließlich die traditionelle Schreibweise „$R(x, y)$“ und nicht „$(x, y) \in R$“

Semantik der Prädikatenlogik

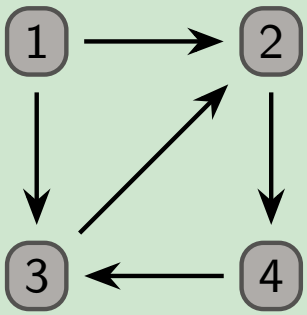
Ein kleines Problem bei der Formalisierung der Semantik

- **Nächstes Ziel:** Formale Semantik der Prädikatenlogik

Beispiel

- Betrachte $\varphi = \forall x \exists y \, E(x, y)$
- Intuitive Bedeutung: jeder Knoten eines Graphen hat eine ausgehende Kante

- Intuitiv klar: φ gilt in folgendem Graphen



- **Frage:** Wie lässt sich das formalisieren?

- **Ansatz:** Wie immer, induktiv über den Formelaufbau
  - ➡ Von kleinen zu immer größeren Teilformeln
- **Problem:** Teilformeln mit freien Variablen

- Um die Semantik von φ induktiv zu definieren, müssen wir zuerst die Semantik von $E(x, y)$ in G definieren
- Ob diese Formel gilt, hängt aber davon ab, welche Werte x und y annehmen

- **Frage:** Wie sollen freie Variablen einer Formel interpretiert werden?

Prädikatenlogische Interpretationen

- Für die Semantik einer PL-Formel φ benötigen wir:
  - eine Struktur, in der die Relations-, Funktions- und Konstantensymbole interpretiert werden, sowie
  - eine Zuordnung von Werten zu den freien Variablen von φ

Definition: Interpretation, Belegung

- Eine **Interpretation** $I = (\mathcal{A}, \beta)$ besteht aus
  - einer Struktur $\mathcal{A}$ mit Grundmenge A und
  - einer **Belegung**, d.h. einer partiellen Abbildung $\beta : \mathrm{PV} \rightharpoonup A$

- Eine Interpretation $(\mathcal{A}, \beta)$ heißt **passend** für eine PL-Formel φ, falls
  - $\mathcal{A}$ passend zur Signatur von φ ist und
  - $\beta(x)$ für jede freie Variable x von φ definiert ist
- Entsprechend sind **passende Terme** definiert

- **Notation:** Ist β eine Belegung mit $\beta(x) = \mathbf{2}$, so schreiben wir statt $(\mathcal{A}, \beta)$ auch $(\mathcal{A}, x \mapsto \mathbf{2})$

Beispiel

- Sei $\mathcal{D} = (U, \textsc{Angestellte})$ die Struktur mit der Grundmenge

$$U = \{\text{Ochs}, \text{Faninal}, \text{Lerchenau}, \text{Valzacchi}, \mathbf{4}, \mathbf{8}, \mathbf{12}, \mathbf{13}, \mathbf{14}, \mathbf{17}, \mathbf{48.000}, \mathbf{63.000}, \mathbf{72.000}, \mathbf{90.000}\}$$

und der 4-stelligen Relation Angestellte wie vorne

- Dann ist $(\mathcal{D}, x \mapsto \text{Ochs})$ eine passende Interpretation für die Formel

$$\exists y_1 \exists y_2 \; \textsc{Angestellte}(x, y_1, y_2, \mathbf{12})$$

Semantik der Prädikatenlogik (1/2)

- Die Semantik der Prädikatenlogik ordnet jeder Formel φ und jeder passenden Interpretation $\boldsymbol{I}$ einen Wahrheitswert $[\![\varphi]\!]_{\boldsymbol{I}}$ zu

- $\underline{\beta[x/a]}$ bezeichne die Belegung $\hat{\beta}$, die aus β wie folgt entsteht:

$$\hat{\beta}(z) \stackrel{\text{def}}{=} \begin{cases} a & \text{falls } z = x \\ \beta(z) & \text{sonst} \end{cases}$$

- Ist $\boldsymbol{I} = (\mathcal{A}, \beta)$, so bezeichnet $\underline{\boldsymbol{I}[x/a]}$ die Interpretation $(\mathcal{A}, \beta[x/a])$

Semantik der Prädikatenlogik (2/2)

Prädikatenlogik: Semantik

- Sei $\mathcal{I} = (\mathcal{A}, \beta)$ eine Interpretation mit Grundmenge A

- Der **Wert** $[\![t]\!]_{\mathcal{I}}$ eines (zu $\mathcal{I}$ passenden) Terms t ist induktiv wie folgt definiert
  - $[\![x]\!]_{\mathcal{I}} \stackrel{\text{def}}{=} \beta(x)$ für Variablen $x \in \mathrm{PV}$
  - $[\![c]\!]_{\mathcal{I}} \stackrel{\text{def}}{=} c^{\mathcal{A}}$ für Konstantensymbole c
  - $[\![f(t_1, \ldots, t_k)]\!]_{\mathcal{I}} \stackrel{\text{def}}{=} f^{\mathcal{A}}([\![t_1]\!]_{\mathcal{I}}, \ldots, [\![t_k]\!]_{\mathcal{I}})$

- Die **Gültigkeit** einer PL-Formel φ für die Interpretation $\mathcal{I}$ ist induktiv definiert
  ☞ Notation: $\mathcal{I} \models \varphi$

(1) $\mathcal{I} \models R(t_1, \ldots, t_k) \stackrel{\text{def}}{\Leftrightarrow} ([\![t_1]\!]_{\mathcal{I}}, \ldots, [\![t_k]\!]_{\mathcal{I}}) \in R^{\mathcal{A}}$

(2)(a) $\mathcal{I} \models \neg\varphi \stackrel{\text{def}}{\Leftrightarrow} \mathcal{I} \not\models \varphi$

(b) $\mathcal{I} \models \varphi_1 \wedge \varphi_2 \stackrel{\text{def}}{\Leftrightarrow}$ $\mathcal{I} \models \varphi_1$ **und** $\mathcal{I} \models \varphi_2$

(c) $\mathcal{I} \models \varphi_1 \vee \varphi_2 \stackrel{\text{def}}{\Leftrightarrow}$ $\mathcal{I} \models \varphi_1$ **oder** $\mathcal{I} \models \varphi_2$

(3)(a) $\mathcal{I} \models \exists x\, \psi \stackrel{\text{def}}{\Leftrightarrow}$ **es gibt** ein $a \in A$ mit $\mathcal{I}[x/a] \models \psi$

(b) $\mathcal{I} \models \forall x\, \psi \stackrel{\text{def}}{\Leftrightarrow}$ **für alle** $a \in A$ gilt $\mathcal{I}[x/a] \models \psi$

- Ist φ geschlossen, so schreiben wir statt $(\mathcal{A}, \beta) \models \varphi$ auch einfach $\mathcal{A} \models \varphi$, da β dann sowieso keinen Einfluss hat

Semantik der Prädikatenlogik: Beispiele (1/2)

Beispiel: Schloss Dreadsbury

- Sei $\mathcal{A} = (U, D, K, H, R, a, b, c)$ wieder die wie folgt definierte Struktur:
  - Sei U die Menge

    $\{\text{Agatha}, \text{Ben}, \text{Chris}, \text{Diana}\}$
  - Sei $a = \text{Agatha}$, $b = \text{Ben}$, $c = \text{Chris}$
  - Seien

D
Agatha
Ben
Claude

K	
Agatha	Ben
Claude	Diane

H	
Agatha	Ben
Ben	Agatha
Claude	Diane

R	
Agatha	Ben
Ben	Claude

- Wir werten die Formel $\neg\exists x \forall y H(x, y)$ in der Struktur $\mathcal{A}$ aus

  ☞ „Niemand hasst jeden“

- Es gilt:
  - $(\mathcal{A}, x \mapsto \text{Agatha}, y \mapsto \text{Chris}) \not\models H(x, y)$

  $\Rightarrow (\mathcal{A}, x \mapsto \text{Agatha}) \not\models \forall y H(x, y)$

- Analog:
  - $(\mathcal{A}, x \mapsto \text{Ben}) \not\models \forall y H(x, y)$
  - $(\mathcal{A}, x \mapsto \text{Chris}) \not\models \forall y H(x, y)$
  - $(\mathcal{A}, x \mapsto \text{Diana}) \not\models \forall y H(x, y)$

$\Rightarrow \mathcal{A} \not\models \exists x \forall y H(x, y)$

$\Rightarrow \mathcal{A} \models \neg\exists x \forall y H(x, y)$

Semantik der Prädikatenlogik: Beispiele (2/2)

- Die Formel φ kann auch in Interpretationen ausgewertet werden, die inhaltlich nichts mit dem Dreadsbury-Beispiel zu tun haben
- Wichtig ist nur, dass die Interpretation **passend** ist

Beispiel: Schloss Dreadsbury mal anders

- Sei $\mathcal{B}$ die Struktur

  $$(\mathbb{N}_0, D^{\mathcal{B}}, K^{\mathcal{B}}, H^{\mathcal{B}}, R^{\mathcal{B}}, a^{\mathcal{B}}, b^{\mathcal{B}}, c^{\mathcal{B}})$$

  wobei die Komponenten wie folgt definiert sind:
  - Grundmenge: $\mathbb{N}_0$
  - Sei $a^{\mathcal{B}} = 0$, $b^{\mathcal{B}} = 1$, $c^{\mathcal{B}} = 5$
  - Sei $D^{\mathcal{B}}$ die Menge aller geraden Zahlen
  - Sei $K^{\mathcal{B}}$ die Menge aller Paare (m, n), für die m ein Teiler von n ist
  - Sei $H^{\mathcal{B}}$ die Menge aller Paare (m, n), für die $m \leq n$ gilt
  - Sei $R^{\mathcal{B}}$ die Menge aller Paare (m, n), für die $m^2 = n$ gilt

- Wir werten die Formel $\neg\exists x \forall y H(x, y)$ in der Struktur $\mathcal{B}$ aus

  ☞ Es gibt keine kleinste Zahl

- Es gilt:
  - $(\mathcal{B}, x \mapsto 0, y \mapsto 0) \models H(x, y)$
  - $(\mathcal{B}, x \mapsto 0, y \mapsto 1) \models H(x, y)$
  - $(\mathcal{B}, x \mapsto 0, y \mapsto 2) \models H(x, y)$
  - ...

$\Rightarrow (\mathcal{B}, x \mapsto 0) \models \forall y H(x, y)$

$\Rightarrow \mathcal{B} \models \exists x \forall y H(x, y)$

$\Rightarrow \mathcal{B} \not\models \neg\exists x \forall y H(x, y)$

Prädikatenlogik mit Gleichheit

- Für das 2-stellige Relationssymbol „$=$“ erlauben wir immer nur die Interpretation

  $$\{(a, a) \mid a \in A\}$$

- Infix-Schreibweise: „$t_1 = t_2$“ statt „$= (t_1, t_2)$“

Beispiel

- $\forall x \forall y_1 \forall y_2 \big((E(x, y_1) \wedge E(x, y_2)) \rightarrow y_1 = y_2\big)$
  drückt aus, dass jeder Knoten eines Graphen höchstens eine ausgehende Kante hat

- Wenn wir die Verwendung von „$=$“ erlauben, sprechen wir von **Prädikatenlogik mit Gleichheit**
- Wenn wir nichts dazu sagen, ist die **Prädikatenlogik ohne Gleichheit** gemeint

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Prädikatenlogische Formeln werden auf Strukturen ausgewertet
  - Strukturen bestehen aus Relationen, Funktionen und Konstanten über einer Grundmenge
  - Viele verschiedene Objekte aus der Informatik und Mathematik lassen sich durch Strukturen repräsentieren
  - Eigenschaften solcher Strukturen lassen sich mit prädikatenlogischen Formeln beschreiben
  - Syntax und Semantik der Prädikatenlogik haben wir formal definiert

Literatur

- Das Beispiel des Mordes in Schloss Dreadsbury ist dem (in der Logik) bekannten **Dreadsbury Mansion Mystery** nachempfunden, in dem Tante Agatha getötet wird, und das auf diese Quelle zurückgeht:
  -

Kapitel C.2:

Modellierung und Normalformen

Inhalt

1. Modellierung: Formeln erstellen und verstehen
2. Äquivalenzen
3. Normalformen
4. Auf dem Weg zu einem Erfüllbarkeitstest
5. Anhang: Weitere Modellierungsbeispiele

Modellierung: Formeln erstellen und verstehen

Prädikatenlogik: Formeln verstehen

Beispiel

- Zur Erinnerung:
  - Schloss Dreadsbury:
    - $D(x)$: x wohnt in Schloss Dreadsbury
    - $K(x, y)$: x ermordet y
    - $H(x, y)$: x hasst y
    - $R(x, y)$: x ist reicher als y
    - $M(x, y)$: x mag y

      ☞ für freundlichere Beispiele
    - a: Ärztin, b: Butler, c: Chauffeur

- $\forall x \forall y \left(D(x) \rightarrow \neg K(y, x)\right)$
  - **Formelähnlich:** Für alle Personen x und y gilt: wenn x in Schloss Dreadsbury wohnt, wird x nicht von y ermordet
  - **Prägnanter:** Bewohner von Schloss Dreadsbury werden nicht ermordet

- $\forall x \exists y \left(M(x, y) \wedge H(x, b)\right)$
  - **Formelähnlich:** Für jede Person x gibt es eine Person y, so dass x Person y mag und x den Butler hasst
  - **Prägnanter:** Jeder mag jemanden und alle hassen den Butler

- $\exists x \forall y \left(D(x) \wedge M(x, y)\right)$
  - **Formelähnlich:** Es gibt eine Person x, so dass für alle Personen y gilt: x wohnt in Schloss Dreadsbury und mag y
  - **Prägnanter:** Schloss Dreadsbury hat einen Bewohner, der alle mag

Prädikatenlogik: Formeln erstellen

- Wir suchen jetzt: Prädikatenlogische Formeln mit Gleichheit für umgangssprachlich formulierte Aussagen

Beispiel

- Schloss Dreadsbury:
  - $D(x)$: x wohnt in Schloss Dreadsbury
  - $K(x, y)$: x ermordet y
  - $H(x, y)$: x hasst y
  - $R(x, y)$: x ist reicher als y
  - $M(x, y)$: x mag y
  - a: Ärztin, b: Butler, c: Chauffeur

- **Jeder Bewohner von Schloss Dreadsbury wird von jemandem gemocht**
  - **Formelähnlich:** Für jede Person x, die in Schloss Dreadsbury wohnt, gibt es eine Person y, die sie mag
  - **Als Formel:**

$$\forall x \left(D(x) \rightarrow \exists y \; M(y, x) \right)$$

- **Wer sich selbst hasst, mag niemanden**
  - **Formelähnlich:** Für jede Person x gilt: wenn sie sich selbst hasst, gibt es keine Person y, die von ihr gemocht wird
  - **Als Formel:**

$$\forall x \left(H(x, x) \rightarrow \neg \exists y \; M(x, y) \right)$$

- **Ein Bewohner von Schloss Dreadsbury ist reicher als der Chauffeur**
  - **Formelähnlich:** Es gibt eine Person x, die in Schloss Dreadsbury wohnt und reicher als der Chauffeur ist
  - **Als Formel:**

$$\exists x \left(D(x) \wedge R(x, c) \right)$$

„Beschränkte Quantoren“

- In der Mathematik ist es üblich, den Wertebereich von Quantoren einzuschränken:

$$\forall \epsilon{>}0\ \exists n_0{\in}\mathbb{N}\ \forall n{\geq}n_0 : |a_n{-}a| < \epsilon$$

- Solche „beschränkten Quantoren“ sind in unserer Definition der Prädikatenlogik nicht erlaubt
- Sie lassen sich aber leicht simulieren

- Vorgehen:
  - $\forall x(R(x) \rightarrow \varphi)$ statt $\forall x{\in}R : \varphi$
  - $\exists x(R(x) \wedge \varphi)$ statt $\exists x{\in}R : \varphi$

Beispiel

„Jeder Bewohner von Schloss Dreadsbury wird von jemandem gemocht“

- **Richtig:**

$$\forall x\ \big(D(x) \rightarrow \exists y\ M(y,x)\big)$$

- **Syntaktisch falsch:**

$$\forall x \in D\ \exists y\ M(y,x)$$

- **Auch falsch:**

$$\forall x\ \big(D(x) \wedge \exists y\ M(y,x)\big)$$

☞ beliebter Fehler

„Es gibt eine Person, die in Schloss Dreadsbury wohnt und reicher als der Chauffeur ist“

- **Richtig:**

$$\exists x\ \big(D(x) \wedge R(x,c)\big)$$

- **Falsch:**

$$\exists x\big(D(x) \rightarrow R(x,c)\big)$$

☞ noch beliebterer Fehler

Äquivalenzen

Äquivalenz von Formeln

- Die Äquivalenz von prädikatenlogischen Formeln definieren wir analog zur Aussagen- und Modallogik

Definition:
Äquivalenz prädikatenlogischer Formeln

- Seien φ, ψ prädikatenlogische Formeln

- φ und ψ heißen **äquivalent**, falls für jede zu φ und ψ passende Interpretation $\boldsymbol{I}$ gilt:

$$\boldsymbol{I} \models \varphi \iff \boldsymbol{I} \models \psi$$

- Schreibweise: $\varphi \equiv \psi$

Äquivalenzen der Prädikatenlogik (1/2)

Satz C.2.1:
Äquivalenzen der Prädikatenlogik (1)

- Für prädikatenlogische Formeln $\varphi, \varphi_1, \varphi_2, \varphi_3$ gelten:

- **Kommutativität**
  - $\varphi_1 \wedge \varphi_2 \equiv \varphi_2 \wedge \varphi_1$
  - $\varphi_1 \vee \varphi_2 \equiv \varphi_2 \vee \varphi_1$

- **Assoziativität**
  - $(\varphi_1 \wedge \varphi_2) \wedge \varphi_3 \equiv \varphi_1 \wedge (\varphi_2 \wedge \varphi_3)$
  - $(\varphi_1 \vee \varphi_2) \vee \varphi_3 \equiv \varphi_1 \vee (\varphi_2 \vee \varphi_3)$

- **Idempotenz**
  - $\varphi \wedge \varphi \equiv \varphi$ und $\varphi \vee \varphi \equiv \varphi$

- **Absorption**
  - $\varphi_1 \wedge (\varphi_1 \vee \varphi_2) \equiv \varphi_1$
  - $\varphi_1 \vee (\varphi_1 \wedge \varphi_2) \equiv \varphi_1$

- **Distributivität**
  - $\varphi_1 \vee (\varphi_2 \wedge \varphi_3) \equiv (\varphi_1 \vee \varphi_2) \wedge (\varphi_1 \vee \varphi_3)$
  - $\varphi_1 \wedge (\varphi_2 \vee \varphi_3) \equiv (\varphi_1 \wedge \varphi_2) \vee (\varphi_1 \wedge \varphi_3)$

- **Doppelnegation**
  - $\neg\neg\varphi \equiv \varphi$

- **De Morgansche Regeln**
  - $\neg(\varphi_1 \wedge \varphi_2) \equiv \neg\varphi_1 \vee \neg\varphi_2$
  - $\neg(\varphi_1 \vee \varphi_2) \equiv \neg\varphi_1 \wedge \neg\varphi_2$

Äquivalenzen der Prädikatenlogik (2/2)

Satz C.2.1: Äquivalenzen der Prädikatenlogik (2)

- Für prädikatenlogische Formeln φ, φ_1, φ_2 gelten:

- **Dualität der Quantoren**
  - $\neg\forall x\varphi \equiv \exists x\neg\varphi$
  - $\neg\exists x\varphi \equiv \forall x\neg\varphi$

- **Kommutativität der Quantoren**
  - $\forall x\forall y\varphi \equiv \forall y\forall x\varphi$
  - $\exists x\exists y\varphi \equiv \exists y\exists x\varphi$

- **Distributivität der Quantoren**
  - $(\forall x\varphi_1 \wedge \forall x\varphi_2) \equiv \forall x(\varphi_1 \wedge \varphi_2)$
  - $(\exists x\varphi_1 \vee \exists x\varphi_2) \equiv \exists x(\varphi_1 \vee \varphi_2)$

- **Schwache Distributivität der Quantoren**
  - Falls x in φ_1 nicht vorkommt:

  (1a) $\forall x(\varphi_1 \wedge \varphi_2) \equiv \varphi_1 \wedge \forall x\varphi_2$
  (1b) $\forall x(\varphi_1 \vee \varphi_2) \equiv \varphi_1 \vee \forall x\varphi_2$
  (2a) $\exists x(\varphi_1 \vee \varphi_2) \equiv \varphi_1 \vee \exists x\varphi_2$
  (2b) $\exists x(\varphi_1 \wedge \varphi_2) \equiv \varphi_1 \wedge \exists x\varphi_2$

Nachweis von Äquivalenzen

- Äquivalenzen lassen sich nachweisen durch:
  - die Überprüfung der Definition
  - das Ersetzungslemma

  ☞ Analog zur Aussagenlogik

Lemma C.2.2: Ersetzungslemma der Prädikatenlogik

- Sei φ_1 eine prädikaten-logische Formel mit einer Teilformel ψ_1
- Sei ψ_2 eine weitere Formel mit $\psi_2 \equiv \psi_1$

- Wenn φ_2 aus φ_1 entsteht, indem ein Vorkommen von ψ_1 durch ψ_2 ersetzt wird,
  dann gilt $\varphi_2 \equiv \varphi_1$

- **Jetzt:** Ein Nachweis von Äquivalenz durch Überprüfung der Definition

Beispiel

- **Zeige:** Wenn x nicht in φ_1 vorkommt, dann gilt:

  $$\forall x(\varphi_1 \vee \varphi_2) \equiv \varphi_1 \vee \forall x \varphi_2$$

- Sei $I = (\mathcal{A}, \beta)$ beliebig
- Dann gilt:

  $I \models \forall x(\varphi_1 \vee \varphi_2)$
  $\Leftrightarrow$ für alle $a \in A$ gilt: $I[x/a] \models \varphi_1 \vee \varphi_2$
  $\Leftrightarrow$ für alle $a \in A$ gilt: $I[x/a] \models \varphi_1$ oder $I[x/a] \models \varphi_2$
  $\Leftrightarrow$ für alle $a \in A$ gilt: $I \models \varphi_1$ oder $I[x/a] \models \varphi_2$

  ☞ da x in φ_1 nicht vorkommt

  $\Leftrightarrow$ es gilt $I \models \varphi_1$ oder für alle $a \in A$ gilt $I[x/a] \models \varphi_2$
  $\Leftrightarrow I \models \varphi_1$ oder $I \models \forall x \varphi_2$
  $\Leftrightarrow I \models \varphi_1 \vee \forall x \varphi_2$

Nachweis von Nicht-Äquivalenz

- **Frage:** Wie lässt sich nachweisen, dass eine Äquivalenz nicht gilt?
- **Ansatz:** Finde ein Gegenbeispiel

Beispiel

- **Behauptung:** Distributivität gilt nicht für die jeweils anderen Operatoren
- **Präziser:** Es gibt prädikatenlogische Formeln φ_1 und φ_2 so dass:

(1) $(\forall x \varphi_1 \vee \forall x \varphi_2) \not\equiv \forall x (\varphi_1 \vee \varphi_2)$
(2) $(\exists x \varphi_1 \wedge \exists x \varphi_2) \not\equiv \exists x (\varphi_1 \wedge \varphi_2)$

- Wir zeigen nur (1)

- Wähle $\varphi_1 = R(x)$ und $\varphi_2 = B(x)$
- **Zu zeigen:** $\forall x R(x) \vee \forall x B(x) \not\equiv \forall x (R(x) \vee B(x))$

- Wähle die Stuktur $\mathcal{A} = (A, R^{\mathcal{A}}, B^{\mathcal{A}})$ mit
  - $A = \{1, 2\}$
  - $R = \{1\}$
  - $B = \{2\}$
- Es gilt:
  - $\mathcal{A} \not\models \forall x R(x) \vee \forall x B(x)$, aber
  - $\mathcal{A} \models \forall x (R(x) \vee B(x))$

Normalformen

Normalformen: Begriffe

Definition: Normalformen

- Eine prädikatenlogische Formel ist in **reduzierter Form**, wenn sie nur $\exists, \vee, \neg$ verwendet, aber nicht $\forall$ und $\wedge$

- Eine prädikatenlogische Formel φ ist in **Negationsnormalform (NNF)**, wenn Negationen in φ nur unmittelbar vor atomaren Formeln vorkommen

- Eine prädikatenlogische Formel φ ist in **Pränexform**, wenn sie die Gestalt

  $$Q_1 x_1 Q_2 x_2 \cdots Q_k x_k \varphi'$$

  hat, wobei
  - die Q_i jeweils entweder $\exists$ oder $\forall$ sind,
  - die x_i paarweise verschieden sind und
  - φ' quantorenfrei ist

Beispiel

- $\exists x \forall y (E(x,y) \wedge \neg E(y,x))$ ist in Pränexform und in NNF, aber nicht in reduzierter Form

- $\exists x \forall y E(x,y) \wedge \neg E(y,x)$
  ist in NNF, aber weder in Pränexform noch in reduzierter Form

  ☞ beliebter Fehler!

- $\exists x \neg \big(\exists y (\neg E(x,y) \vee E(y,x))\big)$
  ist in reduzierter Form, aber weder in NNF noch in Pränexform

Normalformen der Prädikatenlogik

Lemma C.2.3: Normalformen der Prädikatenlogik

- Zu jeder prädikatenlogischen Formel φ gibt es jeweils eine äquivalente Formel in
  (a) reduzierter Form,
  (b) Negationsnormalform und
  (c) Pränexform

Beweisskizze

- (a) und (b): Anwendung der De Morganschen Regel und der Dualität der Quantoren
- (c): Wir geben gleich einen Algorithmus für die Berechnung der Pränexform an

Umwandlung in Pränexform: Variablenumbenennungen

- **Beobachtung:** Bei der Pränexform ist es nicht erlaubt, dass verschiedene Quantoren dieselbe Variable quantifizieren
  - ➡ Benenne Variablen so um, dass alle Quantoren verschiedene Variablen verwenden
- Das ändert nichts an der Bedeutung von Formeln:

Lemma C.2.4

- Sei φ eine Formel, in der die Variable x' nicht vorkommt
- Sei φ' die Formel, die aus φ entsteht, indem jedes freie Vorkommen von x durch x' ersetzt wird

- Dann gelten:

(1) $\exists x \varphi \equiv \exists x' \varphi'$
(2) $\forall x \varphi \equiv \forall x' \varphi'$

Umwandlung in Pränexform: Ein Algorithmus

- Im folgenden Algorithmus steht
  - jedes Q_i und jedes Q'_i für einen Quantor $\exists$ oder $\forall$ und
  - $\overline{Q}_i$ für den zu Q_i dualen Quantor

Algorithmus: Pränexform PF(φ)

Eingabe: Prädikatenlogische Formel φ
Ausgabe: Prädikatenlogische Formel φ' in Pränexform mit $\varphi' \equiv \varphi$

1: Wandle φ äquivalent so um, dass alle quantifizierten Variablen paarweise verschieden und von den freien Variablen verschieden sind
2: **switch** φ ist von der Form
3: **case** atomar
4: **return** φ
5: **case** $\neg\psi$
6: Berechne $Q_1x_1 \cdots Q_kx_k\chi := \text{PF}(\psi)$
7: **return** $\overline{Q_1}x_1 \cdots \overline{Q_k}x_k\neg\chi$
☞ wobei $\overline{\exists} = \forall$ und $\overline{\forall} = \exists$
8: **case** $\psi_1 \vee \psi_2$
9: Berechne $Q_1x_1 \cdots Q_kx_k\chi_1 := \text{PF}(\psi_1)$
10: Berechne $Q'_1y_1 \cdots Q'_ly_l\chi_2 := \text{PF}(\psi_2)$
11: **return** $Q_1x_1 \cdots Q_kx_kQ'_1y_1 \cdots Q'_ly_l(\chi_1 \vee \chi_2)$
12: **case** $\psi_1 \wedge \psi_2$
13: Berechne $Q_1x_1 \cdots Q_kx_k\chi_1 := \text{PF}(\psi_1)$
14: Berechne $Q'_1y_1 \cdots Q'_ly_l\chi_2 := \text{PF}(\psi_2)$
15: **return** $Q_1x_1 \cdots Q_kx_kQ'_1y_1 \cdots Q'_ly_l(\chi_1 \wedge \chi_2)$
16: **case** $\exists x\psi$
17: Berechne $Q_1x_1 \cdots Q_kx_k\chi := \text{PF}(\psi)$
☞ alle $x_i \neq x$
18: **return** $\exists xQ_1x_1 \cdots Q_kx_k\chi$
19: **case** $\forall x\psi$
20: Berechne $Q_1x_1 \cdots Q_kx_k\chi := \text{PF}(\psi)$
☞ alle $x_i \neq x$
21: **return** $\forall xQ_1x_1 \cdots Q_kx_k\chi$

- Die Korrektheit dieses Algorithmus lässt sich durch Induktion nach dem Aufbau der Formel φ beweisen

Berechnung der Pränexform: Beispiel

Beispiel

- Betrachte die Formel:

$$\Big(\boldsymbol{R}(x) \vee \forall y \boldsymbol{P}(x, \boldsymbol{g}(y))\Big) \wedge \neg \forall x \exists y \boldsymbol{P}(\boldsymbol{f}(x), y)$$

- Umbenennung der Variablen:

$$\Big(\boldsymbol{R}(x) \vee \forall y \boldsymbol{P}(x, \boldsymbol{g}(y))\Big) \wedge \neg \forall \boldsymbol{x_1} \exists \boldsymbol{y_1} \boldsymbol{P}(\boldsymbol{f}(\boldsymbol{x_1}), \boldsymbol{y_1})$$

- Da die Formel von der Form $\boldsymbol{\psi_1 \wedge \psi_2}$ ist, wird zunächst für die beiden Teilformeln die Pränexform bestimmt:
  - PF$(\boldsymbol{R}(x) \vee \forall y \boldsymbol{P}(x, \boldsymbol{g}(y)))$:
    - liefert die Formel: $\forall y \Big(\boldsymbol{R}(x) \vee \boldsymbol{P}(x, \boldsymbol{g}(y))\Big)$
  - PF$(\neg \forall \boldsymbol{x_1} \exists \boldsymbol{y_1} \boldsymbol{P}(\boldsymbol{f}(\boldsymbol{x_1}), \boldsymbol{y_1}))$:
    - führt zum Aufruf: PF$(\forall \boldsymbol{x_1} \exists \boldsymbol{y_1} \boldsymbol{P}(\boldsymbol{f}(\boldsymbol{x_1}), \boldsymbol{y_1}))$
    - und liefert dann: $\exists \boldsymbol{x_1} \forall \boldsymbol{y_1} \neg \boldsymbol{P}(\boldsymbol{f}(\boldsymbol{x_1}), \boldsymbol{y_1})$

- Insgesamt erhalten wir also:

$$\forall y \exists \boldsymbol{x_1} \forall \boldsymbol{y_1} \Big(\Big(\boldsymbol{R}(x) \vee \boldsymbol{P}(x, \boldsymbol{g}(y))\Big) \wedge \neg \boldsymbol{P}(\boldsymbol{f}(\boldsymbol{x_1}), \boldsymbol{y_1})\Big)$$

Umwandlung in Pränexform

- Anderer Weg zur Pränexform: Anwendung von Äquivalenzregeln

Beispiel

$$\begin{aligned}
\exists y S(y) \wedge \forall x R(x) &\equiv \forall x \big(\exists y S(y) \wedge R(x)\big) && \text{☞ Schwache Distributivität} \\
&\equiv \forall x \big(R(x) \wedge \exists y S(y)\big) && \text{☞ Kommutativität} \\
&\equiv \forall x \exists y \big(R(x) \wedge S(y)\big) && \text{☞ Schwache Distributivität}
\end{aligned}$$

Auf dem Weg zu einem Erfüllbarkeitstest

Erfüllbarkeit: Grundbegriffe

Definition: Modell, erfüllbar, allgemeingültig

- Sei φ eine prädikatenlogische Formel
- Eine Interpretation I mit $I \models \varphi$ heißt **Modell** von φ

- φ heißt **erfüllbar**, falls φ ein Modell hat, andernfalls **unerfüllbar**

- φ heißt **allgemeingültig**, falls jede zu φ passende Interpretation ein Modell von φ ist

  ☞ andere Bezeichnung: **Tautologie**

- I ist **Modell** einer Menge Φ prädikatenlogischer Formeln, falls $I \models \varphi$, für alle $\varphi \in \Phi$ ☞ $I \models \Phi$

- Die Begriffe „erfüllbar“ und „allgemeingültig“ werden für Mengen Φ von Formeln analog definiert

Beispiel

- $\exists x\ \forall y \big(R(x) \vee \neg R(y)\big)$ ist allgemeingültig
- $\forall x\ \exists y\ \big(\neg R(x) \wedge R(y)\big)$ ist unerfüllbar

Bemerkungen

- Der Zusammenhang zwischen unerfüllbaren und allgemeingültigen Formeln ist genauso wie in der Aussagenlogik:

  φ unerfüllbar $\iff \neg\varphi$ allgemeingültig

- Ist φ eine geschlossene Formel mit Modell $I = (\mathcal{A}, \beta)$, so nennen wir auch $\mathcal{A}$ ein Modell von φ und schreiben $\mathcal{A} \models \varphi$

- **In den nächsten Kapiteln** Erfüllbarkeitstests für prädikatenlogische Formeln
- **Jetzt:** Vorarbeiten

  ➡ „Normalformen“ für prädikatenlogische Formeln

Skolemform (1/4)

- **Zwischenziel:** Vereinfachung von Formeln
  - ➡ Um Erfüllbarkeit einfacher überprüfen zu können

- **Beobachtung:** Um die Erfüllbarkeit einer Formel φ zu testen, kann auch eine Formel ψ getestet werden, die sich bezüglich Erfüllbarkeit gleich verhält
  - ➡ Die Formeln φ und ψ müssen nicht äquivalent sein!
- **Wir werden sehen:** Diese Beobachtung ermöglicht es häufig, Quantoren aus Formeln zu beseitigen

Definition: Erfüllbarkeitsäquivalent

- Zwei prädikatenlogische Formeln φ und ψ heißen **erfüllbarkeitsäquivalent**, wenn gilt
  - φ und ψ sind beide erfüllbar, oder
  - φ und ψ sind beide nicht erfüllbar,

Definition: Skolemform

- Eine prädikatenlogische Formel ist in **Skolemform**, wenn sie
  - in Pränexform ist,
  - im Quantoren-Präfix **nur Allquantoren** vorkommen und

- Die Pränexform garantiert bereits, dass im Quantoren-Präfix keine Variable mehrfach vorkommt
- Formeln in Skolemform sind also von der Form

$$\forall x_1 \cdots \forall x_n \, \psi$$

wobei ψ quantorenfrei ist

Lemma C.2.5

- Es gibt einen Algorithmus, der zu jeder prädikatenlogischen Formel eine erfüllbarkeitsäquivalente, geschlossene Formel in Skolemform berechnet

Skolemform (2/4)

Beispiel

- Betrachte die Formel $\varphi = \forall x \, \exists y \, E(x, y)$
  - ➡ „Jeder Knoten eines Graphen hat mindestens eine ausgehende Kante“
- **Frage:** Wie lässt sich φ in eine erfüllbarkeitsäquivalente Formel in Skolemform umwandeln?

- **Ziel:** Wir müssen den $\exists$-Quantor loswerden, ohne an der Erfüllbarkeit der Formel etwas zu ändern

- Die Formel φ drückt aus, dass es zu jedem Knoten a einen weiteren Knoten gibt, zu dem es eine Kante von a aus gibt
  - ➡ Wenn $G \models \varphi$ für einen Graphen $G = (V, E)$ gilt, dann lässt sich eine Funktion $f : V \mapsto V$ finden, so dass jeder Knoten a eine Kante zum Knoten $f(a)$ hat
  - ➡ Umgekehrt lässt sich eine solche Funktion nur finden, wenn $G \models \varphi$ gilt

- **Also:** Die Formel φ ist erfüllbarkeitsäquivalent zu

  $$\forall x \, E(x, f(x))$$

Skolemform (3/4)

- **Allgemeiner:** Durch Anwendung der folgenden **Skolem-Regel** kann ein Existenz-Quantor „erfüllbarkeitsäquivalent“ aus einer Formel entfernt werden

Skolem-Regel

- Ist $\varphi = \forall x_1 \forall x_2 \cdots \forall x_k \exists y \psi$, und kommt das Funktionssymbol f in ψ nicht vor, so ist die Formel

$$\forall x_1 \forall x_2 \cdots \forall x_k \psi[y/f(x_1, \ldots, x_k)]$$

  erfüllbarkeitsäquivalent zu φ
- Hierbei: $\psi[y/f(x_1, \ldots, x_k)]$ ist die Formel, die aus ψ ensteht, indem y überall durch $f(x_1, \ldots, x_k)$ ersetzt wird

Bemerkungen

- ψ darf weitere Quantoren enthalten
- Steht der $\exists$-Quantor am Anfang ($k = 0$), so wird y durch eine Konstante ersetzt
  - ✎ Zur Erinnerung: Konstanten sind eigentlich 0-stellige Funktionen

- Umwandlung einer prädikatenlogischen Formel φ in eine geschlossene, erfüllbarkeitsäquivalente Formel in Skolemform:

(1) Wandle φ in eine äquivalente Formel φ_1 in **Pränexform** um
(2) Wandle φ_1 in eine erfüllbarkeitsäquivalente, **geschlossene Pränex-Formel** φ_2 um, indem jede freie Variable durch eine neue Konstante ersetzt wird
(3) Wandle φ_2 durch Anwendung der Skolem-Regel (von links nach rechts) in eine erfüllbarkeitsäquivalente **geschlossene Skolemformel** φ_3 um

Skolemform (4/4)

Beispiel

- **Ziel:** Wandle die Formel

$$\exists y \forall z \exists w \Big((\neg P(x, y) \wedge \neg R(y, f(z))) \vee P(g(x, w), x) \Big)$$

in eine geschlossene, erfüllbarkeitsäquivalente Formel in Skolemform um

- **Schritt 1:** Die Formel ist schon in Pränexform

- **Schritt 2:** Ersetzung freier Variablen
  - Wir ersetzen die freie Variable x durch die neue Konstante a:

$$\exists y \forall z \exists w \Big((\neg P(a, y) \wedge \neg R(y, f(z))) \vee P(g(a, w), a) \Big)$$

- **Schritt 3:** $\exists$-Quantoren eliminieren
  - Der erste $\exists$-Quantor wird entfernt und jedes Vorkommen von y durch das Konstantensymbol b ersetzt:

$$\forall z \exists w \Big((\neg P(a, b) \wedge \neg R(b, f(z))) \vee P(g(a, w), a) \Big)$$

  - Der zweite $\exists$-Quantor wird entfernt und jedes Vorkommen von w durch den 1-stelligen Funktionsterm $h(z)$ ersetzt:

$$\forall z \Big((\neg P(a, b) \wedge \neg R(b, f(z))) \vee P(g(a, h(z)), a) \Big)$$

Matrixklauselform

- Der quantorenfreie Teil der Skolemform kann noch weiter vereinfacht werden

- Sei φ eine Formel in Pränexform
- Sei $\psi = \forall y_1 \cdots \forall y_k \chi$ eine zu φ erfüllbarkeitsäquivalente, geschlossene Formel in Skolemform
- Sei χ' eine KNF zu χ
- Die Klauselmenge $\mathcal{K}$ zu χ' nennen wir nun eine **Matrixklauselform** zu φ

- In Matrixklauselform werden die universellen Quantoren nicht notiert

Beispiel

- Betrachte die Formel
$\Big(R(x) \vee \forall y P(x, g(y))\Big) \wedge \neg \forall x \exists y P(f(x), y)$
- In **Pränexform**:
$\forall y_1 \exists x_1 \forall y \Big(\big(R(x) \vee P(x, g(y_1))\big) \wedge \neg P(f(x_1), y)\Big)$

Beispiel

- Ersetzung $x \mapsto a$ führt zu:
$\forall y_1 \exists x_1 \forall y \Big(\big(R(a) \vee P(a, g(y_1))\big) \wedge \neg P(f(x_1), y)\Big)$
- Ersetzung $x_1 \mapsto h(y_1)$ führt zur **Skolemform**:
$\forall y_1 \forall y \Big(\big(R(a) \vee P(a, g(y_1))\big) \wedge \neg P(f(h(y_1)), y)\Big)$
- Die entstehende **Matrixformel** ist bereits in KNF:
$\big(R(a) \vee P(a, g(y_1))\big) \wedge \neg P(f(h(y_1)), y)$
- Insgesamt erhalten wir die **Matrixklauselform**:
$\{\{R(a), P(a, g(y_1))\}, \{\neg P(f(h(y_1)), y)\}\}$

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Beim Erstellen und Analysieren prädikatenlogischer Formeln ist es oft hilfreich, einen Zwischenschritt in eine „formelähnliche, umgangssprachliche Formulierung“ zu machen
  - Prädikatenlogische Formeln lassen sich in äquivalente Formeln in
    - Pränexform,
    - Negationsnormalform, und
    - reduzierte Form

    umwandeln
  - Prädikatenlogische Formeln lassen sich in erfüllbarkeitsäquivalente Formeln in Skolemform umwandeln
  - Die Matrixklauselform ergibt sich als Klauselmenge einer Formel in Skolemform, deren quantorenfreier Anteil in KNF ist

Anhang: Weitere Modellierungsbeispiele

Prädikatenlogik: Formeln für Zeichenketten erstellen

Beispiel

- Zur Erinnerung: Strings w über $\{a, b, c\}$ werden modelliert durch Strukturen mit
  - der Menge $A = \{1, \ldots, n\}$ als Grundmenge, ☞ n = Länge von w
  - einer 2-stelligen Relation $\leq$,
  - 1-stelligen Relationen Z_a, Z_b, Z_c
  - Funktionen ℓ und r für linke und rechte Nachbarpositionen

- Dabei gelten:
  - Eine Position i ist genau dann die erste Position eines Strings, wenn $\ell(i) = i$ gilt
  - Eine Position i ist genau dann die letzte Position eines Strings, wenn $r(i) = i$ gilt

- Wir charakterisieren im Folgenden Mengen von Strings durch PL-Formeln
- Wir betrachten nur nicht-leere Strings, da immer $A \neq \emptyset$ gelten muss

- **Strings von der Form a^*b^***
  - „beliebig viele a gefolgt von beliebig vielen b“
  - **Formelähnlich:** Für alle Positionen x und y gilt: x ist keine c-Position und falls x eine a-Position und y eine b-Position ist, so gilt $x \leq y$
  - **Als Formel:**
    $$\forall x\, \forall y\, [\neg Z_c(x) \wedge \big((Z_a(x) \wedge Z_b(y)) \rightarrow x \leq y\big)]$$

- **Annahme:** Wir betrachten ausschließlich Strings
  - ➡ An keiner Position stehen zwei Zeichen

Prädikatenlogik: Formeln für Zeichenketten erstellen

Beispiel (Fortsetzung)

- **Strings, die mit abc beginnen**
  - **Formelähnlich:** Es gibt eine Position x, die keinen linken Nachbarn hat und an der ein a steht, an deren rechter Nachbarposition ein b steht, und wiederum an deren rechter Nachbarposition ein c steht
  - **Als Formel:**
    $$\exists x \big(\ell(x) = x \wedge \neg(x = r(x)) \wedge \neg(r(x) = r(r(x))) \wedge Z_a(x) \wedge Z_b(r(x)) \wedge Z_c(r(r(x))) \big)$$

- **Strings, die abwechselnd aus a's und b's bestehen**
  - **Formelähnlich:** Für alle unterschiedlichen Positionen x und y, für die y der rechte Nachbar von x ist, steht
    - an Position x ein a und an y ein b, oder
    - an x steht ein b und an y ein a
  - **Als Formel:**
    $$\forall x \forall y \, \big((r(x) = y \wedge \neg(x = y)) \rightarrow [(Z_a(x) \wedge Z_b(y)) \vee (Z_b(x) \wedge Z_a(y))] \big)$$

Prädikatenlogik: Formeln für Zeichenketten verstehen

Beispiel

- $\forall x\ [\neg(r(x){=}x) \rightarrow \neg(Z_a(x) \wedge Z_a(r(x)))]$
  - **Formelähnlich:** Für jede Position x, die nicht die letzte Position ist, gilt, dass nicht an ihr und an ihrem rechten Nachbar ein a steht
  - **Prägnanter:** aa kommt nicht als Teilstring vor

- $\forall x\ [\ell(x){=}x \vee Z_a(x)]$
  - **Formelähnlich:** Für jede Position x gilt, dass sie keinen linken Nachbarn hat oder dass an ihr ein a steht
  - **Prägnanter:** Das erste Zeichen ist beliebig, danach kommen nur noch Positionen mit a

Kapitel C.3:

Erfüllbarkeit: Grundresolution

Inhalt

1. Prädikatenlogische Folgerungen
2. Auf dem Weg zu einem Erfüllbarkeitstest
3. Ein prädikatenlogischer Erfüllbarkeitstest: Grundresolution
4. Vorschau: Prädikatenlogische Resolution
5. Anhang: Formeln mit unendlichen Modellen
6. Anhang: Beweisskizze zum Satz von Herbrand

Prädikatenlogische Folgerungen

Prädikatenlogisches Folgern

- Das semantische Folgern ist für die Prädikatenlogik analog zur Aussagenlogik definiert

Definition:
Prädikatenlogische Folgerung

- Sei Φ eine Menge von PL-Formeln und ψ eine PL-Formel
- ψ **folgt aus** Φ, wenn jede zu $\Phi \cup \{\psi\}$ passende Interpretation, die Modell von Φ ist, auch ein Modell von ψ ist
- Schreibweise: $\Phi \models \psi$

- Der Zusammenhang zwischen Folgern und Unerfüllbarkeit ist in der Prädikatenlogik genauso wie in den anderen Logiken

Proposition C.3.1

- Sei Φ eine Menge von PL-Formeln und ψ eine PL-Formel
- Dann gilt:

$$\Phi \models \psi \iff \Phi \cup \{\neg\psi\} \text{ ist unerfüllbar}$$

- **Ziel dieses Kapitels:** Ein Erfüllbarkeitstest für Prädikatenlogik
- **Zunächst:** Zwei ausführlichere Anwendungsbeispiele für prädikatenlogische Folgerungen

Beispiel: Wissensbasierte Systeme (1/2)

- In **wissensbasierten Systemen** werden Fakten und Regeln gespeichert, aus denen neues Wissen gefolgert werden kann

Beispiel

- Beispiel-Fakten:

```
parent_of(bob,anne).
parent_of(dave,bob).
parent_of(eve,dave).
```

- Beispiel-Regeln:

```
ancestor_of(X,Y) :- parent_of(X,Y).
ancestor_of(X,Y) :-
    parent_of(X,Z),ancestor_of(Z,Y).
```

- Solche Regeln lassen sich durch prädikatenlogische Formeln beschreiben

- Verwende:
  - Konstantensymbole:
    - bob, anne, dave, eve
  - Relationssymbole:
    - $A \equiv$ ancestor_of
    - $P \equiv$ parent_of

- Die Fakten übersetzen sich in die atomaren Formeln

  $P(\text{bob}, \text{anne}), P(\text{dave}, \text{bob}), P(\text{eve}, \text{dave})$

- Die Regel

```
ancestor_of(X,Y) :- parent_of(X,Y)
```

  entspricht der PL-Formel

  $$\forall x \, \forall y \, (P(x, y) \rightarrow A(x, y))$$

- Die Regel

```
ancestor_of(X,Y) :-
    parent_of(X,Z),ancestor_of(Z,Y)
```

  entspricht der PL-Formel

  $$\forall x \, \forall y \, \big(\exists z \, (P(x, z) \wedge A(z, y)) \rightarrow A(x, y)\big)$$

Beispiel: Wissensbasierte Systeme (2/2)

- **Problem:** Wir können nicht annehmen, dass die Wissensbasis alle Fakten enthält
- **Deshalb die Frage:** Welche weiteren Fakten folgen aus den gegebenen Fakten und Regeln?
  - ➡ **In anderen Worten:** Welche Fakten gelten in allen Strukturen, in denen die Fakten und Schlussregeln der Wissensbasis gelten

Beispiel

- Folgt das Faktum

  `ancestor_of(eve,anne)`

  aus den gegebenen Beispiel-Fakten und Beispiel-Regeln?
- Also: Gilt

  $A(\text{eve}, \text{anne})$

  in allen Modellen der Formelmenge, die sich aus den Beispiel-Fakten und Beispiel-Regeln ergibt

- Das ist der Fall, wenn folgende Menge unerfüllbar ist:

$$\{P(\text{bob}, \text{anne}), P(\text{dave}, \text{bob}),$$
$$P(\text{eve}, \text{dave}),$$
$$\forall x\, \forall y\, (P(x, y) \rightarrow A(x, y)),$$
$$\forall x\, \forall y \big(\exists z\, (P(x, z) \wedge A(z, y)) \rightarrow A(x, y)\big),$$
$$\neg A(\text{eve}, \text{anne})\}$$

- Wir nutzen hier:

  $\Phi \models \psi \;\Leftrightarrow\; \Phi \cup \{\neg\psi\}$ ist unerfüllbar
- Also: Semantisches Schließen verhält sich zur Erfüllbarkeit genauso, wie bei den bisher betrachteten Logiken

Beispiel: Mathematische Theoreme (1/2)

- Die Mathematik lässt sich in der Prädikatenlogik formalisieren
- Mathematische Theoreme sind dann zumeist von der Art $\Phi \models \psi$, wobei Φ eine Menge von Axiomen ist und ψ ein „mathematisches Theorem“

Beispiel

- Die **Gruppentheorie** beschäftigt sich mit Strukturen mit einer Verknüpfung $\circ$ und einem ausgezeichneten Einselement e
- Eine **Gruppe** $G = (U^G, \circ^G, e^G)$ besteht aus
  - einer Menge U^G
  - einer binären Verknüpfung $\circ^G$ auf U^G (hier in Infix-Notation geschrieben) und
  - einem **neutralen Element** $e^G \in U^G$

  und erfüllt die folgenden **Axiome**:

  (G1) $\forall x \forall y \forall z\ (x \circ y) \circ z = x \circ (y \circ z)$
  (G2) $\forall x\ e \circ x = x$
  (G3) $\forall x \exists y\ y \circ x = e$

Beispiel: Mathematische Theoreme (2/2)

Beispiel (Fortsetzung)

- **(Einfaches) Theorem:** In einer Gruppe gibt es für je zwei Elemente a, b immer ein Element c mit $a \circ c = b$

- Die Aussage, dass dies ein gültiges Theorem ist, ist gleichbedeutend mit der Aussage

$$\{(G1), (G2), (G3)\} \models \forall x\ \forall y\ \exists z\ x \circ z = y$$

- Diese Aussage ist äquivalent dazu, dass

$$\{(G1), (G2), (G3), \neg\forall x\ \forall y\ \exists z\ x \circ z = y\}$$

unerfüllbar ist

- Das Theorem gilt also gerade dann, wenn es keine passende Struktur gibt, in der die Axiome gelten, aber nicht die zusätzliche Formel

✎ Erfüllbarkeitstests für die Prädikatenlogik können also im Prinzip als Basis für automatische Theorem-Beweiser dienen

Auf dem Weg zu einem Erfüllbarkeitstest

Wiederholung: Vereinfachung prädikatenlogischer Formeln

- Wir haben im letzten Kapitel gesehen:
  - Jede prädikatenlogische Formel φ kann in eine äquivalente Formel φ_1 in Pränexform umgewandelt werden (C.2.3 (c))
  - Zu jeder prädikatenlogischen Formel φ_1 in Pränexform kann eine erfüllbarkeitsäquivalente, geschlossene Formel ψ in Skolemform berechnet werden (C.2.5)
  - Der quantorenfreie Teil jeder geschlossenen prädikatenlogischen Formel ψ in Skolemform kann in konjunktive Normalform gebracht werden und zu dieser kann die Matrixklauselform $\mathcal{K}$ berechnet werden

- Bis jetzt haben wir also Umwandlungen betrachtet, die zu syntaktischen Vereinfachungen geführt haben
- Jetzt werden wir sehen, wie wirklich Erfüllbarkeit getestet werden kann

Erfüllbarkeit prädikatenlogischer Formeln: Vorüberlegungen

- In der **Aussagenlogik** ist Erfüllbarkeit konzeptionell sehr einfach zu testen
- Es genügt, alle passenden Wahrheitsbelegungen auszuprobieren
- Da es nur endlich viele gibt, terminiert dieses Verfahren

  ✎ Wir kennen natürlich noch andere Verfahren

- Bei der **Modallogik** war a priori nicht klar, dass es genügt, begrenzt viele „Modellkandidaten“ zu testen
- Aber: Es reicht, baumförmige Kripkestrukturen einer bestimmten Maximaltiefe zu testen
- Das führt zu einem Verfahren, das garantiert terminiert

- In der **Prädikatenlogik** ist die Lage noch komplizierter
- **Frage:** Wo sollen wir für eine erfüllbare Formel nach Modellen suchen?
  (1) Was könnte die Grundmenge sein?
  (2) Wie sollen die Funktionen und Konstanten gewählt werden?
  (3) Wie sollen die Relationen gewählt werden?

- **Wir werden sehen:**
  - Erste schlechte Nachricht: es gibt prädikatenlogische Formeln, die nur Modelle mit unendlich großer Grundmenge haben
  - Gute Nachricht: (1) und (2) lassen sich trotzdem überraschend einfach beantworten
  - Zweite schlechte Nachricht: (3) ist wirklich eine harte Nuss. . .

Erfüllbarkeit von PL-Formeln: Unendliche Modelle

- Schlechte Nachricht:

  Es gibt prädikatenlogische Formeln, die ausschließlich unendlich große Modelle haben

Beispiel

- Sei $\varphi = \forall x \neg P(x,x)$
  $\wedge \quad \forall x \, \exists y \, P(x,y)$
  $\wedge \quad \forall u \, \forall v \, \forall w$
  $\big((P(u,v) \wedge P(v,w)) \rightarrow P(u,w)\big)$

- φ hat das Modell $\mathcal{A} = (\mathbb{N}, P^{\mathcal{A}})$ mit

  $P^{\mathcal{A}}(n,m) \overset{\text{def}}{\Leftrightarrow} n < m$, für alle $n, m \in \mathbb{N}$

- $\mathcal{A}$ hat eine unendlich große Grundmenge

- **Aber:** φ hat kein Modell mit endlicher Grundmenge

  ☞ Beweis im Anhang

Auf der Suche nach einfachen Modellen: Einstieg

- **Also:** Es kann sein, dass eine Formel nur unendliche Modelle hat

- **Jetzt:** Bei der Suche nach Modellen für eine Formel φ können wir uns auf Strukturen eines einfachen Types beschränken
  - ➡ Diese Strukturen heißen **Herbrand-Strukturen**

- In Herbrand-Strukturen sind...
  - ...die Grundmenge, die Konstanten und die Funktionen durch φ schon eindeutig festgelegt
  - ...die Relationen noch **nicht** festgelegt

- Ein Erfüllbarkeitstest muss dann „nur“ noch überprüfen, ob die Relationen so gewählt werden können, dass φ wahr wird
  - ➡ Dieses Problem wird sich allerdings als sehr schwierig herausstellen...

- **Zur Erinnerung:** es genügt, einen Erfüllbarkeitstest für geschlossene Skolemformeln zu finden
  - ➡ Beliebige Formeln können auf diese zurückgeführt werden

- **Zur Vereinfachung:** Wir betrachten in diesem und dem folgenden Kapitel die Prädikatenlogik **ohne Gleichheit**

Herbrand-Strukturen (1/4)

Beispiel

- Sei $\varphi_1' = \forall x \, \exists y \, \big(\neg E(a,a) \wedge E(x,y)\big)$
- Skolemisierung ergibt die Formel

$$\varphi_1 = \forall x \, \big(\neg E(a,a) \wedge E(x, f(x))\big)$$

- Nehmen wir an, $\mathcal{A}$ ist ein Modell von φ_1
- Was wissen wir dann über $\mathcal{A}$?
  - ➡ $\mathcal{A}$ muss ein Element $a^{\mathcal{A}}$ enthalten
  - ➡ $\mathcal{A}$ muss auch ein Element $f^{\mathcal{A}}(a^{\mathcal{A}})$ enthalten
  - ➡ $\mathcal{A}$ muss Elemente $f^{\mathcal{A}}(f^{\mathcal{A}}(a^{\mathcal{A}}))$, $f^{\mathcal{A}}(f^{\mathcal{A}}(f^{\mathcal{A}}(a^{\mathcal{A}})))$, ... enthalten

- **Kurz gesagt:** Für jeden Term t, der über der Signatur von φ_1 gebildet werden kann, muss $\mathcal{A}$ ein Element $t^{\mathcal{A}}$ enthalten
- **Aber:** Welche dieser Elemente zueinander gleich oder verschieden sind, wissen wir dabei nicht

Herbrand-Strukturen (2/4)

- In einem Modell $\mathcal{A}$ einer Formel φ muss es also zu jedem Grundterm t ein Element $t^{\mathcal{A}}$ in $\mathcal{A}$ geben
- **Erinnerung:** Grundterme sind Terme ohne Variablen

- **Grundidee von Herbrand-Strukturen:**

  Versuche, ein Modell zu finden, in dem die Grundmenge **genau ein Element je Grundterm t** hat
- Noch konkreter: Herbrand-Strukturen $\mathcal{H}$ haben die Menge $\mathrm{GT}(\mathcal{S}(\varphi))$ als Grundmenge

  ➡ Also: Grundterme über Symbolen aus φ

Beispiel

- Eine Herbrand-Struktur $\mathcal{H}$ zu

  $$\varphi_1 = \forall x \left(\neg E(a,a) \wedge E(x, f(x))\right)$$

  hat also die Grundmenge

  $$\{a, f(a), f(f(a)), f(f(f(a))), \ldots\}$$

- Kleine Komplikation: Formeln ohne Konstantensymbole

Beispiel

- $\varphi_2 = \forall x \left(P(x) \wedge \neg P(f(x))\right)$

- Bei Formeln ohne Konstantensymbole nehmen wir immer das Konstantensymbol a als „Basis“ der Grundmenge hinzu

Beispiel

- Die Grundmenge einer Herbrandstruktur für φ_2 ist also (auch)

  $$\{a, f(a), f(f(a)), f(f(f(a))), \ldots\}$$

Herbrand-Strukturen (3/4)

- **Frage:** Wie sollen Konstantensymbole und Funktionensymbole in Herbrand-Strukturen interpretiert werden?
- **Ansatz:** Gehe so naiv wie möglich vor...

- **Konstanten:** Kommt das Konstantensymbol a in einer Formel vor, so kommt a in der Grundmenge jeder Herbrand-Struktur $\mathcal{H}$ vor
  ➡ Wähle $a^{\mathcal{H}} \stackrel{\text{def}}{=} a$
- **Funktionen:** Kommen a, b und f in einer Formel vor, so kommt $f(a,b)$ in der Grundmenge jeder Herbrand-Struktur $\mathcal{H}$ vor
  ➡ Wähle $f^{\mathcal{H}}(a,b) \stackrel{\text{def}}{=} f(a,b)$

- **Allgemein:**

$$f^{\mathcal{H}}(t_1,\ldots,t_k) \stackrel{\text{def}}{=} f(t_1,\ldots,t_k)$$

Definition:
Herbrand-Universum, -Struktur, -Modell

- Sei φ eine geschlossene Skolemformel
- Das **Herbrand-Universum** $H(\varphi)$ von φ sei
  - $\mathrm{GT}(\mathcal{S}(\varphi))$, falls φ ein Konstantensymbol enthält, und
  - $\mathrm{GT}(\mathcal{S}(\varphi) \cup \{a\})$ andernfalls

- Eine Struktur $\mathcal{H}$ heißt **Herbrand-Struktur** für φ, falls gilt:
  - Die Grundmenge von $\mathcal{H}$ ist $H(\varphi)$
  - Ist c ein Konstantensymbol von φ, so ist $c^{\mathcal{H}} = c$
  - Ist f ein k-stelliges Funktionssymbol von φ und sind $t_1,\ldots,t_k \in H(\varphi)$, so ist

$$f^{\mathcal{H}}(t_1,\ldots,t_k) = f(t_1,\ldots,t_k)$$

- Ein **Herbrand-Modell** einer Formel φ ist eine Herbrand-Struktur für φ, die ein Modell für φ ist

Herbrand-Strukturen (4/4)

Definition:
Herbrand-Universum, -Struktur, -Modell

- Sei φ eine geschlossene Skolemformel
- Das **Herbrand-Universum** $H(\varphi)$ von φ sei
  - $\mathrm{GT}(\mathcal{S}(\varphi))$, falls φ ein Konstantensymbol enthält, und
  - $\mathrm{GT}(\mathcal{S}(\varphi) \cup \{a\})$ andernfalls

- Eine Struktur $\mathcal{H}$ heißt **Herbrand-Struktur** für φ, falls gilt:
  - Die Grundmenge von $\mathcal{H}$ ist $H(\varphi)$
  - Ist c ein Konstantensymbol von φ, so ist $c^{\mathcal{H}} = c$
  - Ist f ein k-stelliges Funktionssymbol von φ und sind $t_1, \ldots, t_k \in H(\varphi)$, so ist
    $$f^{\mathcal{H}}(t_1, \ldots, t_k) = f(t_1, \ldots, t_k)$$

- Ein **Herbrand-Modell** einer Formel φ ist eine Herbrand-Struktur für φ, die ein Modell für φ ist

Bemerkungen:

- Alle Herbrand-Strukturen einer gegebenen Formel haben dieselbe Grundmenge, dieselben Konstanten und dieselben Funktionen
- Einziger Freiheitsgrad: Interpretation der Relationssymbole

- Herbrand-Strukturen wurden nur für geschlossene Formeln in Skolemform definiert
- Für allgemeine Formeln ψ vereinbaren wir:

  $\mathcal{H}$ ist Herbrand-Struktur für ψ, wenn es eine Skolemisierung φ von ψ gibt, so dass $\mathcal{H}$ eine Herbrand-Struktur für φ ist

Kurz-Bio: Jacques Herbrand

Beispiel



Jacques Herbrand

Bild: © Gemeinfrei

- Geboren: 12.2.1908 in Paris
- Studium in Paris
- Forschungsaufenthalte in Berlin bei von Neumann, in Hamburg bei Emil Artin und Göttingen bei Emmy Noether
- Am 27.7.1931 beim Bergsteigen in den französischen Alpen tödlich verunglückt

Satz von Löwenheim-Skolem

Satz C.3.2: Satz von Herbrand

- Eine prädikatenlogische Formel ist genau dann erfüllbar, wenn sie ein Herbrand-Modell hat

- **Beobachtung:** Die Menge $\mathrm{GT}(\mathcal{S}(\varphi))$ ist für jede Formel φ höchstens abzählbar unendlich
- Damit ergibt sich als Folgerung das folgende wichtige Resultat:

Satz C.3.3: Löwenheim-Skolem

- Eine prädikatenlogische Formel ist genau dann erfüllbar, wenn sie ein Modell mit höchstens abzählbar unendlicher Grundmenge hat

- Der Satz von Löwenheim-Skolem gilt auch für die Prädikatenlogik mit Gleichheit

Leopold Löwenheim und Thoralf Skolem

Kurz-Bio: Leopold Löwenheim



Leopold Löwenheim
Bild: © kulturimpuls.org

- Geboren: 26.6.1878 in Krefeld
- Studium in Berlin
- Mathematik-Lehrer ab 1906
- Teilnahme am ersten Weltkrieg 1915/1916
- Als „Nicht-Arier“ 1934 aus dem Schuldienst entlassen
- 1946-1949: Lehrer in Berlin
- Gestorben: 5.5.1957 in Berlin

Kurz-Bio: Thoralf Skolem



Thoralf Skolem
Bild: Gemeinfrei

- Geboren: 23.5.1887 in Oslo
- Studium in Oslo (Kristiania) ab 1905
- Dozent in Oslo und Bergen
- Gestorben: 23.3.1963 in Oslo

Ein prädikatenlogischer Erfüllbarkeitstest: Grundresolution

Auf der Suche nach Herbrand-Modellen

Beispiel

- Für $\varphi = \forall x \left(P(x) \wedge \neg P(f(x))\right)$ gilt:

  $$H(\varphi) = \{a, f(a), f(f(a)), f(f(f(a))), \ldots\}$$

- Insbesondere müssen gleichzeitig gelten:

  $\{P(a) \wedge \neg P(f(a)),$ wegen $x \mapsto a$

  $P(f(a)) \wedge \neg P(f(f(a))),$ wegen $x \mapsto f(a)$

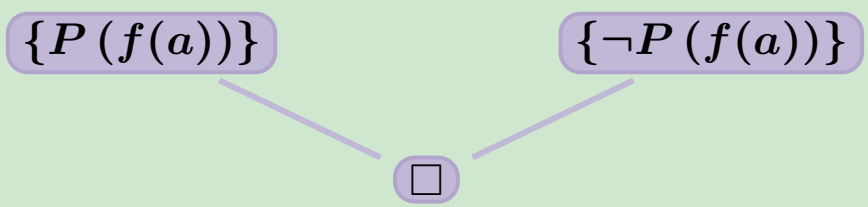
  $P(f(f(a))) \wedge \neg P(f(f(f(a)))),$ wegen $x \mapsto f(f(a))$

  $\ldots\}$

- Aber: $P(f(a))$ und $\neg P(f(a))$ können nicht gleichzeitig gelten
- Deshalb: φ ist unerfüllbar

- **Allgemeiner Ansatz:** Leite solche Widersprüche mit **Grundresolution** her
- **Dazu:** Fasse atomare Formeln $P(a)$, $P(f(a))$, $P(f(f(a)))$, ... als aussagenlogische Variablen auf

- Unerfüllbarkeit via Grundresolution:



Grundresolution: Idee

- **Ziel:** Führe die Frage, ob es geeignete Relationen für ein Herbrand-Modell gibt, auf das aussagenlogische Erfüllbarkeitsproblem zurück

- Sei $\varphi = \forall x_1 \cdots \forall x_\ell \; \psi$ eine geschlossene Skolemformel mit Matrix-Formel ψ
- Damit φ in einem Herbrand-Modell wahr wird, muss ψ für alle Belegungen, die die Variablen $x_1, \ldots, x_\ell$ auf Grundterme $t_1, \ldots, t_\ell$ abbilden, wahr werden

- **Ansatz der Grundresolution:**
  - Ersetze die Variablen in ψ auf alle möglichen Weisen durch Terme
  - Aus ψ entsteht dadurch eine (in der Regel abzählbar unendliche) Menge von Formeln

    ➡ **Herbrand-Expansion**
  - Die Erfüllbarkeit dieser Menge kann dann mit aussagenlogischer Resolution getestet werden

    ➡ **Dazu:** Fasse atomare Formeln als aussagenlogische Variablen auf

Herbrand-Expansionen

Herbrand-Expansion

- Sei $\varphi = \forall x_1 \cdots \forall x_\ell \ \psi$ eine geschlossene Skolemformel mit Matrix-Formel ψ

- Die **Herbrand-Expansion** $E(\varphi)$ von φ sei die folgende Menge von PL-Formeln:

$$\{\psi[x_1/t_1, \ldots, x_\ell/t_\ell] \mid t_1, \ldots, t_\ell \in H(\varphi)\}$$

- $E(\varphi)$ ist **aussagenlogisch erfüllbar**, wenn es eine Belegung der atomaren Formeln von $E(\varphi)$ mit Wahrheitswerten gibt, die alle Formeln in $E(\varphi)$ wahr macht

- Wir erhalten den folgenden Satz, der, gegeben Satz C.3.2, nicht schwierig zu beweisen ist

Satz C.3.4: Gödel, Herbrand, Skolem

- Eine geschlossene Skolemformel φ ist genau dann erfüllbar, wenn $E(\varphi)$ aussagenlogisch erfüllbar ist

Grundresolution: Erstes Beispiel

Beispiel

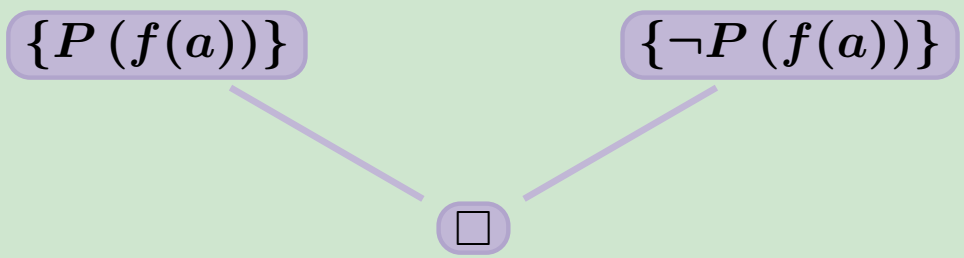
- Für $\varphi = \forall x \left(P(x) \wedge \neg P(f(x))\right)$ gilt:

$$H(\varphi) = \{a, f(a), f(f(a)), f(f(f(a))), \ldots\}$$

$$\begin{aligned} E(\varphi) = \{ & P(a) \wedge \neg P(f(a)), && \text{wegen } x \mapsto a \\ & P(f(a)) \wedge \neg P(f(f(a))), && \text{wegen } x \mapsto f(a) \\ & P(f(f(a))) \wedge \neg P(f(f(f(a)))), && \text{wegen } x \mapsto f(f(a)) \\ & P(f(f(f(a)))) \wedge \neg P(f(f(f(f(a))))), && \text{wegen } x \mapsto f(f(f(a))) \\ & \ldots\} \end{aligned}$$

- Die atomaren Formeln $P(a)$, $P(f(a))$, $P(f(f(a)))$, ... fassen wir nun als aussagenlogische Variablen auf

- Unerfüllbarkeit via Grundresolution:



- Leider wird nicht immer so schnell eine unerfüllbare Klauselmenge erreicht

Grundresolution: Zweites Beispiel

Beispiel

- Sei $\varphi = \forall x\, \forall y \left(R(x) \wedge (P(x) \vee \neg R(y) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,y)))\right)$
- Herbrand-Universum und -Expansion:

$$H(\varphi) = \{a, b, f(a), g(a,a), f(b), g(a,b), g(b,a), g(b,b), f(f(a)), \ldots\}$$

$$E(\varphi) = \{R(a) \wedge (P(a) \vee \neg R(a) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,a)))$$

wegen $x \mapsto a, y \mapsto a$

$$R(f(a)) \wedge (P(f(a)) \vee \neg R(a) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,a)))$$

wegen $x \mapsto f(a), y \mapsto a$

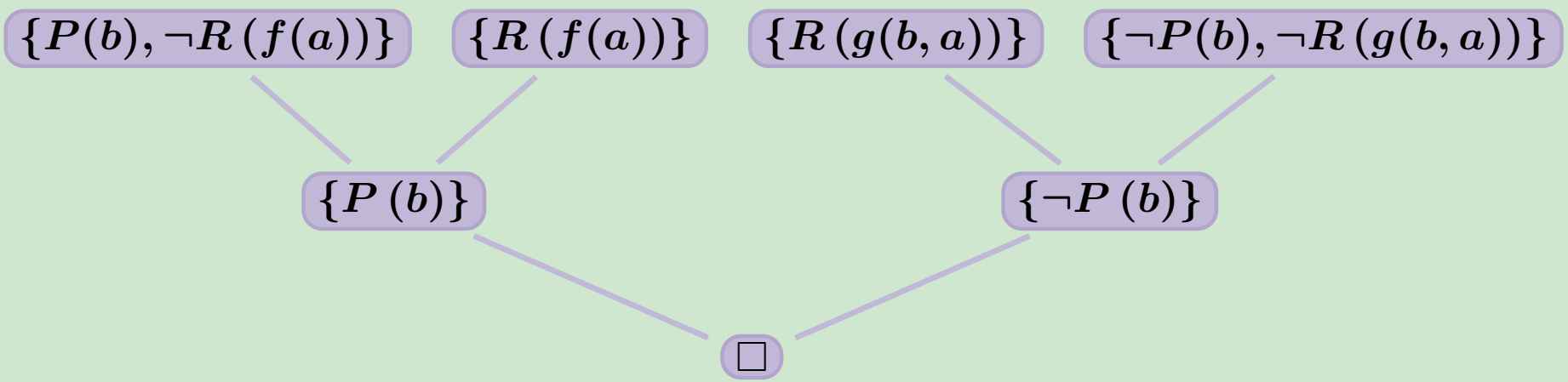
$$R(b) \wedge (P(b) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,f(a))))$$

wegen $x \mapsto b, y \mapsto f(a)$

$$R(g(b,a)) \wedge (P(g(b,a)) \vee \neg R(a) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,a)))$$

wegen $x \mapsto g(b,a), y \mapsto a$

$$\ldots\}$$



Erfüllbarkeitstest für prädikatenlogische Formeln

- Wir erhalten den folgenden Algorithmus

Grundresolutionsalgorithmus

Eingabe: Geschlossene Skolemformel φ
Ausgabe: „unerfüllbar“, falls φ unerfüllbar ist, andernfalls „erfüllbar“ oder keine Termination

1: $M := \emptyset$
2: **for** $\psi \in E(\varphi)$ (in lexikographischer Reihenfolge) **do**
3: $\quad M := M \cup \{\psi\}$
4: $\quad$ **if** M aussagenlogisch unerfüllbar **then**
5: $\quad\quad$ **return** „unerfüllbar“
6: $\quad$ **end if**
7: **end for**
8: **return** „erfüllbar“

- **Bemerkungen:**

- Der Algorithmus terminiert nur, wenn φ unerfüllbar oder $E(\varphi)$ endlich ist
- Wir sagen, dass das Unerfüllbarkeitsproblem für PL-Formeln **semi-entscheidbar** ist, weil es einen Algorithmus gibt, der
  - für unerfüllbare Formeln anhält und „unerfüllbar“ ausgibt und
  - für erfüllbare Formeln möglicherweise nicht terminiert

- Semi-entscheidbare Probleme werden im Modul **Grundbegriffe der Theoretischen Informatik** behandelt

- Weil der Algorithmus nur Grundterme verwendet, heißt er **Grund**resolutionsalgorithmus
  - ➡ . . . im Gegensatz zur prädikatenlogischen Resolution

Vorschau: Prädikatenlogische Resolution

Prädikatenlogische Resolution: Vorschau

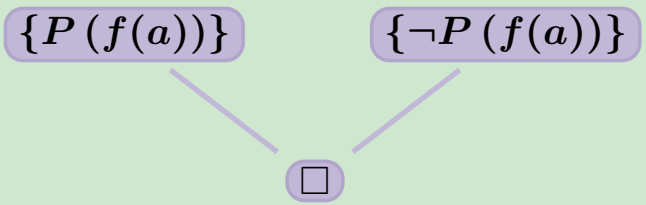
Beispiel: Grundresolution

- Test der Erfüllbarkeit von

$$\varphi_2 = \forall x\, [P(x) \wedge \neg P(f(x))]$$

mit Grundresolution

- Ansatz:
  - $H(\varphi_2) = \{a, f(a), f(f(a)), f(f(f(a))), \ldots\}$
  - $E(\varphi_2) = \{P(a) \wedge \neg P(f(a)), P(f(a)) \wedge \neg P(f(f(a))), \ldots\}$



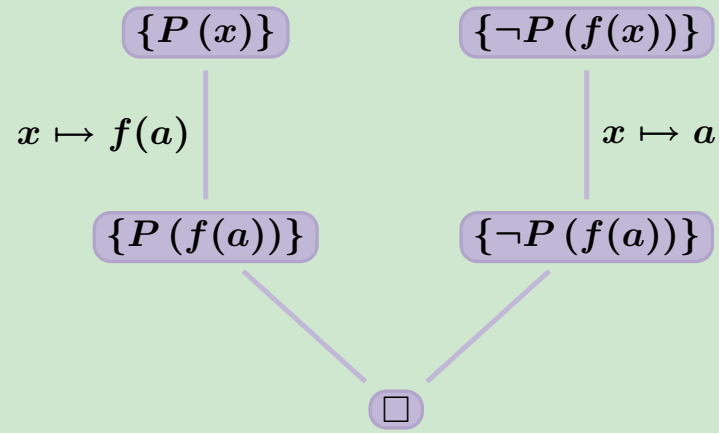
- Grundresolution arbeitet mit Brute-Force:
  - Zunächst werden „wahllos“ Klauseln der Herbrand-Expansion erzeugt und
  - dann wird versucht, passende Klauseln für Resolutionsschritte zu finden

- Können wir zielgerichteter vorgehen?

Beispiel: Prädikatenlogische Resolution

- **Jetzt:** Ein „direkterer“ Resolutionsbeweis für die Unerfüllbarkeit von φ_2
- Matrixklauselform von φ_2:

$$\{P(x)\}, \{\neg P(f(x))\}$$



- **Prädikatenlogische Resolution** versucht, die Box direkt aus Klauseln der Matrixklauselform herzuleiten

☞ nächstes Kapitel

- Resolutionsschritte werden dabei zumeist erst durch Anwendung geeigneter **Substitutionen** ermöglicht

☞ jetzt

Substitutionen: Definition (1/2)

- **Jetzt:** Prädikatenlogische Substitutionen

- Variablen spielen in der Aussagenlogik und der Prädikatenlogik völlig unterschiedliche Rollen:
  - Aussagenlogische Variablen stehen für Aussagen, können also wahr oder falsch sein
  - Prädikatenlogische Variablen stehen für Elemente der Grundmenge

- Dem entsprechend sind Substitutionen in Aussagenlogik und Prädikatenlogik sehr verschieden:
  - **Aussagenlogische Substitutionen** ordnen aussagenlogischen Variablen jeweils aussagenlogische **Formeln** zu
  - **Prädikatenlogische Substitutionen** ordnen prädikatenlogischen Variablen jeweils **Terme** zu

- Wir notieren prädikatenlogische Substitutionen häufig in der Form

$$\sigma = \{x_1 \mapsto t_1, \ldots, x_k \mapsto t_k\}$$

Beispiel

- Sei $L = \neg R(x, y, x)$
- Für $\sigma = \{y \mapsto g(b)\}$ ist

$$\sigma(L) = \neg R(x, g(b), x)$$

- Für $\sigma = \{x \mapsto f(y)\}$ ist

$$\sigma(L) = \neg R(f(y), y, f(y))$$

- Für $\sigma = \{x \mapsto f(y), y \mapsto g(b)\}$ ist

$$\sigma(L) = \neg R(f(y), g(b), f(y))$$

Substitutionen: Definition (2/2)

Definition: Substitution

- Eine prädikatenlogische **Substitution** ist eine partielle Funktion $\sigma : \mathrm{PV} \rightharpoonup \mathrm{PT}$

- Ist σ eine Substitution und L ein Literal, so bezeichnet $\sigma(L)$ das Literal, das aus L durch **simultane** Ersetzung aller Variablen x durch $\sigma(x)$ entsteht
  - ➡ Dabei bleiben Variablen x, für die $\sigma(x)$ undefiniert ist, unverändert

- σ heißt **Grundsubstitution**, falls $\sigma(x)$ für kein x Variablen enthält

- **Wichtig:** Die Ersetzungen finden nicht nacheinander, sondern simultan statt
- Beispiel: Ist $\sigma = \{x \mapsto y, y \mapsto a\}$, so gilt

  $$\sigma(R(x, y)) = R(y, a)$$

Komposition von Substitutionen

Definition: Komposition von Substitutionen

- Sind σ, τ Substitutionen, so bezeichnet $\tau \circ \sigma$ die Substitution, die durch Hintereinanderausführung von (zuerst) σ und (dann) τ entsteht

Beispiel

- $\{x \mapsto a, y \mapsto f(b), z \mapsto y\} \circ \{u \mapsto f(x), v \mapsto g(y), y \mapsto b\}$
  $= \{u \mapsto f(a),$
  $v \mapsto g(f(b)),$
  $y \mapsto b,$
  $x \mapsto a,$
  $z \mapsto y\}$

Beispiel

- $\{x \mapsto y\} \circ \{y \mapsto a\} \neq \{y \mapsto a\} \circ \{x \mapsto y\}$

Definition: Komposition mit Belegungen

- Ist β eine Belegung und σ eine Substitution, so bezeichnet $\beta \circ \sigma$ die Belegung, die durch Hintereinanderausführung von (zuerst) σ und (dann) β entsteht

Beispiel

- Sei $\mathcal{A}$ eine Struktur mit $a^{\mathcal{A}} = 1$ und $f^{\mathcal{A}}(3) = 5$
- Sei $\beta = \{x \mapsto 2, u \mapsto 3\}$ eine Belegung
- Sei $\sigma = \{x \mapsto a, y \mapsto f(u), z \mapsto x\}$ eine Substitution

- Dann gilt
  - $[\![x]\!]_{(\mathcal{A}, \beta \circ \sigma)} = 1$
  - $[\![y]\!]_{(\mathcal{A}, \beta \circ \sigma)} = 5$

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Erfüllbarkeitstests für prädikatenlogische Formeln sind wichtig für die Mathematik und die Informatik
  - Das Erfüllbarkeitsproblem ist für die Prädikatenlogik aber erheblich schwieriger als für die Aussagenlogik und die Modallogik
  - Immerhin genügt es als potentielle Modelle Herbrandstrukturen zu betrachten
  - Dies führt zum Grundresolutionsalgorithmus, der bei erfüllbaren Formeln aber nicht immer terminiert

Anhang: Formeln mit unendlichen Modellen

Erfüllbarkeit von PL-Formeln: unendliche Modelle

- Wir betrachten zunächst die erste schlechte Nachricht:

  es gibt prädikatenlogische Formeln, die nur unendlich große Modelle haben

Beispiel

- Sei $\varphi = \forall x \neg P(x,x) \wedge \forall x\, \exists y\, P(x,y) \wedge \forall u\, \forall v\, \forall w \big((P(u,v) \wedge P(v,w)) \rightarrow P(u,w)\big)$

- φ hat das Modell $\mathcal{A} = (\mathbb{N}, P^{\mathcal{A}})$:
  - $P^{\mathcal{A}}(n,m) \overset{\text{def}}{\Leftrightarrow} n < m$, für alle $n, m \in \mathbb{N}$

  mit unendlicher Grundmenge

Proposition C.3.5

- Die Formel φ aus dem Beispiel hat kein Modell mit endlicher Grundmenge

Beweisskizze

- **Beweis durch Widerspruch:** Angenommen, es gibt ein Modell $\mathcal{B} = (U, P^{\mathcal{B}})$ für φ mit endlicher Grundmenge $U \neq \emptyset$
- Sei a_1 irgendein Element von U
  - ➡ Wegen $\forall y\, \exists z\, P(y,z)$ muss es ein Element a_2 geben mit $P(a_1, a_2)$
  - ➡ Entsprechend muss es ein Element a_3 geben mit $P(a_2, a_3)$ usw.
  - ➡ Es entsteht eine Folge $a_1, a_2, a_3, \ldots$ unendlicher Länge, deren Elemente $P(a_i, a_{i+1})$ für alle i, erfüllen
- Da U nach Annahme endlich ist, muss es $i < j$ geben mit $a_i = a_j$
- Da P transitiv ist und $(a_i, a_{i+1}) \in P$ für jedes i gilt, folgt mit Induktion $(a_i, a_j) \in P$
- Wegen $a_i = a_j$ gilt dann aber die Teilformel $\forall x\, \neg P(x,x)$ nicht

☞ Widerspruch

Anhang: Beweisskizze zum Satz von Herbrand

Satz von Herbrand (1/3)

Satz C.3.2: Satz von Herbrand

- Eine prädikatenlogische Formel ist genau dann erfüllbar, wenn sie ein Herbrand-Modell hat

Beweisskizze

- Sei o.B.d.A. φ geschlossen und in Skolemform
- Die Richtung „$\Leftarrow$“ ist klar
- Für „$\Rightarrow$“ sei $\mathcal{A}$ ein Modell für φ mit Grundmenge A
- Das Herbrand-Modell $\mathcal{H}$ für φ lässt sich wie folgt konstruieren
  - Die Grundmenge $H(\varphi)$, die Konstanten und Funktionen stehen gemäß der Definition von Herbrand-Strukturen fest
  - Die Relationen werden wie folgt aus $\mathcal{A}$ „kopiert“ : Ist R ein k-stelliges Relationssymbol von φ, so definieren wir:

$$R^{\mathcal{H}} \stackrel{\text{def}}{=} \{(t_1, \ldots, t_k) \mid t_1, \ldots, t_k \in H(\varphi), \mathcal{A} \models R(t_1, \ldots, t_k)\}$$

Satz von Herbrand (2/3)

Beispiel: Konstruktion von $\mathcal{H}$

- Sei $\varphi_1 = \forall x \left(\neg E(a,a) \wedge E(x, f(x))\right)$

- φ_1 hat das Modell $\mathcal{A} = (V, E, f, a)$, wobei
  - (V, E) ist der Graph:



  - $f(1) = 2$ und $f(2) = 1$
  - $a^{\mathcal{A}} = 1$

- Wie können wir aus $\mathcal{A}$ ein Herbrand-Modell $\mathcal{H}$ für φ_1 gewinnen?
  - Universum U von $\mathcal{H}$:

$$a, f(a), f(f(a)), \ldots$$

  - Die Konstante $a^{\mathcal{H}}$ und Funktion $f^{\mathcal{H}}$ sind definiert durch:

$$a^{\mathcal{H}} \stackrel{\text{def}}{=} a$$
$$f^{\mathcal{H}}(t) \stackrel{\text{def}}{=} f(t), \text{ für alle } t \in U$$

- Die Relation $E^{\mathcal{H}}$ lässt sich nun mit Hilfe von $E^{\mathcal{A}}$ definieren:

$$E^{\mathcal{H}} \stackrel{\text{def}}{=} \{(t_1, t_2) \mid t_1, t_2 \in H(\varphi), \mathcal{A} \models E(t_1, t_2)\}$$

- $(t_1, t_2) \in E^{\mathcal{H}}$ gilt also genau dann, wenn $(t_1^{\mathcal{A}}, t_2^{\mathcal{A}}) \in E^{\mathcal{A}}$

- Wie sieht die Relation $E^{\mathcal{H}}$ aus?
  - Es gilt:
    - $a^{\mathcal{A}} = 1$
    - $f^{\mathcal{A}}(a^{\mathcal{A}}) = 2$
    - $f^{\mathcal{A}}(f^{\mathcal{A}}(a^{\mathcal{A}})) = 1$
    - $f^{\mathcal{A}}(f^{\mathcal{A}}(f^{\mathcal{A}}(a^{\mathcal{A}}))) = 2$
    - usw.

➡ Die Relation $E^{\mathcal{H}}$ besteht also aus allen Paaren $(f^i(a), f^j(a))$, für die genau eine der Zahlen i, j ungerade ist

Satz von Herbrand (3/3)

Beweisskizze (Fortsetzung)

- **Zur Erinnerung:** Ist R ein k-stelliges Relationssymbol von φ, so definieren wir:

$$R^{\mathcal{H}} \stackrel{\text{def}}{=} \{(t_1, \ldots, t_k) \mid t_1, \ldots, t_k \in H(\varphi), \mathcal{A} \models R(t_1, \ldots, t_k)\}$$

- **Zu zeigen:** $\mathcal{H} \models \varphi$

- Sei $\varphi = \forall x_1 \ldots \forall x_\ell \psi$
- **Es genügt zu zeigen:** Für jede Abbildung β, die $x_1, \ldots, x_\ell$ auf Grundterme $t_1, \ldots, t_\ell$ abbildet, gilt

$$(\mathcal{H}, \beta) \models \psi \iff \mathcal{A} \models \psi[x_1/t_1, \ldots, x_\ell/t_\ell],$$

  wobei $\psi[x_1/t_1, \ldots, x_\ell/t_\ell]$ aus ψ entsteht, indem jedes x_i durch $\beta(x_i)$ ersetzt wird
- **Ansatz:** Induktion über den Aufbau von ψ

- Auf einen formalen Beweis verzichten wir

- Die obige Beweisskizze für den Satz von Herbrand bezieht sich auf geschlossene Formeln in Skolemform
- Der Beweis im allgemeinen Fall skolemisiert die gegebene Formel φ und konstruiert für die sich ergebende Formel φ' ein Herbrand-Modell $\mathcal{H}$

Kapitel C.4:

Prädikatenlogische Resolution

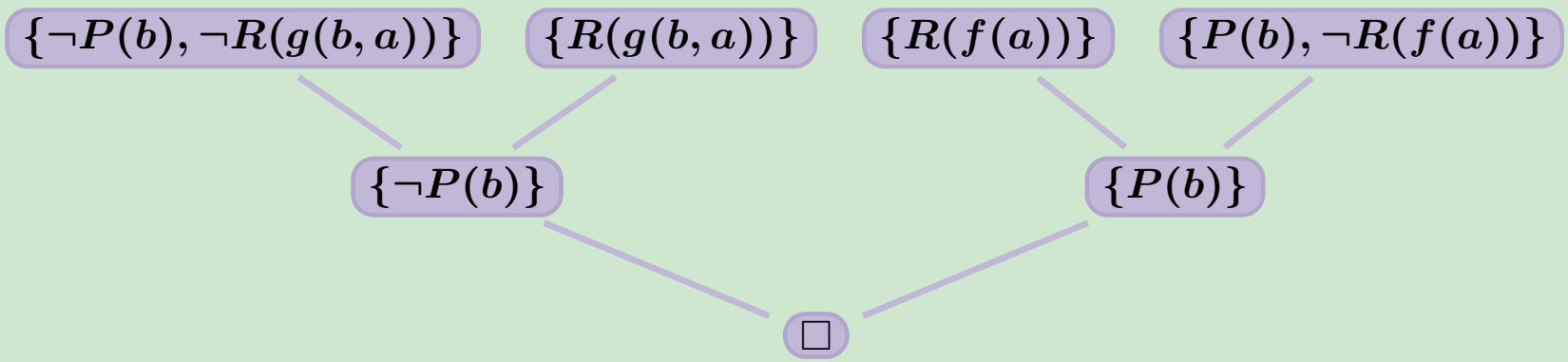
Vorüberlegungen

Prädikatenlogische Resolution: Vorüberlegungen (1/5)

Beispiel: Grundresolution (Wiederholung)

- $\varphi = \forall x\, \forall y \left(R(x) \wedge (P(x) \vee \neg R(y) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,y)))\right)$

- $H(\varphi) = \{a, b, f(a), f(b), g(a,a), g(a,b), g(b,a), g(b,b),$
  $f(f(a)), f(f(b)), f(g(a,a)), f(g(a,b)), \ldots\}$
- $E(\varphi) = \{R(a) \wedge (P(a) \vee \neg R(a) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,a))),$
  $R(b) \wedge (P(b) \vee \neg R(a) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,a))), \ldots\}$

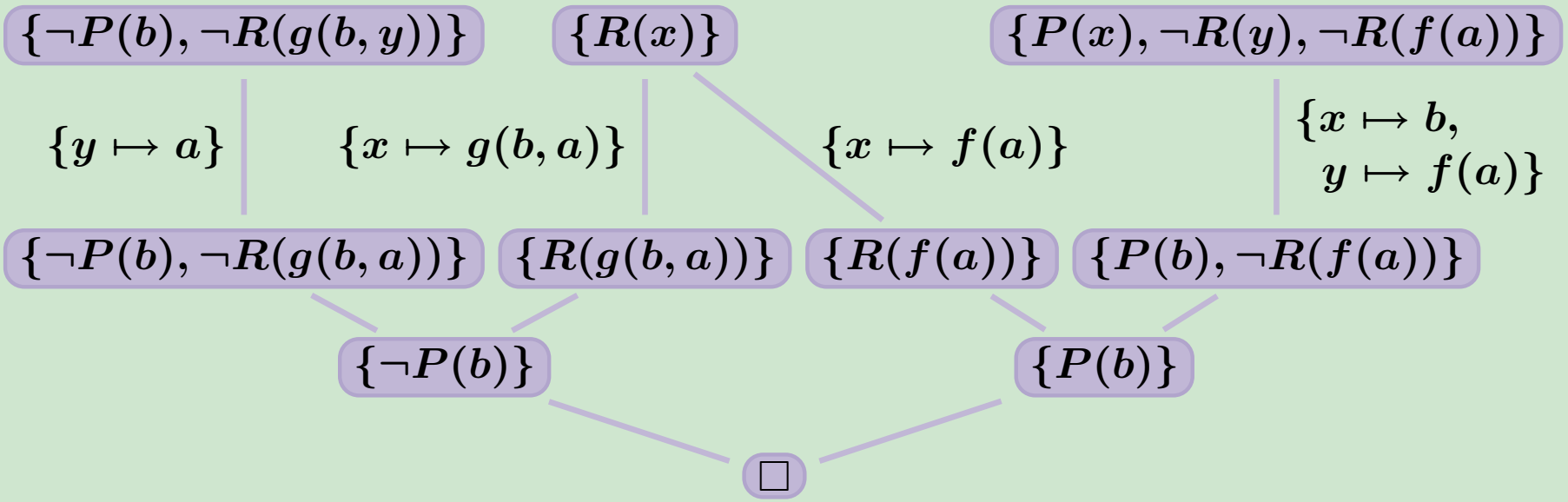


- Prinzipielle Vorgehensweise der Grundresolution:
  - Berechne die Matrixklauselform φ
  - Berechne die Herbrand-Expansion $E(\varphi)$
  - Versuche, einen Resolutionsbeweis zu finden, der aus $E(\varphi)$ die leere Klausel herleitet

Prädikatenlogische Resolution: Vorüberlegungen (2/5)

Beispiel: Grundresolution – eine andere Perspektive

- $\varphi = \forall x\, \forall y\, \big(R(x) \wedge (P(x) \vee \neg R(y) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,y)))\big)$
- Matrixklauselform: $\{\{R(x)\}, \{P(x), \neg R(y), \neg R(f(a))\}, \{\neg P(b), \neg R(g(b,y))\}\}$



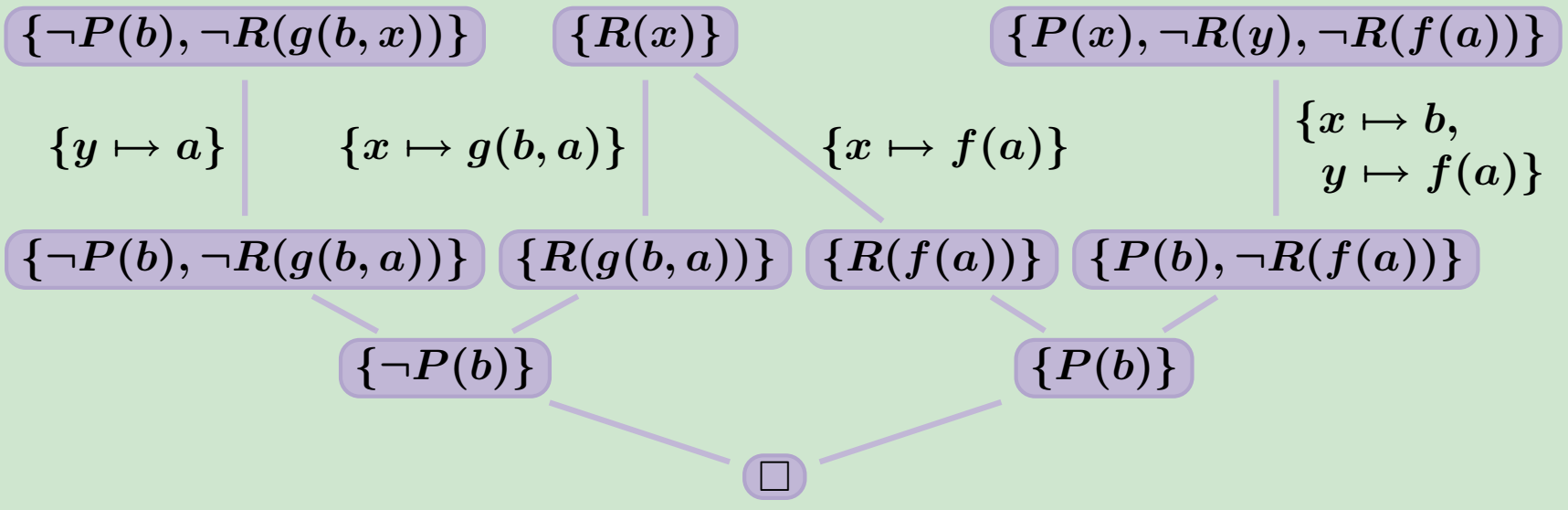
- $E(\varphi)$ ergibt sich aus φ durch Anwendung aller möglichen Grundsubstitutionen
- Ein Resolutionsschritt wird möglich, wenn es
  zu einer Klausel K mit einem Literal A und einer Klausel K' mit einem Literal $\neg A'$ Grundsubstitutionen σ und σ' mit $\sigma(A) = \sigma'(A')$ gibt

- Die Kombination von σ und σ' nennen wir dann einen **Unifikator** für A und A'

Prädikatenlogische Resolution: Vorüberlegungen (3/5)

Beispiel: Grundresolution – eine andere Perspektive (Fortsetzung)

- $\varphi = \forall x\, \forall y\, \Big(R(x) \wedge (P(x) \vee \neg R(y) \vee \neg R(f(a))) \wedge (\neg P(b) \vee \neg R(g(b,y)))\Big)$
- Matrixklauselform: $\{\{R(x)\}, \{P(x), \neg R(y), \neg R(f(a))\}, \{\neg P(b), \neg R(g(b,y))\}\}$



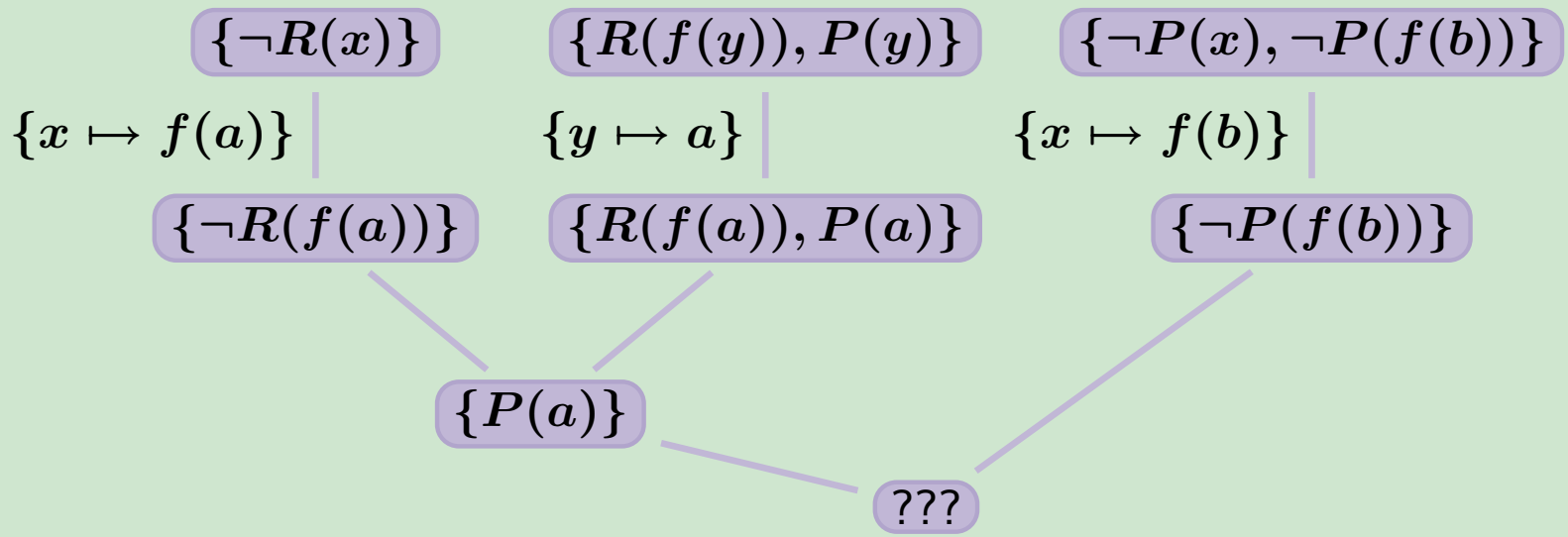
- **Beobachtung:** Für den eigentlichen Resolutionsbeweis werden nur einige Klauseln aus $E(\varphi)$ verwendet
- **Frage:** Geht das auch effizienter?

- **Neuer Ansatz:** Versuche, nur wirklich benötigte Klauseln aus $E(\varphi)$ zu erzeugen
- **Dazu:** Suche gezielt nach Unifikatoren, die zu resolvierbaren Klauselpaaren führen

Prädikatenlogische Resolution: Vorüberlegungen (4/5)

Ein weiteres Beispiel

- Formel: $\forall x \forall y \big(\neg R(x) \wedge (R(f(y) \vee P(y)) \wedge (\neg P(x) \vee \neg P(f(b))) \big)$
- Matrixklauselform: $\{\{\neg R(x)\}, \{R(f(y)), P(y)\}, \{\neg P(x), \neg P(f(b))\}\}$



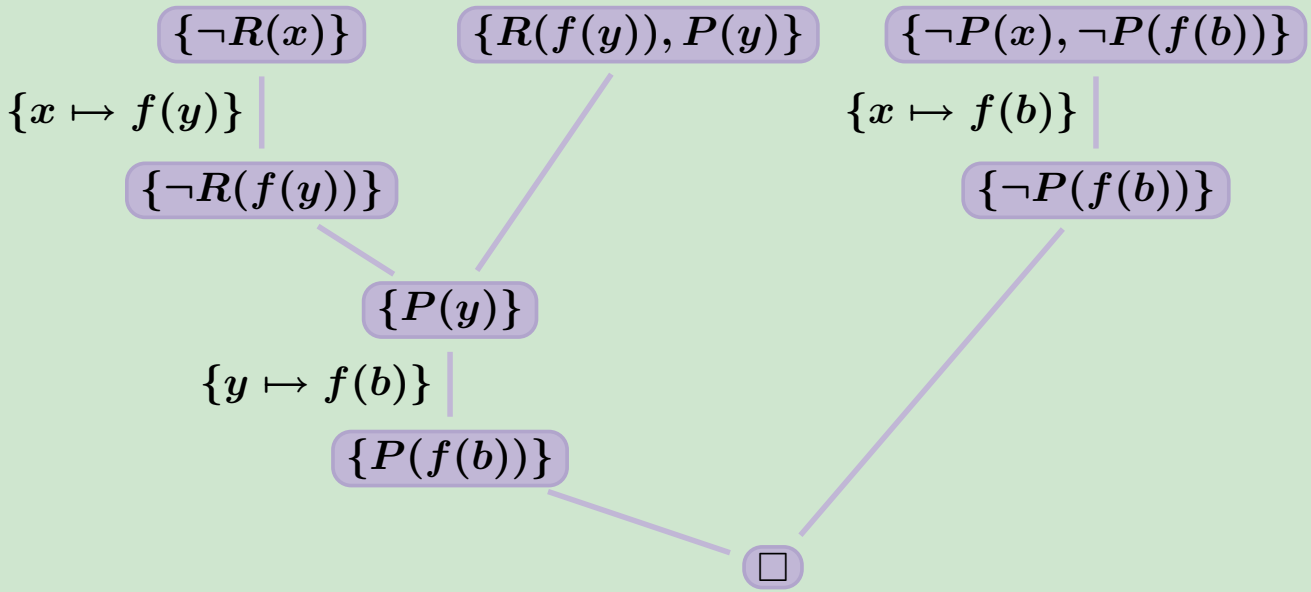
- Welche Unifikatoren für das Herleiten von $\square$ benötigt werden, ist manchmal nicht leicht zu erkennen

- Im obigen Beispiel „scheitert“ die Resolution
- **Aber:** Die Festlegung auf a im ersten Resolutionsschritt war eigentlich unnötig

Prädikatenlogische Resolution: Vorüberlegungen (5/5)

Ein weiteres Beispiel (Fortsetzung)

- Matrixklauselform: $\{\neg R(x)\}$, $\{R(f(y)), P(y)\}$, $\{\neg P(x), \neg P(f(b))\}$



- **Verfolgter Ansatz:** Vermeide voreilige Festlegungen
- **Also:** Bei der Unifikation sollte möglichst zurückhaltend vorgegangen werden
  - ➡ Terme sollten nur durch Konstanten festgelegt werden, wenn für den Resolutionsschritt unbedingt nötig
- Diese Ideen führen von der Grundresolution zur prädikatenlogischen Resolution

Unifikation

Unifikatoren (1/4)

Definition: Unifikator

- Eine Substitution σ heißt **Unifikator** einer Menge $\{A_1, \ldots, A_m\}$ von Atomen, falls gilt:

$$\sigma(A_1) = \cdots = \sigma(A_m)$$

✎ Manchmal sagen wir auch, dass σ ein Unifikator von $A_1, \ldots, A_m$ ist

- Als Hilfsmittel für die prädikatenlogische Resolution müssen wir das folgende algorithmische Problem (möglichst effizient) lösen

Problem: Unifikationsproblem

- **Gegeben:** Menge $\{A_1, \ldots, A_m\}$ von PL-Atomen
- **Gesucht:** Unifikator σ für $\{A_1, \ldots, A_m\}$

✎ Meist muss das Unifikationsproblem nur für zwei Atome gelöst werden

Beispiel

- $A_1 = R(x, a, y)$
- $A_2 = R(f(z), u, b)$
- $A_3 = R(v, u, w)$

- Die folgende Substitution bildet alle Atome auf $R(f(a), a, b)$ ab:

$$\begin{array}{lcl} u & \mapsto & a \\ v & \mapsto & f(a) \\ w & \mapsto & b \\ x & \mapsto & f(a) \\ y & \mapsto & b \\ z & \mapsto & a \end{array}$$

Unifikatoren (2/4)

- Wir interessieren uns für Unifikatoren, die möglichst wenige Festlegungen treffen
  - ➡ Um in Resolutionsbeweisen möglichst viele Möglichkeiten für spätere Resolutionsschritte offen zu halten

- Wie lässt sich „möglichst wenige Festlegungen“ formal definieren?
- **Idee:** Ein Unifikator σ ist „allgemein“, wenn jeder andere Unifikator σ' durch Hintereinanderausführung von σ und einer weiteren Substitution erreicht werden kann

Definition: Allgemeinster Unifikator

- Ein Unifikator σ heißt **allgemeinster Unifikator (MGU)** für $\{A_1, \ldots, A_m\}$, falls für jeden Unifikator σ' von $\{A_1, \ldots, A_m\}$ eine Substitution σ'' mit $\sigma' = \sigma'' \circ \sigma$ existiert

✎ Die Abkürzung **MGU** steht für „most general unifier“

Unifikatoren (3/4)

Definition: Allgemeinster Unifikator (Wdh.)

- Ein Unifikator σ heißt **allgemeinster Unifikator (MGU)** für $\{A_1, \ldots, A_m\}$, falls
  für jeden Unifikator σ' von $\{A_1, \ldots, A_m\}$ eine Substitution σ'' mit $\sigma' = \sigma'' \circ \sigma$ existiert

Beispiel

- $A_1 = R(x, a, y)$
- $A_2 = R(f(z), u, b)$
- $A_3 = R(v, u, w)$

- Die folgende Substitution σ bildet die Atome auf $R(f(z), a, b)$ ab und ist ein MGU:

$$\begin{array}{lcl} u & \mapsto & a \\ v & \mapsto & f(z) \\ w & \mapsto & b \\ x & \mapsto & f(z) \\ y & \mapsto & b \end{array}$$

- Die folgende Substitution σ' bildet alle Atome auf $R(f(a), a, b)$ ab:

☞ von Folie 9

$$\begin{array}{lcl} u & \mapsto & a \\ v & \mapsto & f(a) \\ w & \mapsto & b \\ x & \mapsto & f(a) \\ y & \mapsto & b \\ z & \mapsto & a \end{array}$$

- Sie lässt sich schreiben als:

$$\sigma' = \{z \mapsto a\} \circ \sigma$$

Unifikatoren (4/4)

- Natürlich gibt es nicht für jede Menge von Atomen einen Unifikator

Beispiel

- Beispiele von Atompaaren ohne Unifikator:
  - Verschiedene Relationsnamen:

    $$P(a), R(y)$$

  - Verschiedene Funktionssymbole:

    $$P(a, f(x)), P(y, g(b))$$

  - Variablen-Rekursion:

    $$P(y, x), P(a, f(x))$$

- Der Algorithmus auf der nächsten Folie entscheidet, ob eine gegebene Atommenge einen Unifikator hat und
  - berechnet einen allgemeinsten Unifikator oder
  - gibt „nicht unifizierbar“ aus

Der Unifikationsalgorithmus

Unifikationsalgorithmus

Eingabe: Menge $\mathcal{L} \neq \emptyset$ von PL-Atomen
Ausgabe: Allgemeinster Unifikator σ für $\mathcal{L}$ oder „nicht unifizierbar“

1: **if** es kommen verschiedene Relationssymbole in $\mathcal{L}$ vor **then**
2: **return** „nicht unifizierbar“
3: **end if**
4: $\mathcal{L}' := \mathcal{L}$, $\sigma := \emptyset$
5: **while** $|\mathcal{L}'| > 1$ **do**
6: Wähle zwei Atome A_1, A_2 in $\mathcal{L}'$
7: Seien z_1 (in A_1) und z_2 (in A_2) die ersten Zeichen, an denen A_1 und A_2 verschieden sind
8: **if** weder z_1 noch z_2 ist Variable **then**
9: **return** „nicht unifizierbar“
10: **end if**
11: **if** z_1 ist Variable **then**
12: Sei t der in A_2 mit z_2 beginnende Term
13: **if** z_1 kommt in t vor **then**
14: **return** „nicht unifizierbar“
15: **else**
16: $\sigma := \{z_1 \mapsto t\} \circ \sigma$
17: Wende $z_1 \mapsto t$ auf alle Atome in $\mathcal{L}'$ an
18: **end if**
19: **else**
20: Analog (z_2 ist Variable)
21: **end if**
22: **end while**
23: **return** σ

Unifikationsalgorithmus: Beispiele (1/2)

Beispiel 1

- $\mathcal{L} = \{P(f(x), g(y, h(a, z))), P(f(g(a, b)), g(g(u, v), w))\}$

- **1. Iteration:** $|\mathcal{L}'| = 2$
- Sei
  - $A_1 = P(f(x), g(y, h(a, z)))$
  - $A_2 = P(f(g(a, b)), g(g(u, v), w))$
- $z_1 = x$, $z_2 = g$
  $\Rightarrow t = g(a, b)$ und x kommt nicht in t vor
  $\Rightarrow \sigma := \{x \mapsto g(a, b)\}$
- $\mathcal{L}' = \{P(f(g(a, b)), g(y, h(a, z))), P(f(g(a, b)), g(g(u, v), w))\}$

- **2. Iteration:** $|\mathcal{L}'| = 2$
- Sei
  - $A_1 = P(f(g(a, b)), g(y, h(a, z)))$
  - $A_2 = P(f(g(a, b)), g(g(u, v), w))$
- $z_1 = y$, $z_2 = g$
- $t = g(u, v)$ und y kommt in t nicht vor
  $\Rightarrow \sigma := \{y \mapsto g(u, v)\} \circ \sigma$
- $\mathcal{L}' = \{P(f(g(a, b)), g(g(u, v), h(a, z))), P(f(g(a, b)), g(g(u, v), w))\}$

- **3. Iteration:** $|\mathcal{L}'| = 2$
- Sei
  - $A_1 = P(f(g(a, b)), g(g(u, v), h(a, z)))$
  - $A_2 = P(f(g(a, b)), g(g(u, v), w))$
- $z_1 = h$, $z_2 = w$
  $\Rightarrow t = h(a, z)$ und w kommt nicht in t vor
  $\Rightarrow \sigma := \{w \mapsto h(a, z)\} \circ \sigma$
- $\mathcal{L}' = \{P(f(g(a, b)), g(g(u, v)), h(a, z))\}$

- **4. Iteration:** $|\mathcal{L}'| = 1$
- Ausgabe:
  $\{x \mapsto g(a, b), y \mapsto g(u, v), w \mapsto h(a, z)\}$

Unifikationsalgorithmus: Beispiele (2/2)

Beispiel 2

- $\mathcal{L} = \{P(a, f(x)), P(y, g(b))\}$

- **1. Iteration:** $|\mathcal{L}'| = 2$
- Sei
  - $A_1 = P(a, f(x))$
  - $A_2 = P(y, g(b))$
- $z_1 = a$, $z_2 = y$
  $\Rightarrow t = a$ und y kommt nicht in t vor
  $\Rightarrow \sigma := \{y \mapsto a\}$
- $\mathcal{L}' = \{P(a, f(x)), P(a, g(b))\}$

- **2. Iteration:** $|\mathcal{L}'| = 2$
- Sei
  - $A_1 = P(a, f(x))$
  - $A_2 = P(a, g(b))$
- $z_1 = f$, $z_2 = g$
  $\Rightarrow$ Weder z_1 noch z_2 ist Variable
  $\Rightarrow$ nicht unifizierbar

Beispiel 3

- $\mathcal{L} = \{P(f(x), a), P(x, y)\}$

- **1. Iteration:** $|\mathcal{L}'| = 2$
- Sei
  - $A_1 = P(f(x), a)$
  - $A_2 = P(x, y)$
- $z_1 = f$, $z_2 = x$
  $\Rightarrow t = f(x)$ und $z_2 = x$ kommt in t vor
  $\Rightarrow$ nicht unifizierbar

Unifikationsalgorithmus: Eigenschaften

Proposition C.4.1

- Der Unifikationsalgorithmus hat folgende Eigenschaften:
  - Für die Ausgabe gilt:
    - Wenn eine Menge von Atomen unifizierbar ist, berechnet er einen MGU für die Menge
    - Andernfalls gibt er „nicht unifizierbar“ aus
  - Er terminiert nach höchstens exponentiell vielen Schritten

Folgerung C.4.2

- Jede unifizierbare Menge von Atomen hat einen allgemeinsten Unifikator

✎ Es gibt im Allgemeinen viele MGUs, die sich aber nur durch die Benennung von Variablen unterscheiden

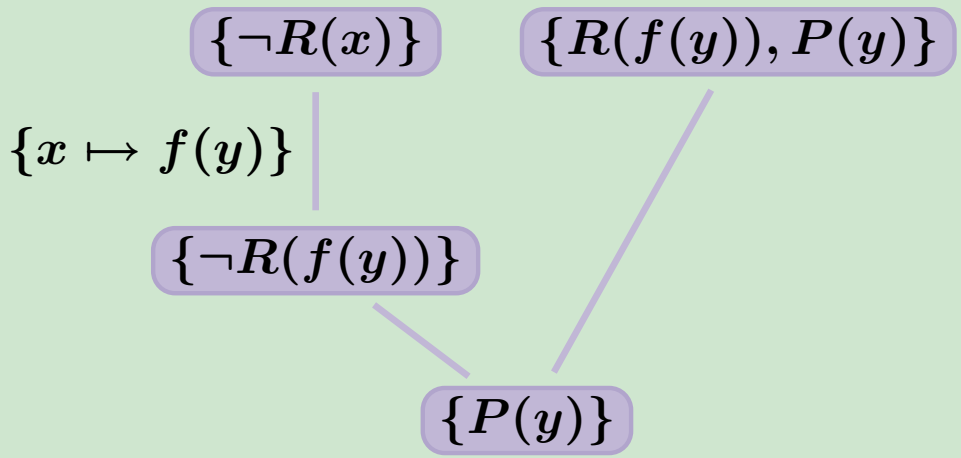
✎ MGUs können **exponentiell groß** im Verhältnis zur Länge der Ausgangsatome werden

☞ Ein Beispiel findet sich im Anhang

Prädikatenlogische Resolution

Prädikatenlogische Resolution: Beispiel

Beispiel



- Ein prädikatenlogischer Resolutionsschritt lässt sich auf zwei Klauseln K und K' anwenden, wenn
  - in K ein Atom A vorkommt,
  - in K' ein negiertes Atom $\neg A'$ vorkommt und
  - A und A' unifizierbar sind

Variablenumbenennungen

- Lassen sich die beiden Klauseln
  - $K = \{R(x, u)\}$
  - $K' = \{\neg P(x, y), \neg R(g(x), z)\}$

  resolvieren?

- Zunächst sind keine Atome unifizierbar:
  - $R(x, u)$ und $P(x, y)$ sowieso nicht
  - $R(x, u)$ und $R(g(x), z)$ auch nicht
- **Beobachtung:** Die Unifikation scheitert daran, dass beide Klauseln x verwenden

- **Aber:** x kommt zwar in beiden Klauseln vor, kann aber in beiden Klauseln unterschiedliche Werte annehmen
- **Denn:** x und die anderen Variablen sind $\forall$-quantifiziert

  ➡ alle Klauseln müssen für **alle** Variablenbelegungen wahr werden

- In der Herbrand-Expansion würden beispielsweise $R(g(a), a)$ und $\neg R(g(a), a)$ entstehen, die resolvierbar sind

- **Daher:** An der Bedeutung der Klauseln ändert sich nichts, wenn in K die Variable x in eine neue Variable umbenannt wird

  ➡ Durch diese Umbenennung werden K und K' resolvierbar

- **Zu beachten:**
  - Innerhalb einer Klausel müssen aber alle Vorkommen der Variablen auf die gleiche Art umbenannt werden

Definition: Variablenumbenennung

- Eine **Variablenumbenennung** ist eine Substitution $\mathrm{PV} \rightharpoonup \mathrm{PV}$

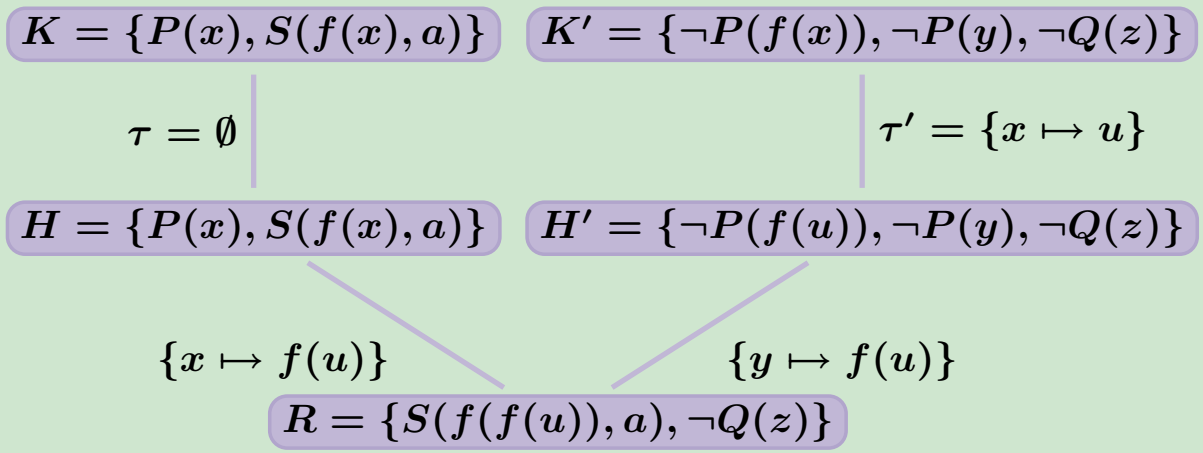
- Variablen werden hier also durch Variablen ersetzt

  ☞ nicht durch beliebige Terme

Prädikatenlogische Resolventen (1/3)

- Ein prädikatenlogischer Resolutionsschritt besteht im Allgemeinen aus:
  - einer Variablenumbenennung,
  - einer Unifikation und
  - dem Bilden der Resolvente

Beispiel



- Zu beachten: Wir teilen hier in der graphischen Notation den Unifikator $\sigma = \{x \mapsto f(u), y \mapsto f(u)\}$ in seine für die beiden Klauseln relevanten Bestandteile auf und kombinieren seine Anwendung mit dem eigentlichen Resolutionsschritt

  ✎ In der Definition von Unifikatoren werden diese beiden Teil-Substitutionen zusammengefasst

Prädikatenlogische Resolventen (2/3)

Definition: Prädikatenlogische Resolvente

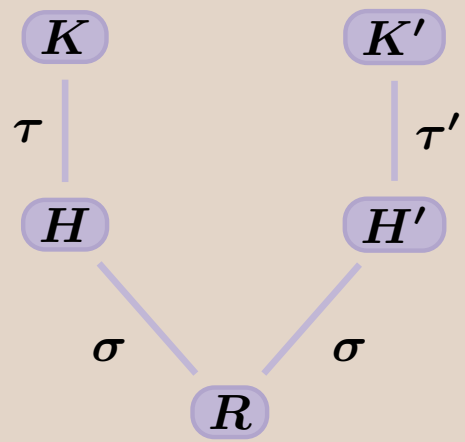
- Seien K, K' prädikatenlogische Klauseln
- Seien τ, τ' Variablenumbenennungen, so dass $H \stackrel{\text{def}}{=} \tau(K)$ und $H' \stackrel{\text{def}}{=} \tau'(K')$ keine Variablen gemeinsam haben
- Seien $A_1, \ldots, A_k$ Atome in H und $\neg A'_1, \ldots, \neg A'_m$ Literale in H', so dass $\{A_1, \ldots, A_k, A'_1, \ldots, A'_m\}$ unifizierbar ist mit MGU σ

- Dann heißt

$$R \stackrel{\text{def}}{=} \sigma((H - \{A_1, \ldots, A_k\}) \cup (H' - \{\neg A'_1, \ldots, \neg A'_m\}))$$

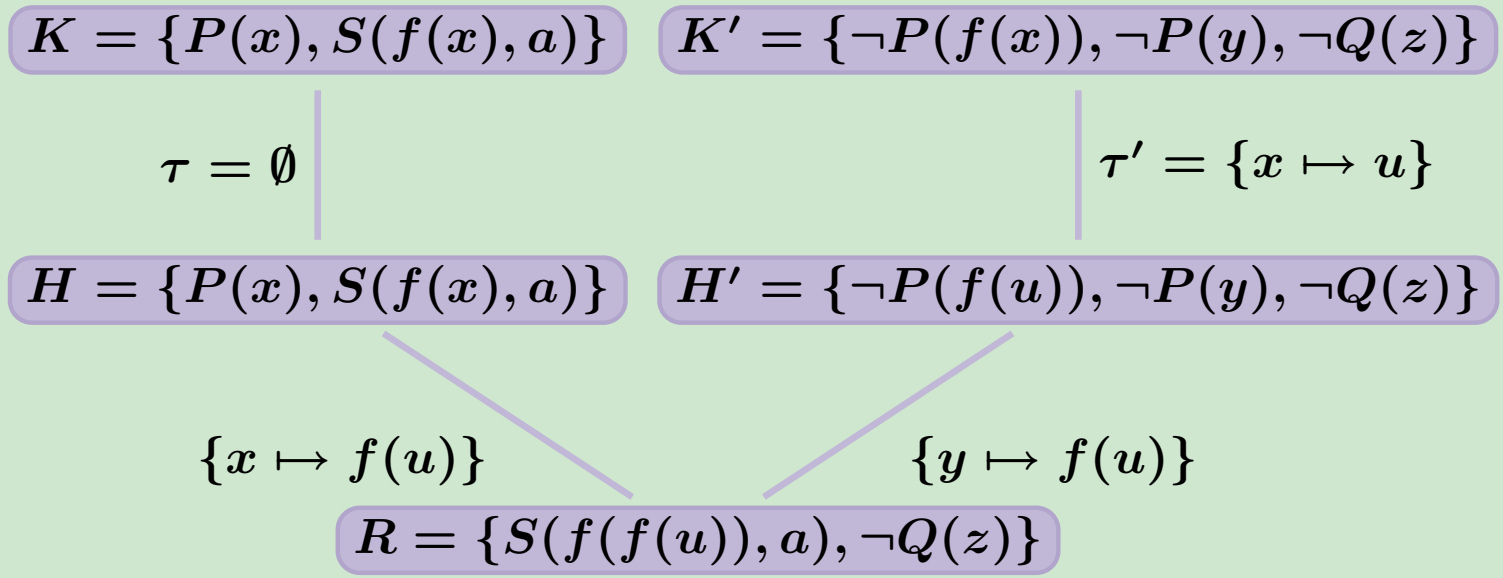
prädikatenlogische Resolvente von K und K'

- Illustration:



Prädikatenlogische Resolventen (3/3)

Beispiel (Fortsetzung)



- Mit den Bezeichnungen aus der Definition gilt hier:
  - $A_1 = P(x)$
  - $A'_1 = P(f(u))$
  - $A'_2 = P(y)$

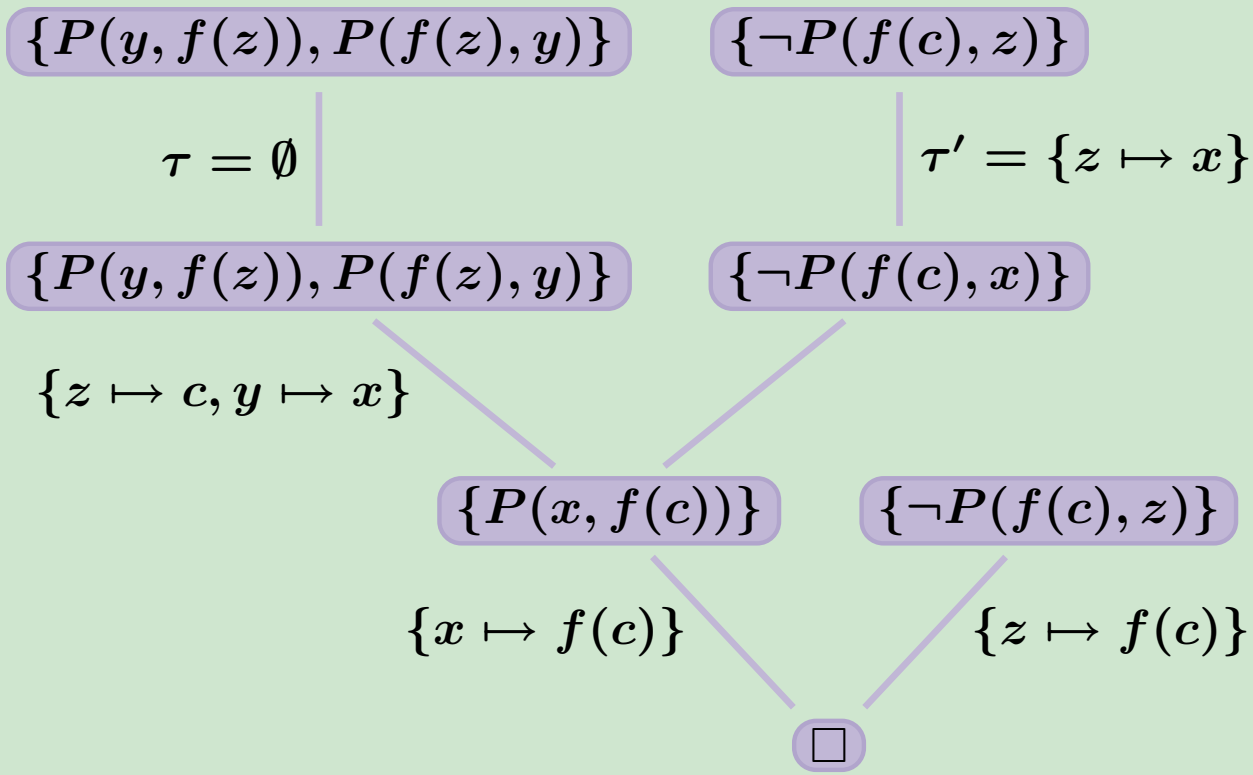
- Diese drei Atome sind unifizierbar mit MGU $\sigma = \{x \mapsto f(u), y \mapsto f(u)\}$

$$\Rightarrow R \overset{\text{def}}{=} \sigma((H - \{A_1\}) \cup (H' - \{\neg A'_1, \neg A'_2\}))$$
$$= \{S(f(f(u)), a), \neg Q(z)\}$$

Prädikatenlogische Resolutionsbeweise: Beispiel

Beispiel

- Sei $\varphi = \exists x \forall y \forall z \big((P(y, f(z)) \vee P(f(z), y)) \wedge \neg P(f(x), z) \big)$
- Matrixklauselform: $\{\{P(y, f(z)), P(f(z), y)\}, \{\neg P(f(c), z)\}\}$



- φ ist unerfüllbar, da wir mit prädikatenlogischer Resolution die leere Klausel herleiten konnten

✎ Beim zweiten Schritt ist keine Variablenumbenennung nötig

Der prädikatenlogische Resolutionssatz

- Für prädikatenlogische Klauselmengen $\mathcal{K}$ definieren wir:
  - $\underline{\mathbf{Res}(\mathcal{K})} \stackrel{\text{def}}{=} \mathcal{K} \cup \{K \mid K \text{ ist PL-Resolvente zweier Klauseln aus } \mathcal{K}\}$
  - $\underline{\mathbf{Res}^0(\mathcal{K})} \stackrel{\text{def}}{=} \mathcal{K}$
  - $\underline{\mathbf{Res}^\ell(\mathcal{K})} \stackrel{\text{def}}{=} \mathbf{Res}(\mathbf{Res}^{\ell-1}(\mathcal{K}))$, für alle $\ell \geq 1$
  - $\underline{\mathbf{Res}^\infty(\mathcal{K})} \stackrel{\text{def}}{=} \bigcup_{\ell \geq 0} \mathbf{Res}^\ell(\mathcal{K})$

Satz C.4.3: Prädikatenlogischer Resolutionssatz

- Eine prädikatenlogische Formel φ ist genau dann unerfüllbar, wenn $\square \in \mathbf{Res}^\infty(\mathcal{K})$ für ihre Matrixklauselform $\mathcal{K}$ gilt:

- Bevor wir einen Blick auf den Beweis des Resolutionssatzes werfen, schauen wir uns einige weitere Beispiele an

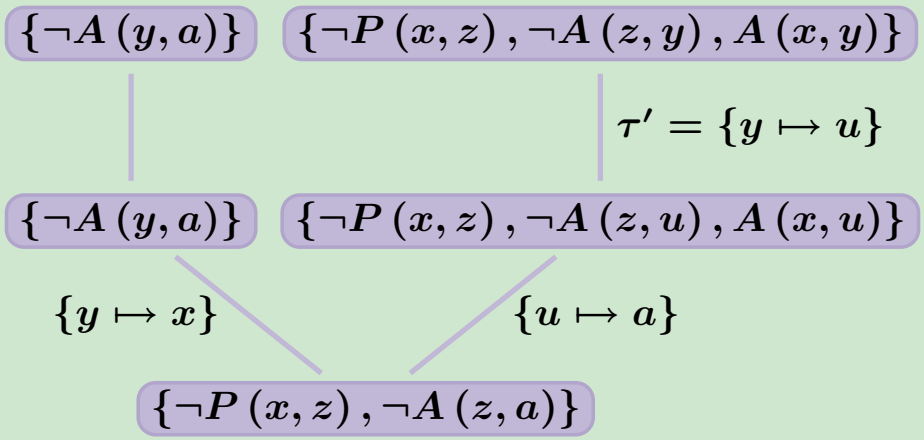
Weitere Bemerkungen und Beispiele

Notation von Resolutionsbeweisen

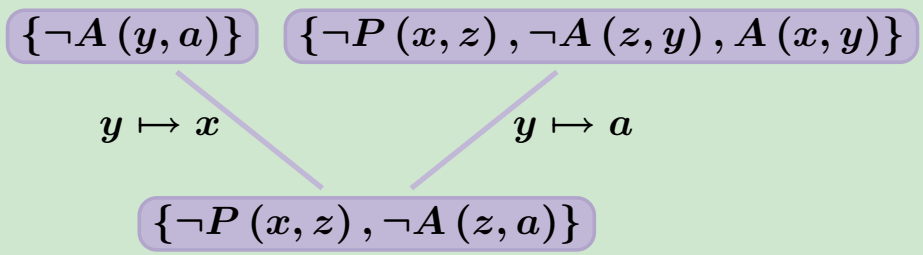
- Im Folgenden werden wir eine kompaktere Darstellung in Resolutionsbeweisen verwenden:
  - Variablenumbenennungen, Substitutionen und Übergang zur Resolventen werden zu einem Schritt kombiniert
  - Mengenklammern bei Substitutionen lassen wir weg

Beispiel

- Statt. . .



- . . . schreiben wir beispielsweise:



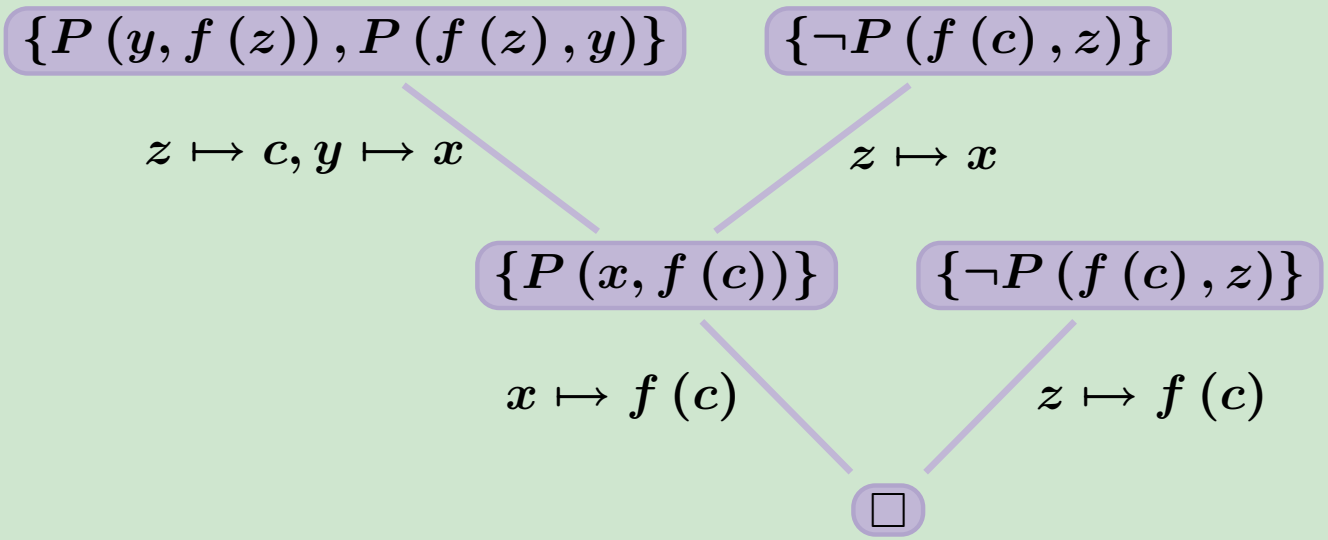
✎ Dass y hier auf zwei Werte abgebildet wird, ist nicht schädlich: Implizit nehmen wir an, dass y vor der Unifikation in einer der beiden Klauseln durch eine neue Variable ersetzt wird

Prädikatenlogische Resolution: Beispiele (1/2)

- In der kompakten Notation ergibt sich für eins der vorherigen Beispiele:

Beispiel

- Sei $\varphi = \exists x \forall y \forall z[(P(y, f(z)) \vee P(f(z), y)) \wedge \neg P(f(x), z)]$
- Matrixklauselform: $\{\{P(y, f(z)), P(f(z), y)\}, \{\neg P(f(c), z)\}\}$

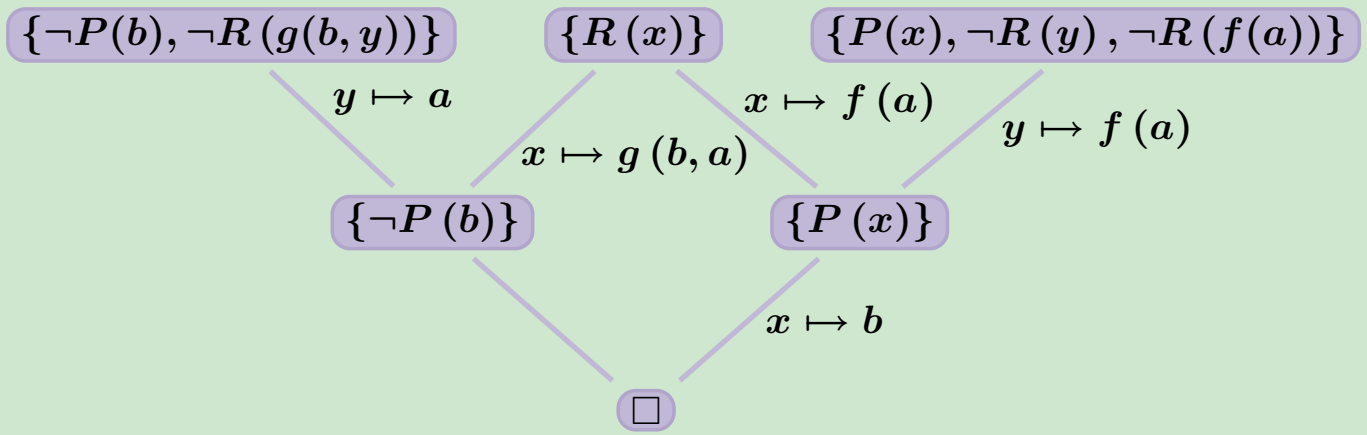


Prädikatenlogische Resolution: Beispiele (2/2)

Beispiel

- Wir betrachten nochmals den prädikatenlogischen Resolutionsbeweis für die Klauselmenge:

$$\{R(x)\}, \{P(x), \neg R(y), \neg R(f(a))\}, \{\neg P(b), \neg R(g(b,y))\}$$



- Dieses Beispiel illustriert, dass bei einem Resolutionsschritt auch mehrere Atome **einer** Klausel unifiziert werden können

  ☞ Hier: $R(y)$ und $R(f(a))$

- Aus diesem Grund haben wir den Unifikationsalgorithmus auch für mehr als zwei Atome und für Atome mit nicht disjunkten Variablenmengen formuliert

Folgern mit prädikatenlogischer Resolution

- **Ziel:** Überprüfen, ob $\Phi \models \psi$ gilt

- **Vorgehen:** Teste dazu die Unerfüllbarkeit von $\Phi \cup \{\neg\psi\}$
  1) Sei φ die Konjunktion der Einzelformeln aus $\Phi \cup \{\neg\psi\}$
  2) Wandle φ in eine äquivalente Formel φ' in Pränexform um
  3) Wandle φ' in eine erfüllbarkeitsäquivalente Formel φ'' in Skolemform um
  4) Wandle φ'' in eine Matrixklauselform $\mathcal{K}$ um
  5) Leite aus $\mathcal{K}$ mit Hilfe der prädikatenlogischen Resolution die Box $\square$ her
  6) Falls dies gelingt, gilt $\Phi \models \psi$

- Eine alternative Möglichkeit ist es, aus Φ und ψ separate Matrixklauselformen $\mathcal{K}(\Phi)$ und $\mathcal{K}(\neg\psi)$ zu berechnen und dann prädikatenlogische Resolution auf $\mathcal{K}(\Phi) \cup \mathcal{K}(\neg\psi)$ anzuwenden

Beispiel

- Gilt $\exists x \forall y R(x,y) \models \forall y \exists x R(x,y)$?

- Teste Unerfüllbarkeit von

$$\{\exists x \forall y R(x,y), \neg\forall y \exists x R(x,y)\}$$

1) $\varphi \overset{\text{def}}{=} \exists x \forall y R(x,y) \wedge \neg\forall y \exists x R(x,y)$

2) $\varphi' \overset{\text{def}}{=} \exists x \forall y \exists y' \forall x' \big(R(x,y) \wedge \neg R(x',y')\big)$

3) $\varphi'' \overset{\text{def}}{=} \forall y \forall x' \big(R(a,y) \wedge \neg R(x', f(y))\big)$

4) $\mathcal{K} \overset{\text{def}}{=} \{\{R(a,y)\}, \{\neg R(x', f(y))\}\}$

5) $\{R(a,y)\}$ $\quad$ $\{\neg R(x', f(y))\}$

$y \mapsto f(y)$ $\qquad$ $x' \mapsto a$

$\square$

6) Deshalb:

$$\exists x \forall y R(x,y) \models \forall y \exists x R(x,y)$$

Zum Beweis des prädikatenlogischen Resolutionssatzes

Zur Erinnerung: Der prädikatenlogische Resolutionssatz

Satz C.4.4: Prädikatenlogischer Resolutionssatz

- Eine prädikatenlogische Formel φ ist genau dann unerfüllbar, wenn $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$ für ihre Matrixklauselform $\mathcal{K}$ gilt:

- Um den Resolutionssatz zu beweisen wird gezeigt:
  - Die prädikatenlogische Resolution ist **korrekt**:

    wenn $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$, dann ist φ unerfüllbar
  - Die prädikatenlogische Resolution ist **vollständig**:

    wenn φ unerfüllbar ist, dann ist $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$

Idee des Beweises der Korrektheit

- **Erinnerung:** Für die Korrektheit der prädikatenlogischen Resolution ist zu zeigen:

  wenn $\square \in \mathbf{Res}^\infty(\mathcal{K})$, dann ist φ unerfüllbar

- Wir werden im Prinzip die Kontraposition zeigen:

  wenn φ erfüllbar ist, dann ist $\square \notin \mathbf{Res}^\infty(\mathcal{K})$

- Dafür werden wir zeigen, dass aus einer erfüllbaren Klauselmenge nur erfüllbare Klauseln resolviert werden können

  ➡ Einzelne Resolutionsschritte sind korrekt

- Das ist der Inhalt des folgenden **prädikatenlogischen Resolutionslemmas**:

  Wenn zwei Klauseln $\boldsymbol{K}$ und $\boldsymbol{K'}$ in einer Struktur für alle Belegungen wahr werden, dann wird auch ihre Resolvente $\boldsymbol{R}$ für alle Belegungen wahr

- Hierbei: Es werden alle Belegungen betrachtet, weil die Variablen in Klauseln $\forall$-quantifiziert sind

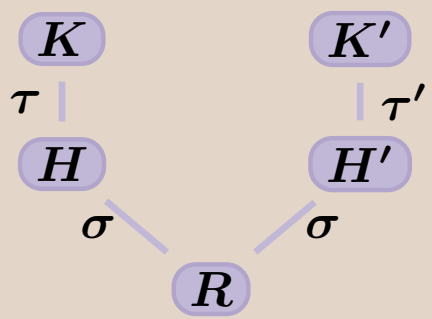
Das prädikatenlogische Resolutionslemma

Definition: Prädikatenlogische Resolvente (Wdh.)

- Seien K, K' prädikatenlogische Klauseln
- Seien τ, τ' Variablenumbenennungen, so dass $H \stackrel{\text{def}}{=} \tau(K)$ und $H' \stackrel{\text{def}}{=} \tau'(K')$ keine Variablen gemeinsam haben
- Seien $A_1, \ldots, A_k$ Atome in H und $\neg A'_1, \ldots, \neg A'_m$ Literale in H', so dass $\{A_1, \ldots, A_k, A'_1, \ldots, A'_m\}$ unifizierbar ist mit MGU σ

- Dann heißt $R \stackrel{\text{def}}{=} \sigma((H - \{A_1, \ldots, A_k\}) \cup (H' - \{\neg A'_1, \ldots, \neg A'_m\}))$ **prädikatenlogische Resolvente** von K und K'

- Illustration:



Lemma C.4.5: PL-Resolutionslemma

- Seien K, K', τ, τ', H, H', σ und R wie in der Definition von prädikatenlogischen Resolventen
- Sei $\mathcal{A}$ eine zu K und K' passende Struktur

- Wenn
  (a) für alle passenden Belegungen β gilt: $(\mathcal{A}, \beta) \models K$ und
  (b) für alle passenden Belegungen β' gilt: $(\mathcal{A}, \beta') \models K'$,
  dann gilt auch für alle passenden Belegungen γ: $(\mathcal{A}, \gamma) \models R$

- Weitere Details zum Beweis der Korrektheit finden sich im Anhang

Idee des Beweises der Vollständigkeit

- **Erinnerung:** Für die Vollständigkeit der prädikatenlogischen Resolution ist zu zeigen:

  wenn φ unerfüllbar ist, dann gilt $\square \in \mathbf{Res}^{\infty}(\mathcal{K})$

- Die **Vollständigkeit** führen wir auf die im letzten Kapitel bewiesene Vollständigkeit der Grundresolution zurück
- **Idee:** Die prädikatenlogische Resolution ist „mindestens so vollständig“ wie die Grundresolution

  ➡ Dazu genügt es zu zeigen, dass sich aus jedem Grundresolutionsbeweis für $\square$ ein prädikatenlogischer Resolutionsbeweis für $\square$ konstruieren lässt

- Dazu beweisen wir das **Lifting-Lemma**:

  Zu jedem Resolutionsschritt eines Grundresolutionsbeweises mit Resolvente $\boldsymbol{R'}$ gibt es einen prädikatenlogischen Resolutionsschritt, aus dessen Resolvente $\boldsymbol{R'}$ durch eine Grundsubstitution gewonnen werden kann

- Dass es zu jedem Grundresolutionsbeweis auch einen prädikatenlogischen Resolutionsbeweis gibt, folgt dann per Induktion

Das Lifting-Lemma

Lemma C.4.6: Lifting-Lemma

- Seien K_1, K_2 prädikatenlogische Klauseln mit Grundsubstitutionen σ_1, σ_2
- Seien $K_1' = \sigma_1(K_1)$ und $K_2' = \sigma_2(K_2)$

- Wenn R' eine aussagenlogische Resolvente von K_1' und K_2' ist,

  dann gibt es eine prädikatenlogische Resolvente R von K_1 und K_2 und eine Grundsubstitution σ mit $\sigma(R) = R'$

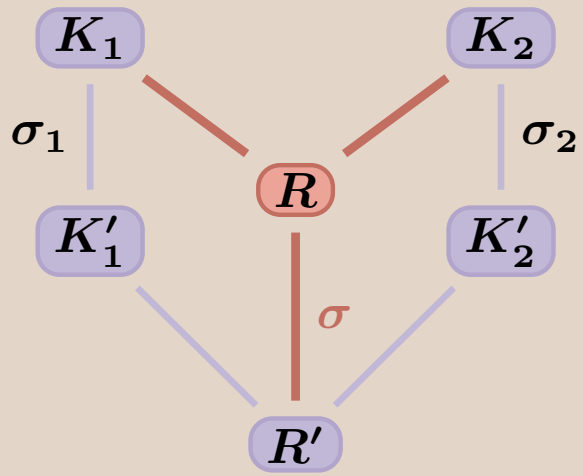
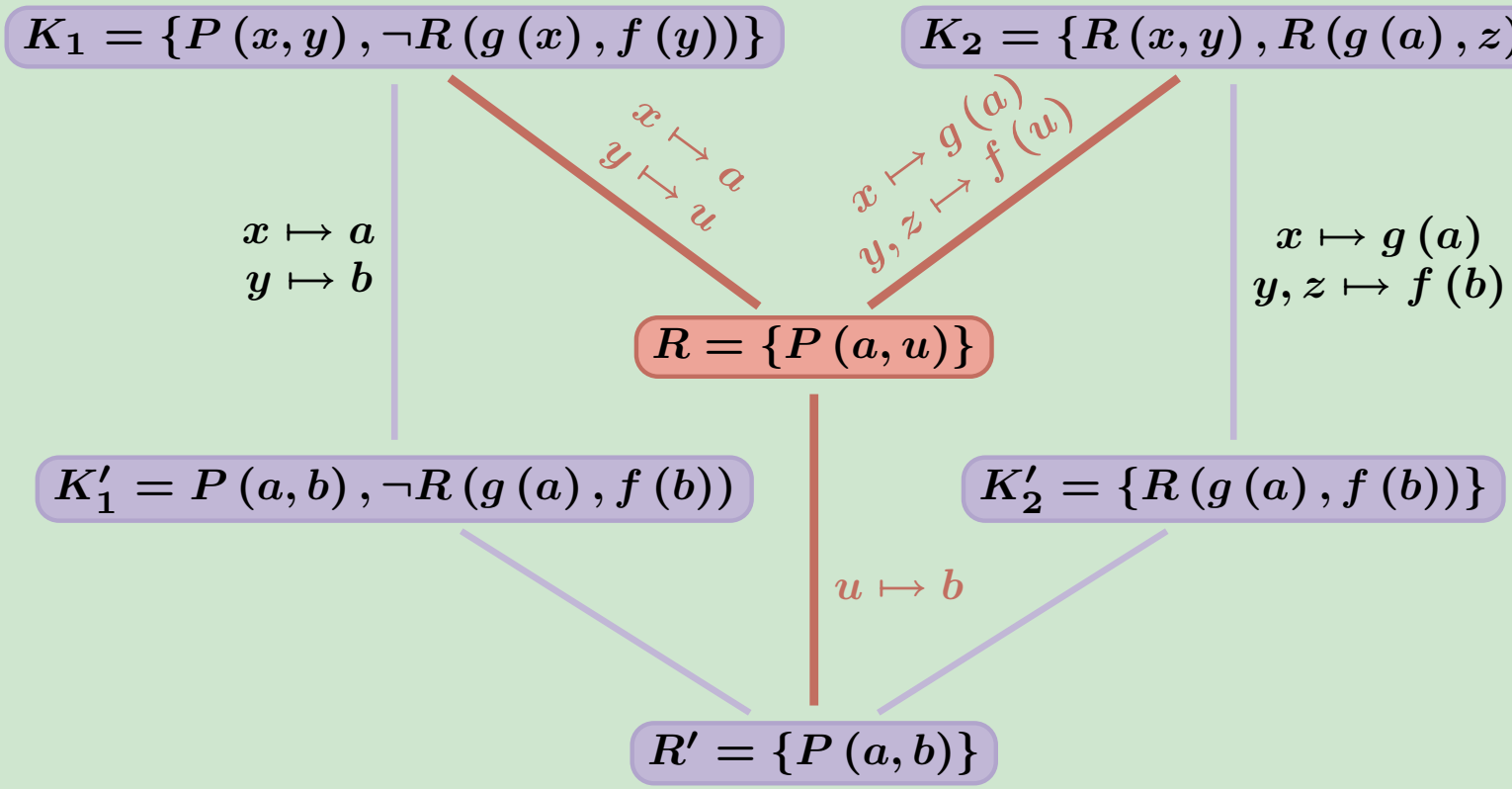


Illustration des Lifting-Lemmas

Beispiel



- Der nicht schwierige, aber etwas technische Beweis des Lifting-Lemmas findet sich im Buch von Kreuzer und Kühling
  - ✎ Zu beachten: Die Verwendung von MGUs ist an dieser Stelle wichtig!

Sehr grobe Beweisskizze für die Vollständigkeit der prädikatenlogischen Resolution

Beweisskizze

- **Zu zeigen:** φ unerfüllbar $\Rightarrow \square \in \mathbf{Res}^{\infty}(\mathcal{K})$

- Sei also φ unerfüllbar

- Mit dem Grundresolutionssatz (Satz 11.5) folgt, dass $\boldsymbol{E}(\varphi)$ unerfüllbar ist
  $\Rightarrow \square \in \mathbf{Res}^{\infty}(\boldsymbol{E}(\varphi))$
  $\Rightarrow$ Es gibt also einen aussagenlogischen Resolutionsbeweis für $\square$, der endlich viele Klauseln aus $\boldsymbol{E}(\varphi)$ verwendet

- Durch Anwendung des Lifting-Lemmas auf jeden einzelnen Resolutionsschritt lässt sich daraus induktiv ein prädikatenlogischer Resolutionsbeweis gewinnen

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Es ist nicht leicht, die Erfüllbarkeit prädikatenlogischer Formeln zu testen
  - Wir haben zwei Semi-Entscheidungsverfahren für die Unerfüllbarkeit kennen gelernt
    - Der Grundresolutionsalgorithmus führt das Problem auf einen Unentscheidbarkeitstest für eine unendliche Menge aussagenlogischer Klauseln zurück
    - Die prädikatenlogische Resolution versucht, „direkt“ einen Unerfüllbarkeitsbeweis zu finden

Anhang: Blow-up bei der Unifikation

Blow-Up durch MGUs

- Atome können nach Anwendung eines MGUs **exponentiell groß** werden

  ☞ im Verhältnis zu ihrer Länge

Beispiel

- Wir betrachten die Menge mit den folgenden zwei Atomen:
  - $R(x_0, f(x_1, x_1), x_1, \ldots, x_{n-1}, f(x_n, x_n))$
  - $R(f(y_1, y_1), y_1, f(y_2, y_2), \ldots, f(y_n, y_n), y_n)$

- Der erste Schritt vom Unifikationsalgorithmus ergibt:
  - $R(f(y_1, y_1), f(x_1, x_1), x_1, \ldots, x_{n-1}, f(x_n, x_n))$
  - $R(f(y_1, y_1), y_1, f(y_2, y_2), \ldots, f(y_n, y_n), y_n)$

- Im zweiten Schritt erhalten wir:
  - $R(f(f(x_1, x_1), f(x_1, x_1)), f(x_1, x_1), x_1, \ldots, x_{n-1}, f(x_n, x_n))$
  - $R(f(f(x_1, x_1), f(x_1, x_1)), f(x_1, x_1), f(y_2, y_2), \ldots, f(y_n, y_n), y_n)$

- Im dritten Schritt erhalten wir:
  - $R(f(f(f(y_2, y_2), f(y_2, y_2)), f(f(y_2, y_2), f(y_2, y_2))),$
    $f(f(y_2, y_2), f(y_2, y_2)), f(y_2, y_2), \ldots, x_{n-1}, f(x_n, x_n))$
  - $R(f(f(f(y_2, y_2), f(y_2, y_2)), f(f(y_2, y_2), f(y_2, y_2))),$
    $f(f(y_2, y_2), f(y_2, y_2)), f(y_2, y_2), \ldots, f(y_n, y_n), y_n)$

- Die Länge des ersten Terms des ersten Atoms wird in jedem Schritt ungefähr verdoppelt

Anhang: Beweisskizze für die Korrektheit der prädikatenlogischen Resolutionssatz

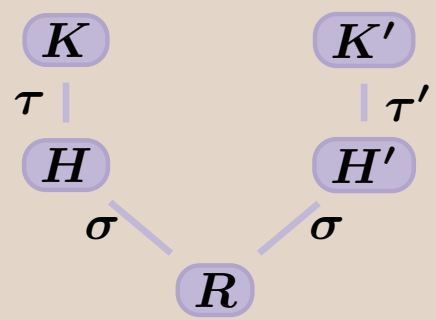
Das prädikatenlogische Resolutionslemma

Definition: Prädikatenlogische Resolvente (Wdh.)

- Seien K, K' prädikatenlogische Klauseln
- Seien τ, τ' Variablenumbenennungen, so dass $H \stackrel{\text{def}}{=} \tau(K)$ und $H' \stackrel{\text{def}}{=} \tau'(K')$ keine Variablen gemeinsam haben
- Seien $A_1, \ldots, A_k$ Atome in H und $\neg A'_1, \ldots, \neg A'_m$ Literale in H', so dass $\{A_1, \ldots, A_k, A'_1, \ldots, A'_m\}$ unifizierbar ist mit MGU σ

- Dann heißt $R \stackrel{\text{def}}{=} \sigma((H - \{A_1, \ldots, A_k\}) \cup (H' - \{\neg A'_1, \ldots, \neg A'_m\}))$ **prädikatenlogische Resolvente** von K und K'

- Illustration:



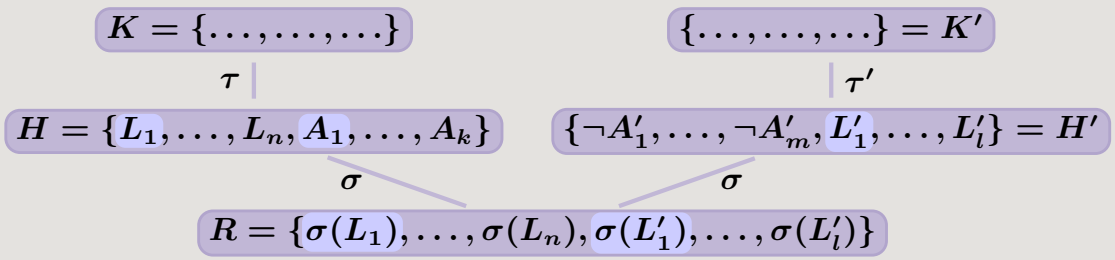
Lemma C.4.7: PL-Resolutionslemma

- Seien K, K', τ, τ', H, H', σ und R wie in der Definition von prädikatenlogischen Resolventen
- Sei $\mathcal{A}$ eine zu K und K' passende Struktur

- Wenn
  (a) für alle passenden Belegungen β gilt: $(\mathcal{A}, \beta) \models K$ und
  (b) für alle passenden Belegungen β' gilt: $(\mathcal{A}, \beta') \models K'$,
  dann gilt auch für alle passenden Belegungen γ: $(\mathcal{A}, \gamma) \models R$

Prädikatenlogisches Resolutionslemma: Beweisidee

Beweisidee



- Sei $\mathcal{A}$ eine Struktur, so dass
  (a) für alle passenden Belegungen β gilt: $(\mathcal{A},\beta) \models K$, und
  (b) für alle passenden Belegungen β' gilt: $(\mathcal{A},\beta') \models K'$

- Sei γ eine passende Belegung für R
- **Zu zeigen:** $(\mathcal{A},\gamma) \models R$

- **Beobachtung:**
  - $\gamma \circ \sigma \circ \tau$ ist passende Belegung von K
  - Wegen (a) gilt: $(\mathcal{A},\gamma \circ \sigma \circ \tau) \models L$ für mindestens ein Literal L aus K

  ➡ Analog für $\gamma \circ \sigma \circ \tau'$ und K'

- **Fall 1:** $\tau(L) = L_i$, für ein i
  $\Rightarrow (\mathcal{A},\gamma \circ \sigma) \models L_i$
  $\Rightarrow (\mathcal{A},\gamma) \models \sigma(L_i)$
  $\Rightarrow (\mathcal{A},\gamma) \models R$

- **Fall 2:** $\tau(L) = A_i$, für ein i
  $\Rightarrow (\mathcal{A},\gamma \circ \sigma) \models A_i$
  $\Rightarrow$ für alle j gilt dann:
  $$(\mathcal{A},\gamma \circ \sigma) \not\models \neg A'_j$$
  ☞ da $\sigma(A_i) = \sigma(A'_j)$
  $\Rightarrow (\mathcal{A},\gamma \circ \sigma \circ \tau') \models L'$ für ein Literal L' aus K' und $\tau'(L') = L'_k$, für ein k
  $\Rightarrow (\mathcal{A},\gamma \circ \sigma) \models L'_k$
  $\Rightarrow (\mathcal{A},\gamma) \models \sigma(L'_k)$
  $\Rightarrow (\mathcal{A},\gamma) \models R$

Beweisskizze für die Korrektheit der prädikatenlogischen Resolution

- Notation $\forall\mathcal{K}$ bezeichne die Formel, die aus der KNF zur Klauselmenge $\mathcal{K}$ durch All-Quantifizierung aller vorkommenden Variablen entsteht

Beweisskizze

- **Zu zeigen:** $\square \in \mathbf{Res}^\infty(\mathcal{K}) \Rightarrow \varphi$ unerfüllbar

- Wir zeigen zuerst, dass für jede prädikatenlogische Klauselmenge $\mathcal{K}'$ gilt:

  $\forall\mathbf{Res}(\mathcal{K}')$ unerfüllbar $\Rightarrow \forall\mathcal{K}'$ unerfüllbar

- Dazu zeigen wir die Kontraposition:

  $\forall\mathcal{K}'$ erfüllbar $\Rightarrow \forall\mathbf{Res}(\mathcal{K}')$ erfüllbar

  - Sei $\mathcal{A}$ ein Modell für $\forall\mathcal{K}'$
  - Für Resolventen $K \in \mathbf{Res}(\mathcal{K}')$ gilt wegen dem Resolutionslemma dann:

    $\mathcal{A} \models \forall\{K\}$

  $\Rightarrow \mathcal{A}$ ist ein Modell für $\forall\mathbf{Res}(\mathcal{K}')$

- Der Rest des Korrektheitsbeweises ist wie beim Korrektheitsbeweis der aussagenlogischen Resolution:

  $\square \in \mathbf{Res}^\infty(\mathcal{K})$
  $\Rightarrow \square \in \mathbf{Res}^\ell(\mathcal{K})$, für ein ℓ
  $\Rightarrow \mathbf{Res}^\ell(\mathcal{K})$ unerfüllbar, für ein ℓ
  $\Rightarrow \mathbf{Res}^{\ell-1}(\mathcal{K})$ unerfüllbar
  $\Rightarrow \ldots$
  $\Rightarrow \mathbf{Res}^0(\mathcal{K})$ unerfüllbar
  $\Rightarrow \mathcal{K}$ unerfüllbar

Kapitel C.5:

Grundlagen der logischen Programmierung

Inhalt

1. Logik-Programme und Prolog: Beispiele
2. SLD-Resolution und Auswertungsstrategien

Logik-Programme und Prolog: Beispiele

Logische Programmierung und Prädikatenlogik (1/7)

Beispiel

- Wir betrachten wieder das folgende Logik-Programm:

```
parent_of(bob,anne).
parent_of(dave,bob).
parent_of(eve,dave).
ancestor_of(X,Y) :- parent_of(X,Y).
ancestor_of(X,Y) :- parent_of(X,Z),
                    ancestor_of(Z,Y).
```

und die Anfrage

```
?- ancestor_of(Y,anne).
```

- Logik-Programme bestehen aus:
  - **Fakten:** `B.`
  - **Regeln:** `B :- A1,...,An.`
  - **Anfragen:** `?- A1,...,An.`

- **Konventionen:**
  - Variablennamen beginnen mit Großbuchstaben
  - Prädikate, Funktionen, Konstanten beginnen mit Kleinbuchstaben

- Praktische Umsetzung von Logik-Programmen: in der Programmiersprache **Prolog**
  - Es gibt verschiedene Dialekte und Implementierungen
  - Logik-Programme sind eine Abstraktion von Prolog-Programmen

Logische Programmierung und Prädikatenlogik (2/7)

Beispiel

```
parent_of(bob,anne).
parent_of(dave,bob).
parent_of(eve,dave).
ancestor_of(X,Y) :- parent_of(X,Y).
ancestor_of(X,Y) :- parent_of(X,Z),
                    ancestor_of(Z,Y).

?- ancestor_of(Y,anne).
```

- Informell lassen sich Logik-Programme gut verstehen
- **Fakten** `A.`:

  Atomare Aussagen, die als wahr angesehen werden
- **Regeln** `B :- A1,...,An.`:

  Wenn durch eine Substitution der Variablen alle Atome der rechten Seite zu wahren Fakten werden, dann wird auch das Atom `B` im Kopf der Regel wahr

Beispiel

- Die erste Regel (Zeile 4) liefert mit $X \mapsto$ bob und $Y \mapsto$ anne das Faktum `ancestor_of(bob,anne).`
- Die zweite Regel liefert mit $X \mapsto$ dave, $Y \mapsto$ anne und $Z \mapsto$ bob das Faktum `ancestor_of(dave,anne).`

- **Anfragen** `?- A1,...,An.`:

  Wenn durch eine Substitution der Variablen alle Atome der rechten Seite zu wahren Fakten werden, dann ist die Substitution der Variablen eine Antwort des Programmes

Beispiel

- Die Substitution $Y \mapsto$ bob liefert die Antwort bob

Logische Programmierung und Prädikatenlogik (3/7)

Beispiel

- Beispiel-Fakten:

```
parent_of(bob,anne).
parent_of(dave,bob).
parent_of(eve,dave).
```

- Beispiel-Regeln:

```
ancestor_of(X,Y) :-
                    parent_of(X,Y).
ancestor_of(X,Y) :-
   parent_of(X,Z),ancestor_of(Z,Y).
```

- Solche Regeln lassen sich durch prädikatenlogische Formeln beschreiben

- Verwende:
  - Konstantensymbole:
    - bob, anne, dave, eve
  - Relationssymbole:
    - $A \equiv$ ancestor_of
    - $P \equiv$ parent_of

- Die Fakten übersetzen sich in die atomaren Formeln

  $P(\text{bob}, \text{anne}), P(\text{dave}, \text{bob}), P(\text{eve}, \text{dave})$

- Die Regel

```
ancestor_of(X,Y) :- parent_of(X,Y)
```

  entspricht der PL-Formel

  $\forall x \, \forall y \, (P(x, y) \rightarrow A(x, y))$

- Die Regel

```
ancestor_of(X,Y) :-
   parent_of(X,Z),ancestor_of(Z,Y)
```

  entspricht der PL-Formel

  $\forall x \, \forall y$
  $\big(\exists z \, (P(x, z) \wedge A(z, y)) \rightarrow A(x, y)\big)$

- Die Anfrage

```
?- ancestor_of(Y,anne).
```

  entspricht der Formel

  $\exists y \, A(y, a)$

Logische Programmierung und Prädikatenlogik (4/7)

Beispiel

```
parent_of(bob,anne).
parent_of(dave,bob).
parent_of(eve,dave).
ancestor_of(X,Y) :- parent_of(X,Y).
ancestor_of(X,Y) :- parent_of(X,Z),ancestor_of(Z,Y).

?- ancestor_of(Y,anne).
```

- Durch das Programm gestellte Frage: Gilt die folgende Schlussfolgerung?

  Wenn alle Fakten und Regeln gelten,
  dann gibt es einen Wert für `Y`, der die Anfrage wahr macht

- In prädikatenlogische Begriffe übersetzt:

  Folgt die prädikatenlogische Formel für die Anfrage aus den Formeln für die Regeln und Fakten:

$$\{P(b,a), P(d,b), P(e,d), \varphi_1, \varphi_2\} \overset{?}{\models} \exists y\ A(y,a)$$

  wobei φ_1 und φ_2 die Formeln für die Regeln sind
  und a, b, d, e für anne, bob, dave, eve stehen

Logische Programmierung und Prädikatenlogik (5/7)

Beispiel

```
parent_of(bob,anne).
parent_of(dave,bob).
parent_of(eve,dave).
ancestor_of(X,Y) :- parent_of(X,Y).
ancestor_of(X,Y) :- parent_of(X,Z),ancestor_of(Z,Y).
?- ancestor_of(Y,anne).
```

- Wir wissen, dass

  $$\{P(b,a), P(d,b), P(e,d), \varphi_1, \varphi_2\} \models \exists y A(y,a)$$

  genau dann gilt, wenn die folgende Formelmenge unerfüllbar ist:

  $$\{P(b,a), P(d,b), P(e,d), \varphi_1, \varphi_2, \neg\exists y A(y,a)\}$$

  $$\equiv \{P(b,a), P(d,b), P(e,d), \varphi_1, \varphi_2, \forall y \neg A(y,a)\}$$

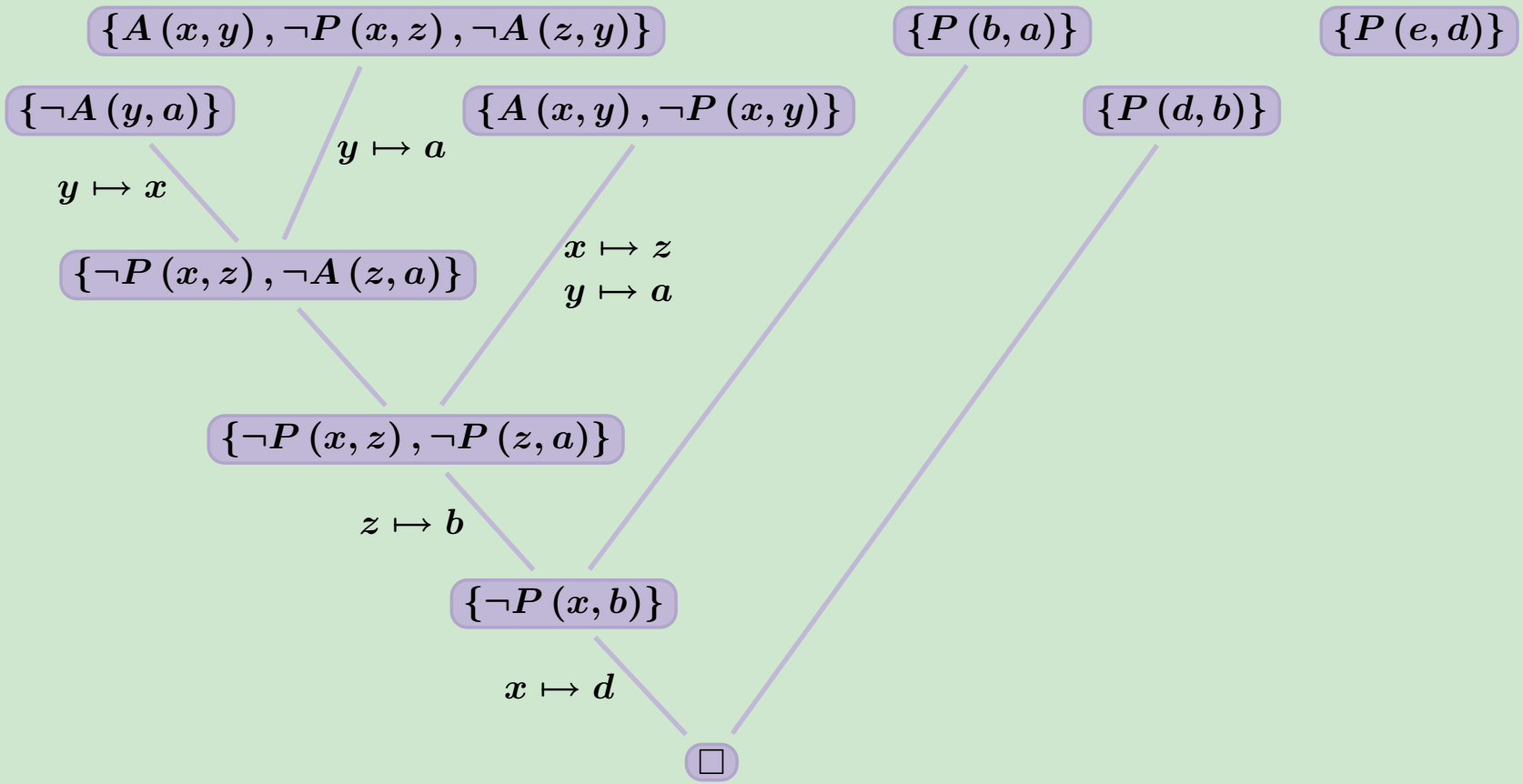
- Es ist also die Unerfüllbarkeit der folgenden Matrixklauselform zu testen:

  $$\{\{P(b,a)\}, \{P(d,b)\}, \{P(e,d)\}, \{A(x,y), \neg P(x,y)\}, \{A(x,y), \neg P(x,z), \neg A(z,y)\}, \{\neg A(y,a)\}\}$$

➡ Das geht mit prädikatenlogischer Resolution...

Logische Programmierung und Prädikatenlogik (6/7)

Beispiel



- Eine Antwort auf die Anfrage kann aus dem Resolutionsbeweis abgelesen werden: Die Komposition der Substitutionen für y liefert $y \mapsto d$
- Also: Dave ist ein Vorfahre von Anne

  (Erinnerung: `parent_of(bob,anne). parent_of(dave,bob). parent_of(eve,dave).`)

Logische Programmierung und Prädikatenlogik (7/7)

- Das eben betrachtete Prolog-Programm für Verwandtschaftsverhältnisse ist relativ einfach, da es keine zusammengesetzten Terme verwendet
- Ein Beispiel mit Termen werden wir gleich betrachten

- In der folgenden Tabelle ist der Zusammenhang zwischen Logik-Programmen und prädikatenlogischen Klauseln zusammengefasst:

Logik-Programme		Klauseln	
Faktum	`B.`	$\{B\}$	Tatsachenklausel
Regel	`B :- A1,...,An.`	$\{B, \neg A_1, \ldots, \neg A_n\}$	Prozedurklausel
Anfrage	`?- A1,...,An.`	$\{\neg A_1, \ldots, \neg A_n\}$	Zielklausel

- Aus Logik-Programmen resultierende prädikatenlogische Formeln haben also eine spezielle Form:
  - Die aus Fakten und Regeln entstehenden Klauseln in der KNF haben jeweils nur ein positives Literal ☞ linke Seite der Regel
  - Die aus der Anfrage entstehenden Klauseln haben gar keine positiven Literale
- Solche Klauseln haben wir für aussagenlogische Formeln bereits betrachtet

Prädikatenlogische Hornformeln

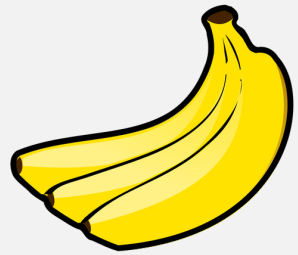
Definition: Hornklauseln, Hornformeln

- Eine disjunktive prädikatenlogische Klausel, die höchstens ein positives Literal hat, heißt **Hornklausel**
- Eine prädikatenlogische Formel in Pränexform, deren quantorenfreier Teil in KNF ist und nur aus Hornklauseln besteht, heißt **Hornformel**

- Die Umwandlung von Logik-Programmen in prädikatenlogische Formeln ergibt also immer Hornformeln

- Umgekehrt lässt sich die Erfüllbarkeit von Hornformeln als Logik-Programm formulieren
  - Bei der Skolemisierung entstehen aus Hornformeln wieder Hornformeln
    ☞ ohne Existenzquantoren
  - Solche universellen Hornformeln lassen sich in Logik-Programme umwandeln

Ein komplizierteres Logik-Programm (1/10)



Ein Affe ist in einem Raum eingeschlossen.

An der Decke hängt eine Bananenstaude.

Es steht ein Stuhl im Raum.

Kann der Affe die Bananen erreichen?





Ein komplizierteres Logik-Programm (2/10)

- Nehmen wir an, wir wissen das Folgende über Affen, Bananen und Stühle:
  (1) Wenn der Affe die Banane ergriffen hat, kann er sie essen
  (2) Ist der Stuhl in der Mitte des Raumes und ist der Affe auf dem Stuhl, so kann er die Banane greifen
  (3) Befinden sich der Affe und der Stuhl an derselben Position des Raumes, so kann der Affe auf den Stuhl steigen
  (4) Befinden sich der Affe und der Stuhl an derselben Position des Raumes, so kann der Affe den Stuhl zu einer anderen Position schieben
  (5) Der Affe kann zur Position des Stuhles laufen

- **Frage:** Kann der Affe die Bananen essen?
- Falls ja: wie muss er dabei vorgehen?

- **Ziel:** Ein Logik-Programm, das testet
  - ob der Affe die Banane essen kann und
  - wenn ja, auch eine Aktionsfolge findet, die den Affen zum Essen der Banane führt

Ein komplizierteres Logik-Programm (3/10)

Beispiel (Fortsetzung)

- Das Programm verwendet Terme zur Repräsentation von Zuständen des Szenarios
- Diese Terme verwenden das vierstellige Funktionssymbol `state`
- Der Term `state(window,false,door,false)` repräsentiert beispielsweise die Situation, dass
  - der Affe am Fenster ist,
  - der Affe nicht auf dem Stuhl steht,
  - der Stuhl an der Tür steht und
  - der Affe nicht die Banane hat
- **Allgemein:** state(PosAffe, AufStuhl, PosStuhl, HatBanane)

PosAffe	AufStuhl	PosStuhl	HatBanane
door	true	door	true
window	false	window	false
middle		middle	

Ein komplizierteres Logik-Programm (4/10)

Beispiel (Fortsetzung)

- **Einzelne Aktionen** repräsentieren wir durch Funktionssymbole und Konstanten
- Aktions**folgen** werden dann durch verschachtelte Terme repräsentiert
  - Ein Aktionsfolgenterm `f(Act)` hat die intendierte Bedeutung:

    Führe zuerst die Aktion `f` aus und dann die Aktionsfolge `Act`

- Beispiele:
  - Die Konstante `eat` steht für die Aktion, die Banane zu essen
  - `grasp(eat)` steht für die Aktionsfolge: Ergreife die Banane und iss sie

- Einige Aktionen haben noch einen weiteren Parameter

- Im Einzelnen verwenden wir die folgenden Funktionssymbole:
  - `grasp(Act)`:

    Ergreife zuerst die Banane und führe danach die durch `Act` gegeben Aktionsfolge aus
  - `climb(Act)`:

    Klettere auf den Stuhl und führe dann `Act` aus
  - `push(NewPos,Act)`:

    Schiebe den Stuhl zu Position `NewPos` und führe dann `Act` aus
  - `go(NewPos,Act)`:

    Gehe zu Position `NewPos` und führe dann `Act` aus
  - `eat`: Esse die Banane

Ein komplizierteres Logik-Programm (5/10)

Beispiel (Fortsetzung)

- Außerdem verwenden wir zwei Relationssymbole mit der folgenden Bedeutung:
  - `can_eat(State,Act)`:

    Von `State` aus kann der Affe mit der Aktionsfolge `Act` die Banane essen
  - `reach(Act)`:

    Der Affe kann von der Tür aus und mit dem Stuhl am Fenster mit der Aktionsfolge `Act` die Banane essen

- Das Programm verwendet Regeln der Art:

```
can_eat(s2,f(Act)) :- can_eat(s1,Act).
```

- Eine solche Regel bedeutet:

  Wenn vom Zustand `s1` aus mit der Aktionsfolge `Act` die Banane gegessen werden kann,
  dann auch vom Zustand `s2` aus mit `f(Act)`
- Intuitiv: Vom Zustand `s2` ist mit Aktion `f` ein Übergang in den Zustand `s1` möglich

Ein komplizierteres Logik-Programm (6/10)

- Jetzt übersetzen wir unser Wissen über die Situation in Regeln eines logischen Programms

Beispiel (Fortsetzung)

state(PosAffe, AufStuhl, PosStuhl, HatBanane)

door	true	door	true
window	false	window	false
middle		middle	

(1) „Wenn der Affe die Banane ergriffen hat, kann er sie essen“

```
can_eat(state(X,Y,Z,true),eat).
```

(2) „Ist der Stuhl in der Mitte und der Affe auf dem Stuhl, so kann er die Banane greifen“

```
can_eat(state(middle,true,middle,false),grasp(Act))
    :- can_eat(state(middle,true,middle,true),Act).
```

(3) „Sind Affe und Stuhl an derselben Position, so kann der Affe auf den Stuhl steigen“

```
can_eat(state(Pos,false,Pos,HasB),climb(Act))
    :- can_eat(state(Pos,true,Pos,HasB),Act).
```

Ein komplizierteres Logik-Programm (7/10)

Beispiel (Fortsetzung)

state(PosAffe, AufStuhl, PosStuhl, HatBanane)

PosAffe	AufStuhl	PosStuhl	HatBanane
door	true	door	true
window	false	window	false
middle		middle	

(4) „Sind Affe und Stuhl an derselben Position, so kann der Affe ihn zu einer anderen Position schieben“

```
can_eat(state(OldPos,false,OldPos,HasB),push(NewPos,Act))
    :- can_eat(state(NewPos,false,NewPos,HasB),Act).
```

(5) „Der Affe kann zu einer neuen Position laufen“

```
can_eat(state(OldPos,false,ChairPos,HasB),go(NewPos,Act))
    :- can_eat(state(NewPos,false,ChairPos,HasB),Act).
```

(6) Zusammenfassung des Gesamtziels:

```
reach_goal(Act)
    :- can_eat(state(door,false,window,false),Act).
```

(7) Anfrage: Kann der Affe die Banane essen? Falls ja, wie?

```
?- reach_goal(Act).
```

Ein komplizierteres Logik-Programm (8/10)

Beispiel (Fortsetzung)

- Für die Übersetzung des Programms in die Prädikatenlogik verwenden wir die Konstanten und Funktionssymbole:
  - s für `state`, e für `eat`, gr für `grasp`
  - c für `climb`, p für `push`, go für `go`
  - d für `door`, w für `window`, m für `middle`
  - t für `true`, f für `false`

- Außerdem verwenden wir die Relationssymbole C und R für `can_eat` und `reach_goal`

Ein komplizierteres Logik-Programm (9/10)

Beispiel (Fortsetzung)

- Die Übersetzung des Programms in die Prädikatenlogik ergibt die folgenden Klauseln:

(1) `can_eat(state(X,Y,Z,true),eat).` ➡ $\{C(s(x,y,z,t),e)\}$

(2) `can_eat(state(middle,true,middle,false),grasp(Act))`
`:- can_eat(state(middle,true,middle,true),Act).`
➡ $\{C(s(m,t,m,f),gr(x)), \neg C(s(m,t,m,t),x)\}$

(3) `can_eat(state(Pos,false,Pos,HasB),climb(Act))`
`:- can_eat(state(Pos,true,Pos,HasB),Act).`
➡ $\{C(s(y,f,y,u),c(x)), \neg C(s(y,t,y,u),x)\}$

(4) `can_eat(state(OldPos,false,OldPos,HasB),push(NewPos,Act))`
`:- can_eat(state(NewPos,false,NewPos,HasB),Act).`
➡ $\{C(s(y,f,y,u),p(z,x)), \neg C(s(z,f,z,u),x)\}$

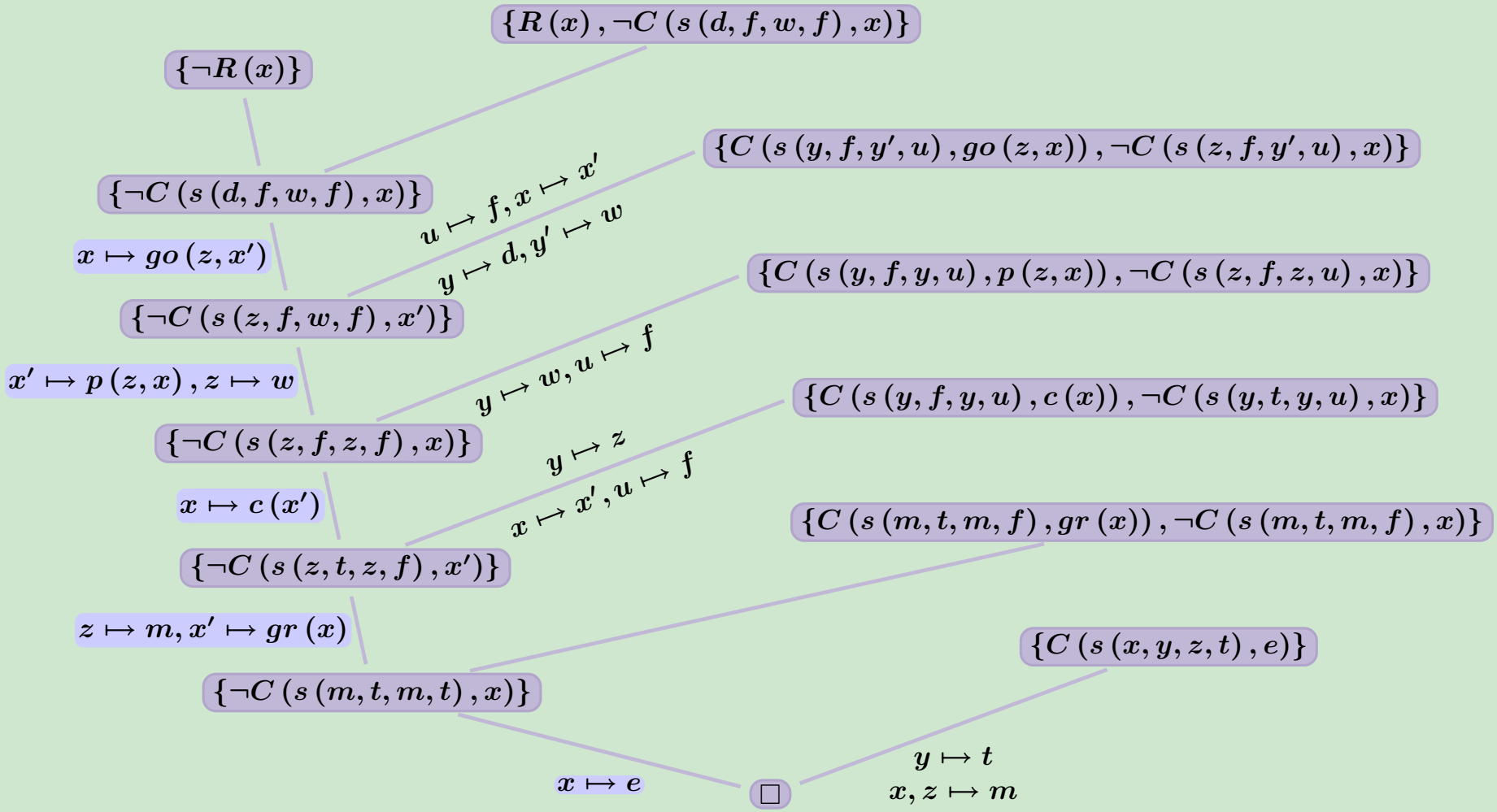
(5) `can_eat(state(Pos2,false,ChairPos,HasB),go(NewPos1,Act4))`
`:- can_eat(state(NewPos1,false,ChairPos,HasB),Act4).`
➡ $\{C(s(y,f,y',u),go(z,x)), \neg C(s(z,f,y',u),x)\}$

(6) `reach_goal(Act) :- can_eat(state(door,false,window,false),Act).`
➡ $\{R(x), \neg C(s(d,f,w,f),x)\}$

(7) `?- reach_goal(Act).` ➡ $\{\neg R(x)\}$

Ein komplizierteres Logik-Programm (10/10)

Beispiel (Fortsetzung)



- Durch Komposition der Substitutionen ergibt sich die Lösung:

  $x \mapsto go(w, p(m, c(gr(e))))$

SLD-Resolution und Auswertungsstrategien

Logik-Programme und Hornklauseln

Logik-Programme		Klauseln	
Faktum	`B.`	$\{B\}$	Tatsachenklausel
Regel	`B :- A1,...,An.`	$\{B, \neg A_1, \ldots, \neg A_n\}$	Prozedurklausel
Anfrage	`?- A1,...,An.`	$\{\neg A_1, \ldots, \neg A_n\}$	Zielklausel

- **Wir haben gesehen:**
  - Die Auswertung von Logik-Programmen lässt sich auf die prädikatenlogische Resolution zurückführen
  - Aus Zielklausel, Prozedur- und Tatsachenklauseln wird versucht die Box herzuleiten
- **Probleme:**
  - Die prädikatenlogische Resolution ist zwar vollständig, aber wir haben nur einen Semi-Entscheidungsalgorithmus
    - ➡ Wenn sich die Box nicht herleiten lässt, dann hält dieser Algorithmus nicht notwendig
  - Es gibt oft sehr viele resolvierbare Klauseln
    - ➡ Viele Wege, die (möglicherweise) nicht zum Ziel führen
- **Lösungsansatz:** Nutze aus, dass Logik-Programme nur Hornklauseln verwenden

Prädikatenlogische Resolution: Einschränkungen

- **Idee:** Nutze aus, dass Logik-Programme nur Hornklauseln verwenden
  - ➡ Bei der Resolution kann relativ zielgerichteter vorgegangen werden

- Um die Effizienz zu steigern betrachten wir Einschränkungen der prädikatenlogischen Resolution, die
  - den Suchraum der möglichen Resolutionsbeweise einschränken
  - und dabei für Mengen von Hornklauseln vollständig bleiben

- Eine Einschränkung ist **vollständig**, wenn es für jede unerfüllbare Formel in Matrixklauselform einen prädikatenlogischen Resolutionsbeweis gibt, der die Einschränkung respektiert und $\square$ erreicht

- Wir werden sehen:
  - Nicht alle betrachteten Einschränkungen sind vollständig
  - Aber alle sind zumindest für Hornformeln vollständig
    - ➡ Das reicht für Logik-Programme

✎ Wir verwenden im Folgenden häufig $A, B, \ldots$ als Notation für Atome, wenn es auf die vorkommenden Terme nicht ankommt

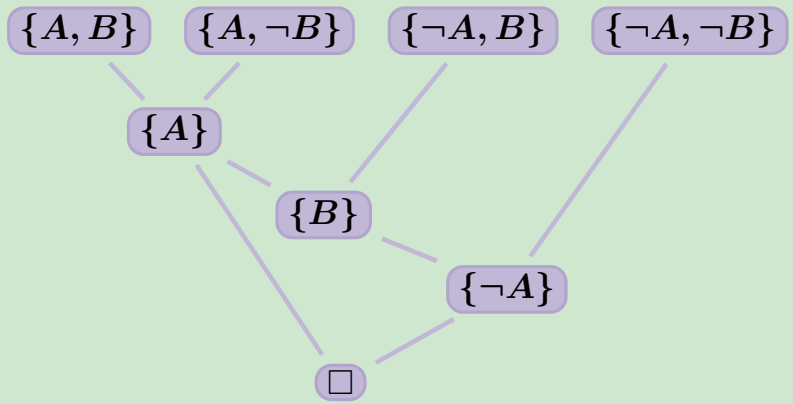
✎ Die folgenden Betrachtungen gelten auch für aussagenlogische Logik-Programme

✎ Wir folgen jetzt weitgehend dem Schöning-Buch [Sch]

Prädikatenlogische Resolution: Vollständige Restriktionen (1/2)

- $\mathcal{K}$ bezeichnet im Folgenden immer eine gegebene Klauselmenge
- Wir betrachten zunächst folgende Einschränkungen (**Restriktionen**) der prädikatenlogischen Resolution
- **P-Restriktion:** eine der beiden in einem Resolutionsschritt verwendeten Klauseln darf nur positive Literale enthalten
- **N-Restriktion:** eine der beiden in einem Resolutionsschritt verwendeten Klauseln darf nur negative Literale enthalten
- **Lineare Restriktion:** Der Resolutionsbeweis besteht aus einer Folge $K_0, \ldots, K_n$ und es gelten:
  - $K_0 \in \mathcal{K}$,
  - für jedes $i > 0$ ist K_i Resolvente von K_{i-1} und einer anderen Klausel
    ☞ K_j mit $j < i$ oder aus $\mathcal{K}$

Beispiel: Lineare Restriktion



Satz C.5.1

- Die folgenden Einschränkungen der prädikatenlogischen Resolution sind vollständig:
  - Resolution mit P-Restriktion
  - Resolution mit N-Restriktion
  - Resolution mit linearer Restriktion

Beweisidee (siehe [Sch])

- Zunächst: Nachweis für Aussagenlogik
- Übertragung auf Prädikatenlogik mit Hilfe des Lifting-Lemma

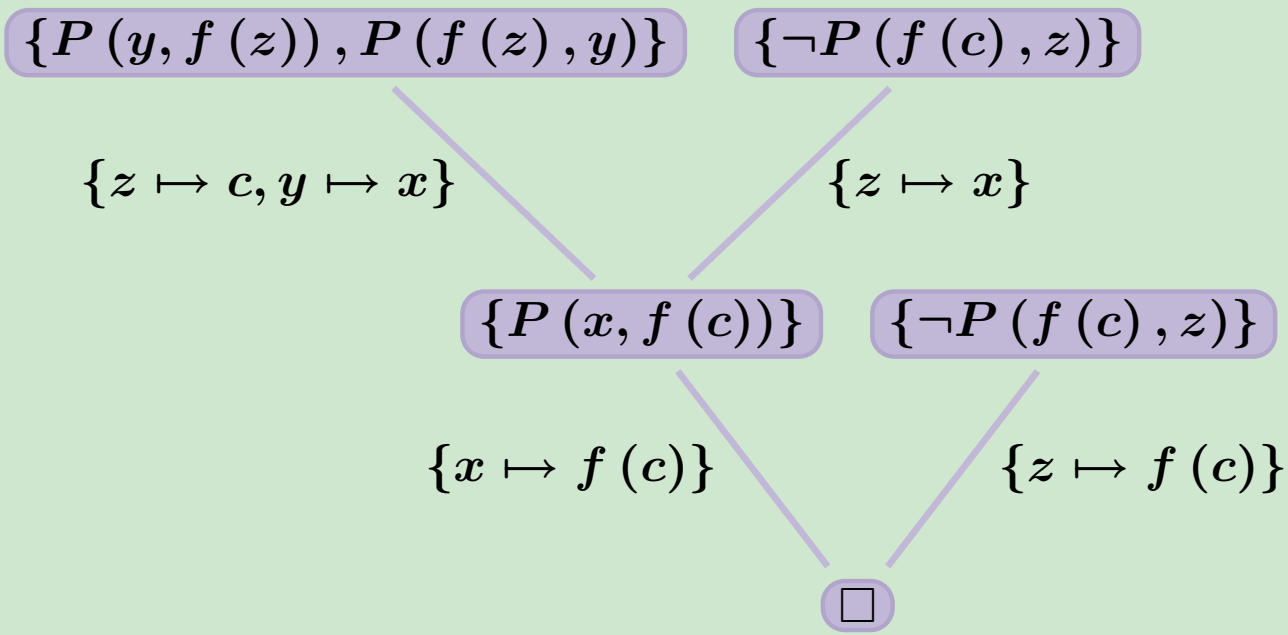
Prädikatenlogische Resolution: Vollständige Restriktionen (2/2)

Beispiel: P-Restriktion und N-Restriktion

- Sei $\varphi = \exists x \forall y \forall z \Big((P(y, f(z)) \vee P(f(z), y)) \wedge \neg P(f(x), z) \Big)$

- Matrixklauselform:

$$\{\{P(y, f(z)), P(f(z), y)\}, \{\neg P(f(c), z)\}\}$$



PL-Resolution: Vollständige Restriktionen für Hornklauseln

- Einige weitere Einschränkungen:

- **Input-Restriktion:** Mindestens eine der beiden verwendeten Klauseln muss immer aus $\mathcal{K}$ sein

- **Einheits-Restriktion:** Mindestens eine der beiden verwendeten Klauseln enthält nur ein Literal

- **SLD-Resolution:**
  - Lineare Restriktion
  - Input-Restriktion
  - Die erste Klausel K_0 ist eine Zielklausel und auch die jeweilige Resolvente ist dann auch wieder eine Zielklausel

☞ Mehr Details: nächste Folien

✎ Beim Affe-Stuhl-Banane-Beispiel haben wir die SLD-Resolution schon verwendet

Satz C.5.2

- Die folgenden Einschränkungen der prädikatenlogischen Resolution sind im Allgemeinen unvollständig, aber vollständig für Hornklauseln:
  - Resolution mit Input-Restriktion
  - Resolution mit Einheits-Restriktion
  - Resolution mit SLD-Restriktion

- Da Logik-Programme nur aus Hornklauseln bestehen, lassen sich mit der SLD-Resolution also alle Lösungen für Logik-Programme finden

- Die Auswertung von PROLOG-Programmen erfolgt deshalb meist mit SLD-Resolution

SLD-Resolution: Vorgehensweise (1/2)

- Vorgehen der **SLD-Resolution**:

- Beginne mit der Zielklausel für die Anfrage
- In jedem Schritt: Resolviere die aktuelle Zielklausel mit einer Klausel des Programms
  - Wähle dazu aus:
    - Ein Literal der aktuellen Zielklausel, mit dem resolviert werden soll, und
    - Eine Prozedur- oder Tatsachenklausel, deren positives Literal sich mit dem ausgewählten Literal unifizieren lässt
- Die entstehende Klausel ist wieder eine Zielklausel und wird zur neuen aktuellen Zielklausel
- Dieses Vorgehen wird wiederholt, bis die leere Klausel erreicht wird

- Dabei gilt: Variablenumbenennungen werden immer nur in der Prozedur- bzw. Tatsachenklausel vorgenommen

Beispiel

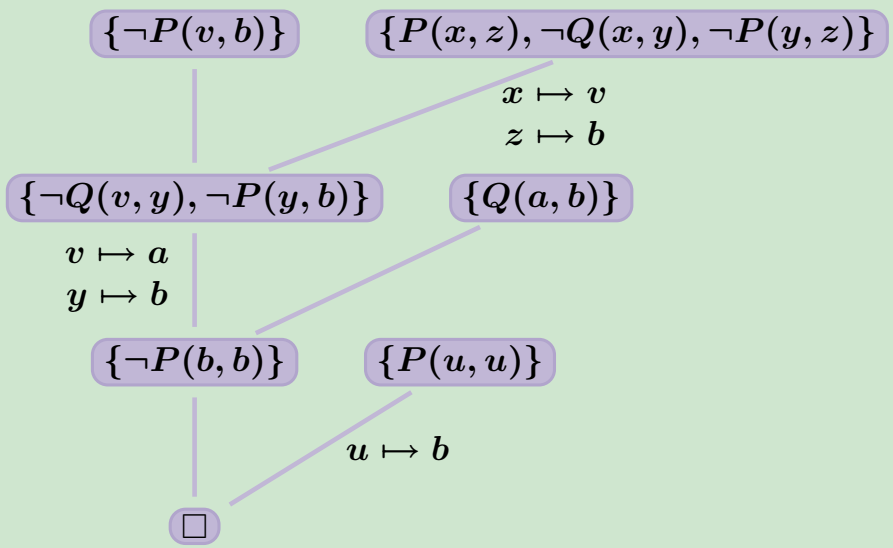
- Beispiel-Programm:

```
p(X,Z) :- q(X,Y), p(Y,Z).
p(U,U).
q(a,b).
?- p(V,b).
```

- Als Klauseln:
  - Prozedurklausel:

$$\{P(x,z), \neg Q(x,y), \neg P(y,z)\}$$

  - Tatsachenklauseln: $\{P(u,u)\}$, $\{Q(a,b)\}$
  - Zielklausel: $\{\neg P(v,b)\}$



SLD-Resolution: Vorgehensweise (2/2)

Beispiel

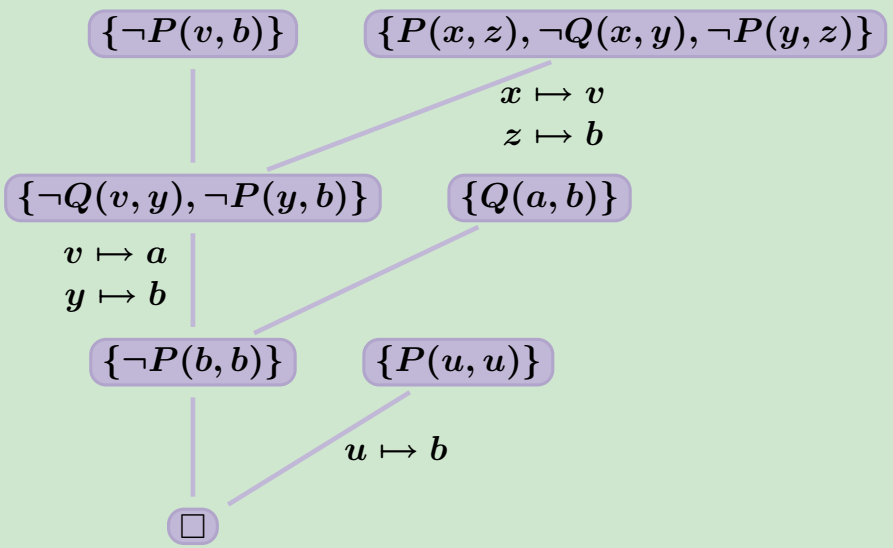
- Beispiel-Programm:

```
p(X,Z) :- q(X,Y), p(Y,Z).
p(U,U).
q(a,b).
?- p(V,b).
```

- Als Klauseln:
  - Prozedurklausel:

$$\{P(x,z), \neg Q(x,y), \neg P(y,z)\}$$

  - Tatsachenklauseln: $\{P(u,u)\}$, $\{Q(a,b)\}$
  - Zielklausel: $\{\neg P(v,b)\}$



- Bezogen auf die Regeln eines Logik-Programms geht die SLD-Resolution also wie folgt vor:

- Sei G die Menge der Atome der Anfrage
- Resolviere G mit einer Regel oder einem Fakt
  - Wähle dazu aus:
    - Ein Atom A in G
    - Einen Fakt oder eine Regel mit einer linken Seite, die zu A passt
- Entferne A aus G
- Falls eine Regel verwendet wurde, füge die Atome ihrer rechten Seite (unter Anwendung der nötigen Umbenennungen) zu G hinzu
- Wiederhole dies, bis G leer wird

- Wir betrachten nun diese Vorgehensweise noch etwas genauer, insbesondere die Protokollierung der Variablenumbenennungen

SLD-Resolution: Berechnungen (1/2)

- Wenn wir einen SLD-Beweis schrittweise aufbauen, haben wir jeweils
  - eine aktuelle Zielklausel G und
  - eine Substitution σ, die die Substitutionen der bisherigen Resolutionsschritte „zusammenfasst“

  ➡ **Konfiguration:** (G, σ)

- Statt der prädikatenlogischen Notation $\{\neg A_1, \ldots, \neg A_n\}$ für Zielklauseln verwenden wir dabei die Logik-Programm-Notation:

```
?- A1,...,An
```

- Wir schreiben $(G_1, \sigma_1) \vdash_{\mathcal{K}} (G_2, \sigma_2)$, falls
  - G_2 durch einen Resolutionsschritt aus G_1 und einer Klausel aus $\mathcal{K}$ hervorgeht und
  - $\sigma_2 = \sigma \circ \sigma_1$ ist, wobei σ die auf G_1 angewendete Substitution bezeichnet

- Eine **Berechnung** zu einem Logik-Programm $\mathcal{K}$ und einer Zielklausel G ist eine (möglicherweise unendliche) Folge

$$(G, [\,]) \vdash_{\mathcal{K}} (G_1, \sigma_1) \vdash_{\mathcal{K}} (G_2, \sigma_2) \vdash_{\mathcal{K}} \cdots$$

- **Erfolgreiche Berechnung:**

$$(G, [\,]) \vdash_{\mathcal{K}} (G_1, \sigma_1) \vdash_{\mathcal{K}} \cdots \vdash_{\mathcal{K}} (\square, \sigma_n)$$

  - σ_n heißt dann eine **Lösung**

SLD-Resolution: Berechnungen (2/2)

Beispiel

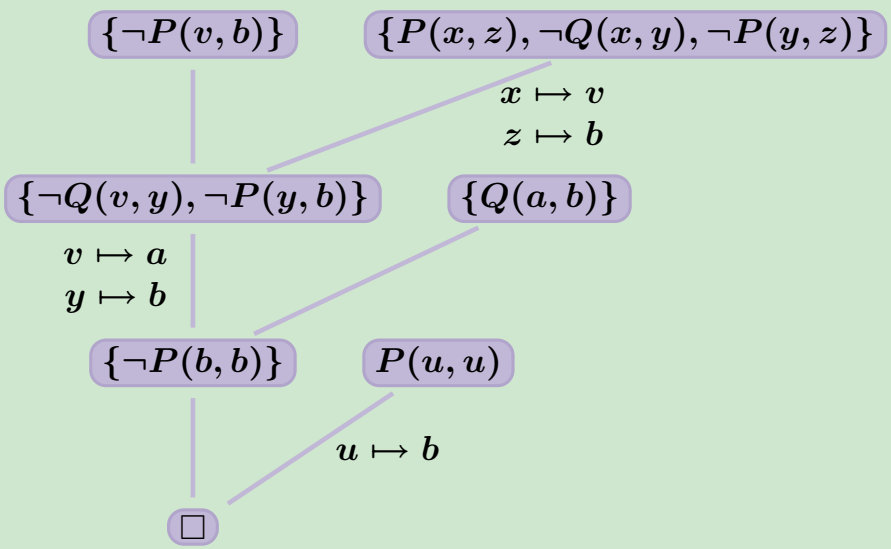
- Beispiel-Programm:

```
p(X,Z) :- q(X,Y), p(Y,Z).
p(U,U).
q(a,b).
?- p(V,b).
```

- Als Klauseln:
  - Prozedurklausel:

$$\{P(x,z), \neg Q(x,y), \neg P(y,z)\}$$

  - Tatsachenklauseln: $\{P(u,u)\}$, $\{Q(a,b)\}$
  - Zielklausel: $\{\neg P(v,b)\}$



- In Konfigurationen-Schreibweise:

$(\texttt{?- p(V,b)}, [\,])$

$\vdash_{\mathcal{K}} (\texttt{?- q(V,Y), p(Y,b)}, [\,])$

$\vdash_{\mathcal{K}} (\texttt{?- p(b,b)}, [\texttt{V} \mapsto \texttt{a}])$

$\vdash_{\mathcal{K}} (\square, [\texttt{V} \mapsto \texttt{a}])$

- Lösung: $\sigma = [\texttt{V} \mapsto \texttt{a}]$

✎ Es gibt hier natürlich noch andere erfolgreiche Berechnungen

✎ Wir notieren in σ nur den für die Lösung relevanten Teil

Vollständigkeit der SLD-Resolution: Präzisierung

- **Ziel:** Die Folgerungen aus einem Logik-Programm lassen sich gerade durch erfolgreiche Berechnungen finden

- Für eine Formel φ und eine Substitution τ, die jeder Variablen von φ einen Grundterm zuordnet, nennen wir $\tau(\varphi)$ eine **Grundinstanz** von φ

Satz C.5.3: Satz von Clark

(a) **Korrektheit:**

Wenn

- $\mathcal{K}$ die Klauselmenge eines Logik-Programms,
- G eine Zielklausel ?- $A_1, \ldots, A_k$ und
- $(G, [\,]) \vdash_{\mathcal{K}} \cdots \vdash_{\mathcal{K}} (\square, \sigma)$ eine erfolgreiche Berechnung ist,

dann ist jede Grundinstanz von $\sigma(A_1 \wedge \cdots \wedge A_k)$ eine Folgerung aus $\mathcal{K}$

(b) **Vollständigkeit:**

Wenn für ein σ jede Grundinstanz von $\sigma(A_1 \wedge \cdots \wedge A_k)$ eine Folgerung aus $\mathcal{K}$ ist, dann gibt es eine erfolgreiche Berechnung

$$(G, [\,]) \vdash_{\mathcal{K}} \cdots \vdash_{\mathcal{K}} (\square, \sigma')$$

und eine Substitution σ'' mit

$$\sigma(A_1 \wedge \cdots \wedge A_k) = \sigma''(\sigma'(A_1 \wedge \cdots \wedge A_k))$$

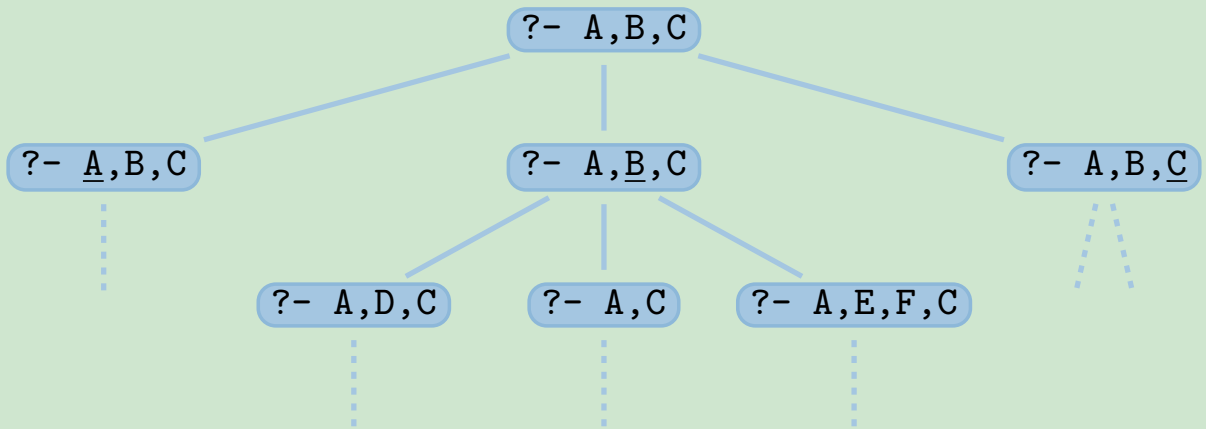
- Ein Beweis findet sich in [Sch]

Auswertungsstrategien (1/3)

- Zu einem Logik-Programm und einer Zielklausel kann es viele verschiedene Berechnungen geben
- In jedem Schritt gibt es zwei Wahlmöglichkeiten:
  (1) Auswahl eines Literals in der (aktuellen) Zielklausel
  (2) Auswahl der passenden Prozedur- oder Tatsachenklausel

Beispiel: Auswertung

```
A :- B,C.
B :- D.
B.
B :- E,F.
C :- A,C.
C.
?- A,B,C.
```



- Die Reihenfolge der Auswertung hinsichtlich (1) ist egal
  - Zu jeder erfolgreichen Berechnung gibt es eine äquivalente Berechnung, in der immer mit dem ersten Literal resolviert wird

  ➡ Wir nennen solche Berechnungen **kanonische Berechnungen**

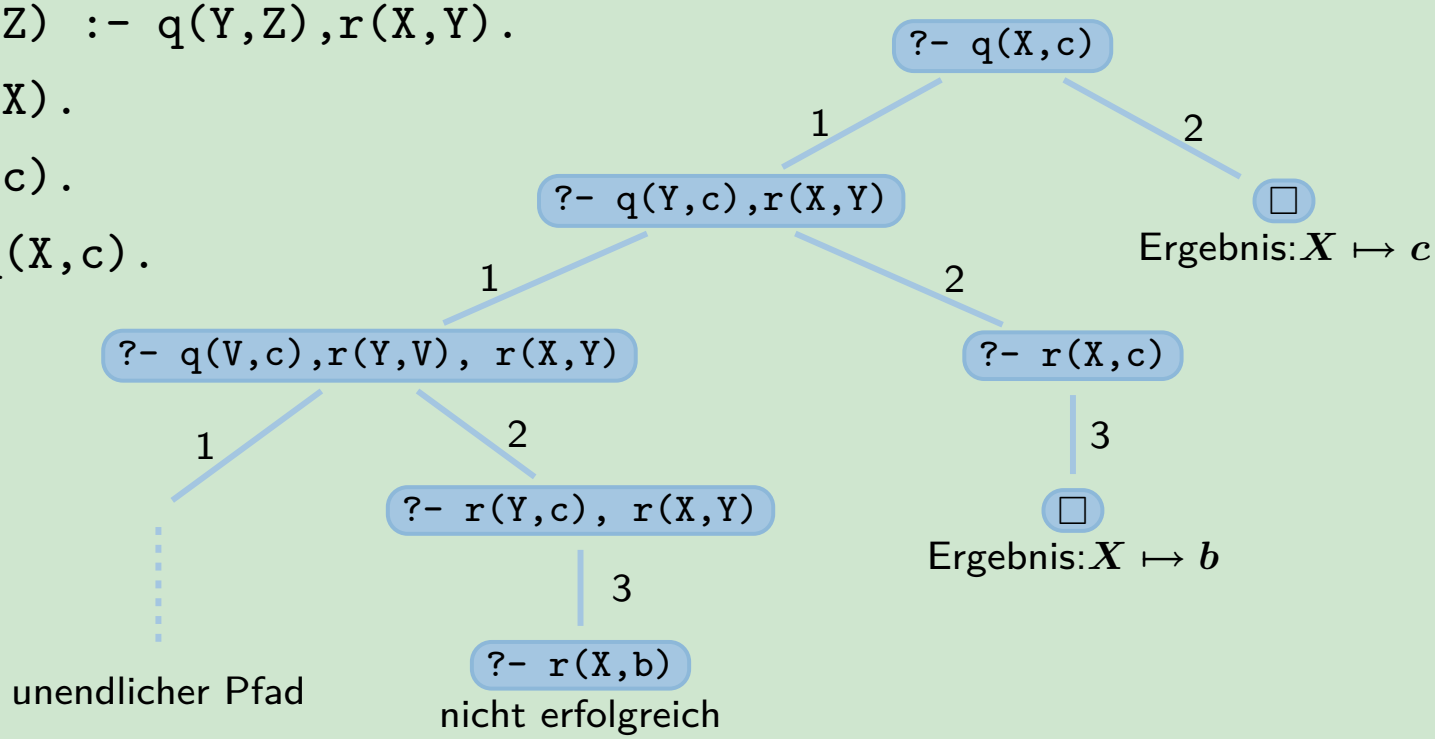
- Prolog verwendet kanonische Berechnungen

Auswertungsstrategien (2/3)

- Die möglichen kanonischen Berechnungen lassen sich noch etwas kompakter darstellen:
  - ➡ Da die Links-nach-rechts-Auswertung vorgegeben ist, sind keine Verzweigungen zur Auswahl eines Literals in der aktuellen Anfrage nötig

Beispiel: Kompakte Auswertung

```
1: q(X,Z) :- q(Y,Z),r(X,Y).
2: q(X,X).
3: r(b,c).
Z: ?- q(X,c).
```



- Der Baum kann mit Backtracking durchsucht werden

Auswertungsstrategien (3/3)

- Zur systematischen Durchsuchung des Baumes bieten sich (wie immer) zwei Strategien an:

- **Breitensuche:**
  - Zuerst alle Kinder der Wurzel, dann alle Kinder der Kinder usw.
  - Auf diese Weise wird jede Lösung gefunden
  - Aber es ist sehr aufwändig

- **Tiefensuche:**
  - Immer zuerst im ersten Teilbaum suchen
  - Wenn er ganz durchsucht ist, im zweiten Teilbaum weitersuchen usw.

- **Problem:** Bei der Tiefensuche terminiert die Auswertung eines unendlichen Teilbaumes nicht!

- Die Tiefensuche ist aber meist effizienter und wird deshalb in Prolog trotz der Gefahr der Nicht-Termination üblicherweise verwendet
- Die Auswertungsreihenfolge kann dabei eine große Rolle spielen

- Prolog probiert die Prozedur- und Tatsachenklauseln nach ihrer Reihenfolge im Programm aus
- Eventuell kann die Umordnung der Regeln oder der Literale einer Regel dazu führen, dass ein zuvor nicht terminierendes Programm eine Lösung findet
- Insbesondere ist es meist sinnvoll, die Fakten (Tatsachenklauseln) **vor** den Regeln (Prozedurklauseln) zu platzieren!

- Wenn eine Lösung gefunden wurde, können durch eine Fortsetzung der Tiefensuche möglicherweise weitere Lösungen gefunden werden

Zusammenfassung

- **In diesem Vorlesungskapitel haben wir gesehen:**
  - Logische Programmierung erlaubt es
    - Wissensbasen, d.h. Fakten und Regeln, zu formalisieren und
    - Anfragen zu stellen
  - Fakten, Regeln und Anfragen in Logik-Programmen entsprechen prädikatenlogischen Hornformeln
  - Zur Anfrageauswertung kann prädikatenlogische Resolution verwendet werden
  - Um die Auswertungseffizienz zu erhöhen, wird versucht einen SLD-Resolutionsbeweis zu finden

Kapitel C.6:

(Un-)Vollständigkeit und weitere Ergebnisse

Übersicht über dieses Kapitel

- **Gödelscher Vollständigkeitssatz:** Wir zeigen den Gödelschen Vollständigkeitssatz, eines der wichtigsten Ergebnisse der Prädikatenlogik
- **Hilberts Programm:** Wir lernen einige Resultate kennen, die Grenzen der Beweisbarkeit von Aussagen über die Arithmetik aufzeigen

  ➡ Gödelsche Unvollständigkeitssätze

- **Algorithmen:** Wir werfen einen kurzen Blick auf algorithmische Probleme im Zusammenhang der Prädikatenlogik
- **Vergleich der Logiken:** Wir vergleichen die Prädikatenlogik mit der Aussagenlogik und der Modallogik
- **Schloss Dreadsbury:** Wir lösen auf
- **Andere Beweiskalküle:** Außer der Resolution gibt es noch viele andere Beweiskalküle für die Prädikatenlogik. Wir betrachten zwei dieser Kalküle:

  ➡ Den **Hilbertkalkül** und den **Sequenzenkalkül**

Inhalt

1. Gödelsche Sätze und Hilberts Programm
2. Entscheidungsprobleme der Prädikatenlogik
3. Vergleich der betrachteten Logiken
4. Auflösung Schloss Dreadsbury
5. Hilbert- und Sequenzenkalkül

Gödelsche Sätze und Hilberts Programm

Folgern und Beweisen

- **Zur Erinnerung:** Für eine Menge Φ prädikatenlogischer Formeln und eine prädikatenlogische Formel ψ schreiben wir $\Phi \models \psi$, wenn für jede passende Struktur $\mathcal{A}$ gilt:

  Wenn $\mathcal{A} \models \Phi$ dann $\mathcal{A} \models \psi$

- Dies ist der **semantische** Folgerungsbegriff der Prädikatenlogik

- Wir definieren auch wieder einen **syntaktischen** Beweisbegriff, basierend auf dem Resolutionskalkül

Definition: Beweisbar

- Sei Φ eine Menge prädikatenlogischer Formeln und ψ eine prädikatenlogische Formel

- ψ ist aus Φ **beweisbar** falls es eine endliche Teilmenge $\Phi' \subseteq \Phi$ gibt, sodass

  $$\square \in \mathbf{Res}^{\infty}(\mathcal{K})$$

  für die Matrixklauselformel $\mathcal{K}$ von $\varphi \wedge \neg\psi$ gilt, wobei φ die Konjunktion der Formeln aus Φ' ist

- Notation: $\Phi \vdash \psi$

- $\Phi \vdash \psi$ bedeutet also, dass es einen prädikatenlogischen Resolutionsbeweis für die Unerfüllbarkeit von $\Phi \cup \{\neg\psi\}$ gibt

✎ Da Resolutionsbeweise immer endlich sind, genügt es in der Definition endliche Teilmengen Φ' von Φ zu betrachten

Vollständigkeitssatz für endliche Formelmengen

- **Ziel:** $\Phi \models \psi \iff \Phi \vdash \psi$

- Um diese Aussage für beliebige Formelmengen zu beweisen, bräuchten wir das Auswahlaxiom
- Hier: Beweis für endliche und abzählbar unendliche Formelmengen

- Wir gehen wie folgt vor:
  - Erst beweisen wir den Vollständigkeitssatz für endliche Formelmengen
  - Dann beweisen wir ihn für abzählbar unendliche Formelmengen
    - ➡ Dazu benötigen wir den Endlichkeitssatz der Prädikatenlogik

Satz C.6.1: Endlicher Vollständigkeitssatz

- Sei Φ eine endliche Menge prädikatenlogischer Formeln und ψ eine prädikatenlogische Formel
- Dann gilt: $\Phi \models \psi \iff \Phi \vdash \psi$

Beweis

„$\Phi \models \psi \Rightarrow \Phi \vdash \psi$“:

- Wenn $\Phi \models \psi$, dann ist $\Phi \cup \{\neg\psi\}$ unerfüllbar
- Sei φ die Konjunktion der Formeln aus Φ

  $\Rightarrow \varphi \wedge \neg\psi$ ist unerfüllbar
  $\Rightarrow \square \in \mathbf{Res}^\infty(\mathcal{K})$ für die Matrixklauselform $\mathcal{K}$ von $\varphi \wedge \neg\psi$

  ☞ Prädikatenlogischer Resolutionssatz

  $\Rightarrow \Phi \vdash \psi$

„$\Phi \vdash \psi \Rightarrow \Phi \models \psi$“:

- Ist $\Phi \vdash \psi$, so gibt eine Konjunktion φ von Formeln aus Φ, so dass für die Matrixklauselform $\mathcal{K}$ von $\varphi \wedge \neg\psi$ gilt: $\square \in \mathbf{Res}^\infty(\mathcal{K})$

  $\Rightarrow \varphi \wedge \neg\psi$ ist unerfüllbar

  ☞ Prädikatenlogischer Resolutionssatz

  $\Rightarrow \Phi \cup \{\neg\psi\}$ ist unerfüllbar
  $\Rightarrow \Phi \models \psi$

Der Endlichkeitssatz der Prädikatenlogik

Satz C.6.2: Endlichkeitssatz

- Sei Φ eine abzählbar unendliche Menge prädikatenlogischer Formeln über einer endlichen Signatur σ
- Dann sind äquivalent:
  - (i) Φ ist erfüllbar
  - (ii) Jede endliche Teilmenge von Φ ist erfüllbar

Beweisskizze

- Es genügt zu zeigen:

  Wenn jede endliche Teilmenge von Φ erfüllbar ist, dann ist Φ erfüllbar

- **Idee:** Rückführung auf den Endlichkeitssatz der Aussagenlogik mit Hilfe der Grundresolution
- Sei $\Phi = \{\varphi_1, \varphi_2, \ldots\}$ eine abzählbar unendliche Menge prädikatenlogischer Formeln über einer endlichen Signatur σ
- Zu jeder Formel φ_i sei φ'_i eine erfüllbarkeitsäquivalente Skolemformel

  ➡ Bei der Skolemisierung verwenden wir für jede Formel φ_i eigene Funktionssymbole $f_1^i, f_2^i, \ldots$
- Sei $\Phi' \stackrel{\text{def}}{=} \{\varphi'_1, \varphi'_2, \ldots\}$
- Da jede endliche Teilmenge von Φ erfüllbar ist, ist auch jede endliche Konjunktion χ von Formeln aus Φ' erfüllbar

  $\Rightarrow$ Die Herbrand-Expansion $E(\chi)$ ist für jede solche Formel erfüllbar

  ☞ Grundresolutionssatz

  $\Rightarrow$ Die Herbrand-Expansion $E(\Phi')$ aller Formeln aus Φ' ist erfüllbar

  ☞ Endlichkeitssatz der Aussagenlogik

  $\Rightarrow$ Es gibt ein Modell für Φ (das sich aus einem Modell für $E(\Phi')$ gewinnen lässt)

  $\Rightarrow$ Φ ist erfüllbar

Gödelscher Vollständigkeitssatz

Satz C.6.3: Vollständigkeitssatz **[Gödel 1929]**

- Sei Φ eine endliche oder abzählbar unendliche Menge prädikatenlogischer Formeln über einer endlichen Signatur σ und ψ eine prädikatenlogische Formel
- Dann gilt: $\Phi \models \psi \iff \Phi \vdash \psi$

Beweisskizze

„$\Phi \vdash \psi \Rightarrow \Phi \models \psi$“:

- Wie im Beweis vom endlichen Vollständigkeitssatz

„$\Phi \models \psi \Rightarrow \Phi \vdash \psi$“:

- Es gelte $\Phi \models \psi$
  $\Rightarrow \Phi \cup \{\neg\psi\}$ ist unerfüllbar
- Aus dem Endlichkeitssatz folgt, dass es eine endliche Teilmenge Φ' von Φ gibt, so dass $\Phi' \cup \{\neg\psi\}$ unerfüllbar ist
  $\Rightarrow \Phi' \vdash \psi$ ☞ Prädikatenlogischer Resolutionssatz
  $\Rightarrow \Phi \vdash \psi$ nach Definition von „$\vdash$“

✎ Die Aussage gilt auch für beliebige Formelmengen Φ

Prädikatenlogik mit Gleichheit

- **Bislang:** Prädikatenlogik ohne Gleichheit (zur Einfachheit)
- **Frage:** Lassen sich die gefundenen Resultate auf die Prädikatenlogik mit Gleichheit übertragen?

Beispiel

- $\varphi = \forall x\ f(x) = x$
- Klar: Diese Formel ist erfüllbar, aber sie hat kein Herbrand-Modell in unserem Sinne

- Der **Vollständigkeitssatz** gilt aber auch für die Prädikatenlogik mit Gleichheit

- Der Beweis erfordert eine Modifikation der Definition von Herbrand-Modellen:
  - Statt der Menge aller Terme wird die Menge aller Äquivalenzklassen von Termen verwendet bezüglich der Äquivalenzrelation

    $$s \sim t \stackrel{\text{def}}{=} (\varphi \vdash s = t)$$

  - Das macht den Beweis natürlich nicht gerade einfacher...

- Der **Endlichkeitssatz** gilt entsprechend ebenfalls für die Prädikatenlogik mit Gleichheit

- Der **Satz von Löwenheim-Skolem** gilt auch:

  Für jede erfüllbare Formel gibt es ein abzählbar unendliches oder ein endliches Modell

Gödel und Hilbert

Kurz-Bio: Kurt Gödel



Kurt Gödel
Bild: Gemeinfrei

- Geboren: 28.4.1906 in Brno (Brünn)
- Studium in Wien (Physik, Mathematik)
- Promotion 1930: Vollständigkeitssatz
- 1931: Unvollständigkeitssätze
- Ab 1933 häufig (ab 1940 dauernd) in Princeton
- Zusammenarbeit unter anderem mit von Neumann und Einstein
- Gestorben: 14.1.1978 in Princeton

Kurz-Bio: David Hilbert



David Hilbert
Bild: Gemeinfrei

- Geboren: 23.1.1862 in Königsberg
- Studium in Königsberg
- Professor in Göttingen von 1895-1934
- Bedeutendster Mathematiker zu Beginn des 20. Jahrhunderts
- Fundamentale Beiträge zu den meisten Gebieten der Mathematik
- **Hilberts Programm**
- **Hilberts 23 Probleme**
- Gestorben: 14.2.1943 in Göttingen

Hilberts Programm

- Gödels 1929 bewiesener Vollständigkeitssatz steht im engen Zusammenhang mit **Hilberts Programm**

- Um 1900 durchlief die Mathematik eine schwere Grundlagenkrise
  - ➡ Russels Paradoxon (1903)
- **Hilberts Ziel:** Ein sicheres, formales Fundament der Mathematik

- **Startpunkt:** Ein Fundament für die Arithmetik
  - ➡ Viele Gebiete der Mathematik lassen sich auf diese zurückführen
- Ein **wahrer Satz der Arithmetik** ist eine geschlossene prädikatenlogische Formel ψ mit $(\mathbb{N}, +, \times, \mathbf{0}, \mathbf{1}) \models \psi$
- **Gesucht wurde:** Ein widerspruchsfreies, entscheidbares Axiomensystem für die wahren Sätze der Arithmetik

- Das heißt, es wurde eine Menge Φ prädikatenlogischer Formeln und ein Beweisbegriff „$\vdash$“ mit den folgenden Eigenschaften gesucht:
  - Ist ψ ein Satz der Arithmetik, so gilt $\Phi \vdash \psi$ ☞ Vollständigkeit
  - Es gibt keine prädikatenlogische Formel ψ mit $\Phi \vdash \psi$ und $\Phi \vdash \neg\psi$ ☞ Widerspruchsfreiheit
  - Φ ist endlich oder es lässt sich algorithmisch feststellen, ob eine gegebene Formel χ in Φ ist

- Außerdem wurde ein Verfahren gesucht, mit dem sich entscheiden lässt, ob für eine gegebene Formel ψ gilt:

  $$(\mathbb{N}, +, \times, \mathbf{0}, \mathbf{1}) \models \psi$$

- Diese Frage ist als **Entscheidungsproblem der Arithmetik** bekannt

Ein Kandidat: Die Peano-Arithmetik

- **Gesucht:** Ein widerspruchsfreies, entscheidbares Axiomensystem für die wahren Sätze der Arithmetik

- **Kandidat:** Peano-Arithmetik
- **Axiome der Peano-Arithmetik:**
  - **Arithmetische Axiome:**
    - $\forall x(\neg \mathbf{0} = S(x))$
    - $\forall x \forall y(S(x) = S(y) \rightarrow x = y)$
    - $\forall x(x + \mathbf{0} = x)$
    - $\forall x \forall y(x + S(y) = S(x + y))$
    - $\forall x(x \times \mathbf{0} = \mathbf{0})$
    - $\forall x \forall y\ (x \times S(y) = x \times y + x)$
  - **Induktionsaxiome:** Für jede Formel $\varphi(x, \bar{y})$ das folgende Axiom

$$\forall \bar{y}\left(\left(\varphi(\mathbf{0}, \bar{y}) \wedge \forall x\Big(\varphi(x, \bar{y}) \rightarrow \varphi(S(x), \bar{y})\Big)\right) \rightarrow \forall x \varphi(x, \bar{y})\right)$$

- Hierbei: $S(x)$ ist die Nachfolgerfunktion (aus $+$ definierbar)

- Klar: $(\mathbb{N}, +, \times, \mathbf{0}, \mathbf{1})$ erfüllt alle diese Axiome
- **Frage:** Sind die Peano-Axiome (oder eine Erweiterung davon) ein Axiomensystem für die Arithmetik?

Das Scheitern von Hilberts Programm

- 1930 bewies Gödel zwei **Unvollständigkeitssätze**, die Hilberts Programm zerstörten

- **Erster Gödelscher Unvollständigkeitssatz**:

  Wenn Φ widerspruchsfrei und „mindestens so ausdrucksstark wie die Peano-Arithmetik“ ist, dann gibt es eine geschlossene Formel ψ der Arithmetik mit $\Phi \nvdash \psi$ und $\Phi \nvdash \neg\psi$

- Ein Beweissystem, wie von Hilbert erhofft, kann es also nicht geben

- Gödel zeigte in derselben Arbeit, dass auch hinsichtlich der Widerspruchsfreiheit Hilberts Ansatz nicht funktionieren kann

- **Zweiter Gödelscher Unvollständigkeitssatz**:

  Wenn Φ widerspruchsfrei und „mindestens so ausdrucksstark wie die Peano-Arithmetik“ ist, dann lässt sich die Widerspruchsfreiheit von Φ nicht aus Φ beweisen

- 1936 zeigten Alan Turing und Alonzo Church (unabhängig voneinander), dass das Hilbertsche „Entscheidungsproblem“ auch nicht gelöst werden kann

- **Unentscheidbarkeit der Arithmetik:**

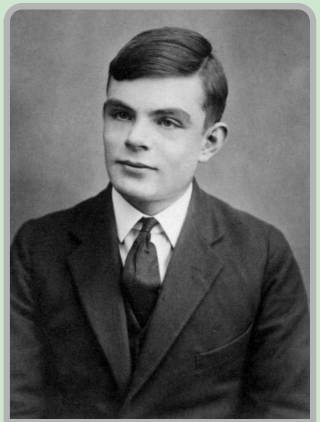
  Es gibt keinen Algorithmus, mit dem sich entscheiden lässt, ob für eine gegebene Formel ψ gilt: $(\mathbb{N}, +, \times, \mathbf{0}, \mathbf{1}) \models \psi$

Alonso Church und Alan Turing

Kurz-Bio: Alonzo Church

- Geboren: 14.6.1903 in Washington
- Studium in Princeton, Ph.D.: 1927
- Sein Beweis der Unlösbarkeit des „Entscheidungsproblems“ führte außerdem den Lambda-Kalkül ein
  - ➡ Grundlage der funktionalen Programmierung
- Gestorben am 11.8.1995 in Princeton

Kurz-Bio: Alan Turing



Alan Turing

Bild: © Gemeinfrei

- Geboren: 23.6.1912 in Paddington, London
- 1931-1934: King's College, Cambridge
- 1936: *On computable numbers with an application to the Entscheidungsproblem*
- 1938-1945: maßgeblich an der Entschlüsselung der Enigma-Chiffrier-Maschine beteiligt
- 1945-1950: Mitarbeit an Entwicklung und Programmierung eines der ersten Computer
- 1948: Marathon in 2h46
- 7.6.1954: Selbstmord

Entscheidungsprobleme der Prädikatenlogik

Entscheidungsprobleme für die Prädikatenlogik (1/4)

- **Wir haben:** Ein Verfahren, das die Unerfüllbarkeit prädikatenlogischer Formeln nachweist
- Bei Eingabe einer prädikatenlogischen Formel φ hat es
  - die Ausgabe „unerfüllbar“, falls φ unerfüllbar ist und
  - keine Ausgabe, falls φ erfüllbar ist

- **Wir hätten gerne:** Ein Verfahren, das bei Eingabe einer prädikatenlogischen Formel φ
  - die Ausgabe „unerfüllbar“ hat, falls φ unerfüllbar ist und
  - die Ausgabe „erfüllbar“ hat, falls φ erfüllbar ist

- Für die Aussagenlogik gibt es solche Verfahren:
  - Wahrheitstabelle
  - Resolution
  - und viele andere

Entscheidungsprobleme für die Prädikatenlogik (2/4)

- **Entscheidungsproblem** zu einer Menge L:
  - **Gegeben:** „Kandidat“ x
  - **Frage:** Ist $x \in L$?

- L heißt **entscheidbar**, falls es einen Algorithmus gibt, der **immer anhält** und
  - bei Eingabe $x \in L$ die Ausgabe „ja“ hat und
  - bei Eingabe $x \notin L$ die Ausgabe „nein“ hat

- L heißt **semi-entscheidbar**, falls es einen Algorithmus gibt, der
  - bei Eingabe $x \in L$ **anhält** und die Ausgabe „ja“ hat und
  - bei Eingabe $x \notin L$ die Ausgabe „nein“ hat oder **nicht anhält**

✎ Der Begriff **Algorithmus** wird in der Veranstaltung GTI präzisiert

Entscheidungsprobleme für die Prädikatenlogik (3/4)

- Die Menge der prädikatenlogischen Formeln lässt sich wie folgt strukturieren:



- Wir betrachten jeweils das Entscheidungsproblem zu den folgenden Mengen prädikatenlogischer Formeln:
  - $\boldsymbol{L_1} \stackrel{\text{def}}{=}$ Menge der erfüllbaren Formeln
  - $\boldsymbol{L_2} \stackrel{\text{def}}{=}$ Menge der unerfüllbaren Formeln
  - $\boldsymbol{L_3} \stackrel{\text{def}}{=}$ Menge der allgemeingültigen Formeln

    ☞ Tautologien
  - $\boldsymbol{L_4} \stackrel{\text{def}}{=}$ Menge der durch endliche Modelle erfüllbaren Formeln

- Bemerkung: $\boldsymbol{L_3 \subseteq L_4 \subseteq L_1}$

Entscheidungsprobleme für die Prädikatenlogik (4/4)

Satz C.6.4

(a) Die Menge L_1 der erfüllbaren prädikatenlogischen Formeln ist nicht semi-entscheidbar (und damit insbesondere auch nicht entscheidbar)

(b) Die folgenden Mengen prädikatenlogischer Formeln sind semi-entscheidbar, aber nicht entscheidbar:

- die Menge L_2 der unerfüllbaren Formeln
- die Menge L_3 der allgemein gültigen Formeln
- die Menge L_4 der durch endliche Modelle erfüllbaren Formeln

- Es ist also kein Zufall, dass der prädikatenlogische Resolutionsalgorithmus für erfüllbare Formeln nicht terminiert

Beweisidee

- Dass L_2 und L_3 semi-entscheidbar sind, folgt aus dem Resolutionssatz
- Die Semi-Entscheidbarkeit von L_4 folgt, da die endlichen Strukturen systematisch aufgezählt werden können bis ein endliches Modell gefunden ist
- Die Unentscheidbarkeit von L_1, L_2, L_3, und L_4 folgt letztlich aus der Unentscheidbarkeit des Halteproblems für Turingmaschinen

- **Genaueres:** In der GTI

- Satz C.6.4 gilt für Signaturen, die ein mindestens 2-stelliges Relations- oder ein mindestens 1-stelliges Funktionssymbol enthalten

Vergleich der betrachteten Logiken

Prädikatenlogik vs. Aussagenlogik

- Vieles ist ähnlich in Aussagenlogik und Prädikatenlogik
  - ➡ Aber offensichtlich ist die Prädikatenlogik mächtiger
- Wie ist das genaue Verhältnis zwischen den beiden?

- Aussagenlogik kann als Prädikatenlogik auf Strukturen aufgefasst werden, die ausschließlich 0-stellige Relationssymbole haben:

  Aussagenvariablen $\equiv$ 0-stellige Relationssymbole

✎ **Vorsicht:** Manches, was ähnlich heißt, hat in Aussagenlogik und Prädikatenlogik unterschiedliche Bedeutung!

- Aussagenlogische Variablen entsprechen **nicht** prädikatenlogischen Variablen
- Aussagenlogische und prädikatenlogische Substitutionen entsprechen sich ebenfalls **nicht**

☞ und die Notation ist verschieden

Prädikatenlogik vs. Modallogik (1/2)

- Gemäß Definition ist die Modallogik eine Erweiterung der Aussagenlogik
- Statt nur über eine Welt kann sie über verschiedene Welten sprechen

- Die Modallogik lässt sich andererseits in der Prädikatenlogik „simulieren“
- **Ansatz:** Simuliere $\Diamond$ durch $\exists$ und $\Box$ durch $\forall$
- **Formales Vorgehen:**
  - Modelliere Kripkestrukturen durch logische Strukturen
  - Übersetze modallogische Formeln induktiv in prädikatenlogische Formeln

- Definiere für jede Kripkestruktur $\mathcal{K} = (V, E, P)$ eine logische Struktur $\mathcal{K}'$ mit
  - Universum V,
  - 2-stelliger Relation E und
  - 1-stelligen Relationen P_X für jede aussagenlogische Variable X, die in einer Menge $P(s)$ für eine Welt s der Kripkestruktur vorkommt

Prädikatenlogik vs. Modallogik (2/2)

Satz C.6.5

- Zu jeder modallogischen Formel φ gibt es eine prädikatenlogische Formel $\varphi'(x)$, mit nur zwei Variablen x, y, so dass für jede Kripkestruktur $\mathcal{K}$ und jede Welt s von $\mathcal{K}$ gilt:

$$\mathcal{K}, s \models \varphi \iff (\mathcal{K}', x \mapsto s) \models \varphi'$$

Beweisskizze

- **Idee:** Eine modallogische Formel φ kann induktiv in eine prädikatenlogische Formel φ' übersetzt werden

- **Intuition:**
  - $\Diamond$ wird durch $\exists$ simuliert
  - $\Box$ wird durch $\forall$ simuliert

- Übersetze:
  - A in $P_A(x)$
  - $\neg\chi$ in $\neg\chi'$
  - $\chi \wedge \psi$ in $\chi' \wedge \psi'$
  - $\Diamond\chi$ in $\exists y(E(x,y) \wedge \chi'')$, wobei χ'' die Formel ist, die aus χ' durch Vertauschen aller Vorkommen von x und y entsteht

- Dass die Konstruktion korrekt ist, lässt sich durch Induktion nach dem Aufbau von φ beweisen

Beispiel

- Die Formel $\Diamond\Box A$ lässt sich beispielsweise in die folgende Formel $\varphi'(x)$ übersetzen:

$$\exists y \left(E(x,y) \wedge (\forall x \, (E(y,x) \rightarrow P_A(x)))\right)$$

Auflösung Schloss Dreadsbury

Schloss Dreadsbury: Auflösung (1/3)

- Aus dem Dreadsbury-Beispiel in Kapitel C.1 ergeben sich die folgenden Klauseln

- $\{\neg K(x,y), H(x,y)\}$
  - Ein Mörder hasst sein Opfer immer
- $\{\neg K(x,y), \neg R(x,y)\}$
  - Ein Mörder ... ist aber niemals reicher als sein Opfer
- $\{\neg K(x,y), H(x,y)\}$
  - Ein Mörder hasst sein Opfer immer
- $\{\neg H(a,x), \neg H(c,x)\}$
  - Der Chauffeur hasst niemanden, den der Arzt hasst
- $\{R(x,a), H(b,x)\}$
  - Der Butler hasst jeden, der nicht reicher als der Arzt ist
- $\{\neg H(a,x), H(b,x)\}$
  - Der Butler hasst jeden, den der Arzt hasst

Schloss Dreadsbury: Auflösung (2/3)

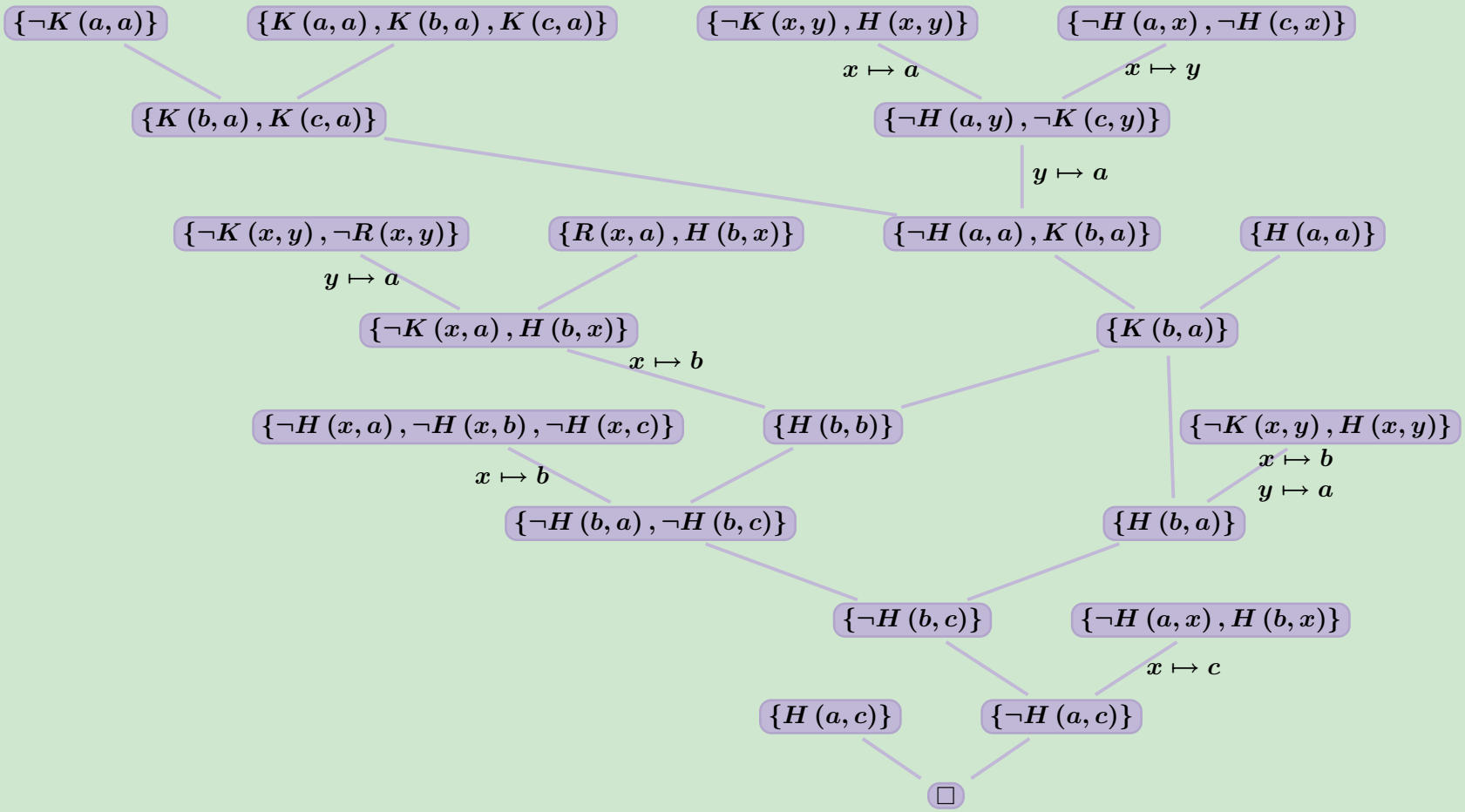
- Die folgenden Klauseln entstehen durch sinnvolle Ersetzung der Aussagen, die Gleichheitsatome beinhalten

- $\{K(a,a), K(b,a), K(c,a)\}$
  - Ein Bewohner von Schloss Dreadsbury hat den dortigen Arzt ermordet
  - $\exists x \Big(D(x) \wedge K(x,a) \Big)$
  - $D(a) \wedge D(b) \wedge D(c)$
  - $\forall x \Big(D(x) \rightarrow \big(x = a \vee x = b \vee x = c \big) \Big)$
- $\{H(a,a)\}, \{H(a,c)\}$
  - Der Arzt hasst jeden, der nicht der Butler ist
  - $\forall x \Big(\neg (x = b) \rightarrow H(a,x) \Big)$
- $\{\neg H(x,a), \neg H(x,b), \neg H(x,c)\}$
  - Niemand hasst jeden (aus Schloss Dreadsbury)

- Wir wollen zeigen: Der Arzt hat Selbstmord begangen
- Zu zeigen: Unerfüllbarkeit der obigen Klauseln ergänzt um $\{\neg K(a,a)\}$

Schloss Dreadsbury: Auflösung (3/3)

Beispiel



- Da sich aus der Annahme $\neg K(a,a)$ die Box herleiten lässt, folgt also $K(a,a)$

Hilbert- und Sequenzenkalkül

Andere Beweiskalküle

- Außer dem Resolutionskalkül gibt es viele verschiedene Beweiskalküle für die Prädikatenlogik
- Wir werfen einen Blick auf zwei solche Kalküle

- **Hilbertkalkül**
  - Der Hilbert-Kalkül hat nur wenige Axiome und Regeln
  - Es ist aber ziemlich schwierig, Ableitungen für Formeln zu finden

- **Sequenzenkalkül**
  - Es gibt eine Vielzahl von Kalkülen aus der Familie der Sequenzenkalküle
  - Der hier betrachtete Sequenzenkalkül hat nur ein Axiom aber zehn Schlussregeln
  - Es ist meist erheblich einfacher, einen Beweis im Sequenzenkalkül als im Hilbertkalkül zu finden
  - Sequenzenkalküle werden häufig für automatische Beweise eingesetzt

Andere Beweiskalküle: Hilbertkalkül

Aussagenlogischer Hilbert-Kalkül

- Die Menge AX der Axiome enthalte alle Formeln der folgenden Form:

  (A1) $[\varphi_1 \rightarrow (\varphi_2 \rightarrow \varphi_3)] \rightarrow [(\varphi_1 \rightarrow \varphi_2) \rightarrow (\varphi_1 \rightarrow \varphi_3)]$

  (A2) $\varphi_1 \rightarrow (\varphi_2 \rightarrow \varphi_1)$

  (A3) $(\neg\varphi_1 \rightarrow \neg\varphi_2) \rightarrow (\varphi_2 \rightarrow \varphi_1)$

- Die einzige **Schlussregel** ist:
  - **Modus ponens**: $\dfrac{\psi \qquad \psi \rightarrow \psi'}{\psi'}$

- ψ ist **Hilbert-beweisbar** aus Φ, falls es eine Folge $\psi_1, \ldots, \psi_n$ gibt, mit:
  - $\psi_n = \psi$ und
  - jedes ψ_i ist entweder
    - aus AX oder aus Φ oder
    - entsteht durch Anwendung des Modus Ponens auf ψ_j und ψ_k mit $j, k < i$
- Notation: $\Phi \vdash_H \psi$

- Der Kalkül ist vollständig und korrekt für die Aussagenlogik
- Er kann auf die Prädikatenlogik verallgemeinert werden

- Obwohl die Regeln des Hilbertkalküls recht nahe liegend wirken, ist es recht schwierig, mit ihm Beweise zu finden

Beispiel

- Wir zeigen: $\emptyset \vdash_H \varphi \rightarrow \varphi$
  - ➡ D.h. $\varphi \rightarrow \varphi$ ist für jede Formel φ eine Tautologie

(1) $[\varphi \rightarrow ((\varphi \rightarrow \varphi) \rightarrow \varphi)] \rightarrow [(\varphi \rightarrow (\varphi \rightarrow \varphi)) \rightarrow (\varphi \rightarrow \varphi)]$ ☞ aus (A1)

(2) $\varphi \rightarrow ((\varphi \rightarrow \varphi) \rightarrow \varphi)$ ☞ aus (A2)

(3) $(\varphi \rightarrow (\varphi \rightarrow \varphi)) \rightarrow (\varphi \rightarrow \varphi)$ ☞ MP aus (1) und (2)

(4) $\varphi \rightarrow (\varphi \rightarrow \varphi)$ ☞ aus (A2)

(5) $\varphi \rightarrow \varphi$ ☞ MP aus (3) und (4)

Andere Beweiskalküle: Sequenzenkalkül (1/3)

- Der **Sequenzenkalkül** verwendet Aussagen der Art $\Gamma \Rightarrow \Delta$, wobei Γ und Δ Mengen von Formeln sind
  - Diese Aussagen heißen **Sequenzen**
  - Mengenklammern werden dabei meistens weggelassen

- Auch der Sequenzenkalkül besteht aus Axiomen und Schlussregeln
  - Axiome sind alle Sequenzen der Art $\Gamma, \varphi \Rightarrow \Delta, \varphi$
  - Für jeden logischen Operator und Quantor gibt es jeweils zwei Schlussregeln

✎ Bemerkung: es gibt viele Varianten des Sequenzenkalküls mit unterschiedlicher Notation

- Eine Sequenz $\Gamma \Rightarrow \Delta$ heißt **gültig**, falls für jede passende Struktur $\mathcal{A}$ gilt:
  - Wenn $\mathcal{A} \models \Gamma$ dann gibt es **mindestens** eine Formel $\varphi \in \Delta$ mit $\mathcal{A} \models \varphi$

- Beobachtung: Axiome sind gültig
- **Idee:** Schlussregeln leiten gültige Sequenzen aus gültigen Sequenzen ab

- **Anwendung:** Um $\Phi \models \psi$ nachzuweisen, wird gezeigt, dass sich $\Phi \Rightarrow \psi$ mit den Schlussregeln aus den Axiomen ableiten lässt

- Es ist oft einfacher, Beweise im Sequenzenkalkül zu finden als im Hilbertkalkül

Andere Beweiskalküle: Sequenzenkalkül (2/3)

Aussagenlogischer Sequenzenkalkül

- **Axiome:** $\Gamma, \varphi \Rightarrow \Delta, \varphi$
- **„Aussagenlogische“ Schlussregeln:**
  - $\dfrac{\Gamma, \varphi \Rightarrow \Delta \qquad \Gamma, \psi \Rightarrow \Delta}{\Gamma, \varphi \vee \psi \Rightarrow \Delta}$ $(\vee \Rightarrow)$
  - $\dfrac{\Gamma \Rightarrow \Delta, \varphi, \psi}{\Gamma \Rightarrow \Delta, \varphi \vee \psi}$ $(\Rightarrow \vee)$
  - $\dfrac{\Gamma, \varphi, \psi \Rightarrow \Delta}{\Gamma, \varphi \wedge \psi \Rightarrow \Delta}$ $(\wedge \Rightarrow)$
  - $\dfrac{\Gamma \Rightarrow \Delta, \varphi \qquad \Gamma \Rightarrow \Delta, \psi}{\Gamma \Rightarrow \Delta, \varphi \wedge \psi}$ $(\Rightarrow \wedge)$
  - $\dfrac{\Gamma \Rightarrow \Delta, \varphi}{\Gamma, \neg\varphi \Rightarrow \Delta}$ $(\neg \Rightarrow)$
  - $\dfrac{\Gamma, \varphi \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, \neg\varphi}$ $(\Rightarrow \neg)$

Prädikatenlogischer Sequenzenkalkül

- **„Prädikatenlogische“ Schlussregeln:**
  - Falls c nicht in Γ, Δ oder φ vorkommt:
    - $\dfrac{\Gamma, \varphi[x/c] \Rightarrow \Delta}{\Gamma, \exists x\, \varphi(x) \Rightarrow \Delta}$ $(\exists \Rightarrow)$
    - $\dfrac{\Gamma \Rightarrow \Delta, \varphi(c)}{\Gamma \Rightarrow \Delta, \forall x\, \varphi(x)}$ $(\Rightarrow \forall)$
  - Falls x die einzige freie Variable von φ ist und t ein Grundterm ist:
    - $\dfrac{\Gamma \Rightarrow \Delta, \varphi[x/t]}{\Gamma \Rightarrow \Delta, \exists x\, \varphi(x)}$ $(\Rightarrow \exists)$
    - $\dfrac{\Gamma, \varphi[x/t] \Rightarrow \Delta}{\Gamma, \forall x\, \varphi(x) \Rightarrow \Delta}$ $(\forall \Rightarrow)$

✎ Dies ist der Sequenzenkalkül für die Prädikatenlogik ohne Gleichheit

Andere Beweiskalküle: Sequenzenkalkül (3/3)

- Um eine Sequenz $\Gamma \Rightarrow \Delta$ zu beweisen, wird versucht, ausgehend von $\Gamma \Rightarrow \Delta$, durch **Rückwärtsanwendung von Schlussregeln** zu einem (oder bei Verzweigung mehreren) Axiomen zu gelangen

Beispiel

- Wir betrachten einen Beweis der Aussage

$$\exists x\, \forall y\, R(x,y) \models \forall y\, \exists x\, R(x,y)$$

$$\begin{array}{r} \exists x \forall y R(x,y) \implies \forall y \exists x R(x,y) \\ \big| \; (\exists \implies) \\ \forall y R(c,y) \implies \forall y \exists x R(x,y) \\ \big| \; (\implies \forall) \\ \forall y R(c,y) \implies \exists x R(x,d) \\ \big| \; (\forall \implies) \\ R(c,d) \implies \exists x R(x,d) \\ \big| \; (\implies \exists) \\ R(c,d) \implies R(c,d) \end{array}$$

Zusammenfassung

- **Gute Nachrichten:**
  - $\Phi \models \psi$ und $\Phi \vdash \psi$ sind äquivalente Aussagen, jede Folgerung lässt sich also beweisen
  - Um die Unerfüllbarkeit einer Formelmenge zu erkennen, genügt es, ihre endlichen Teilmengen zu betrachten
  - Diese Aussagen gelten auch für die Prädikatenlogik mit Gleichheit
  - Prädikatenlogische Formeln können auf endlichen Strukturen algorithmisch ausgewertet werden

- **Schlechte Nachrichten:**
  - Fragen hinsichtlich der Erfüllbarkeit oder Gültigkeit von PL-Formeln sind algorithmisch nicht lösbar
  - Die Gödelschen Unvollständigkeitssätze geben weitere Grenzen der formalen Methode an

- **Literaturempfehlung:**
  - D.W. Hoffmann. *Grenzen der Mathematik: eine Reise durch die Kerngebiete der Mathematischen Logik.* Spektrum-Akademischer Vlg, 2011

Logik in der Informatik: Übersicht

